

```
// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf
// File: WearManager.java
```

```
package com.android.chileaf;
```

```
import android.annotation.SuppressLint;
import android.bluetooth.BluetoothDevice;
import android.bluetooth.BluetoothGatt;
import android.bluetooth.BluetoothGattCharacteristic;
import android.bluetooth.BluetoothGattService;
import android.content.Context;
import android.os.ParcelUuid;
import android.text.TextUtils;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import com.android.chileaf.fitness.FitnessManager;
import com.android.chileaf.fitness.callback.AccelerometerCallback;
import com.android.chileaf.fitness.callback.BloodOxygenCallback;
import com.android.chileaf.fitness.callback.BluetoothStatusCallback;
import com.android.chileaf.fitness.callback.BodyHealthCallback;
import com.android.chileaf.fitness.callback.BodySportCallback;
import com.android.chileaf.fitness.callback.BodySportHealthCallback;
import com.android.chileaf.fitness.callback.CustomDataReceivedCallback;
import com.android.chileaf.fitness.callback.HeartRateAlarmCallback;
import com.android.chileaf.fitness.callback.HeartRateMaxCallback;
import com.android.chileaf.fitness.callback.HeartRateStatusCallback;
import com.android.chileaf.fitness.callback.HistoryOf3DDataCallback;
import com.android.chileaf.fitness.callback.HistoryOfHRDataCallback;
import com.android.chileaf.fitness.callback.HistoryOfHRRRecordCallback;
import com.android.chileaf.fitness.callback.HistoryOfRRRDataCallback;
import com.android.chileaf.fitness.callback.HistoryOfRRRRecordCallback;
import com.android.chileaf.fitness.callback.HistoryOfSingleRecordCallback;
import com.android.chileaf.fitness.callback.HistoryOfSleepCallback;
import com.android.chileaf.fitness.callback.HistoryOfSportCallback;
import com.android.chileaf.fitness.callback.IntervalStepCallback;
import com.android.chileaf.fitness.callback.Sensor3DFrequencyCallback;
import com.android.chileaf.fitness.callback.Sensor3DStatusCallback;
import com.android.chileaf.fitness.callback.Sensor6DFrequencyCallback;
import com.android.chileaf.fitness.callback.Sensor6DRawDataCallback;
import com.android.chileaf.fitness.callback.SingleTapRecordCallback;
import com.android.chileaf.fitness.callback.TemperatureCallback;
import com.android.chileaf.fitness.callback.UserInfoCallback;
import com.android.chileaf.fitness.callback.WearManagerCallbacks;
import com.android.chileaf.fitness.callback.WearReceivedDataCallback;
import com.android.chileaf.fitness.common.FilterScanCallback;
import com.android.chileaf.fitness.common.heart.BodySensorLocationDataCallback;
import com.android.chileaf.fitness.common.heart.HeartRateMeasurementCallback;
import
com.android.chileaf.fitness.common.heart.HeartRateMeasurementDataCallback;
import com.android.chileaf.fitness.common.parser.BodySensorLocationParser;
import com.android.chileaf.fitness.common.parser.HeartRateMeasurementParser;
```

```

import com.android.chileaf.model.HistoryOf3D;
import com.android.chileaf.model.HistoryOfHeartRate;
import com.android.chileaf.model.HistoryOfRecord;
import com.android.chileaf.model.HistoryOfRespiratoryRate;
import com.android.chileaf.model.HistoryOfSport;
import com.android.chileaf.model.HistorySleep;
import com.android.chileaf.model.IntervalStep;
import com.android.chileaf.util.DateUtil;
import com.android.chileaf.util.HexUtil;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.Locale;
import java.util.UUID;
import no.nordicsemi.android.ble.BleManager;
import no.nordicsemi.android.ble.callback.DataReceivedCallback;
import no.nordicsemi.android.ble.data.Data;
import no.nordicsemi.android.support.v18.scanner.BluetoothLeScannerCompat;
import no.nordicsemi.android.support.v18.scanner.ScanCallback;
import no.nordicsemi.android.support.v18.scanner.ScanFilter;
import no.nordicsemi.android.support.v18.scanner.ScanResult;
import no.nordicsemi.android.support.v18.scanner.ScanSettings;

@SuppressLint("MissingPermission")
public class WearManager extends FitnessManager<WearManagerCallbacks> {
    private static final UUID HR_SERVICE_UUID =
    UUID.fromString("0000180D-0000-1000-8000-00805f9b34fb");
    private static final UUID BODY_SENSOR_LOCATION_CHARACTERISTIC_UUID =
    UUID.fromString("00002A38-0000-1000-8000-00805f9b34fb");
    private static final UUID HEART_RATE_MEASUREMENT_CHARACTERISTIC_UUID =
    UUID.fromString("00002A37-0000-1000-8000-00805f9b34fb");
    private static final String[] MODE_NAMES = new String[]{"CL831", "CL880N"};
    private WearManager.WearScanCallback mScanCallback;
    private static WearManager managerInstance = null;
    private String[] mFilterNames = null;
    private BluetoothGattCharacteristic mBodySensorLocationCharacteristic;
    private BluetoothGattCharacteristic mHeartRateCharacteristic;
    private UserInfoCallback mUserInfoCallback;
    private AccelerometerCallback mAccelerometerCallback;
    private BodySportCallback mBodySportCallback;
    private BodyHealthCallback mBodyHealthCallback;
    private HeartRateMeasurementCallback mHeartRateMeasurementCallback;
    private HistoryOfSportCallback mHistoryOfSportCallback;
    private HistoryOfHRRRecordCallback mHistoryOfHRRRecordCallback;
    private HistoryOfHRDataCallback mHistoryOfHRDataCallback;
    private HistoryOfRRRRecordCallback mHistoryOfRRRRecordCallback;
    private HistoryOfRRRDataCallback mHistoryOfRRRDataCallback;
    private IntervalStepCallback mIntervalStepsCallback;
    private SingleTapRecordCallback mSingleTapRecordCallback;
    private BluetoothStatusCallback mBluetoothStatusCallback;
    private CustomDataReceivedCallback mCustomDataReceivedCallback;
    private HeartRateStatusCallback mHeartRateStatusCallback;
    private BloodOxygenCallback mBloodOxygenCallback;
    private TemperatureCallback mTemperatureCallback;

```

```

private HistoryOfSingleRecordCallback mHistoryOfSingleRecordCallback;
private HeartRateAlarmCallback mHeartRateAlarmCallback;
private HeartRateMaxCallback mHeartRateMaxCallback;
private HistoryOfSleepCallback mHistoryOfSleepCallback;
private Sensor3DFrequencyCallback mSensor3DFrequencyCallback;
private Sensor3DStatusCallback mSensor3DStatusCallback;
private Sensor6DFrequencyCallback mSensor6DFrequencyCallback;
private Sensor6DRawDataCallback mSensor6DRawDataCallback;
private BodySportHealthCallback mBodySportHealthCallback;
private HistoryOf3DDataCallback mHistoryOf3DDataCallback;
private final BodySensorLocationDataCallback mBodySensorLocationDataCallback
= new BodySensorLocationDataCallback() {
    public void onBodySensorLocationReceived(@NonNull final BluetoothDevice
device, final int sensorLocation) {

((WearManagerCallbacks)WearManager.this.mCallbacks).onBodySensorLocationReceived
(device, sensorLocation);
    }

    public void onDataReceived(@NonNull final BluetoothDevice device, @NonNull
final Data data) {
        WearManager.this.log(3, String.format("%s received",
BodySensorLocationParser.parse(data)));
        super.onDataReceived(device, data);
    }
};
private final HeartRateMeasurementDataCallback mHeartRateMeasureDataCallback
= new HeartRateMeasurementDataCallback() {
    public void onHeartRateMeasurementReceived(@NonNull BluetoothDevice
device, int heartRate, @Nullable Boolean contactDetected, @Nullable Integer
energyExpanded, @Nullable List<Integer> rrIntervals) {

((WearManagerCallbacks)WearManager.this.mCallbacks).onHeartRateMeasurementReceiv
ed(device, heartRate, contactDetected, energyExpanded, rrIntervals);
        if (WearManager.this.mHeartRateMeasurementCallback != null) {

WearManager.this.mHeartRateMeasurementCallback.onHeartRateMeasurementReceived(de
vice, heartRate, contactDetected, energyExpanded, rrIntervals);
        }

    }

    public void onDataReceived(@NonNull final BluetoothDevice device, @NonNull
final Data data) {
        WearManager.this.log(3, HeartRateMeasurementParser.parse(data) + "
received");
        super.onDataReceived(device, data);
    }
};
private final WearReceivedDataCallback mReceivedDataCallback = new
WearReceivedDataCallback() {
    public void onSportReceived(@NonNull BluetoothDevice device, int step, int
distance, int calorie) {

```

```

((WearManagerCallbacks)WearManager.this.mCallbacks).onSportReceived(device,
step, distance, calorie);
    if (WearManager.this.mBodySportCallback != null) {
        WearManager.this.mBodySportCallback.onSportReceived(device, step,
distance, calorie);
    }

}

    public void onHealthReceived(@NonNull BluetoothDevice device, int vo2Max,
int breathRate, int emotionLevel, int stressPercent, int stamina, float tp,
float lf, float hf) {
        if (WearManager.this.mBodyHealthCallback != null) {
            WearManager.this.mBodyHealthCallback.onHealthReceived(device,
vo2Max, breathRate, emotionLevel, stressPercent, stamina, tp, lf, hf);
        }

    }

    public void onSportHealthReceived(@NonNull BluetoothDevice device, int
vo2Max, int breathRate, int emotion, int pressure, int stamina) {
        if (WearManager.this.mBodySportHealthCallback != null) {

WearManager.this.mBodySportHealthCallback.onSportHealthReceived(device, vo2Max,
breathRate, emotion, pressure, stamina);
        }

    }

    public void onBluetoothStatusReceived(@NonNull BluetoothDevice device,
boolean enabled) {

((WearManagerCallbacks)WearManager.this.mCallbacks).onBluetoothStatusReceived(de
vice, enabled);
        if (WearManager.this.mBluetoothStatusCallback != null) {

WearManager.this.mBluetoothStatusCallback.onBluetoothStatusReceived(device,
enabled);
        }

    }

    public void onUserInfoReceived(@NonNull BluetoothDevice device, int age,
int sex, int weight, int height, long userId) {
        if (WearManager.this.mUserInfoCallback != null) {
            WearManager.this.mUserInfoCallback.onUserInfoReceived(device, age,
sex, weight, height, userId);
        }

    }

    public void onHistoryOfSportReceived(@NonNull BluetoothDevice device,
List<HistoryOfSport> sports) {
        if (WearManager.this.mHistoryOfSportCallback != null) {

```

```

WearManager.this.mHistoryOfSportCallback.onHistoryOfSportReceived(device,
sports);
    }

    }

    public void onHistoryOfHRRRecordReceived(@NonNull BluetoothDevice device,
List<HistoryOfRecord> records) {
        if (WearManager.this.mHistoryOfHRRRecordCallback != null) {

WearManager.this.mHistoryOfHRRRecordCallback.onHistoryOfHRRRecordReceived(device,
records);
        }

    }

    public void onHistoryOfHRDataReceived(@NonNull BluetoothDevice device,
List<HistoryOfHeartRate> heartRates) {
        if (WearManager.this.mHistoryOfHRDataCallback != null) {

WearManager.this.mHistoryOfHRDataCallback.onHistoryOfHRDataReceived(device,
heartRates);
        }

    }

    public void onHistoryOfRRRRecordReceived(@NonNull BluetoothDevice device,
List<HistoryOfRecord> records) {
        if (WearManager.this.mHistoryOfRRRRecordCallback != null) {

WearManager.this.mHistoryOfRRRRecordCallback.onHistoryOfRRRRecordReceived(device,
records);
        }

    }

    public void onHistoryOfRRRDataReceived(@NonNull BluetoothDevice device,
List<HistoryOfRespiratoryRate> respiratoryRates) {
        if (WearManager.this.mHistoryOfRRRDataCallback != null) {

WearManager.this.mHistoryOfRRRDataCallback.onHistoryOfRRRDataReceived(device,
respiratoryRates);
        }

    }

    public void onIntervalStepReceived(@NonNull BluetoothDevice device,
List<IntervalStep> steps) {
        if (WearManager.this.mIntervalStepsCallback != null) {

WearManager.this.mIntervalStepsCallback.onIntervalStepReceived(device, steps);
        }

```

```

    }

    public void onSingleTapRecordReceived(@NonNull BluetoothDevice device,
List<HistoryOfRecord> records) {
        if (WearManager.this.mSingleTapRecordCallback != null) {

WearManager.this.mSingleTapRecordCallback.onSingleTapRecordReceived(device,
records);
        }

    }

    public void onHeartRateStatusReceived(@NonNull BluetoothDevice device, int
min, int max, int goal) {
        if (WearManager.this.mHeartRateStatusCallback != null) {

WearManager.this.mHeartRateStatusCallback.onHeartRateStatusReceived(device, min,
max, goal);
        }

    }

    public void onHistoryOf3DDataReceived(@NonNull BluetoothDevice device,
HistoryOf3D history, boolean finish) {
        if (WearManager.this.mHistoryOf3DDataCallback != null) {

WearManager.this.mHistoryOf3DDataCallback.onHistoryOf3DDataReceived(device,
history, finish);
        }

    }

    public void onBloodOxygenReceived(@NonNull BluetoothDevice device, int
bSwitch, String value, int gesture, int piValue, int onwrist) {
        if (WearManager.this.mBloodOxygenCallback != null) {
            WearManager.this.mBloodOxygenCallback.onBloodOxygenReceived(device,
bSwitch, value, gesture, piValue, onwrist);
        }

    }

    public void onTemperatureReceived(@NonNull BluetoothDevice device, float
environment, float wrist, float body) {
        if (WearManager.this.mTemperatureCallback != null) {
            WearManager.this.mTemperatureCallback.onTemperatureReceived(device,
environment, wrist, body);
        }

    }

    public void onHistorySingleRecordReceived(@NonNull BluetoothDevice device,
long stamp, long step, long distance, long calorie) {
        if (WearManager.this.mHistoryOfSingleRecordCallback != null) {

```

```

WearManager.this.mHistoryOfSingleRecordCallback.onHistorySingleRecordReceived(device, stamp, step, distance, calorie);
    }

    }

    public void onHeartRateAlarmReceived(@NonNull BluetoothDevice device, long stamp, boolean enabled) {
        if (WearManager.this.mHeartRateAlarmCallback != null) {

WearManager.this.mHeartRateAlarmCallback.onHeartRateAlarmReceived(device, stamp, enabled);
        }

    }

    public void onAccelerometerReceived(@NonNull BluetoothDevice device, int x, int y, int z) {
        if (WearManager.this.mAccelerometerCallback != null) {

WearManager.this.mAccelerometerCallback.onAccelerometerReceived(device, x, y, z);
        }

    }

    public void onHeartRateMaxReceived(@NonNull BluetoothDevice device, int max) {
        if (WearManager.this.mHeartRateMaxCallback != null) {

WearManager.this.mHeartRateMaxCallback.onHeartRateMaxReceived(device, max);
        }

    }

    public void onHistoryOfSleepReceived(@NonNull BluetoothDevice device, List<HistorySleep> sleeps) {
        if (WearManager.this.mHistoryOfSleepCallback != null) {

WearManager.this.mHistoryOfSleepCallback.onHistoryOfSleepReceived(device, sleeps);
        }

    }

    public void onSensor3DFrequencyReceived(@NonNull BluetoothDevice device, int frequency) {
        if (WearManager.this.mSensor3DFrequencyCallback != null) {

WearManager.this.mSensor3DFrequencyCallback.onSensor3DFrequencyReceived(device, frequency);
        }

    }

```

```

        public void onSensor3DStatusReceived(@NonNull BluetoothDevice device,
boolean enabled) {
            if (WearManager.this.mSensor3DStatusCallback != null) {

WearManager.this.mSensor3DStatusCallback.onSensor3DStatusReceived(device,
enabled);
            }

        }

        public void onSensor6DFrequencyReceived(@NonNull BluetoothDevice device,
int sensor) {
            if (WearManager.this.mSensor6DFrequencyCallback != null) {

WearManager.this.mSensor6DFrequencyCallback.onSensor6DFrequencyReceived(device,
sensor);
            }

        }

        public void onSensor6DRawDataReceived(@NonNull BluetoothDevice device,
long utc, int sequence, int gyroscopeX, int gyroscopeY, int gyroscopeZ, int
accelerometerX, int accelerometerY, int accelerometerZ) {
            if (WearManager.this.mSensor6DRawDataCallback != null) {

WearManager.this.mSensor6DRawDataCallback.onSensor6DRawDataReceived(device, utc,
sequence, gyroscopeX, gyroscopeY, gyroscopeZ, accelerometerX, accelerometerY,
accelerometerZ);
            }

        }
    };

    public static synchronized WearManager getInstance(final Context context) {
        if (managerInstance == null) {
            managerInstance = new WearManager(context);
        }

        return managerInstance;
    }

    public WearManager(final Context context) {
        super(context);
    }

    public void checkModel(String modelName, boolean isCL833) {
        boolean cl833 = this.checkMode(modelName);
        this.mReceivedDataCallback.setCL833(cl833 && isCL833);
        this.log(3, String.format("Check modelName:%s isCL833:%s", modelName,
isCL833));
    }

    private boolean checkMode(String name) {

```



```

String[] var2 = MODE_NAMES;
int var3 = var2.length;

for(int var4 = 0; var4 < var3; ++var4) {
    String device = var2[var4];
    if (device.equalsIgnoreCase(name)) {
        return true;
    }
}

return false;
}

@NonNull
protected BleManager.BleManagerGattCallback getGattCallback() {
    return new WearManager.WearManagerGattCallback();
}

public void addBodySportCallback(final BodySportCallback callback) {
    this.mBodySportCallback = callback;
}

public void addBodyHealthCallback(final BodyHealthCallback callback) {
    this.mBodyHealthCallback = callback;
}

public void addHeartRateMeasurementCallback(final
HeartRateMeasurementCallback callback) {
    this.mHeartRateMeasurementCallback = callback;
}

public void setBluetoothStatusCallback(BluetoothStatusCallback callback) {
    this.mBluetoothStatusCallback = callback;
}

public void addUserInfoCallback(final UserInfoCallback callback) {
    this.mUserInfoCallback = callback;
}

public void addHistoryOfSportCallback(final HistoryOfSportCallback callback)
{
    this.mHistoryOfSportCallback = callback;
}

public void addHistoryOfHRRRecordCallback(final HistoryOfHRRRecordCallback
callback) {
    this.mHistoryOfHRRRecordCallback = callback;
}

public void addHistoryOfHRDataCallback(final HistoryOfHRDataCallback
callback) {
    this.mHistoryOfHRDataCallback = callback;
}

```

```

    public void addHistoryOfRRRRecordCallback(final HistoryOfRRRRecordCallback
callback) {
        this.mHistoryOfRRRRecordCallback = callback;
    }

    public void addHistoryOfRRRDataCallback(final HistoryOfRRRDataCallback
callback) {
        this.mHistoryOfRRRDataCallback = callback;
    }

    public void addIntervalStepCallback(final IntervalStepCallback callback) {
        this.mIntervalStepsCallback = callback;
    }

    public void addSingleTapRecordCallback(final SingleTapRecordCallback
callback) {
        this.mSingleTapRecordCallback = callback;
    }

    public void addHistoryOf3DDDataCallback(final HistoryOf3DDDataCallback
callback) {
        this.mHistoryOf3DDDataCallback = callback;
    }

    public void addHeartRateStatusCallback(HeartRateStatusCallback callback) {
        this.mHeartRateStatusCallback = callback;
    }

    public void addBloodOxygenCallback(final BloodOxygenCallback callback) {
        this.mBloodOxygenCallback = callback;
    }

    public void addTemperatureCallback(final TemperatureCallback callback) {
        this.mTemperatureCallback = callback;
    }

    public void addHistoryOfSingleRecordCallback(final
HistoryOfSingleRecordCallback callback) {
        this.mHistoryOfSingleRecordCallback = callback;
    }

    public void addHeartRateAlarmCallback(HeartRateAlarmCallback callback) {
        this.mHeartRateAlarmCallback = callback;
    }

    public void addAccelerometerCallback(final AccelerometerCallback callback) {
        this.mAccelerometerCallback = callback;
    }

    public void addHeartRateMaxCallback(HeartRateMaxCallback callback) {
        this.mHeartRateMaxCallback = callback;
    }

    public void addHistoryOfSleepCallback(HistoryOfSleepCallback callback) {

```

```

        this.mHistoryOfSleepCallback = callback;
    }

    public void addSensor3DFrequencyCallback(Sensor3DFrequencyCallback callback)
    {
        this.mSensor3DFrequencyCallback = callback;
    }

    public void addSensor3DStatusCallback(Sensor3DStatusCallback callback) {
        this.mSensor3DStatusCallback = callback;
    }

    public void addSensor6DFrequencyCallback(final Sensor6DFrequencyCallback
callback) {
        this.mSensor6DFrequencyCallback = callback;
    }

    public void addSensor6DRawDataCallback(final Sensor6DRawDataCallback
callback) {
        this.mSensor6DRawDataCallback = callback;
    }

    public void addBodySportHealthCallback(final BodySportHealthCallback
callback) {
        this.mBodySportHealthCallback = callback;
    }

    public void setFilterNames(String... filterNames) {
        this.mFilterNames = filterNames;
    }

    public void startScan(final FilterScanCallback callback) {
        this.mScanCallback = new WearManager.WearScanCallback(callback);
        BluetoothLeScannerCompat scanner = BluetoothLeScannerCompat.getScanner();
        ScanSettings settings = (new
ScanSettings.Builder()).setLegacy(false).setReportDelay(1000L).setUseHardwareBat
chingIfSupported(false).setScanMode(2).build();
        List<ScanFilter> filters = new ArrayList();
        ParcelUuid uuid = new ParcelUuid(HR_SERVICE_UUID);
        filters.add((new ScanFilter.Builder()).setServiceUuid(uuid).build());
        scanner.startScan(filters, settings, this.mScanCallback);
    }

    public void stopScan() {
        if (this.mScanCallback != null) {
            BluetoothLeScannerCompat scanner =
BluetoothLeScannerCompat.getScanner();
            scanner.stopScan(this.mScanCallback);
        }
    }

    private void sendCommand(final byte cmd, final int... values) {
        int unique = true;

```

```

        byte[] result;
        byte[] command;
        if (values != null) {
            int len = values.length + 4;
            command = HexUtil.compose(255, len, cmd);
            byte[] bytes = HexUtil.compose(values);
            result = HexUtil.append(command, bytes);
        } else {
            result = HexUtil.compose(255, 4, cmd);
        }

        byte check = this.checkSum(result);
        command = HexUtil.append(result, check);
        this.writeTxCharacteristic(command);
    }

    protected int[] utc2Bytes(final long stamp) {
        int[] utcArray = new int[]{(int)(stamp >> 24), (int)(stamp >> 16),
(int)(stamp >> 8), (int)stamp};
        return utcArray;
    }

    public void setUTCTime() {
        this.setUTCTime(DateUtil.getZoneUTC());
    }

    public void setUTCTime(final long stamp) {
        this.sendCommand((byte)8, this.utc2Bytes(stamp));
    }

    public void shutdown() {
        this.sendCommand((byte)-15, 0);
    }

    public void restoration() {
        this.sendCommand((byte)-13, 0);
    }

    public void setBluetoothDisabled() {
        this.sendCommand((byte)63, 2);
    }

    public void getHistoryOfSport() {
        this.mReceivedDataCallback.clearType(2);
        this.sendCommand((byte)22, 0);
    }

    public void getHistoryOfHRRRecord() {
        this.mReceivedDataCallback.clearType(4);
        this.sendCommand((byte)33, 0);
    }

    public void getHistoryOfHRData(final long stamp) {
        this.mReceivedDataCallback.clearType(6);
    }

```

```

        this.sendCommand((byte)34, HexUtil.append((int)1,
(int[])this.utc2Bytes(stamp)));
    }

    public void getHistoryOfRRRRecord() {
        this.mReceivedDataCallback.clearType(8);
        this.sendCommand((byte)36);
    }

    public void getHistoryOfRRRData(final long stamp) {
        this.mReceivedDataCallback.clearType(16);
        this.sendCommand((byte)37, HexUtil.append((int)1,
(int[])this.utc2Bytes(stamp)));
    }

    public void getIntervalSteps() {
        this.mReceivedDataCallback.clearType(18);
        this.sendCommand((byte)64, 0);
    }

    public void getSingleTapRecords() {
        this.mReceivedDataCallback.clearType(20);
        this.sendCommand((byte)66, 0);
    }

    public void getHistoryOfSleep() {
        this.mReceivedDataCallback.clearType(22);
        this.sendCommand((byte)5, 2);
    }

    public void getUserInfo() {
        this.sendCommand((byte)3, 0);
    }

    public void setUserInfo(final int age, final int sex, final int weight, final
int height, final long userId) {
        int[] command = new int[]{(byte)age, (byte)sex, (byte)weight,
(byte)height, (byte)((int)(userId >> 32 & 255L)), (byte)((int)(userId >> 24 &
255L)), (byte)((int)(userId >> 16 & 255L)), (byte)((int)(userId >> 8 & 255L)),
(byte)((int)(userId & 255L))};
        this.sendCommand((byte)4, command);
    }

    public void getHeartRateStatus() {
        this.sendCommand((byte)70, 0);
    }

    public void setHeartRateStatus(int min, int max, int goal) {
        this.sendCommand((byte)70, 1, (byte)min, (byte)max, (byte)goal);
    }

    public void setHeartRateMax(int max) {
        this.sendCommand((byte)116, 0, 6, (byte)max);
    }

```

```

public void getHeartRateMax() {
    this.sendCommand((byte)117, 0, 6);
}

public void setBloodOxygen(final int mode) {
    int[] command = new int[]{(byte)mode, 0};
    this.sendCommand((byte)55, command);
}

public void setHeartRateAlarm(final boolean alarm) {
    this.sendCommand((byte)87, alarm ? 1 : 0);
}

public void getHeartRateAlarm() {
    this.sendCommand((byte)91, 0);
}

public void getHistoryOfSingleRecord(final long stamp) {
    this.sendCommand((byte)73, this.utc2Bytes(stamp));
}

public void set3DFrequency(@IntRange(from = 0L,to = 4L) int frequency) {
    this.sendCommand((byte)116, 0, 11, (byte)frequency);
}

public void get3DFrequency() {
    this.sendCommand((byte)117, 0, 11);
}

public void set3DEnabled(boolean enabled) {
    this.sendCommand((byte)116, 0, 12, enabled ? 1 : 0);
}

public void get3DStatus() {
    this.sendCommand((byte)117, 0, 12);
}

public void get6DFrequency() {
    this.sendCommand((byte)97, 0);
}

public void set6DFrequency(@IntRange(from = 0L,to = 3L) int frequency) {
    this.sendCommand((byte)98, frequency);
}

public void setCustomDataReceivedCallback(CustomDataReceivedCallback
customDataReceivedCallback) {
    this.mCustomDataReceivedCallback = customDataReceivedCallback;
}

public void getHistoryOf3D() {
    this.sendCommand((byte)119, 0);
}

```

```

public String dfuMode() {
    String address = this.getDFUAddress();
    byte[] command = new byte[]{-1, 4, 39, 0};
    command[3] = this.checkSum(command);
    this.writeTxCharacteristic(command);
    return address;
}

private String getDFUAddress() {
    BluetoothDevice device = this.getBluetoothDevice();
    if (device == null) {
        return null;
    } else {
        String deviceAddress = device.getAddress();
        String firstBytes = deviceAddress.substring(0, 15);
        String lastByte = deviceAddress.substring(15);
        String lastByteIncremented = String.format(Locale.US, "%02X",
Integer.valueOf(lastByte, 16) + 1 & 255);
        return firstBytes + lastByteIncremented;
    }
}

private final class WearManagerGattCallback extends
FitnessManager<WearManagerCallbacks>.FitnessManagerGattCallback {
    private WearManagerGattCallback() {
        super();
    }

    protected void initialize() {
        super.initialize();

WearManager.this.readCharacteristic(WearManager.this.mBodySensorLocationCharacte
ristic).with((DataReceivedCallback)WearManager.this.mBodySensorLocationDataCallb
ack).fail((device, status) -> {
    WearManager.this.log(5, "Body Sensor Location characteristic not
found");
}).enqueue();

WearManager.this.setNotificationCallback(WearManager.this.mHeartRateCharacterist
ic).with(WearManager.this.mHeartRateMeasureDataCallback);

WearManager.this.enableNotifications(WearManager.this.mHeartRateCharacteristic).
enqueue();

WearManager.this.setNotificationCallback(WearManager.this.mRXCharacteristic).wit
h(WearManager.this.mReceivedDataCallback);

WearManager.this.enableNotifications(WearManager.this.mRXCharacteristic).done((d
evice) -> {
    WearManager.this.log(3, "Rx notifications enabled");
}).fail((device, status) -> {
    WearManager.this.log(5, "Rx characteristic not found");
}).enqueue();

```

```

        if (WearManager.this.mCustomRxCharacteristic != null) {

WearManager.this.setNotificationCallback(WearManager.this.mCustomRxCharacteristic).with((device, data) -> {
            if (WearManager.this.mCustomDataReceivedCallback != null) {

WearManager.this.mCustomDataReceivedCallback.onDataReceived(device,
data.getValue());
            }

        });

WearManager.this.enableNotifications(WearManager.this.mCustomRxCharacteristic).enqueue();
    }

    public boolean isRequiredServiceSupported(@NonNull BluetoothGatt gatt) {
        super.isRequiredServiceSupported(gatt);
        BluetoothGattService hrService =
gatt.getService(WearManager.HR_SERVICE_UUID);
        if (hrService != null) {
            WearManager.this.mHeartRateCharacteristic =
hrService.getCharacteristic(WearManager.HEART_RATE_MEASUREMENT_CHARACTERISTIC_UUID);
        }

        return WearManager.this.mHeartRateCharacteristic != null;
    }

    protected boolean isOptionalServiceSupported(@NonNull final BluetoothGatt gatt) {
        super.isOptionalServiceSupported(gatt);
        BluetoothGattService service =
gatt.getService(WearManager.HR_SERVICE_UUID);
        if (service != null) {
            WearManager.this.mBodySensorLocationCharacteristic =
service.getCharacteristic(WearManager.BODY_SENSOR_LOCATION_CHARACTERISTIC_UUID);
        }

        return WearManager.this.mBodySensorLocationCharacteristic != null;
    }

    protected void onDeviceReady() {
        super.onDeviceReady();
        WearManager.this.setUTCTime();
    }

    protected void onDeviceDisconnected() {
        super.onDeviceDisconnected();
        WearManager.this.mBodySensorLocationCharacteristic = null;
        WearManager.this.mHeartRateCharacteristic = null;
        WearManager.this.mReceivedDataCallback.setCL833(false);
    }

```



```

    }

    protected void onServicesInvalidated() {
        super.onServicesInvalidated();
        WearManager.this.mBodySensorLocationCharacteristic = null;
        WearManager.this.mHeartRateCharacteristic = null;
        WearManager.this.mReceivedDataCallback.setCL833(false);
    }

    // $FF: synthetic method
    WearManagerGattCallback(Object x1) {
        this();
    }
}

private final class WearScanCallback extends ScanCallback {
    private final FilterScanCallback mCallback;

    private WearScanCallback(FilterScanCallback callback) {
        this.mCallback = callback;
    }

    private boolean matchDeviceName(BluetoothDevice device) {
        if (WearManager.this.mFilterNames == null) {
            return true;
        } else {
            if (device != null) {
                String name = device.getName();
                if (name != null && !TextUtils.isEmpty(name)) {
                    String[] var3 = WearManager.this.mFilterNames;
                    int var4 = var3.length;

                    for(int var5 = 0; var5 < var4; ++var5) {
                        String filterName = var3[var5];
                        if (name.contains(filterName)) {
                            return true;
                        }
                    }
                }
            }

            return false;
        }
    }

    public void onScanResult(int callbackType, @NonNull ScanResult result) {
        super.onScanResult(callbackType, result);
        if (this.mCallback != null) {
            this.mCallback.onScanResult(callbackType, result);
        }
    }

    public void onBatchScanResults(@NonNull List<ScanResult> results) {

```

```

        List<ScanResult> scanDevices = new ArrayList();
        Iterator var3 = results.iterator();

        while(var3.hasNext()) {
            ScanResult result = (ScanResult)var3.next();
            BluetoothDevice device = result.getDevice();
            if (this.matchDeviceName(device)) {
                scanDevices.add(result);
            }
        }

        if (this.mCallback != null) {
            this.mCallback.onBatchScanResults(results);
            this.mCallback.onFilterScanResults(scanDevices);
        }
    }

    public void onScanFailed(int errorCode) {
        super.onScanFailed(errorCode);
        if (this.mCallback != null) {
            this.mCallback.onScanFailed(errorCode);
        }
    }

    // $FF: synthetic method
    WearScanCallback(FilterScanCallback x1, Object x2) {
        this(x1);
    }
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness
// File: FitnessManager.java

package com.android.chileaf.fitness;

import android.bluetooth.BluetoothDevice;
import android.bluetooth.BluetoothGatt;
import android.bluetooth.BluetoothGattCharacteristic;
import android.bluetooth.BluetoothGattService;
import android.content.Context;
import android.os.Build.VERSION;
import android.text.TextUtils;
import androidx.annotation.NonNull;
import com.android.chileaf.fitness.common.battery.BatteryLevelDataCallback;
import com.android.chileaf.util.HexUtil;
import com.android.chileaf.util.LogUtil;
import java.util.Iterator;
import java.util.List;
import java.util.UUID;

```

```

import no.nordicsemi.android.ble.BleManager;
import no.nordicsemi.android.ble.LegacyBleManager;
import no.nordicsemi.android.ble.callback.DataReceivedCallback;
import no.nordicsemi.android.ble.callback.RssiCallback;
import no.nordicsemi.android.ble.data.Data;
import no.nordicsemi.android.ble.utils.ParserUtils;

public abstract class FitnessManager<T extends FitnessManagerCallbacks> extends
LegacyBleManager<T> {
    protected static final UUID SERVICE_UUID =
UUID.fromString("AAE28F00-71B5-42A1-8C3C-F9CF6AC969D0");
    protected static final UUID RX_CHAR_UUID =
UUID.fromString("AAE28F01-71B5-42A1-8C3C-F9CF6AC969D0");
    protected static final UUID TX_CHAR_UUID =
UUID.fromString("AAE28F02-71B5-42A1-8C3C-F9CF6AC969D0");
    protected static final String SPEC_CHAR_UUID =
"AAE21541-71B5-42A1-8C3C-F9CF6AC969D0";
    protected static final String CUSTOM_CHAR_UUID =
"AAE21542-71B5-42A1-8C3C-F9CF6AC969D0";
    private static final UUID BATTERY_SERVICE_UUID =
UUID.fromString("0000180F-0000-1000-8000-00805f9b34fb");
    private static final UUID BATTERY_LEVEL_CHARACTERISTIC_UUID =
UUID.fromString("00002A19-0000-1000-8000-00805f9b34fb");
    private static final UUID PROFILE_SERVICE_UUID =
UUID.fromString("0000180A-0000-1000-8000-00805f9b34fb");
    private static final UUID PROFILE_SYSTEM_CHARACTERISTIC_UUID =
UUID.fromString("00002A23-0000-1000-8000-00805f9b34fb");
    private static final UUID PROFILE_MODEL_CHARACTERISTIC_UUID =
UUID.fromString("00002A24-0000-1000-8000-00805f9b34fb");
    private static final UUID PROFILE_SERIAL_CHARACTERISTIC_UUID =
UUID.fromString("00002A25-0000-1000-8000-00805f9b34fb");
    private static final UUID PROFILE_FIRMWARE_CHARACTERISTIC_UUID =
UUID.fromString("00002A26-0000-1000-8000-00805f9b34fb");
    private static final UUID PROFILE_HARDWARE_CHARACTERISTIC_UUID =
UUID.fromString("00002A27-0000-1000-8000-00805f9b34fb");
    private static final UUID PROFILE_SOFTWARE_CHARACTERISTIC_UUID =
UUID.fromString("00002A28-0000-1000-8000-00805f9b34fb");
    private static final UUID PROFILE_VENDOR_CHARACTERISTIC_UUID =
UUID.fromString("00002A29-0000-1000-8000-00805f9b34fb");
    protected boolean isContainCL833 = false;
    protected BluetoothGattCharacteristic mRXCharacteristic;
    protected BluetoothGattCharacteristic mTXCharacteristic;
    protected BluetoothGattCharacteristic mCustomRxCharacteristic;
    private BluetoothGattCharacteristic mBatteryLevelCharacteristic;
    private BluetoothGattCharacteristic mProfileSystemCharacteristic;
    private BluetoothGattCharacteristic mProfileModelCharacteristic;
    private BluetoothGattCharacteristic mProfileSerialCharacteristic;
    private BluetoothGattCharacteristic mProfileFirmwareCharacteristic;
    private BluetoothGattCharacteristic mProfileHardwareCharacteristic;
    private BluetoothGattCharacteristic mProfileSoftwareCharacteristic;
    private BluetoothGattCharacteristic mProfileVendorCharacteristic;
    private Integer mRssi;
    private Integer mBatteryLevel;
    private String mSystemId;

```

```

private String mModelName;
private String mSerialNumber;
private String mFirmwareVersion;
private String mHardwareVersion;
private String mSoftwareVersion;
private String mVendorName;
private final RssiCallback mRssiCallback = (device, rssi) -> {
    ((FitnessManagerCallbacks)this.mCallbacks).onRssiRead(device, rssi);
    this.mRssi = rssi;
};
private final DataReceivedCallback mSystemCallBack = (device, data) -> {
    if (data.size() > 0 && data.getValue() != null) {
        String systemId = HexUtil.byteArrayToString(data.getValue());
        if (!TextUtils.isEmpty(systemId)) {
            this.log(4, "System Id: " + systemId);
            ((FitnessManagerCallbacks)this.mCallbacks).onSystemId(device,
systemId);
            this.mSystemId = systemId;
        }
    }
};
private final DataReceivedCallback mModelCallBack = (device, data) -> {
    if (data.size() > 0 && data.getValue() != null) {
        String modelName = HexUtil.byteArrayToString(data.getValue());
        if (!TextUtils.isEmpty(modelName)) {
            this.log(4, "Model Name: " + modelName);
            this.checkModel(modelName, this.isContainCL833);
            ((FitnessManagerCallbacks)this.mCallbacks).onModelName(device,
modelName);
            this.mModelName = modelName;
        }
    }
};
private final DataReceivedCallback mSerialNumberCallBack = (device, data) ->
{
    if (data.size() > 0 && data.getValue() != null) {
        String serialNumber = HexUtil.byteArrayToString(data.getValue());
        if (!TextUtils.isEmpty(serialNumber)) {
            this.log(4, "Serial Number: " + serialNumber);
            ((FitnessManagerCallbacks)this.mCallbacks).onSerialNumber(device,
serialNumber);
            this.mSerialNumber = serialNumber;
        }
    }
};
private final DataReceivedCallback mFirmwareCallBack = (device, data) -> {
    if (data.size() > 0 && data.getValue() != null) {
        String firmware = HexUtil.byteArrayToString(data.getValue());
        if (!TextUtils.isEmpty(firmware)) {
            this.log(4, "Firmware Version: " + firmware);
            ((FitnessManagerCallbacks)this.mCallbacks).onFirmwareVersion(device,

```

```

firmware);
        this.mFirmwareVersion = firmware;
    }
}

};
private final DataReceivedCallback mHardwareCallBack = (device, data) -> {
    if (data.size() > 0 && data.getValue() != null) {
        String value = HexUtil.byteArrayToString(data.getValue());
        if (!TextUtils.isEmpty(value)) {
            this.log(4, "Hardware Version: " + value);
            ((FitnessManagerCallbacks)this.mCallbacks).onHardwareVersion(device,
value);
            this.mHardwareVersion = value;
        }
    }
};
private final DataReceivedCallback mSoftwareCallBack = (device, data) -> {
    if (data.size() > 0 && data.getValue() != null) {
        String value = HexUtil.byteArrayToString(data.getValue());
        if (!TextUtils.isEmpty(value)) {
            this.log(4, "Software Version: " + value);
            ((FitnessManagerCallbacks)this.mCallbacks).onSoftwareVersion(device,
value);
            this.mSoftwareVersion = value;
        }
    }
};
private final DataReceivedCallback mVendorCallBack = (device, data) -> {
    if (data.size() > 0 && data.getValue() != null) {
        String value = HexUtil.byteArrayToString(data.getValue());
        if (!TextUtils.isEmpty(value)) {
            this.log(4, "Vendor Name: " + value);
            ((FitnessManagerCallbacks)this.mCallbacks).onVendorName(device,
value);
            this.mVendorName = value;
        }
    }
};
private final DataReceivedCallback mBatteryLevelDataCallback = new
BatteryLevelDataCallback() {
    public void onBatteryLevelChanged(@NonNull final BluetoothDevice device,
final int batteryLevel) {
        FitnessManager.this.log(4, "Battery Level received: " + batteryLevel +
"%");
        ((FitnessManagerCallbacks)FitnessManager.this.mCallbacks).onBatteryLevelChanged(
device, batteryLevel);
        FitnessManager.this.mBatteryLevel = batteryLevel;
        if (FitnessManager.this.isReadRssi()) {

```

```

FitnessManager.this.readRssi().with(FitnessManager.this.mRssiCallback).enqueue()
;
    }

    }

    public void onInvalidDataReceived(@NonNull final BluetoothDevice device,
@NonNull final Data data) {
        FitnessManager.this.log(5, "Invalid Battery Level data received: " +
data);
    }
};

    public FitnessManager(@NonNull final Context context) {
        super(context);
    }

    public void setManagerCallbacks(@NonNull final T callbacks) {
        super.setGattCallbacks(callbacks);
        this.mCallbacks = callbacks;
    }

    @NonNull
    protected BleManager.BleManagerGattCallback getGattCallback() {
        return new BleManager.BleManagerGattCallback() {
            protected boolean isRequiredServiceSupported(@NonNull BluetoothGatt
gatt) {
                return false;
            }

            protected void onServicesInvalidated() {
            }
        };
    }

    public void setDebug(boolean debug) {
        LogUtil.setDebug(debug);
    }

    public void log(final int priority, @NonNull final String message) {
        LogUtil.log(6, priority, message);
    }

    protected byte checksum(final byte[] data) {
        int result = 0;
        byte[] var3 = data;
        int var4 = data.length;

        for(int var5 = 0; var5 < var4; ++var5) {
            byte item = var3[var5];
            result += item;
        }

        result = -result;
    }

```

```

        result ^= 58;
        return (byte)(result & 255);
    }

    public Integer getRssi() {
        return this.mRssi;
    }

    public Integer getBatteryLevel() {
        return this.mBatteryLevel;
    }

    public String getSystemId() {
        return this.mSystemId;
    }

    public String getModelName() {
        return this.mModelName;
    }

    public String getSerialNumber() {
        return this.mSerialNumber;
    }

    public String getFirmwareVersion() {
        return this.mFirmwareVersion;
    }

    public String getHardwareVersion() {
        return this.mHardwareVersion;
    }

    public String getSoftwareVersion() {
        return this.mSoftwareVersion;
    }

    public String getVendorName() {
        return this.mVendorName;
    }

    public boolean isReadRssi() {
        return true;
    }

    public abstract void checkModel(String modelName, boolean isCL833);

    public void readProfileCharacteristic() {
        if (this.isConnected()) {
            this.readCharacteristic(this.mProfileSystemCharacteristic).with(this.mSystemCall
            Back).fail((device, status) -> {
                this.log(5, "Profile system characteristic not found");
            }).enqueue();
        }
    }

```

```

this.readCharacteristic(this.mProfileModelCharacteristic).with(this.mModelCallBa
ck).fail((device, status) -> {
    this.log(5, "Profile model characteristic not found");
}).enqueue();

this.readCharacteristic(this.mProfileSerialCharacteristic).with(this.mSerialNumb
erCallBack).fail((device, status) -> {
    this.log(5, "Profile serial characteristic not found");
}).enqueue();

this.readCharacteristic(this.mProfileFirmwareCharacteristic).with(this.mFirmware
CallBack).fail((device, status) -> {
    this.log(5, "Profile firmware characteristic not found");
}).enqueue();

this.readCharacteristic(this.mProfileHardwareCharacteristic).with(this.mHardware
CallBack).fail((device, status) -> {
    this.log(5, "Profile hardware characteristic not found");
}).enqueue();

this.readCharacteristic(this.mProfileSoftwareCharacteristic).with(this.mSoftware
CallBack).fail((device, status) -> {
    this.log(5, "Profile software characteristic not found");
}).enqueue();

this.readCharacteristic(this.mProfileVendorCharacteristic).with(this.mVendorCall
Back).fail((device, status) -> {
    this.log(5, "Profile vendor characteristic not found");
}).enqueue();
}

}

public void readBatteryLevelCharacteristic() {
    if (this.isConnected()) {

this.readCharacteristic(this.mBatteryLevelCharacteristic).with(this.mBatteryLeve
lDataCallback).fail((device, status) -> {
    this.log(5, "Battery Level characteristic not found");
}).enqueue();
}

}

public void enableBatteryLevelCharacteristicNotifications() {
    if (this.isConnected()) {

this.setNotificationCallback(this.mBatteryLevelCharacteristic).with(this.mBatter
yLevelDataCallback);

this.enableNotifications(this.mBatteryLevelCharacteristic).done((device) -> {
    this.log(3, "Battery Level notifications enabled");
}).fail((device, status) -> {
    this.log(5, "Battery Level characteristic not found");
}

```



```

        }).enqueue();
    }

}

public void disableBatteryLevelCharacteristicNotifications() {
    if (this.isConnected()) {
this.disableNotifications(this.mBatteryLevelCharacteristic).done((device) -> {
    this.log(3, "Battery Level notifications disabled");
    }).enqueue();
    }

}

public void connectDevice(BluetoothDevice device) {
    this.connect(device).useAutoConnect(false).enqueue();
}

public void disconnectDevice() {
    this.disconnect().enqueue();
}

protected void writeTxCharacteristic(final byte[] command) {
    if (this.isConnected() && this.mTXCharacteristic != null) {
        this.writeCharacteristic(this.mTXCharacteristic, command).with((device,
data) -> {
            this.log(2, "Send:" + ParserUtils.parse(data.getValue()));
        }).done((device) -> {
            this.log(3, "Tx writeCharacteristic success");
        }).fail((device, status) -> {
            this.log(5, "Tx writeCharacteristic failure");
        }).enqueue();
    }

}

protected void sendCommand(final byte[] bytes) {
    this.sendCommand(bytes, false);
}

protected void sendCommand(final byte[] bytes, boolean isChecksum) {
    byte[] command;
    if (isChecksum) {
        byte check = this.checkSum(bytes);
        command = HexUtil.append(bytes, check);
    } else {
        command = bytes;
    }

    this.writeTxCharacteristic(command);
}

protected abstract class FitnessManagerGattCallback extends

```

```

BleManager.BleManagerGattCallback {
    protected void initialize() {
        if (VERSION.SDK_INT >= 21) {
            FitnessManager.this.requestConnectionPriority(1).enqueue();
        }

        FitnessManager.this.readProfileCharacteristic();
        FitnessManager.this.readBatteryLevelCharacteristic();
        FitnessManager.this.enableBatteryLevelCharacteristicNotifications();
    }

    public boolean isRequiredServiceSupported(@NonNull final BluetoothGatt
gatt) {
        BluetoothGattService service =
gatt.getService(FitnessManager.SERVICE_UUID);
        if (service != null) {
            FitnessManager.this.mRXCharacteristic =
service.getCharacteristic(FitnessManager.RX_CHAR_UUID);
            FitnessManager.this.mTXCharacteristic =
service.getCharacteristic(FitnessManager.TX_CHAR_UUID);
            List<BluetoothGattCharacteristic> characteristics =
service.getCharacteristics();
            Iterator var4 = characteristics.iterator();

            while(var4.hasNext()) {
                BluetoothGattCharacteristic characteristic =
(BluetoothGattCharacteristic)var4.next();
                if (characteristic != null && characteristic.getUuid() != null &&
characteristic.getUuid().toString().equalsIgnoreCase("AAE21541-71B5-42A1-8C3C-F9
CF6AC969D0")) {
                    FitnessManager.this.mCustomRxCharacteristic =
service.getCharacteristic(UUID.fromString("AAE21542-71B5-42A1-8C3C-F9CF6AC969D0"
));
                    FitnessManager.this.isContainCL833 = true;
                }
            }
        }

        boolean writeRequest = false;
        boolean writeCommand = false;
        if (FitnessManager.this.mTXCharacteristic != null) {
            int txProperties =
FitnessManager.this.mTXCharacteristic.getProperties();
            writeRequest = (txProperties & 8) > 0;
            writeCommand = (txProperties & 4) > 0;
            if (writeRequest) {
                FitnessManager.this.mTXCharacteristic.setWriteType(2);
                FitnessManager.this.log(3, "TXCharacteristic notifications
WRITE_TYPE_DEFAULT");
            }
        }

        return FitnessManager.this.mRXCharacteristic != null &&
FitnessManager.this.mTXCharacteristic != null && (writeCommand || writeRequest);
    }
}

```

```

    }

    protected boolean isOptionalServiceSupported(@NonNull final BluetoothGatt
gatt) {
        BluetoothGattService batteryService =
gatt.getService(FitnessManager.BATTERY_SERVICE_UUID);
        if (batteryService != null) {
            FitnessManager.this.mBatteryLevelCharacteristic =
batteryService.getCharacteristic(FitnessManager.BATTERY_LEVEL_CHARACTERISTIC_UUID);
        }

        boolean isBatteryService =
FitnessManager.this.mBatteryLevelCharacteristic != null;
        BluetoothGattService profileService =
gatt.getService(FitnessManager.PROFILE_SERVICE_UUID);
        if (profileService != null) {
            FitnessManager.this.mProfileSystemCharacteristic =
profileService.getCharacteristic(FitnessManager.PROFILE_SYSTEM_CHARACTERISTIC_UUID);
            FitnessManager.this.mProfileModelCharacteristic =
profileService.getCharacteristic(FitnessManager.PROFILE_MODEL_CHARACTERISTIC_UUID);
            FitnessManager.this.mProfileSerialCharacteristic =
profileService.getCharacteristic(FitnessManager.PROFILE_SERIAL_CHARACTERISTIC_UUID);
            FitnessManager.this.mProfileFirmwareCharacteristic =
profileService.getCharacteristic(FitnessManager.PROFILE_FIRMWARE_CHARACTERISTIC_UUID);
            FitnessManager.this.mProfileHardwareCharacteristic =
profileService.getCharacteristic(FitnessManager.PROFILE_HARDWARE_CHARACTERISTIC_UUID);
            FitnessManager.this.mProfileSoftwareCharacteristic =
profileService.getCharacteristic(FitnessManager.PROFILE_SOFTWARE_CHARACTERISTIC_UUID);
            FitnessManager.this.mProfileVendorCharacteristic =
profileService.getCharacteristic(FitnessManager.PROFILE_VENDOR_CHARACTERISTIC_UUID);
        }

        boolean isProfileService =
FitnessManager.this.mProfileSystemCharacteristic != null &&
FitnessManager.this.mProfileModelCharacteristic != null &&
FitnessManager.this.mProfileSerialCharacteristic != null &&
FitnessManager.this.mProfileFirmwareCharacteristic != null &&
FitnessManager.this.mProfileHardwareCharacteristic != null &&
FitnessManager.this.mProfileSoftwareCharacteristic != null &&
FitnessManager.this.mProfileVendorCharacteristic != null;
        return isBatteryService && isProfileService;
    }

    protected void onDeviceDisconnected() {
        FitnessManager.this.mBatteryLevelCharacteristic = null;
        FitnessManager.this.mCustomRxCharacteristic = null;
    }

```

```

        FitnessManager.this.mRXCharacteristic = null;
        FitnessManager.this.mTXCharacteristic = null;
        FitnessManager.this.mBatteryLevel = null;
        FitnessManager.this.isContainCL833 = false;
    }

    protected void onServicesInvalidated() {
        FitnessManager.this.mBatteryLevelCharacteristic = null;
        FitnessManager.this.mCustomRxCharacteristic = null;
        FitnessManager.this.mRXCharacteristic = null;
        FitnessManager.this.mTXCharacteristic = null;
        FitnessManager.this.mBatteryLevel = null;
        FitnessManager.this.isContainCL833 = false;
    }
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness
// File: FitnessManagerCallbacks.java

package com.android.chileaf.fitness;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import com.android.chileaf.fitness.common.battery.BatteryLevelCallback;
import com.android.chileaf.fitness.common.profile.ProfileCallback;
import no.nordicsemi.android.ble.BleManagerCallbacks;
import no.nordicsemi.android.ble.callback.RssiCallback;

public interface FitnessManagerCallbacks extends BleManagerCallbacks,
RssiCallback, BatteryLevelCallback, ProfileCallback {
    default void onDeviceConnecting(@NonNull final BluetoothDevice device) {
    }

    void onDeviceConnected(@NonNull final BluetoothDevice device);

    default void onDeviceDisconnecting(@NonNull final BluetoothDevice device) {
    }

    void onDeviceDisconnected(@NonNull final BluetoothDevice device);

    default void onLinkLossOccurred(@NonNull final BluetoothDevice device) {
    }

    default void onServicesDiscovered(@NonNull final BluetoothDevice device,
final boolean optionalServicesFound) {
    }

    default void onDeviceReady(@NonNull final BluetoothDevice device) {
    }
}

```

```

    /** @deprecated */
    @Deprecated
    default boolean shouldEnableBatteryLevelNotifications(@NonNull final
BluetoothDevice device) {
        return false;
    }

    /** @deprecated */
    @Deprecated
    default void onBatteryValueReceived(@NonNull final BluetoothDevice device,
@IntRange(from = 0L,to = 100L) final int value) {
    }

    default void onBondingRequired(@NonNull final BluetoothDevice device) {
    }

    default void onBonded(@NonNull final BluetoothDevice device) {
    }

    default void onBondingFailed(@NonNull final BluetoothDevice device) {
    }

    default void onError(@NonNull final BluetoothDevice device, @NonNull final
String message, final int errorCode) {
    }

    default void onDeviceNotSupported(@NonNull final BluetoothDevice device) {
    }

    default void onRssiRead(@NonNull final BluetoothDevice device, final int
rssi) {
    }

    default void onBatteryLevelChanged(@NonNull final BluetoothDevice device,
final int batteryLevel) {
    }

    default void onSystemId(@NonNull final BluetoothDevice device, final String
systemId) {
    }

    default void onModelName(@NonNull final BluetoothDevice device, final String
modelName) {
    }

    default void onSerialNumber(@NonNull final BluetoothDevice device, final
String serialNumber) {
    }

    default void onFirmwareVersion(@NonNull final BluetoothDevice device, final
String firmware) {
    }

    default void onHardwareVersion(@NonNull final BluetoothDevice device, final

```

```

String hardware) {
    }

    default void onSoftwareVersion(@NonNull final BluetoothDevice device, final
String software) {
    }

    default void onVendorName(@NonNull final BluetoothDevice device, final String
vendorName) {
    }
}

// Directory: app
// Subdirectory: classes.jar_Decompiler.com\com\android\chileaf\fitness\callback
// File: AccelerometerCallback.java

package com.android.chileaf.fitness.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;

public interface AccelerometerCallback {
    void onAccelerometerReceived(@NonNull final BluetoothDevice device, int x,
int y, int z);
}

// Directory: app
// Subdirectory: classes.jar_Decompiler.com\com\android\chileaf\fitness\callback
// File: BloodOxygenCallback.java

package com.android.chileaf.fitness.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;

public interface BloodOxygenCallback {
    void onBloodOxygenReceived(@NonNull final BluetoothDevice device, int
bSwitch, String value, int gesture, int piValue, int Onwrist);
}

// Directory: app
// Subdirectory: classes.jar_Decompiler.com\com\android\chileaf\fitness\callback
// File: BluetoothStatusCallback.java

package com.android.chileaf.fitness.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;

public interface BluetoothStatusCallback {
    void onBluetoothStatusReceived(@NonNull final BluetoothDevice device, boolean

```

```
enabled);  
}
```

```
// Directory: app  
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback  
// File: BodyHealthCallback.java
```

```
package com.android.chileaf.fitness.callback;
```

```
import android.bluetooth.BluetoothDevice;  
import androidx.annotation.NonNull;
```

```
public interface BodyHealthCallback {  
    void onHealthReceived(@NonNull final BluetoothDevice device, final int  
vo2Max, final int breathRate, final int emotionLevel, final int stressPercent,  
final int stamina, final float tp, final float lf, final float hf);  
}
```

```
// Directory: app  
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback  
// File: BodySportCallback.java
```

```
package com.android.chileaf.fitness.callback;
```

```
import android.bluetooth.BluetoothDevice;  
import androidx.annotation.IntRange;  
import androidx.annotation.NonNull;
```

```
public interface BodySportCallback {  
    void onSportReceived(@NonNull final BluetoothDevice device, @IntRange(from =  
0L) final int step, @IntRange(from = 0L) final int distance, @IntRange(from =  
0L) final int calorie);  
}
```

```
// Directory: app  
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback  
// File: BodySportHealthCallback.java
```

```
package com.android.chileaf.fitness.callback;
```

```
import android.bluetooth.BluetoothDevice;  
import androidx.annotation.NonNull;
```

```
public interface BodySportHealthCallback {  
    void onSportHealthReceived(@NonNull final BluetoothDevice device, final int  
vo2Max, final int breathRate, final int emotion, final int pressure, final int  
stamina);  
}
```

```
// Directory: app
```

```
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback
// File: CustomDataReceivedCallback.java
```

```
package com.android.chileaf.fitness.callback;
```

```
import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;
```

```
public interface CustomDataReceivedCallback {
    void onDataReceived(@NonNull final BluetoothDevice device, final byte[]
data);
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback
// File: HeartRateAlarmCallback.java
```

```
package com.android.chileaf.fitness.callback;
```

```
import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;
```

```
public interface HeartRateAlarmCallback {
    void onHeartRateAlarmReceived(@NonNull final BluetoothDevice device, long
stamp, boolean enabled);
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback
// File: HeartRateMaxCallback.java
```

```
package com.android.chileaf.fitness.callback;
```

```
import android.bluetooth.BluetoothDevice;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
```

```
public interface HeartRateMaxCallback {
    void onHeartRateMaxReceived(@NonNull final BluetoothDevice device,
@IntRange(from = 0L) final int max);
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback
// File: HeartRateStatusCallback.java
```

```
package com.android.chileaf.fitness.callback;
```

```
import android.bluetooth.BluetoothDevice;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
```



```

public interface HeartRateStatusCallback {
    void onHeartRateStatusReceived(@NonNull final BluetoothDevice device,
    @IntRange(from = 0L) final int min, @IntRange(from = 0L) final int max,
    @IntRange(from = 0L) final int goal);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback
// File: HistoryOf3DDataCallback.java

package com.android.chileaf.fitness.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;
import com.android.chileaf.model.HistoryOf3D;

public interface HistoryOf3DDataCallback {
    void onHistoryOf3DDataReceived(@NonNull final BluetoothDevice device,
    HistoryOf3D history, boolean finish);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback
// File: HistoryOfHRDataCallback.java

package com.android.chileaf.fitness.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;
import com.android.chileaf.model.HistoryOfHeartRate;
import java.util.List;

public interface HistoryOfHRDataCallback {
    void onHistoryOfHRDataReceived(@NonNull final BluetoothDevice device,
    List<HistoryOfHeartRate> heartRates);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback
// File: HistoryOfHRRRecordCallback.java

package com.android.chileaf.fitness.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;
import com.android.chileaf.model.HistoryOfRecord;
import java.util.List;

public interface HistoryOfHRRRecordCallback {
    void onHistoryOfHRRRecordReceived(@NonNull final BluetoothDevice device,

```

```
List<HistoryOfRecord> records);  
}
```

```
// Directory: app  
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback  
// File: HistoryOfRRDataCallback.java
```

```
package com.android.chileaf.fitness.callback;
```

```
import android.bluetooth.BluetoothDevice;  
import androidx.annotation.NonNull;  
import com.android.chileaf.model.HistoryOfRespiratoryRate;  
import java.util.List;
```

```
public interface HistoryOfRRDataCallback {  
    void onHistoryOfRRDataReceived(@NonNull final BluetoothDevice device,  
    List<HistoryOfRespiratoryRate> respiratoryRates);  
}
```

```
// Directory: app  
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback  
// File: HistoryOfRRRecordCallback.java
```

```
package com.android.chileaf.fitness.callback;
```

```
import android.bluetooth.BluetoothDevice;  
import androidx.annotation.NonNull;  
import com.android.chileaf.model.HistoryOfRecord;  
import java.util.List;
```

```
public interface HistoryOfRRRecordCallback {  
    void onHistoryOfRRRecordReceived(@NonNull final BluetoothDevice device,  
    List<HistoryOfRecord> records);  
}
```

```
// Directory: app  
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback  
// File: HistoryOfSingleRecordCallback.java
```

```
package com.android.chileaf.fitness.callback;
```

```
import android.bluetooth.BluetoothDevice;  
import androidx.annotation.IntRange;  
import androidx.annotation.NonNull;
```

```
public interface HistoryOfSingleRecordCallback {  
    void onHistorySingleRecordReceived(@NonNull final BluetoothDevice device,  
    final long stamp, @IntRange(from = 0L) final long step, @IntRange(from = 0L)  
    final long distance, @IntRange(from = 0L) final long calorie);  
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompiler.com\com\android\chileaf\fitness\callback
// File: HistoryOfSleepCallback.java
```

```
package com.android.chileaf.fitness.callback;
```

```
import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;
import com.android.chileaf.model.HistorySleep;
import java.util.List;
```

```
public interface HistoryOfSleepCallback {
    void onHistoryOfSleepReceived(@NonNull final BluetoothDevice device,
    List<HistorySleep> sleeps);
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompiler.com\com\android\chileaf\fitness\callback
// File: HistoryOfSportCallback.java
```

```
package com.android.chileaf.fitness.callback;
```

```
import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;
import com.android.chileaf.model.HistoryOfSport;
import java.util.List;
```

```
public interface HistoryOfSportCallback {
    void onHistoryOfSportReceived(@NonNull final BluetoothDevice device,
    List<HistoryOfSport> sports);
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompiler.com\com\android\chileaf\fitness\callback
// File: IntervalStepCallback.java
```

```
package com.android.chileaf.fitness.callback;
```

```
import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;
import com.android.chileaf.model.IntervalStep;
import java.util.List;
```

```
public interface IntervalStepCallback {
    void onIntervalStepReceived(@NonNull final BluetoothDevice device,
    List<IntervalStep> steps);
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompiler.com\com\android\chileaf\fitness\callback
```

```
// File: Sensor3DFrequencyCallback.java

package com.android.chileaf.fitness.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;

public interface Sensor3DFrequencyCallback {
    void onSensor3DFrequencyReceived(@NonNull final BluetoothDevice device,
    @IntRange(from = 0L,to = 4L) final int frequency);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback
// File: Sensor3DStatusCallback.java

package com.android.chileaf.fitness.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;

public interface Sensor3DStatusCallback {
    void onSensor3DStatusReceived(@NonNull final BluetoothDevice device, boolean
enabled);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback
// File: Sensor6DFrequencyCallback.java

package com.android.chileaf.fitness.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;

public interface Sensor6DFrequencyCallback {
    void onSensor6DFrequencyReceived(@NonNull final BluetoothDevice device,
    @IntRange(from = 0L,to = 3L) final int frequency);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback
// File: Sensor6DRawDataCallback.java

package com.android.chileaf.fitness.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;
```

```

public interface Sensor6DRawDataCallback {
    void onSensor6DRawDataReceived(@NonNull final BluetoothDevice device, final
long stamp, final int sequence, final int gyroscopeX, final int gyroscopeY,
final int gyroscopeZ, final int accelerometerX, final int accelerometerY, final
int accelerometerZ);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback
// File: SingleTapRecordCallback.java

package com.android.chileaf.fitness.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;
import com.android.chileaf.model.HistoryOfRecord;
import java.util.List;

public interface SingleTapRecordCallback {
    void onSingleTapRecordReceived(@NonNull final BluetoothDevice device,
List<HistoryOfRecord> records);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback
// File: TemperatureCallback.java

package com.android.chileaf.fitness.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;

public interface TemperatureCallback {
    void onTemperatureReceived(@NonNull final BluetoothDevice device, float
environment, float wrist, float body);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback
// File: UserInfoCallback.java

package com.android.chileaf.fitness.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;

public interface UserInfoCallback {
    void onUserInfoReceived(@NonNull final BluetoothDevice device, @IntRange(from
= 0L) final int age, @IntRange(from = 0L) final int sex, @IntRange(from = 0L)
final int weight, @IntRange(from = 0L) final int height, final long userId);
}

```

```
}
```

```
// Directory: app  
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback  
// File: WearManagerCallbacks.java
```

```
package com.android.chileaf.fitness.callback;
```

```
import android.bluetooth.BluetoothDevice;  
import androidx.annotation.NonNull;  
import androidx.annotation.Nullable;  
import com.android.chileaf.fitness.FitnessManagerCallbacks;  
import com.android.chileaf.fitness.common.heart.BodySensorLocationCallback;  
import com.android.chileaf.fitness.common.heart.HeartRateMeasurementCallback;  
import java.util.List;
```

```
public interface WearManagerCallbacks extends FitnessManagerCallbacks,  
BodySensorLocationCallback, HeartRateMeasurementCallback, BodySportCallback,  
BluetoothStatusCallback {
```

```
    default void onBodySensorLocationReceived(@NonNull final BluetoothDevice  
device, final int sensorLocation) {  
    }
```

```
    default void onHeartRateMeasurementReceived(@NonNull final BluetoothDevice  
device, final int heartRate, @Nullable final Boolean contactDetected, @Nullable  
final Integer energyExpanded, @Nullable final List<Integer> rrIntervals) {  
    }
```

```
    default void onSportReceived(@NonNull final BluetoothDevice device, final int  
step, final int distance, final int calorie) {  
    }
```

```
    default void onBluetoothStatusReceived(@NonNull final BluetoothDevice device,  
boolean enabled) {  
    }  
}
```

```
// Directory: app  
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\callback  
// File: WearReceivedDataCallback.java
```

```
package com.android.chileaf.fitness.callback;
```

```
import android.bluetooth.BluetoothDevice;  
import android.os.Parcel;  
import androidx.annotation.NonNull;  
import com.android.chileaf.model.HistoryOf3D;  
import com.android.chileaf.model.HistoryOfHeartRate;  
import com.android.chileaf.model.HistoryOfRecord;  
import com.android.chileaf.model.HistoryOfRespiratoryRate;  
import com.android.chileaf.model.HistoryOfSport;  
import com.android.chileaf.model.HistorySleep;
```

```

import com.android.chileaf.model.IntervalStep;
import com.android.chileaf.util.DateUtil;
import com.android.chileaf.util.HexUtil;
import com.android.chileaf.util.LogUtil;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import no.nordicsemi.android.ble.callback.profile.ProfileReadResponse;
import no.nordicsemi.android.ble.data.Data;
import no.nordicsemi.android.ble.utils.ParserUtils;

public abstract class WearReceivedDataCallback extends ProfileReadResponse
implements UserInfoCallback, BodySportCallback, BluetoothStatusCallback,
HistoryOfSportCallback, HistoryOfHRRRecordCallback, HistoryOfHRRDataCallback,
HistoryOfRRRRecordCallback, HistoryOfRRRDataCallback, IntervalStepCallback,
SingleTapRecordCallback, HeartRateStatusCallback, BloodOxygenCallback,
TemperatureCallback, HistoryOfSingleRecordCallback, HeartRateAlarmCallback,
AccelerometerCallback, HeartRateMaxCallback, HistoryOfSleepCallback,
Sensor3DFrequencyCallback, Sensor3DStatusCallback, HistoryOf3DDataCallback,
BodyHealthCallback, Sensor6DFrequencyCallback, Sensor6DRawDataCallback,
BodySportHealthCallback {
    private static final long END_TAG = 4294967295L;
    private long mStamp = 0L;
    private boolean isCL833 = false;
    private boolean isStamp = false;
    private List<Data> mPackages = new ArrayList();
    private List<HistoryOfSport> mHistoryOfSports;
    private List<HistoryOfRecord> mHistoryOfRecords;
    private List<HistoryOfHeartRate> mHistoryOfHeartRates;
    private List<HistoryOfRecord> mRespiratoryRatesRecords;
    private List<HistoryOfRespiratoryRate> mHistoryOfRespiratoryRates;
    private List<IntervalStep> mIntervalSteps;
    private List<HistoryOfRecord> mSingleTapRecords;
    private List<HistorySleep> mHistoryOfSleeps;
    public static final int TYPE_SPORT = 2;
    public static final int TYPE_HEART = 4;
    public static final int TYPE_HEARTS = 6;
    public static final int TYPE_HEART_RR = 8;
    public static final int TYPE_HEART_RRS = 16;
    public static final int TYPE_INTERVAL = 18;
    public static final int TYPE_SINGLE_TAP = 20;
    public static final int TYPE_SLEEP = 22;
    public static final int TYPE_HISTORY_3D = 24;

    public WearReceivedDataCallback() {
    }

    protected WearReceivedDataCallback(final Parcel in) {
        super(in);
    }

    public void onDataReceived(@NonNull final BluetoothDevice device, @NonNull

```

```

final Data data) {
    super.onDataReceived(device, data);
    boolean validate = data.getIntValue(17, 1) == data.size();
    if (!validate) {
        this.onInvalidDataReceived(device, data);
        LogUtil.w("onDataReceived:length:%s", data.size());
    }

    byte[] value = data.getValue();
    int mode = data.getIntValue(17, 2);

    try {
        int cmd;
        int vo2Max;
        int breathRate;
        int emotionLevel;
        long stamp;
        if (mode == 3) {
            cmd = this.getIntParse(value, 5, 1);
            vo2Max = this.getIntParse(value, 6, 1);
            breathRate = this.getIntParse(value, 7, 1);
            emotionLevel = this.getIntParse(value, 8, 1);
            stamp = this.getLongParse(value, 9, 5);
            this.onUserInfoReceived(device, cmd, vo2Max, breathRate,
emotionLevel, stamp);
        } else {
            int accelerometerY;
            long utc;
            if (mode == 5) {
                cmd = this.getIntParse(value, 3, 1);
                if (cmd == 3) {
                    if (this.mHistoryOfSleeps == null) {
                        this.mHistoryOfSleeps = new ArrayList();
                    }

                    for(vo2Max = 4; vo2Max < value.length; ++vo2Max) {
                        int len = value[vo2Max];
                        if (len >= 1) {
                            ++vo2Max;
                            utc = this.getLongParse(value, vo2Max, 4);
                            vo2Max += 4;
                            utc *= 1000L;
                            utc -= 28800000L;
                            int[] actions = new int[len];

                            for(accelerometerY = 0; accelerometerY < len;
++accelerometerY) {
                                actions[accelerometerY] = this.getIntParse(value,
accelerometerY + vo2Max, 1);
                            }

                            vo2Max += len - 1;
                            HistorySleep historySleep = new HistorySleep(utc,
actions);

```



```

        this.mHistoryOfSleeps.add(historySleep);
        if (vo2Max == value.length - 2) {
            break;
        }
    }
}

this.onHistoryOfSleepReceived(device, this.mHistoryOfSleeps);
this.mHistoryOfSleeps.clear();
}
} else {
    byte[] slice;
    int stressPercent;
    int stamina;
    if (mode == 12) {
        slice = this.subSlice(3, value);

        for(breathRate = 0; breathRate < slice.length / 6;
++breathRate) {
            cmd = breathRate * 6;
            emotionLevel = data.getIntValue(34, cmd + 3);
            stressPercent = data.getIntValue(34, cmd + 5);
            stamina = data.getIntValue(34, cmd + 7);
            this.onAccelerometerReceived(device, emotionLevel,
stressPercent, stamina);
        }
    } else if (mode != 18) {
        if (mode == 19) {
            cmd = this.getIntParse(value, 3, 1);
            vo2Max = this.getIntParse(value, 4, 1);
            breathRate = this.getIntParse(value, 5, 1);
            emotionLevel = this.getIntParse(value, 6, 1);
            stressPercent = this.getIntParse(value, 7, 1);
            this.onSportHealthReceived(device, cmd, vo2Max, breathRate,
emotionLevel, stressPercent);
        } else if (mode == 21) {
            cmd = this.getIntParse(value, 3, 3);
            vo2Max = this.getIntParse(value, 6, 3);
            breathRate = this.getIntParse(value, 9, 3);
            this.onSportReceived(device, cmd, vo2Max, breathRate);
        } else if (mode == 63) {
            cmd = this.getIntParse(value, 3, 1);
            this.onBluetoothStatusReceived(device, cmd == 1);
        } else {
            long stamp;
            byte[] slice;
            if (mode == 22) {
                if (this.mHistoryOfSports == null) {
                    this.mHistoryOfSports = new ArrayList();
                }

                stamp = this.getLongParse(value, 3, 4);
                if (this.isCL833) {
                    slice = this.subSlice(3, value);

```

```

        this.parseSportHistory(slice);
        Collections.reverse(this.mHistoryOfSports);
        this.onHistoryOfSportReceived(device,
this.mHistoryOfSports);
        this.mHistoryOfSports.clear();
    } else if (stamp != 4294967295L) {
        this.mPackages.add(data);
    } else {
        for(emotionLevel = 0; emotionLevel <
this.mPackages.size(); ++emotionLevel) {
            slice = this.subSlice(3,
((Data)this.mPackages.get(emotionLevel)).getValue());
            this.parseSportHistory(slice);
            LogUtil.d("HistoryOfSport index:%d values:%s",
emotionLevel, ParserUtils.parse(slice));
        }

        this.onHistoryOfSportReceived(device,
this.mHistoryOfSports);
        this.mHistoryOfSports.clear();
        this.mPackages.clear();
    }
} else {
    long stamp;
    long record;
    if (mode == 33) {
        if (this.mHistoryOfRecords == null) {
            this.mHistoryOfRecords = new ArrayList();
        }

        stamp = this.getLongParse(value, 3, 4);
        if (stamp != 4294967295L) {
            this.mPackages.add(data);
        } else {
            for(stressPercent = 0; stressPercent <
this.mPackages.size(); ++stressPercent) {
                byte[] slice = this.subSlice(3,
((Data)this.mPackages.get(stressPercent)).getValue());
                LogUtil.d("mHistoryOfRecords index:%d
values:%s", stressPercent, ParserUtils.parse(slice));

                for(stamina = 0; stamina < slice.length / 4;
++stamina) {
                    breathRate = stamina * 4;
                    record = this.getLongParse(slice,
breathRate, 4);

                    stamp = DateUtil.restoreZoneUTC(record);
                    this.mHistoryOfRecords.add(new
HistoryOfRecord(record, stamp));
                    LogUtil.d("mHistoryOfRecords index:%d
record:%s", stamina, record);
                }
            }
        }
    }
}

```

```

        this.onHistoryOfHRRRecordReceived(device,
this.mHistoryOfRecords);
        this.mHistoryOfRecords.clear();
        this.mPackages.clear();
    }
} else {
    byte[] slice;
    long stamp;
    if (mode != 34 && mode != 35) {
        int accelerometerZ;
        if (mode == 36) {
            if (this.mRespiratoryRatesRecords == null) {
                this.mRespiratoryRatesRecords = new
ArrayList();
            }

            stamp = this.getLongParse(value, 3, 4);
            if (stamp != 4294967295L) {
                this.mPackages.add(data);
            } else {
                int length = 4;

                for(stamina = 0; stamina <
this.mPackages.size(); ++stamina) {
                    slice = this.subSlice(3,
((Data)this.mPackages.get(stamina)).getValue());
                    LogUtil.d("mRespiratoryRatesRecords
index:%d values:%s", stamina, ParserUtils.parse(slice));

                    for(accelerometerY = 0; accelerometerY <
slice.length / length; ++accelerometerY) {
                        accelerometerZ = accelerometerY *
length;
                        stamp = this.getLongParse(slice,
accelerometerZ, 4);
                        long record =
DateUtil.restoreZoneUTC(stamp);
                        HistoryOfRecord historyRecord = new
HistoryOfRecord(stamp, record);
                        this.mRespiratoryRatesRecords.add(historyRecord);
                    }
                }

                this.onHistoryOfRRRRecordReceived(device,
this.mRespiratoryRatesRecords);
                LogUtil.d("onHistoryOfRRRRecordReceived
size:%d", this.mRespiratoryRatesRecords.size());
                this.mRespiratoryRatesRecords.clear();
                this.mPackages.clear();
            }
        } else {
            byte length;
            if (mode != 37 && mode != 38) {

```

```

        if (mode == 55) {
            if (value[1] <= 6) {
                return;
            }

            cmd = this.getIntParse(value, 3, 1);
            if (value[1] <= 8) {
                return;
            }

            vo2Max = this.getIntParse(value, 4, 1);
            breathRate = this.getIntParse(value, 5,
1);
            emotionLevel = this.getIntParse(value, 6,
1);
            stressPercent = this.getIntParse(value,
7, 1);

            this.onBloodOxygenReceived(device, cmd,
String.valueOf(vo2Max), breathRate, emotionLevel, stressPercent);
        } else if (mode == 56) {
            float BadSituation =
(float)this.getIntParse(value, 3, 2) / 10.0F;
            float wrist =
(float)this.getIntParse(value, 5, 2) / 10.0F;
            float body =
(float)this.getIntParse(value, 7, 2) / 10.0F;
            this.onTemperatureReceived(device,
BadSituation, wrist, body);
        } else if (mode != 64 && mode != 65) {
            if (mode != 66 && mode != 67) {
                if (mode == 70) {
                    cmd = this.getIntParse(value, 4,
1);
                    vo2Max = this.getIntParse(value, 5,
1);
                    breathRate =
this.getIntParse(value, 6, 1);

                    this.onHeartRateStatusReceived(device, cmd, vo2Max, breathRate);
                } else if (mode == 73) {
                    stamp =
DateUtil.restoreZoneUTC(this.getLongParse(value, 3, 4));
                    long step =
this.getLongParse(value, 7, 3);
                    stamp = this.getLongParse(value,
10, 3);
                    record = this.getLongParse(value,
13, 3);

                    this.onHistorySingleRecordReceived(device, stamp, step, stamp, record);
                } else if (mode == 91) {
                    stamp =
DateUtil.restoreZoneUTC(this.getLongParse(value, 3, 4));
                    breathRate =

```

```

this.getIntParse(value, 7, 1);

this.onHeartRateAlarmReceived(device, stamp, breathRate == 1);
                                } else if (mode == 96) {
                                cmd = this.getIntParse(value, 3,
1);
                                this.parseSensorRawList(device,
cmd, new Data(this.subSlice(4, value)));
                                } else if (mode == 97) {
                                cmd = this.getIntParse(value, 3,
1);

this.onSensor6DFrequencyReceived(device, cmd);
                                } else if (mode == 100) {
                                stamp = this.getLongParse(value, 3,
4);
                                breathRate =
this.getIntParse(value, 7, 2);
                                utc =
DateUtil.restoreZoneUTCTimeInMillis(stamp * 1000L + (long)breathRate);
                                stamina = this.getIntParse(value,
9, 1);
                                this.parseSensorRawList(device,
utc, stamina, data);
                                } else if (mode == 117) {
                                cmd = this.getIntParse(value, 4,
1);
                                if (cmd == 6) {
                                vo2Max = this.getIntParse(value,
5, 1);

this.onHeartRateMaxReceived(device, vo2Max);
                                } else if (cmd == 11) {
                                vo2Max = this.getIntParse(value,
5, 1);

this.onSensor3DFrequencyReceived(device, vo2Max);
                                } else if (cmd == 12) {
                                boolean enabled =
this.getIntParse(value, 5, 1) == 1;

this.onSensor3DStatusReceived(device, enabled);
                                } else if (cmd == 15) {
                                vo2Max = this.getIntParse(value,
5, 1);
                                breathRate =
this.getIntParse(value, 6, 1);
                                emotionLevel =
this.getIntParse(value, 7, 1);
                                stressPercent =
this.getIntParse(value, 8, 1);
                                stamina =
this.getIntParse(value, 9, 1);
                                float tp =

```

```

(float)this.getLongParse(value, 10, 4) / 1000.0F;
float lf =
(float)this.getLongParse(value, 14, 4) / 1000.0F;
float hf =
(float)this.getLongParse(value, 18, 4) / 1000.0F;
this.onHealthReceived(device,
vo2Max, breathRate, emotionLevel, stressPercent, stamina, tp, lf, hf);
}
} else if ((mode == 119 || mode ==
120) && (mode == 119 || mode == 120)) {
length = 6;
slice = this.subSlice(3,
data.getValue());

for(emotionLevel = 0; emotionLevel
< slice.length / length; ++emotionLevel) {
stressPercent = emotionLevel *
length;
stamina = this.getSInt16(slice,
stressPercent);
accelerometerY =
this.getSInt16(slice, stressPercent + 2);
accelerometerZ =
this.getSInt16(slice, stressPercent + 4);
HistoryOf3D history = new
HistoryOf3D(stamina, accelerometerY, accelerometerZ);
this.onHistoryOf3DDataReceived(device, history, mode == 120);
}
} else {
if (this.mSingleTapRecords == null) {
this.mSingleTapRecords = new
ArrayList();

if (mode == 66) {
this.mPackages.add(data);
}

if (mode == 67) {
for(vo2Max = 0; vo2Max <
this.mPackages.size(); ++vo2Max) {
slice = this.subSlice(3,
((Data)this.mPackages.get(vo2Max)).getValue());
LogUtil.d("mSingleTapRecords
index:%d values:%s", vo2Max, ParserUtils.parse(slice));

for(emotionLevel = 0;
emotionLevel < slice.length / 4; ++emotionLevel) {
breathRate = emotionLevel *
4;
stamp =
this.getLongParse(slice, breathRate, 4);

```

```

record =
DateUtil.restoreZoneUTC(stamp);

this.mSingleTapRecords.add(new HistoryOfRecord(stamp, record));
    }
}

this.onSingleTapRecordReceived(device, this.mSingleTapRecords);
    this.mSingleTapRecords.clear();
    this.mPackages.clear();
}
} else {
    if (this.mIntervalSteps == null) {
        this.mIntervalSteps = new ArrayList();
    }

    if (mode == 64) {
        this.mPackages.add(data);
    }

    if (mode == 65) {
        for(emotionLevel = 0; emotionLevel <
this.mPackages.size(); ++emotionLevel) {
            slice = this.subSlice(3,
((Data)this.mPackages.get(emotionLevel)).getValue());
            LogUtil.d("mIntervalSteps index:%d
values:%s", emotionLevel, ParserUtils.parse(slice));

            for(stressPercent = 0;
stressPercent < slice.length / 8; ++stressPercent) {
                cmd = stressPercent * 8;
                stamp =
DateUtil.restoreZoneUTC(this.getLongParse(slice, cmd, 4));
                accelerometerZ =
this.getIntParse(slice, cmd + 4, 4);
                IntervalStep intervalStep = new
IntervalStep(stamp, accelerometerZ);
                this.mIntervalSteps.add(intervalStep);
            }
        }

        this.onIntervalStepReceived(device,
this.mIntervalSteps);

        this.mIntervalSteps.clear();
        this.mPackages.clear();
    }
} else {
    if (this.mHistoryOfRespiratoryRates == null)
{
        this.mHistoryOfRespiratoryRates = new

```

```

ArrayList();

        }

        if (mode == 37) {
            if (!this.isStamp) {
                this.mStamp = this.getLongParse(value,
3, 4);

                this.isStamp = true;
            }

            this.mPackages.add(data);
        }

        if (mode == 38) {
            length = 2;

            for(emotionLevel = 0; emotionLevel <
this.mPackages.size(); ++emotionLevel) {
                slice = this.subSlice(7,
((Data)this.mPackages.get(emotionLevel)).getValue());
                LogUtil.d("index:%d
HistoryOfRespiratoryRates mValues:%s", emotionLevel, ParserUtils.parse(slice));

                for(stressPercent = 0; stressPercent <
slice.length / length; ++stressPercent) {
                    stamina = stressPercent * length;
                    accelerometerY =
this.getIntParse(slice, stamina, 2);
                    long stamp =
DateUtil.restoreZoneUTC(this.mStamp);
                    HistoryOfRespiratoryRate
respiratoryRate = new HistoryOfRespiratoryRate(stamp, accelerometerY);
this.mHistoryOfRespiratoryRates.add(respiratoryRate);
                    ++this.mStamp;
                }
            }

            LogUtil.d("onHistoryOfRRDataReceived
:%s", this.mHistoryOfRespiratoryRates.toString());
            this.onHistoryOfRRDataReceived(device,
this.mHistoryOfRespiratoryRates);

            this.mHistoryOfRespiratoryRates.clear();
            this.mPackages.clear();
            this.isStamp = false;
            this.mStamp = 0L;
        }
    }
} else {
    if (this.mHistoryOfHeartRates == null) {
        this.mHistoryOfHeartRates = new ArrayList();
    }
}

```



```

        if (mode == 34) {
            if (!this.isStamp) {
                this.mStamp = this.getLongParse(value, 3,
                    this.isStamp = true;
            }
            this.mPackages.add(data);
        }

        if (mode == 35) {
            for(breathRate = 0; breathRate <
this.mPackages.size(); ++breathRate) {
                slice = this.subSlice(3,
((Data)this.mPackages.get(breathRate)).getValue());
                LogUtil.d("mHistoryOfHeartRates index:%d
values:%s", breathRate, ParserUtils.parse(slice));

                for(emotionLevel = 4; emotionLevel <
slice.length; ++emotionLevel) {
                    stressPercent = this.getIntParse(slice,
emotionLevel, 1);
                    stamp =
DateUtil.restoreZoneUTC(this.mStamp);
                    HistoryOfHeartRate heartRate = new
HistoryOfHeartRate(stamp, stressPercent);
                    this.mHistoryOfHeartRates.add(heartRate);
                    ++this.mStamp;
                }
            }

            this.onHistoryOfHRDataReceived(device,
this.mHistoryOfHeartRates);

            this.mHistoryOfHeartRates.clear();
            this.mPackages.clear();
            this.isStamp = false;
            this.mStamp = 0L;
        }
    }
}

} catch (Exception var18) {
    var18.printStackTrace();
}

}

public void setCL833(boolean isCL833) {
    this.isCL833 = isCL833;
}

```

```

    private int getSInt16(byte[] data, int offset) {
        return this.unsignedToSigned(this.unsignedBytesToInt(data[offset],
data[offset + 1]), 16);
    }

    private int unsignedByteToInt(final byte b) {
        return b & 255;
    }

    private int unsignedBytesToInt(final byte b0, final byte b1) {
        return this.unsignedByteToInt(b0) + (this.unsignedByteToInt(b1) << 8);
    }

    private int unsignedToSigned(int unsigned, final int size) {
        if ((unsigned & 1 << size - 1) != 0) {
            unsigned = -1 * ((1 << size - 1) - (unsigned & (1 << size - 1) - 1));
        }

        return unsigned;
    }

    public void clearType(int type) {
        switch(type) {
            case 2:
                if (this.mHistoryOfSports != null) {
                    this.mHistoryOfSports.clear();
                }
            case 3:
            case 5:
            case 7:
            case 9:
            case 10:
            case 11:
            case 12:
            case 13:
            case 14:
            case 15:
            case 17:
            case 19:
            case 21:
            case 23:
            case 24:
            default:
                break;
            case 4:
                if (this.mHistoryOfRecords != null) {
                    this.mHistoryOfRecords.clear();
                }
                break;
            case 6:
                if (this.mHistoryOfHeartRates != null) {
                    this.mHistoryOfHeartRates.clear();
                }
        }
    }

```

```

        break;
    case 8:
        if (this.mRespiratoryRatesRecords != null) {
            this.mRespiratoryRatesRecords.clear();
        }
        break;
    case 16:
        if (this.mHistoryOfRespiratoryRates != null) {
            this.mHistoryOfRespiratoryRates.clear();
        }
        break;
    case 18:
        if (this.mIntervalSteps != null) {
            this.mIntervalSteps.clear();
        }
        break;
    case 20:
        if (this.mSingleTapRecords != null) {
            this.mSingleTapRecords.clear();
        }
        break;
    case 22:
        if (this.mHistoryOfSleeps != null) {
            this.mHistoryOfSleeps.clear();
        }
    }

    this.mPackages.clear();
}

private void parseSportHistory(final byte[] value) {
    LogUtil.d("HistoryOfSport length:%d values:%s", value.length,
ParserUtils.parse(value));

    for(int i = 0; i < value.length / 10; ++i) {
        int offset = i * 10;
        long stamp = this.getLongParse(value, offset, 4);
        long step = this.getLongParse(value, offset + 4, 3);
        long calorie = this.getLongParse(value, offset + 7, 3);
        stamp = DateUtil.restoreZoneUTC(stamp);
        this.mHistoryOfSports.add(new HistoryOfSport(stamp, step, calorie));
    }
}

private synchronized void parseSensorRawList(BluetoothDevice device, final
int sequence, final Data data) {
    LogUtil.d("parseSensorRawList values:%s",
ParserUtils.parse(data.getValue()));

    for(int i = 0; i < data.size() / 12; ++i) {
        int offset = i * 12;
        int gyroscopeX = data.getIntValue(34, offset);
        int gyroscopeY = data.getIntValue(34, offset + 2);
    }
}

```

```

        int gyroscopeZ = data.getIntValue(34, offset + 4);
        int accelerometerX = data.getIntValue(34, offset + 6);
        int accelerometerY = data.getIntValue(34, offset + 8);
        int accelerometerZ = data.getIntValue(34, offset + 10);
        this.onSensor6DRawDataReceived(device, 255L, sequence, gyroscopeX,
gyroscopeY, gyroscopeZ, accelerometerX, accelerometerY, accelerometerZ);
    }

}

private synchronized void parseSensorRawList(BluetoothDevice device, final
long utc, final int sequence, final Data data) {
    for(int i = 0; i < data.size() / 12; ++i) {
        int offset = i * 12;
        int gyroscopeX = data.getIntValue(34, offset + 10);
        int gyroscopeY = data.getIntValue(34, offset + 12);
        int gyroscopeZ = data.getIntValue(34, offset + 14);
        int accelerometerX = data.getIntValue(34, offset + 16);
        int accelerometerY = data.getIntValue(34, offset + 18);
        int accelerometerZ = data.getIntValue(34, offset + 20);
        this.onSensor6DRawDataReceived(device, utc, sequence, gyroscopeX,
gyroscopeY, gyroscopeZ, accelerometerX, accelerometerY, accelerometerZ);
    }

}

private synchronized byte[] subSlice(final int start, final byte[] value) {
    return HexUtil.subByte(value, start, value.length - 1);
}

private long getLongParse(byte[] bytes, int pos, int len) {
    long val = 0L;
    len += pos;

    for(int i = pos; i < len; ++i) {
        val <<= 8;
        val |= (long)bytes[i] & 255L;
    }

    return val;
}

private int getIntParse(byte[] bytes, int pos, int len) {
    int val = 0;
    len += pos;

    for(int i = pos; i < len; ++i) {
        val <<= 8;
        val |= bytes[i] & 255;
    }

    return val;
}

```

```
    @Retention(RetentionPolicy.SOURCE)
    public @interface DataType {
    }
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\common
// File: DateTimeCallback.java
```

```
package com.android.chileaf.fitness.common;
```

```
import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;
import java.util.Calendar;
```

```
public interface DateTimeCallback {
    void onDateTimeReceived(@NonNull final BluetoothDevice device, @NonNull final
Calendar calendar);
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\common
// File: DateTimeDataCallback.java
```

```
package com.android.chileaf.fitness.common;
```

```
import android.bluetooth.BluetoothDevice;
import android.os.Parcel;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import java.util.Calendar;
import no.nordicsemi.android.ble.callback.profile.ProfileReadResponse;
import no.nordicsemi.android.ble.data.Data;
```

```
public abstract class DateTimeDataCallback extends ProfileReadResponse
implements DateTimeCallback {
    public DateTimeDataCallback() {
    }
}
```

```
    protected DateTimeDataCallback(final Parcel in) {
        super(in);
    }
}
```

```
    public void onDataReceived(@NonNull final BluetoothDevice device, @NonNull
final Data data) {
        super.onDataReceived(device, data);
        Calendar calendar = readDateTime(data, 0);
        if (calendar == null) {
            this.onInvalidDataReceived(device, data);
        } else {
            this.onDateTimeReceived(device, calendar);
        }
    }
}
```

```

    }

    @Nullable
    public static Calendar readDateTime(@NonNull final Data data, final int
offset) {
        if (data.size() < offset + 7) {
            return null;
        } else {
            Calendar calendar = Calendar.getInstance();
            int year = data.getIntValue(18, offset);
            int month = data.getIntValue(17, offset + 2);
            int day = data.getIntValue(17, offset + 3);
            if (year > 0) {
                calendar.set(1, year);
            } else {
                calendar.clear(1);
            }

            if (month > 0) {
                calendar.set(2, month - 1);
            } else {
                calendar.clear(2);
            }

            if (day > 0) {
                calendar.set(5, day);
            } else {
                calendar.clear(5);
            }

            calendar.set(11, data.getIntValue(17, offset + 4));
            calendar.set(12, data.getIntValue(17, offset + 5));
            calendar.set(13, data.getIntValue(17, offset + 6));
            calendar.set(14, 0);
            return calendar;
        }
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\fitness\common
// File: FilterScanCallback.java

```

```

package com.android.chileaf.fitness.common;

import androidx.annotation.NonNull;
import java.util.List;
import no.nordicsemi.android.support.v18.scanner.ScanResult;

public interface FilterScanCallback {
    void onFilterScanResults(@NonNull final List<ScanResult> results);

    default void onScanResult(final int callbackType, @NonNull final ScanResult

```

```

result) {
    }

    default void onBatchScanResults(@NonNull final List<ScanResult> results) {
    }

    default void onScanFailed(final int errorCode) {
    }
}

```

```

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\com\android\chileaf\fitness\common\battery
// File: BatteryLevelCallback.java

```

```

package com.android.chileaf.fitness.common.battery;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;

public interface BatteryLevelCallback {
    void onBatteryLevelChanged(@NonNull final BluetoothDevice device,
@IntRange(from = 0L,to = 100L) final int batteryLevel);
}

```

```

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\com\android\chileaf\fitness\common\battery
// File: BatteryLevelDataCallback.java

```

```

package com.android.chileaf.fitness.common.battery;

import android.bluetooth.BluetoothDevice;
import android.os.Parcel;
import androidx.annotation.NonNull;
import no.nordicsemi.android.ble.callback.profile.ProfileReadResponse;
import no.nordicsemi.android.ble.data.Data;

public abstract class BatteryLevelDataCallback extends ProfileReadResponse
implements BatteryLevelCallback {
    public BatteryLevelDataCallback() {
    }

    protected BatteryLevelDataCallback(final Parcel in) {
        super(in);
    }

    public void onDataReceived(@NonNull final BluetoothDevice device, @NonNull
final Data data) {
        super.onDataReceived(device, data);
        if (data.size() == 1) {

```

```

        int batteryLevel = data.getIntValue(17, 0);
        if (batteryLevel >= 0 && batteryLevel <= 100) {
            this.onBatteryLevelChanged(device, batteryLevel);
            return;
        }
    }
    this.onInvalidDataReceived(device, data);
}
}

```

```

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\com\android\chileaf\fitness\common\heart
// File: BodySensorLocation.java

```

```

package com.android.chileaf.fitness.common.heart;

```

```

import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;

```

```

@Retention(RetentionPolicy.SOURCE)
public @interface BodySensorLocation {
}

```

```

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\com\android\chileaf\fitness\common\heart
// File: BodySensorLocationCallback.java

```

```

package com.android.chileaf.fitness.common.heart;

```

```

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;

```

```

public interface BodySensorLocationCallback {
    int SENSOR_LOCATION_OTHER = 0;
    int SENSOR_LOCATION_CHEST = 1;
    int SENSOR_LOCATION_WRIST = 2;
    int SENSOR_LOCATION_FINGER = 3;
    int SENSOR_LOCATION_HAND = 4;
    int SENSOR_LOCATION_EAR_LOBE = 5;
    int SENSOR_LOCATION_FOOT = 6;
    int SENSOR_LOCATION_FIRST = 0;
    int SENSOR_LOCATION_LAST = 6;

    void onBodySensorLocationReceived(@NonNull final BluetoothDevice device,
final int sensorLocation);
}

```

```

// Directory: app

```



```
// Subdirectory:
classes.jar_Decompile.com\com\android\chileaf\fitness\common\heart
// File: BodySensorLocationDataCallback.java

package com.android.chileaf.fitness.common.heart;

import android.bluetooth.BluetoothDevice;
import android.os.Parcel;
import androidx.annotation.NonNull;
import no.nordicsemi.android.ble.callback.profile.ProfileReadResponse;
import no.nordicsemi.android.ble.data.Data;

public abstract class BodySensorLocationDataCallback extends ProfileReadResponse
implements BodySensorLocationCallback {
    public BodySensorLocationDataCallback() {
    }

    protected BodySensorLocationDataCallback(final Parcel in) {
        super(in);
    }

    public void onDataReceived(@NonNull final BluetoothDevice device, @NonNull
final Data data) {
        super.onDataReceived(device, data);
        if (data.size() < 1) {
            this.onInvalidDataReceived(device, data);
        } else {
            int sensorLocation = data.getIntValue(17, 0);
            this.onBodySensorLocationReceived(device, sensorLocation);
        }
    }
}
}
```

```
// Directory: app
// Subdirectory:
classes.jar_Decompile.com\com\android\chileaf\fitness\common\heart
// File: HeartRateMeasurementCallback.java

package com.android.chileaf.fitness.common.heart;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import java.util.List;

public interface HeartRateMeasurementCallback {
    void onHeartRateMeasurementReceived(@NonNull final BluetoothDevice device,
@IntRange(from = 0L) final int heartRate, @Nullable final Boolean
contactDetected, @Nullable @IntRange(from = 0L) final Integer energyExpanded,
@Nullable final List<Integer> rrIntervals);
}
```

```

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\com\android\chileaf\fitness\common\heart
// File: HeartRateMeasurementDataCallback.java

package com.android.chileaf.fitness.common.heart;

import android.bluetooth.BluetoothDevice;
import android.os.Parcel;
import androidx.annotation.NonNull;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import no.nordicsemi.android.ble.callback.profile.ProfileReadResponse;
import no.nordicsemi.android.ble.data.Data;

public abstract class HeartRateMeasurementDataCallback extends
ProfileReadResponse implements HeartRateMeasurementCallback {
    public HeartRateMeasurementDataCallback() {
    }

    protected HeartRateMeasurementDataCallback(final Parcel in) {
        super(in);
    }

    public void onDataReceived(@NonNull final BluetoothDevice device, @NonNull
final Data data) {
        super.onDataReceived(device, data);
        if (data.size() < 2) {
            this.onInvalidDataReceived(device, data);
        } else {
            int offset = 0;
            int flags = data.getIntValue(17, offset);
            int hearRateType = (flags & 1) == 0 ? 17 : 18;
            int sensorContactStatus = (flags & 6) >> 1;
            boolean sensorContactSupported = sensorContactStatus == 2 ||
sensorContactStatus == 3;
            boolean sensorContactDetected = sensorContactStatus == 3;
            boolean energyExpandedPresent = (flags & 8) != 0;
            boolean rrIntervalsPresent = (flags & 16) != 0;
            int offset = offset + 1;
            if (data.size() < 1 + (hearRateType & 15) + (energyExpandedPresent ? 2
: 0) + (rrIntervalsPresent ? 2 : 0)) {
                this.onInvalidDataReceived(device, data);
            } else {
                Boolean sensorContact = sensorContactSupported ?
sensorContactDetected : null;
                int heartRate = data.getIntValue(hearRateType, offset);
                offset += hearRateType & 15;
                Integer energyExpanded = null;
                if (energyExpandedPresent) {
                    energyExpanded = data.getIntValue(18, offset);
                    offset += 2;
                }
            }
        }
    }
}

```

```

    }

    List<Integer> rrIntervals = null;
    if (rrIntervalsPresent) {
        int count = (data.size() - offset) / 2;
        List<Integer> intervals = new ArrayList(count);

        for(int i = 0; i < count; ++i) {
            intervals.add(data.getIntValue(18, offset));
            offset += 2;
        }

        rrIntervals = Collections.unmodifiableList(intervals);
    }

    this.onHeartRateMeasurementReceived(device, heartRate,
    sensorContact, energyExpanded, rrIntervals);
    }
    }
}

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\com\android\chileaf\fitness\common\parser
// File: BloodPressureMeasurementParser.java

package com.android.chileaf.fitness.common.parser;

import no.nordicsemi.android.ble.data.Data;

public class BloodPressureMeasurementParser {
    public static String parse(final Data data) {
        StringBuilder builder = new StringBuilder();
        int offset = 0;
        int offset = offset + 1;
        int flags = data.getIntValue(17, offset);
        int unitType = flags & 1;
        boolean timestampPresent = (flags & 2) > 0;
        boolean pulseRatePresent = (flags & 4) > 0;
        boolean userIdPresent = (flags & 8) > 0;
        boolean statusPresent = (flags & 16) > 0;
        float systolic = data.getFloatValue(50, offset);
        float diastolic = data.getFloatValue(50, offset + 2);
        float meanArterialPressure = data.getFloatValue(50, offset + 4);
        String unit = unitType == 0 ? " mmHg" : " kPa";
        offset += 6;
        builder.append("Systolic: ").append(systolic).append(unit);
        builder.append(" Diastolic: ").append(diastolic).append(unit);
        builder.append(" Mean AP: ").append(meanArterialPressure).append(unit);
        if (timestampPresent) {
            builder.append(" Timestamp: ").append(DateTimeParser.parse(data,
offset));

```

```

        offset += 7;
    }

    if (pulseRatePresent) {
        float pulseRate = data.getFloatValue(50, offset);
        offset += 2;
        builder.append(" Pulse: ").append(pulseRate).append(" bpm");
    }

    int status;
    if (userIdPresent) {
        status = data.getIntValue(17, offset);
        ++offset;
        builder.append(" User ID: ").append(status);
    }

    if (statusPresent) {
        status = data.getIntValue(18, offset);
        if ((status & 1) > 0) {
            builder.append(" Body movement detected");
        }

        if ((status & 2) > 0) {
            builder.append(" Cuff too loose");
        }

        if ((status & 4) > 0) {
            builder.append(" Irregular pulse detected");
        }

        if ((status & 24) == 8) {
            builder.append(" Pulse rate exceeds upper limit");
        }

        if ((status & 24) == 16) {
            builder.append(" Pulse rate is less than lower limit");
        }

        if ((status & 24) == 24) {
            builder.append(" Pulse rate range: Reserved for future use ");
        }

        if ((status & 32) > 0) {
            builder.append(" Improper measurement position");
        }
    }

    return builder.toString();
}
}

```

```

// Directory: app
// Subdirectory:

```

```
classes.jar_Decompile.com\com\android\chileaf\fitness\common\parser
// File: BodySensorLocationParser.java
```

```
package com.android.chileaf.fitness.common.parser;
```

```
import no.nordicsemi.android.ble.data.Data;
```

```
public class BodySensorLocationParser {
    public static String parse(final Data data) {
        int value = data.getIntValue(17, 0);
        switch(value) {
            case 0:
            default:
                return "Other";
            case 1:
                return "Chest";
            case 2:
                return "Wrist";
            case 3:
                return "Finger";
            case 4:
                return "Hand";
            case 5:
                return "Ear Lobe";
            case 6:
                return "Foot";
        }
    }
}
```

```
// Directory: app
```

```
// Subdirectory:
```

```
classes.jar_Decompile.com\com\android\chileaf\fitness\common\parser
```

```
// File: CSCMeasurementParser.java
```

```
package com.android.chileaf.fitness.common.parser;
```

```
import no.nordicsemi.android.ble.data.Data;
```

```
public class CSCMeasurementParser {
    private static final byte WHEEL_REV_DATA_PRESENT = 1;
    private static final byte CRANK_REV_DATA_PRESENT = 2;

    public static String parse(final Data data) {
        int offset = 0;
        int flags = data.getBytes(offset);
        int offset = offset + 1;
        boolean wheelRevPresent = (flags & 1) > 0;
        boolean crankRevPreset = (flags & 2) > 0;
        int wheelRevolutions = 0;
        int lastWheelEventTime = 0;
        if (wheelRevPresent) {
            wheelRevolutions = data.getIntValue(20, offset);
        }
    }
}
```

```

        offset += 4;
        lastWheelEventTime = data.getIntValue(18, offset);
        offset += 2;
    }

    int crankRevolutions = 0;
    int lastCrankEventTime = 0;
    if (crankRevPreset) {
        crankRevolutions = data.getIntValue(18, offset);
        offset += 2;
        lastCrankEventTime = data.getIntValue(18, offset);
    }

    StringBuilder builder = new StringBuilder();
    if (wheelRevPresent) {
        builder.append("Wheel rev: ").append(wheelRevolutions).append(",");
        builder.append("Last wheel event time:
").append(lastWheelEventTime).append(",");
    }

    if (crankRevPreset) {
        builder.append("Crank rev: ").append(crankRevolutions).append(",");
        builder.append("Last crank event time:
").append(lastCrankEventTime).append(",");
    }

    if (!wheelRevPresent && !crankRevPreset) {
        builder.append("No wheel or crank data");
    }

    builder.setLength(builder.length() - 2);
    return builder.toString();
}
}

```

```

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\com\android\chileaf\fitness\common\parser
// File: DateTimeParser.java

```

```

package com.android.chileaf.fitness.common.parser;

import com.android.chileaf.fitness.common.DateTimeDataCallback;
import java.util.Calendar;
import java.util.Locale;
import no.nordicsemi.android.ble.data.Data;

public class DateTimeParser {
    public static String parse(final Data data) {
        return parse(data, 0);
    }

    static String parse(final Data data, final int offset) {

```

```

        Calendar calendar = DateTimeDataCallback.readDateTime(data, offset);
        return String.format(Locale.US, "%1$te %1$tb %1$tY, %1$tH:%1$tM:%1$tS",
calendar);
    }
}

```

```

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\com\android\chileaf\fitness\common\parser
// File: HeartRateMeasurementParser.java

```

```

package com.android.chileaf.fitness.common.parser;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.Locale;
import no.nordicsemi.android.ble.data.Data;

public class HeartRateMeasurementParser {
    private static final byte HEART_RATE_VALUE_FORMAT = 1;
    private static final byte SENSOR_CONTACT_STATUS = 6;
    private static final byte ENERGY_EXPANDED_STATUS = 8;
    private static final byte RR_INTERVAL = 16;

    public static String parse(final Data data) {
        int offset = 0;
        int offset = offset + 1;
        int flags = data.getIntValue(17, offset);
        boolean value16bit = (flags & 1) > 0;
        int sensorContactStatus = (flags & 6) >> 1;
        boolean energyExpandedStatus = (flags & 8) > 0;
        boolean rrIntervalStatus = (flags & 16) > 0;
        int heartRateValue = data.getIntValue(value16bit ? 18 : 17, offset++);
        if (value16bit) {
            ++offset;
        }

        int energyExpanded = -1;
        if (energyExpandedStatus) {
            energyExpanded = data.getIntValue(18, offset);
        }

        offset += 2;
        List<Float> rrIntervals = new ArrayList();
        if (rrIntervalStatus) {
            for(int o = offset; o < data.getValue().length; o += 2) {
                int units = data.getIntValue(18, o);
                rrIntervals.add((float)units * 1000.0F / 1024.0F);
            }
        }

        StringBuilder builder = new StringBuilder();
    }
}

```

```

        builder.append("Heart Rate Measurement: ").append(heartRateValue).append("
bpm");
        switch(sensorContactStatus) {
            case 0:
            case 1:
                builder.append(",Sensor Contact Not Supported");
                break;
            case 2:
                builder.append(",Contact is NOT Detected");
                break;
            case 3:
                builder.append(",Contact is Detected");
        }

        if (energyExpandedStatus) {
            builder.append(",Energy Expanded: ").append(energyExpanded).append("
kJ");
        }

        if (rrIntervalStatus) {
            builder.append(",RR Interval: ");
            Iterator var15 = rrIntervals.iterator();

            while(var15.hasNext()) {
                Float interval = (Float)var15.next();
                builder.append(String.format(Locale.US, "%.02f ms, ", interval));
            }

            builder.setLength(builder.length() - 2);
        }

        return builder.toString();
    }
}

```

// Directory: app

// Subdirectory:

classes.jar_Decompile.com\com\android\chileaf\fitness\common\profile

// File: ProfileCallback.java

```
package com.android.chileaf.fitness.common.profile;
```

```
import android.bluetooth.BluetoothDevice;
```

```
import androidx.annotation.NonNull;
```

```
public interface ProfileCallback {
```

```
    void onSystemId(@NonNull final BluetoothDevice device, final String
systemId);
```

```
    void onModelName(@NonNull final BluetoothDevice device, final String
modelName);
```

```
    void onSerialNumber(@NonNull final BluetoothDevice device, final String
```



```

serialNumber);

    void onFirmwareVersion(@NonNull final BluetoothDevice device, final String
firmware);

    void onHardwareVersion(@NonNull final BluetoothDevice device, final String
hardware);

    void onSoftwareVersion(@NonNull final BluetoothDevice device, final String
software);

    void onVendorName(@NonNull final BluetoothDevice device, final String
vendorName);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\model
// File: HistoryOf3D.java

package com.android.chileaf.model;

import android.os.Parcel;
import android.os.Parcelable;
import android.os.Parcelable.Creator;

public class HistoryOf3D implements Parcelable {
    public int accX;
    public int accY;
    public int accZ;
    public static final Creator<HistoryOf3D> CREATOR = new Creator<HistoryOf3D>()
{
    public HistoryOf3D createFromParcel(Parcel in) {
        return new HistoryOf3D(in);
    }

    public HistoryOf3D[] newArray(int size) {
        return new HistoryOf3D[size];
    }
};

    public HistoryOf3D(int accX, int accY, int accZ) {
        this.accX = accX;
        this.accY = accY;
        this.accZ = accZ;
    }

    protected HistoryOf3D(Parcel in) {
        this.accX = in.readInt();
        this.accY = in.readInt();
        this.accZ = in.readInt();
    }

    public String toString() {

```

```

        return "HistoryOf3D{ accX=" + this.accX + ", accY=" + this.accY + ",
accZ=" + this.accZ + '}'';
    }

```

```

    public int describeContents() {
        return 0;
    }

```

```

    public void writeToParcel(Parcel dest, int flags) {
        dest.writeInt(this.accX);
        dest.writeInt(this.accY);
        dest.writeInt(this.accZ);
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\model
// File: HistoryOfHeartRate.java

```

```

package com.android.chileaf.model;

```

```

import android.os.Parcel;
import android.os.Parcelable;
import android.os.Parcelable.Creator;

```

```

public class HistoryOfHeartRate implements Parcelable {
    public long stamp;
    public int heartRate;
    public static final Creator<HistoryOfHeartRate> CREATOR = new
Creator<HistoryOfHeartRate>() {
        public HistoryOfHeartRate createFromParcel(Parcel in) {
            return new HistoryOfHeartRate(in);
        }

        public HistoryOfHeartRate[] newArray(int size) {
            return new HistoryOfHeartRate[size];
        }
    };
}

```

```

    public HistoryOfHeartRate(long stamp, int heartRate) {
        this.stamp = stamp;
        this.heartRate = heartRate;
    }

```

```

    public String toString() {
        return "HistoryOfHeartRate{startTime=" + this.stamp + ", heartRate=" +
this.heartRate + '}'';
    }

```

```

    protected HistoryOfHeartRate(Parcel in) {
        this.stamp = in.readLong();
        this.heartRate = in.readInt();
    }

```

```

    public void writeToParcel(Parcel dest, int flags) {
        dest.writeLong(this.stamp);
        dest.writeInt(this.heartRate);
    }

    public int describeContents() {
        return 0;
    }
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\model
// File: HistoryOfRecord.java

package com.android.chileaf.model;

import android.os.Parcel;
import android.os.Parcelable;
import android.os.Parcelable.Creator;

public class HistoryOfRecord implements Parcelable {
    public long stamp;
    public long record;
    public static final Creator<HistoryOfRecord> CREATOR = new
Creator<HistoryOfRecord>() {
        public HistoryOfRecord createFromParcel(Parcel in) {
            return new HistoryOfRecord(in);
        }

        public HistoryOfRecord[] newArray(int size) {
            return new HistoryOfRecord[size];
        }
    };

    public HistoryOfRecord(long stamp, long record) {
        this.stamp = stamp;
        this.record = record;
    }

    public String toString() {
        return "HistoryOfSport{startTime=" + this.stamp + ", record=" +
this.record + '}';
    }

    protected HistoryOfRecord(Parcel in) {
        this.stamp = in.readLong();
        this.record = in.readLong();
    }

    public void writeToParcel(Parcel dest, int flags) {
        dest.writeLong(this.stamp);
        dest.writeLong(this.record);
    }
}

```

```

    }

    public int describeContents() {
        return 0;
    }
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\model
// File: HistoryOfRespiratoryRate.java

package com.android.chileaf.model;

import android.os.Parcel;
import android.os.Parcelable;
import android.os.Parcelable.Creator;

public class HistoryOfRespiratoryRate implements Parcelable {
    public long stamp;
    public int respiratoryRate;
    public static final Creator<HistoryOfRespiratoryRate> CREATOR = new
Creator<HistoryOfRespiratoryRate>() {
        public HistoryOfRespiratoryRate createFromParcel(Parcel in) {
            return new HistoryOfRespiratoryRate(in);
        }

        public HistoryOfRespiratoryRate[] newArray(int size) {
            return new HistoryOfRespiratoryRate[size];
        }
    };

    public HistoryOfRespiratoryRate(long stamp, int respiratoryRate) {
        this.stamp = stamp;
        this.respiratoryRate = respiratoryRate;
    }

    public String toString() {
        return "HistoryOfHeartRate{stamp=" + this.stamp + ", respiratoryRate=" +
this.respiratoryRate + '}';
    }

    protected HistoryOfRespiratoryRate(Parcel in) {
        this.stamp = in.readLong();
        this.respiratoryRate = in.readInt();
    }

    public void writeToParcel(Parcel dest, int flags) {
        dest.writeLong(this.stamp);
        dest.writeInt(this.respiratoryRate);
    }

    public int describeContents() {
        return 0;
    }
}

```

```
}  
}
```

```
// Directory: app  
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\model  
// File: HistoryOfSport.java
```

```
package com.android.chileaf.model;
```

```
import android.os.Parcel;  
import android.os.Parcelable;  
import android.os.Parcelable.Creator;
```

```
public class HistoryOfSport implements Parcelable {
```

```
    public long startTime;
```

```
    public long endTime;
```

```
    public long step;
```

```
    public long calorie;
```

```
    public static final Creator<HistoryOfSport> CREATOR = new
```

```
Creator<HistoryOfSport>() {
```

```
    public HistoryOfSport createFromParcel(Parcel in) {
```

```
        return new HistoryOfSport(in);
```

```
    }
```

```
    public HistoryOfSport[] newArray(int size) {
```

```
        return new HistoryOfSport[size];
```

```
    }
```

```
};
```

```
public HistoryOfSport(long startTime, long step, long calorie) {
```

```
    this.startTime = startTime;
```

```
    this.step = step;
```

```
    this.calorie = calorie;
```

```
}
```

```
public HistoryOfSport(long startTime, long endTime, long step, long calorie)  
{
```

```
    this.startTime = startTime;
```

```
    this.endTime = endTime;
```

```
    this.step = step;
```

```
    this.calorie = calorie;
```

```
}
```

```
public String toString() {
```

```
    return "HistoryOfSport{startTime=" + this.startTime + ", endTime=" +  
this.endTime + ", step=" + this.step + ", calorie=" + this.calorie + '}';  
}
```

```
protected HistoryOfSport(Parcel in) {
```

```
    this.startTime = in.readLong();
```

```
    this.step = in.readLong();
```

```
    this.calorie = in.readLong();
```

```
    this.endTime = in.readLong();
```

```

    }

    public void writeToParcel(Parcel dest, int flags) {
        dest.writeLong(this.startTime);
        dest.writeLong(this.step);
        dest.writeLong(this.calorie);
        dest.writeLong(this.endTime);
    }

    public int describeContents() {
        return 0;
    }
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\model
// File: HistorySleep.java

package com.android.chileaf.model;

import android.os.Parcel;
import android.os.Parcelable;
import android.os.Parcelable.Creator;

public class HistorySleep implements Parcelable {
    public long utc;
    public int[] actions;
    public static final Creator<HistorySleep> CREATOR = new
Creator<HistorySleep>() {
        public HistorySleep createFromParcel(Parcel in) {
            return new HistorySleep(in);
        }

        public HistorySleep[] newArray(int size) {
            return new HistorySleep[size];
        }
    };

    public HistorySleep() {
    }

    public HistorySleep(long utc, int[] actions) {
        this.utc = utc;
        this.actions = actions;
    }

    protected HistorySleep(Parcel in) {
        this.utc = in.readLong();
        this.actions = in.createIntArray();
    }

    public void writeToParcel(Parcel dest, int flags) {
        dest.writeLong(this.utc);
    }
}

```

```

        dest.writeIntArray(this.actions);
    }

    public int describeContents() {
        return 0;
    }
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\model
// File: IntervalStep.java

package com.android.chileaf.model;

import android.os.Parcel;
import android.os.Parcelable;
import android.os.Parcelable.Creator;

public class IntervalStep implements Parcelable {
    public long stamp;
    public int steps;
    public static final Creator<IntervalStep> CREATOR = new
Creator<IntervalStep>() {
        public IntervalStep createFromParcel(Parcel in) {
            return new IntervalStep(in);
        }

        public IntervalStep[] newArray(int size) {
            return new IntervalStep[size];
        }
    };

    public IntervalStep(long stamp, int steps) {
        this.stamp = stamp;
        this.steps = steps;
    }

    public String toString() {
        return "IntervalStep{startTime=" + this.stamp + ", steps=" + this.steps +
    }';
    }

    protected IntervalStep(Parcel in) {
        this.stamp = in.readLong();
        this.steps = in.readInt();
    }

    public void writeToParcel(Parcel dest, int flags) {
        dest.writeLong(this.stamp);
        dest.writeInt(this.steps);
    }

    public int describeContents() {

```

```

        return 0;
    }
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\util
// File: DateUtil.java

package com.android.chileaf.util;

import java.util.Calendar;

public class DateUtil {
    public static long getZoneUTC() {
        Calendar calendar = Calendar.getInstance();
        int zoneOffset = calendar.get(15);
        int dstOffset = calendar.get(16);
        calendar.add(14, zoneOffset + dstOffset);
        return calendar.getTimeInMillis() / 1000L;
    }

    public static long restoreZoneUTC(final long stamp) {
        Calendar calendar = Calendar.getInstance();
        calendar.setTimeInMillis(stamp * 1000L);
        int zoneOffset = calendar.get(15);
        int dstOffset = calendar.get(16);
        calendar.add(14, -(zoneOffset + dstOffset));
        return calendar.getTimeInMillis();
    }

    public static long restoreZoneUTCTimeInMillis(final long stamp) {
        Calendar calendar = Calendar.getInstance();
        calendar.setTimeInMillis(stamp);
        int zoneOffset = calendar.get(15);
        int dstOffset = calendar.get(16);
        calendar.add(14, -(zoneOffset + dstOffset));
        return calendar.getTimeInMillis();
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\util
// File: HexUtil.java

package com.android.chileaf.util;

import java.nio.charset.Charset;

public class HexUtil {
    private static final char[] DIGITS_LOWER = new char[]{'0', '1', '2', '3',
'4', '5', '6', '7', '8', '9', 'a', 'b', 'c', 'd', 'e', 'f'};
    private static final char[] DIGITS_UPPER = new char[]{'0', '1', '2', '3',

```



```

'4', '5', '6', '7', '8', '9', 'A', 'B', 'C', 'D', 'E', 'F'};

    public static char[] encodeHex(final byte[] data) {
        return encodeHex(data, true);
    }

    public static char[] encodeHex(final byte[] data, final boolean toLowerCase)
{
        return encodeHex(data, toLowerCase ? DIGITS_LOWER : DIGITS_UPPER);
    }

    protected static char[] encodeHex(final byte[] data, final char[] toDigits) {
        if (data == null) {
            return new char[0];
        } else {
            int l = data.length;
            char[] out = new char[l << 1];
            int i = 0;

            for(int var5 = 0; i < l; ++i) {
                out[var5++] = toDigits[(240 & data[i]) >>> 4];
                out[var5++] = toDigits[15 & data[i]];
            }

            return out;
        }
    }

    public static String encodeHexStr(final byte[] data) {
        return encodeHexStr(data, true);
    }

    public static String encodeHexStr(final byte[] data, final boolean
toLowerCase) {
        return encodeHexStr(data, toLowerCase ? DIGITS_LOWER : DIGITS_UPPER);
    }

    protected static String encodeHexStr(final byte[] data, final char[]
toDigits) {
        return new String(encodeHex(data, toDigits));
    }

    public static byte[] decodeHex(char[] data) {
        int len = data.length;
        if ((len & 1) != 0) {
            throw new RuntimeException("Odd number of characters.");
        } else {
            byte[] out = new byte[len >> 1];
            int i = 0;

            for(int j = 0; j < len; ++j) {
                int f = toDigit(data[j], j) << 4;
                ++j;
                f |= toDigit(data[j], j);
            }
        }
    }

```

```

        ++j;
        out[i] = (byte)(f & 255);
    }

    return out;
}

}

public static int toDigit(char ch, int index) {
    int digit = Character.digit(ch, 16);
    if (digit == -1) {
        throw new RuntimeException("Illegal hexadecimal character " + ch + " at
index " + index);
    } else {
        return digit;
    }
}

public static byte[] hexStringToBytes(String hexString) {
    if (hexString != null && !hexString.equals("")) {
        hexString = hexString.toUpperCase();
        int length = hexString.length() / 2;
        char[] hexChars = hexString.toCharArray();
        byte[] d = new byte[length];

        for(int i = 0; i < length; ++i) {
            int pos = i * 2;
            d[i] = (byte)(charToByte(hexChars[pos]) << 4 |
charToByte(hexChars[pos + 1]));
        }

        return d;
    } else {
        return new byte[0];
    }
}

public static byte[] fromHexString(final String hexString) {
    if (null != hexString && !"".equals(hexString.trim())) {
        byte[] bytes = new byte[hexString.length() / 2];

        for(int i = 0; i < hexString.length() / 2; ++i) {
            String hex = hexString.substring(i * 2, i * 2 + 2);
            bytes[i] = (byte)Integer.parseInt(hex, 16);
        }

        return bytes;
    } else {
        return new byte[0];
    }
}

public static byte charToByte(char c) {
    return (byte)"0123456789ABCDEF".indexOf(c);
}

```

```

}

public static String byteArrayToString(final byte[] data) {
    StringBuilder sb = new StringBuilder();
    byte[] var2 = data;
    int var3 = data.length;

    for(int var4 = 0; var4 < var3; ++var4) {
        byte datum = var2[var4];
        char temp = (char)datum;
        if (temp != 0) {
            sb.append(temp);
        }
    }

    return sb.toString();
}

public static String splitToHexString(final byte[] data) {
    StringBuilder sb = new StringBuilder();

    for(int i = 0; i < data.length; ++i) {
        byte datum = data[i];
        String hex = Integer.toHexString(datum & 255);
        if (hex.length() < 2) {
            sb.append(0);
        }

        if (datum == 44) {
            char ch = (char)datum;
            sb.append(ch);
        } else {
            sb.append(hex);
        }
    }

    return sb.toString();
}

public static String getAsciiString(final byte[] data, final int offset,
final int length) {
    return new String(data, offset, length, Charset.forName("US-ASCII"));
}

public static String getAsciiString(final byte[] data) {
    return getAsciiString(data, 0, data.length);
}

public static byte[] stringToBytes(final String data) {
    byte[] array = new byte[data.length()];

    for(int i = 0; i < data.length(); ++i) {
        array[i] = (byte)data.charAt(i);
    }
}

```

```

    return array;
}

public static String bytes2HexString(final byte[] bytes) {
    StringBuffer sb = new StringBuffer();

    for(int i = 0; i < bytes.length; ++i) {
        String hex = Integer.toHexString(bytes[i] & 255);
        if (hex.length() < 2) {
            sb.append(0);
        }

        sb.append(hex);
    }

    return sb.toString();
}

public static String hex2String(final String hex) {
    StringBuilder sb = new StringBuilder();
    StringBuilder temp = new StringBuilder();

    for(int i = 0; i < hex.length() - 1; i += 2) {
        String output = hex.substring(i, i + 2);
        int str = Integer.parseInt(output, 16);
        sb.append((char)str);
        temp.append(str);
    }

    return sb.toString();
}

public static byte[] compose(final int... bytes) {
    byte[] dest = new byte[bytes.length];

    for(int i = 0; i < bytes.length; ++i) {
        dest[i] = (byte)(bytes[i] & 255);
    }

    return dest;
}

public static byte[] compose(final byte... bytes) {
    byte[] dest = new byte[bytes.length];
    System.arraycopy(bytes, 0, dest, 0, bytes.length);
    return dest;
}

public static byte[] subByte(final byte[] source, final int length) {
    byte[] dest = new byte[length];
    System.arraycopy(source, 0, dest, 0, dest.length);
    return dest;
}

```

```

    public static byte[] subByte(final byte[] source, final int start, final int
end) {
        byte[] dest = new byte[end - start];
        System.arraycopy(source, start, dest, 0, dest.length);
        return dest;
    }

    public static byte[] subByteByLength(final byte[] source, final int start,
final int length) {
        byte[] dest = new byte[length];
        System.arraycopy(source, start, dest, 0, dest.length);
        return dest;
    }

    public static int[] append(final int source, final int[] dest) {
        int[] result = new int[1 + dest.length];
        result[0] = source;
        System.arraycopy(dest, 0, result, 1, dest.length);
        return result;
    }

    public static byte[] append(final byte source, final byte[] dest) {
        byte[] temp = new byte[]{source};
        return append(temp, dest);
    }

    public static byte[] append(final byte[] source, final byte dest) {
        byte[] result = new byte[source.length + 1];
        byte[] temp = new byte[]{dest};
        System.arraycopy(source, 0, result, 0, source.length);
        System.arraycopy(temp, 0, result, result.length - 1, 1);
        return result;
    }

    public static byte[] append(final byte[] source, final byte[] dest) {
        byte[] result = new byte[source.length + dest.length];
        System.arraycopy(source, 0, result, 0, source.length);
        int offset = result.length - dest.length;
        System.arraycopy(dest, 0, result, offset, dest.length);
        return result;
    }

    public static byte[] append(final byte[]... bytes) {
        byte[] dest = new byte[0];
        byte[][] var2 = bytes;
        int var3 = bytes.length;

        for(int var4 = 0; var4 < var3; ++var4) {
            byte[] outs = var2[var4];
            dest = append(dest, outs);
        }

        return dest;
    }

```

```
    }  
}
```

```
// Directory: app  
// Subdirectory: classes.jar_Decompile.com\com\android\chileaf\util  
// File: LogUtil.java
```

```
package com.android.chileaf.util;
```

```
import android.os.Build.VERSION;  
import android.util.Log;  
import androidx.annotation.NonNull;  
import androidx.annotation.Nullable;  
import java.io.PrintWriter;  
import java.io.StringWriter;  
import java.util.regex.Matcher;  
import java.util.regex.Pattern;
```

```
public final class LogUtil {  
    private static final LogUtil.Tree DEBUG_TREE = new LogUtil.DebugTree();  
  
    public static void v(@NonNull String message, Object... args) {  
        DEBUG_TREE.v(message, args);  
    }  
  
    public static void v(Throwable t, @NonNull String message, Object... args) {  
        DEBUG_TREE.v(t, message, args);  
    }  
  
    public static void v(Throwable t) {  
        DEBUG_TREE.v(t);  
    }  
  
    public static void d(@NonNull String message, Object... args) {  
        DEBUG_TREE.d(message, args);  
    }  
  
    public static void d(Throwable t, @NonNull String message, Object... args) {  
        DEBUG_TREE.d(t, message, args);  
    }  
  
    public static void d(Throwable t) {  
        DEBUG_TREE.d(t);  
    }  
  
    public static void i(@NonNull String message, Object... args) {  
        DEBUG_TREE.i(message, args);  
    }  
  
    public static void i(Throwable t, @NonNull String message, Object... args) {  
        DEBUG_TREE.i(t, message, args);  
    }  
}
```

```

public static void i(Throwable t) {
    DEBUG_TREE.i(t);
}

public static void w(@NonNull String message, Object... args) {
    DEBUG_TREE.w(message, args);
}

public static void w(Throwable t, @NonNull String message, Object... args) {
    DEBUG_TREE.w(t, message, args);
}

public static void w(Throwable t) {
    DEBUG_TREE.w(t);
}

public static void e(@NonNull String message, Object... args) {
    DEBUG_TREE.e(message, args);
}

public static void e(Throwable t, @NonNull String message, Object... args) {
    DEBUG_TREE.e(t, message, args);
}

public static void e(Throwable t) {
    DEBUG_TREE.e(t);
}

public static void wtf(@NonNull String message, Object... args) {
    DEBUG_TREE.wtf(message, args);
}

public static void wtf(Throwable t, @NonNull String message, Object... args)
{
    DEBUG_TREE.wtf(t, message, args);
}

public static void wtf(Throwable t) {
    DEBUG_TREE.wtf(t);
}

public static void log(int invoke, int priority, @NonNull String message,
Object... args) {
    DEBUG_TREE.log(invoke, priority, message, args);
}

public static void log(int priority, @NonNull String message, Object... args)
{
    DEBUG_TREE.log(priority, message, args);
}

public static void log(int priority, Throwable t, @NonNull String message,
Object... args) {
    DEBUG_TREE.log(priority, t, message, args);
}

```

```

    }

    public static void log(int priority, Throwable t) {
        DEBUG_TREE.log(priority, t);
    }

    @NonNull
    public static LogUtil.Tree tag(String tag) {
        DEBUG_TREE.explicitTag.set(tag);
        return DEBUG_TREE;
    }

    public static void setDebug(boolean debug) {
        DEBUG_TREE.setDebug(debug);
    }

    private LogUtil() {
        throw new AssertionError("No instances.");
    }

    public static class DebugTree extends LogUtil.Tree {
        private static final int MAX_LOG_LENGTH = 4000;
        private static final int MAX_TAG_LENGTH = 23;
        private static final int CALL_STACK_INDEX = 5;
        private static final Pattern ANONYMOUS_CLASS =
Pattern.compile("(\\$\\d+)+$");

        @Nullable
        protected String createStackElementTag(@NonNull StackTraceElement element)
{
            String tag = element.getClassName();
            Matcher m = ANONYMOUS_CLASS.matcher(tag);
            if (m.find()) {
                tag = m.replaceAll("");
            }

            tag = tag.substring(tag.lastIndexOf(46) + 1);
            return tag.length() > 23 && VERSION.SDK_INT < 24 ? tag.substring(0, 23)
: tag;
        }

        final String getTag() {
            String tag = super.getTag();
            if (tag != null) {
                return tag;
            } else {
                StackTraceElement[] stackTrace = (new Throwable()).getStackTrace();
                if (stackTrace.length <= 5) {
                    throw new IllegalStateException("Synthetic stacktrace didn't have
enough elements: are you using proguard?");
                } else {
                    return this.createStackElementTag(stackTrace[5]);
                }
            }
        }
    }

```



```

    }

    protected void log(int priority, String tag, @NonNull String message,
        Throwable t) {
        if (message.length() < 4000) {
            if (priority == 7) {
                Log.wtf(tag, message);
            } else {
                Log.println(priority, tag, message);
            }
        }

        } else {
            int i = 0;

            int end;
            for(int length = message.length(); i < length; i = end + 1) {
                int newline = message.indexOf(10, i);
                newline = newline != -1 ? newline : length;

                do {
                    end = Math.min(newline, i + 4000);
                    String part = message.substring(i, end);
                    if (priority == 7) {
                        Log.wtf(tag, part);
                    } else {
                        Log.println(priority, tag, part);
                    }
                }

                i = end;
            } while(end < newline);
        }

    }
}

```

```

public abstract static class Tree {
    private boolean isDebug;
    private final ThreadLocal<String> explicitTag = new ThreadLocal();

    @Nullable
    String getTag() {
        String tag = (String)this.explicitTag.get();
        if (tag != null) {
            this.explicitTag.remove();
        }

        return tag;
    }

    public void v(String message, Object... args) {
        this.prepareLog(5, 2, (Throwable)null, message, args);
    }
}

```

```
public void v(Throwable t, String message, Object... args) {
    this.prepareLog(5, 2, t, message, args);
}

public void v(Throwable t) {
    this.prepareLog(5, 2, t, (String)null);
}

public void d(String message, Object... args) {
    this.prepareLog(5, 3, (Throwable)null, message, args);
}

public void d(Throwable t, String message, Object... args) {
    this.prepareLog(5, 3, t, message, args);
}

public void d(Throwable t) {
    this.prepareLog(5, 3, t, (String)null);
}

public void i(String message, Object... args) {
    this.prepareLog(5, 4, (Throwable)null, message, args);
}

public void i(Throwable t, String message, Object... args) {
    this.prepareLog(5, 4, t, message, args);
}

public void i(Throwable t) {
    this.prepareLog(5, 4, t, (String)null);
}

public void w(String message, Object... args) {
    this.prepareLog(5, 5, (Throwable)null, message, args);
}

public void w(Throwable t, String message, Object... args) {
    this.prepareLog(5, 5, t, message, args);
}

public void w(Throwable t) {
    this.prepareLog(5, 5, t, (String)null);
}

public void e(String message, Object... args) {
    this.prepareLog(5, 6, (Throwable)null, message, args);
}

public void e(Throwable t, String message, Object... args) {
    this.prepareLog(5, 6, t, message, args);
}

public void e(Throwable t) {
    this.prepareLog(5, 6, t, (String)null);
}
```

```

    }

    public void wtf(String message, Object... args) {
        this.prepareLog(5, 7, (Throwable)null, message, args);
    }

    public void wtf(Throwable t, String message, Object... args) {
        this.prepareLog(5, 7, t, message, args);
    }

    public void wtf(Throwable t) {
        this.prepareLog(5, 7, t, (String)null);
    }

    public void log(int invoke, int priority, String message, Object... args)
{
        this.prepareLog(invoke, priority, (Throwable)null, message, args);
    }

    public void log(int priority, String message, Object... args) {
        this.prepareLog(5, priority, (Throwable)null, message, args);
    }

    public void log(int priority, Throwable t, String message, Object... args)
{
        this.prepareLog(5, priority, t, message, args);
    }

    public void log(int priority, Throwable t) {
        this.prepareLog(5, priority, t, (String)null);
    }

    public void setDebug(boolean debug) {
        this.isDebugEnabled = debug;
    }

    private void prepareLog(int invoke, int priority, Throwable t, String
message, Object... args) {
        if (this.isDebugEnabled) {
            if (message != null && message.length() == 0) {
                message = null;
            }

            if (message == null) {
                if (t == null) {
                    return;
                }

                message = this.getStackTraceString(t);
            } else {
                if (args != null && args.length > 0) {
                    message = this.formatMessage(message, args);
                }
            }
        }
    }

```

```

        if (t != null) {
            message = message + "\n" + this.getStackTraceString(t);
        }
    }

    String tag = null;
    String prefix = null;

    try {
        StackTraceElement stackTrace =
Thread.currentThread().getStackTrace()[invoke];
        String fileName = stackTrace.getFileName();
        int lineNumber = stackTrace.getLineNumber();
        tag = fileName.substring(0, fileName.lastIndexOf("."));
        prefix = "(" + fileName + ":" + lineNumber + ") ";
    } catch (Exception var11) {
        var11.printStackTrace();
    }

    if (tag == null) {
        tag = this.getTag();
    }

    if (prefix != null) {
        message = prefix + message;
    }

    this.log(priority, tag, message, t);
}

private String formatMessage(@NonNull String message, @NonNull Object[]
args) {
    return String.format(message, args);
}

private String getStackTraceString(Throwable t) {
    StringWriter sw = new StringWriter(256);
    PrintWriter pw = new PrintWriter(sw, false);
    t.printStackTrace(pw);
    pw.flush();
    return sw.toString();
}

protected abstract void log(int priority, @Nullable String tag, @NonNull
String message, @Nullable Throwable t);
}

// Directory: app
// Subdirectory: classes.jar_Decompiler.com\nordicsemi\android\ble
// File: AwaitingRequest.java

```

```

package no.nordicsemi.android.ble;

import android.bluetooth.BluetoothGattCharacteristic;
import android.bluetooth.BluetoothGattDescriptor;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import java.util.concurrent.CancellationException;
import no.nordicsemi.android.ble.exception.BluetoothDisabledException;
import no.nordicsemi.android.ble.exception.DeviceDisconnectedException;
import no.nordicsemi.android.ble.exception.InvalidRequestException;
import no.nordicsemi.android.ble.exception.RequestFailedException;

public abstract class AwaitingRequest<T> extends TimeoutableValueRequest<T> {
    private static final int NOT_STARTED = -123456;
    private static final int STARTED = -123455;
    private Request trigger;
    private int triggerStatus = 0;

    AwaitingRequest(@NonNull Request.Type type) {
        super(type);
    }

    AwaitingRequest(@NonNull Request.Type type, @Nullable
BluetoothGattCharacteristic characteristic) {
        super(type, characteristic);
    }

    AwaitingRequest(@NonNull Request.Type type, @Nullable BluetoothGattDescriptor
descriptor) {
        super(type, descriptor);
    }

    @NonNull
    public AwaitingRequest<T> trigger(@NonNull Operation trigger) {
        if (trigger instanceof Request) {
            this.trigger = (Request)trigger;
            this.triggerStatus = -123456;
            this.trigger.internalBefore((device) -> {
                this.triggerStatus = -123455;
            });
            this.trigger.internalSuccess((device) -> {
                this.triggerStatus = 0;
            });
            this.trigger.internalFail((device, status) -> {
                this.triggerStatus = status;
                this.syncLock.open();
                this.notifyFail(device, status);
            });
        }

        return this;
    }

    @NonNull

```

```

    public <E extends T> E await(@NonNull E response) throws
RequestFailedException, DeviceDisconnectedException, BluetoothDisabledException,
InvalidRequestException, InterruptedException, CancellationException {
    assertNotMainThread();

    try {
        if (this.trigger != null && this.trigger.enqueue() {
            throw new IllegalStateException("Trigger request already enqueued");
        } else {
            super.await(response);
            return response;
        }
    } catch (RequestFailedException var3) {
        if (this.triggerStatus != 0) {
            throw new RequestFailedException(this.trigger, this.triggerStatus);
        } else {
            throw var3;
        }
    }
}

@Nullable
Request getTrigger() {
    return this.trigger;
}

boolean isTriggerPending() {
    return this.triggerStatus == -123456;
}

boolean isTriggerCompleteOrNull() {
    return this.triggerStatus != -123455;
}
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: BleManager.java

```

```

package no.nordicsemi.android.ble;

import android.annotation.SuppressLint;
import android.bluetooth.BluetoothDevice;
import android.bluetooth.BluetoothGatt;
import android.bluetooth.BluetoothGattCharacteristic;
import android.bluetooth.BluetoothGattDescriptor;
import android.bluetooth.BluetoothGattServer;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.content.IntentFilter;
import android.os.Handler;
import android.os.Looper;

```

```

import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.annotation.RequiresApi;
import androidx.annotation.RequiresPermission;
import androidx.annotation.RestrictTo;
import androidx.annotation.StringRes;
import androidx.annotation.RestrictTo.Scope;
import java.util.UUID;
import no.nordicsemi.android.ble.callback.ConnectionParametersUpdatedCallback;
import no.nordicsemi.android.ble.data.Data;
import no.nordicsemi.android.ble.data.DataProvider;
import no.nordicsemi.android.ble.observer.BondingObserver;
import no.nordicsemi.android.ble.observer.ConnectionObserver;
import no.nordicsemi.android.ble.utils.ILogger;
import no.nordicsemi.android.ble.utils.ParserUtils;

@SuppressLint({"MissingPermission"})
public abstract class BleManager implements ILogger {
    static final UUID CLIENT_CHARACTERISTIC_CONFIG_DESCRIPTOR_UUID =
    UUID.fromString("00002902-0000-1000-8000-00805f9b34fb");
    static final UUID BATTERY_SERVICE =
    UUID.fromString("0000180F-0000-1000-8000-00805f9b34fb");
    static final UUID BATTERY_LEVEL_CHARACTERISTIC =
    UUID.fromString("00002A19-0000-1000-8000-00805f9b34fb");
    static final UUID GENERIC_ATTRIBUTE_SERVICE =
    UUID.fromString("00001801-0000-1000-8000-00805f9b34fb");
    static final UUID SERVICE_CHANGED_CHARACTERISTIC =
    UUID.fromString("00002A05-0000-1000-8000-00805f9b34fb");
    public static final int PAIRING_VARIANT_PIN = 0;
    public static final int PAIRING_VARIANT_PASSKEY = 1;
    public static final int PAIRING_VARIANT_PASSKEY_CONFIRMATION = 2;
    public static final int PAIRING_VARIANT_CONSENT = 3;
    public static final int PAIRING_VARIANT_DISPLAY_PASSKEY = 4;
    public static final int PAIRING_VARIANT_DISPLAY_PIN = 5;
    public static final int PAIRING_VARIANT_OOB_CONSENT = 6;
    private final Context context;
    private BleServerManager serverManager;
    @NonNull
    final BleManager.BleManagerGattCallback requestHandler;
    /** @deprecated */
    @Deprecated
    protected BleManagerCallbacks callbacks;
    @Nullable
    BondingObserver bondingObserver;
    @Nullable
    ConnectionObserver connectionObserver;
    private final BroadcastReceiver mPairingRequestBroadcastReceiver;

    public BleManager(@NonNull Context context) {
        this(context, new Handler(Looper.getMainLooper()));
    }

    public BleManager(@NonNull Context context, @NonNull Handler handler) {

```

```

        this.mPairingRequestBroadcastReceiver = new BroadcastReceiver() {
            public void onReceive(Context context, Intent intent) {
                BluetoothDevice device =
                (BluetoothDevice)intent.getParcelableExtra("android.bluetooth.device.extra.DEVICE");
                BluetoothDevice bluetoothDevice =
                BleManager.this.requestHandler.getBluetoothDevice();
                if (bluetoothDevice != null && device != null &&
                device.getAddress().equals(bluetoothDevice.getAddress())) {
                    int variant =
                    intent.getIntExtra("android.bluetooth.device.extra.PAIRING_VARIANT", 0);
                    int key =
                    intent.getIntExtra("android.bluetooth.device.extra.PAIRING_KEY", -1);
                    BleManager.this.log(3, "[Broadcast] Action received:
                    android.bluetooth.device.action.PAIRING_REQUEST, pairing variant: " +
                    ParserUtils.pairingVariantToString(variant) + " (" + variant + "); key: " +
                    key);
                    BleManager.this.onPairingRequestReceived(device, variant, key);
                }
            }
        };
        this.context = context;
        this.requestHandler = this.getGattCallback();
        this.requestHandler.init(this, handler);
        context.registerReceiver(this.mPairingRequestBroadcastReceiver, new
        IntentFilter("android.bluetooth.device.action.PAIRING_REQUEST"));
    }

    protected void initialize() {
        this.requestHandler.initialize();
    }

    protected boolean isRequiredServiceSupported(@NonNull BluetoothGatt gatt) {
        return this.requestHandler.isRequiredServiceSupported(gatt);
    }

    protected boolean isOptionalServiceSupported(@NonNull BluetoothGatt gatt) {
        return this.requestHandler.isOptionalServiceSupported(gatt);
    }

    protected void onServerReady(@NonNull BluetoothGattServer server) {
        this.requestHandler.onServerReady(server);
    }

    protected void onServicesInvalidated() {
        this.requestHandler.onServicesInvalidated();
    }

    protected void onDeviceReady() {
        this.requestHandler.onDeviceReady();
    }

    protected void onManagerReady() {
        this.requestHandler.onManagerReady();
    }

```



```

}

public void close() {
    try {
        this.context.unregisterReceiver(this.mPairingRequestBroadcastReceiver);
    } catch (Exception var2) {
    }

    if (this.serverManager != null) {
        this.serverManager.removeManager(this);
    }

    this.requestHandler.close();
}

protected void runOnCallbackThread(@NonNull Runnable runnable) {
    this.requestHandler.post(runnable);
}

/** @deprecated */
@Deprecated
public void setGattCallbacks(@NonNull BleManagerCallbacks callbacks) {
    this.callbacks = callbacks;
}

public final void setConnectionObserver(@Nullable ConnectionObserver
callback) {
    this.connectionObserver = callback;
}

@RestrictTo({Scope.LIBRARY_GROUP})
@Nullable
public final ConnectionObserver getConnectionObserver() {
    return this.connectionObserver;
}

public final void setBondingObserver(@Nullable BondingObserver callback) {
    this.bondingObserver = callback;
}

@RestrictTo({Scope.LIBRARY_GROUP})
@Nullable
public final BondingObserver getBondingObserver() {
    return this.bondingObserver;
}

public final void useServer(@NonNull BleServerManager server) {
    if (this.serverManager != null) {
        this.serverManager.removeManager(this);
    }

    this.serverManager = server;
    server.addManager(this);
    this.requestHandler.useServer(server);
}

```

```

    }

    final void closeServer() {
        this.serverManager = null;
        this.requestHandler.useServer((BleServerManager)null);
    }

    protected void onPairingRequestReceived(@NonNull BluetoothDevice device, int
variant, int key) {
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    protected BleManager.BleManagerGattCallback getGattCallback() {
        return new BleManager.BleManagerGattCallback() {
            protected boolean isRequiredServiceSupported(@NonNull BluetoothGatt
gatt) {
                return false;
            }

            protected void onServicesInvalidated() {
            }
        };
    }

    @NonNull
    protected final Context getContext() {
        return this.context;
    }

    @Nullable
    public BluetoothDevice getBluetoothDevice() {
        return this.requestHandler.getBluetoothDevice();
    }

    public final boolean isConnected() {
        return this.requestHandler.isConnected();
    }

    public final boolean isReady() {
        return this.requestHandler.isReady();
    }

    protected final boolean isBonded() {
        BluetoothDevice bluetoothDevice =
this.requestHandler.getBluetoothDevice();
        return bluetoothDevice != null && bluetoothDevice.getBondState() == 12;
    }

    public final int getConnectionState() {
        return this.requestHandler.getConnectionState();
    }

```

```

/** @deprecated */
@Deprecated
@IntRange(
    from = -1L,
    to = 100L
)
public final int getBatteryValue() {
    return this.requestHandler.getBatteryValue();
}

public int getMinLogPriority() {
    return 4;
}

public void log(int priority, @NonNull String message) {
}

public void log(int priority, @StringRes int messageRes, @Nullable Object...
params) {
    String message = this.context.getString(messageRes, params);
    this.log(priority, message);
}

/** @deprecated */
@Deprecated
protected boolean shouldAutoConnect() {
    return false;
}

protected boolean shouldClearCacheWhenDisconnected() {
    return false;
}

@IntRange(
    from = 0L
)
protected int getServiceDiscoveryDelay(boolean bonded) {
    return bonded ? 1600 : 300;
}

@NonNull
public final ConnectRequest connect(@NonNull BluetoothDevice device) {
    return
Request.connect(device).useAutoConnect(this.shouldAutoConnect()).setRequestHandl
er(this.requestHandler);
}

/** @deprecated */
@Deprecated
@NonNull
public final ConnectRequest connect(@NonNull BluetoothDevice device, int phy)
{
    return
Request.connect(device).usePreferredPhy(phy).useAutoConnect(this.shouldAutoConne

```

```

ct()).setRequestHandler(this.requestHandler);
}

@NonNull
public final DisconnectRequest disconnect() {
    return Request.disconnect().setRequestHandler(this.requestHandler);
}

public void attachClientConnection(BluetoothDevice client) {
    this.requestHandler.attachClientConnection(client);
}

/** @deprecated */
@Deprecated
@NonNull
@RequiresPermission("android.permission.BLUETOOTH_ADMIN")
protected Request createBond() {
    return this.createBondInsecure();
}

@NonNull
@RequiresPermission("android.permission.BLUETOOTH_ADMIN")
protected Request createBondInsecure() {
    return Request.createBond().setRequestHandler(this.requestHandler);
}

@RequiresPermission("android.permission.BLUETOOTH_ADMIN")
protected Request ensureBond() {
    return Request.ensureBond().setRequestHandler(this.requestHandler);
}

@NonNull
@RequiresPermission("android.permission.BLUETOOTH_ADMIN")
protected Request removeBond() {
    return Request.removeBond().setRequestHandler(this.requestHandler);
}

@NonNull
protected ValueChangedCallback setNotificationCallback(@Nullable
BluetoothGattCharacteristic characteristic) {
    return this.requestHandler.getValueChangedCallback(characteristic);
}

@NonNull
protected ValueChangedCallback setIndicationCallback(@Nullable
BluetoothGattCharacteristic characteristic) {
    return this.setNotificationCallback(characteristic);
}

@NonNull
protected ValueChangedCallback setWriteCallback(@Nullable
BluetoothGattCharacteristic serverCharacteristic) {
    return this.requestHandler.getValueChangedCallback(serverCharacteristic);
}

```

```

    @NonNull
    protected ValueChangedCallback setWriteCallback(@Nullable
BluetoothGattDescriptor serverDescriptor) {
        return this.requestHandler.getValueChangedCallback(serverDescriptor);
    }

    protected void removeNotificationCallback(@Nullable
BluetoothGattCharacteristic characteristic) {
        this.requestHandler.removeValueChangedCallback(characteristic);
    }

    protected void removeIndicationCallback(@Nullable BluetoothGattCharacteristic
characteristic) {
        this.removeNotificationCallback(characteristic);
    }

    protected void removeWriteCallback(@Nullable BluetoothGattCharacteristic
serverCharacteristic) {
        this.requestHandler.removeValueChangedCallback(serverCharacteristic);
    }

    protected void removeWriteCallback(@Nullable BluetoothGattDescriptor
serverDescriptor) {
        this.requestHandler.removeValueChangedCallback(serverDescriptor);
    }

    @NonNull
    protected WaitForValueChangedRequest waitForNotification(@Nullable
BluetoothGattCharacteristic characteristic) {
        return
Request.newWaitForNotificationRequest(characteristic).setRequestHandler(this.req
uestHandler);
    }

    @NonNull
    protected WaitForValueChangedRequest waitForIndication(@Nullable
BluetoothGattCharacteristic characteristic) {
        return
Request.newWaitForIndicationRequest(characteristic).setRequestHandler(this.reque
stHandler);
    }

    @NonNull
    protected WaitForValueChangedRequest waitForWrite(@Nullable
BluetoothGattCharacteristic serverCharacteristic) {
        return
Request.newWaitForWriteRequest(serverCharacteristic).setRequestHandler(this.requ
estHandler);
    }

    @NonNull
    protected WaitForValueChangedRequest waitForWrite(@Nullable
BluetoothGattDescriptor serverDescriptor) {

```

```

        return
Request.newWaitForWriteRequest(serverDescriptor).setRequestHandler(this.requestH
andler);
    }

    @NonNull
    protected ConditionalWaitRequest<Void> waitIf(@NonNull
ConditionalWaitRequest.Condition<Void> condition) {
        return Request.newConditionalWaitRequest(condition,
(Object)null).setRequestHandler(this.requestHandler);
    }

    @NonNull
    protected <T> ConditionalWaitRequest<T> waitIf(@Nullable T parameter,
@NonNull ConditionalWaitRequest.Condition<T> condition) {
        return Request.newConditionalWaitRequest(condition,
parameter).setRequestHandler(this.requestHandler);
    }

    @NonNull
    protected ConditionalWaitRequest<Void> waitUntil(@NonNull
ConditionalWaitRequest.Condition<Void> condition) {
        return this.waitIf(condition).negate();
    }

    @NonNull
    protected <T> ConditionalWaitRequest<T> waitUntil(@Nullable T parameter,
@NonNull ConditionalWaitRequest.Condition<T> condition) {
        return this.waitIf(parameter, condition).negate();
    }

    @NonNull
    protected ConditionalWaitRequest<BluetoothGattCharacteristic>
waitUntilNotificationsEnabled(@Nullable BluetoothGattCharacteristic
serverCharacteristic) {
        return this.waitUntil(serverCharacteristic, (characteristic) -> {
            if (characteristic == null) {
                return false;
            } else {
                BluetoothGattDescriptor cccd =
characteristic.getDescriptor(CLIENT_CHARACTERISTIC_CONFIG_DESCRIPTOR_UUID);
                if (cccd == null) {
                    return false;
                } else {
                    byte[] value = this.requestHandler.getDescriptorValue(cccd);
                    return value != null && value.length == 2 && (value[0] & 1) == 1;
                }
            }
        });
    }

    @NonNull
    protected ConditionalWaitRequest<BluetoothGattCharacteristic>
waitUntilIndicationsEnabled(@Nullable BluetoothGattCharacteristic

```

```

serverCharacteristic) {
    return this.waitForReadRequest(serverCharacteristic, (characteristic) -> {
        if (characteristic == null) {
            return false;
        } else {
            BluetoothGattDescriptor cccd =
characteristic.getDescriptor(CLIENT_CHARACTERISTIC_CONFIG_DESCRIPTOR_UUID);
            if (cccd == null) {
                return false;
            } else {
                byte[] value = this.requestHandler.getDescriptorValue(cccd);
                return value != null && value.length == 2 && (value[0] & 2) == 2;
            }
        }
    });
}

@NonNull
protected WaitForReadRequest waitForRead(@Nullable
BluetoothGattCharacteristic serverCharacteristic) {
    return
Request.newWaitForReadRequest(serverCharacteristic).setRequestHandler(this.reque
stHandler);
}

@NonNull
protected WaitForReadRequest waitForRead(@Nullable
BluetoothGattCharacteristic serverCharacteristic, @Nullable byte[] data) {
    return Request.newWaitForReadRequest(serverCharacteristic,
data).setRequestHandler(this.requestHandler);
}

@NonNull
protected WaitForReadRequest waitForRead(@Nullable
BluetoothGattCharacteristic serverCharacteristic, @Nullable byte[] data, int
offset, int length) {
    return Request.newWaitForReadRequest(serverCharacteristic, data, offset,
length).setRequestHandler(this.requestHandler);
}

@NonNull
protected WaitForReadRequest waitForRead(@Nullable BluetoothGattDescriptor
serverDescriptor) {
    return
Request.newWaitForReadRequest(serverDescriptor).setRequestHandler(this.requestHa
ndler);
}

@NonNull
protected WaitForReadRequest waitForRead(@Nullable BluetoothGattDescriptor
serverDescriptor, @Nullable byte[] data) {
    return Request.newWaitForReadRequest(serverDescriptor,
data).setRequestHandler(this.requestHandler);
}

```

```

    @NonNull
    protected WaitForReadRequest waitForRead(@Nullable BluetoothGattDescriptor
serverDescriptor, @Nullable byte[] data, int offset, int length) {
        return Request.newWaitForReadRequest(serverDescriptor, data, offset,
length).setRequestHandler(this.requestHandler);
    }

    protected void setCharacteristicValue(@Nullable BluetoothGattCharacteristic
serverCharacteristic, @Nullable DataProvider provider) {
        this.requestHandler.setCharacteristicValue(serverCharacteristic,
provider);
    }

    @NonNull
    protected SetValueRequest setCharacteristicValue(@Nullable
BluetoothGattCharacteristic serverCharacteristic, @Nullable Data data) {
        return Request.newSetValueRequest(serverCharacteristic, data != null ?
data.getValue() : null).setRequestHandler(this.requestHandler);
    }

    @NonNull
    protected SetValueRequest setCharacteristicValue(@Nullable
BluetoothGattCharacteristic serverCharacteristic, @Nullable byte[] data) {
        return Request.newSetValueRequest(serverCharacteristic,
data).setRequestHandler(this.requestHandler);
    }

    @NonNull
    protected SetValueRequest setCharacteristicValue(@Nullable
BluetoothGattCharacteristic serverCharacteristic, @Nullable byte[] data, int
offset, int length) {
        return Request.newSetValueRequest(serverCharacteristic, data, offset,
length).setRequestHandler(this.requestHandler);
    }

    protected void setDescriptorValue(@Nullable BluetoothGattDescriptor
serverDescriptor, @Nullable DataProvider provider) {
        this.requestHandler.setDescriptorValue(serverDescriptor, provider);
    }

    @NonNull
    protected SetValueRequest setDescriptorValue(@Nullable
BluetoothGattDescriptor serverDescriptor, @Nullable Data data) {
        return Request.newSetValueRequest(serverDescriptor, data != null ?
data.getValue() : null).setRequestHandler(this.requestHandler);
    }

    @NonNull
    protected SetValueRequest setDescriptorValue(@Nullable
BluetoothGattDescriptor serverDescriptor, @Nullable byte[] data) {
        return Request.newSetValueRequest(serverDescriptor,
data).setRequestHandler(this.requestHandler);
    }

```



```

    @NonNull
    protected SetValueRequest setDescriptorValue(@Nullable
BluetoothGattDescriptor serverDescriptor, @Nullable byte[] data, int offset, int
length) {
        return Request.newSetValueRequest(serverDescriptor, data, offset,
length).setRequestHandler(this.requestHandler);
    }

```

```

    @NonNull
    protected WriteRequest enableNotifications(@Nullable
BluetoothGattCharacteristic characteristic) {
        return
Request.newEnableNotificationsRequest(characteristic).setRequestHandler(this.req
uestHandler);
    }

```

```

    @NonNull
    protected WriteRequest disableNotifications(@Nullable
BluetoothGattCharacteristic characteristic) {
        return
Request.newDisableNotificationsRequest(characteristic).setRequestHandler(this.re
questHandler);
    }

```

```

    @NonNull
    protected WriteRequest enableIndications(@Nullable
BluetoothGattCharacteristic characteristic) {
        return
Request.newEnableIndicationsRequest(characteristic).setRequestHandler(this.reque
stHandler);
    }

```

```

    @NonNull
    protected WriteRequest disableIndications(@Nullable
BluetoothGattCharacteristic characteristic) {
        return
Request.newDisableIndicationsRequest(characteristic).setRequestHandler(this.requ
estHandler);
    }

```

```

    @NonNull
    protected ReadRequest readCharacteristic(@Nullable
BluetoothGattCharacteristic characteristic) {
        return
Request.newReadRequest(characteristic).setRequestHandler(this.requestHandler);
    }

```

```

    @NonNull
    protected WriteRequest writeCharacteristic(@Nullable
BluetoothGattCharacteristic characteristic, @Nullable Data data, int writeType)
{
        return Request.newWriteRequest(characteristic, data != null ?
data.getValue() : null, writeType).setRequestHandler(this.requestHandler);
    }

```

```

    }

    @NonNull
    protected WriteRequest writeCharacteristic(@Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] data, int
writeType) {
        return Request.newWriteRequest(characteristic, data,
writeType).setRequestHandler(this.requestHandler);
    }

    @NonNull
    protected WriteRequest writeCharacteristic(@Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] data, int offset,
int length, int writeType) {
        return Request.newWriteRequest(characteristic, data, offset, length,
writeType).setRequestHandler(this.requestHandler);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    protected WriteRequest writeCharacteristic(@Nullable
BluetoothGattCharacteristic characteristic, @Nullable Data data) {
        return Request.newWriteRequest(characteristic, data != null ?
data.getValue() : null).setRequestHandler(this.requestHandler);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    protected WriteRequest writeCharacteristic(@Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] data) {
        return Request.newWriteRequest(characteristic,
data).setRequestHandler(this.requestHandler);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    protected WriteRequest writeCharacteristic(@Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] data, int offset,
int length) {
        return Request.newWriteRequest(characteristic, data, offset,
length).setRequestHandler(this.requestHandler);
    }

    @NonNull
    protected ReadRequest readDescriptor(@Nullable BluetoothGattDescriptor
descriptor) {
        return
Request.newReadRequest(descriptor).setRequestHandler(this.requestHandler);
    }

    @NonNull

```

```

    protected WriteRequest writeDescriptor(@Nullable BluetoothGattDescriptor
descriptor, @Nullable Data data) {
        return Request.newWriteRequest(descriptor, data != null ? data.getValue()
: null).setRequestHandler(this.requestHandler);
    }

    @NonNull
    protected WriteRequest writeDescriptor(@Nullable BluetoothGattDescriptor
descriptor, @Nullable byte[] data) {
        return Request.newWriteRequest(descriptor,
data).setRequestHandler(this.requestHandler);
    }

    @NonNull
    protected WriteRequest writeDescriptor(@Nullable BluetoothGattDescriptor
descriptor, @Nullable byte[] data, int offset, int length) {
        return Request.newWriteRequest(descriptor, data, offset,
length).setRequestHandler(this.requestHandler);
    }

    @NonNull
    protected WriteRequest sendNotification(@Nullable BluetoothGattCharacteristic
serverCharacteristic, @Nullable Data data) {
        return Request.newNotificationRequest(serverCharacteristic, data != null ?
data.getValue() : null).setRequestHandler(this.requestHandler);
    }

    @NonNull
    protected WriteRequest sendNotification(@Nullable BluetoothGattCharacteristic
serverCharacteristic, @Nullable byte[] data) {
        return Request.newNotificationRequest(serverCharacteristic,
data).setRequestHandler(this.requestHandler);
    }

    @NonNull
    protected WriteRequest sendNotification(@Nullable BluetoothGattCharacteristic
serverCharacteristic, @Nullable byte[] data, int offset, int length) {
        return Request.newNotificationRequest(serverCharacteristic, data, offset,
length).setRequestHandler(this.requestHandler);
    }

    @NonNull
    protected WriteRequest sendIndication(@Nullable BluetoothGattCharacteristic
serverCharacteristic, @Nullable Data data) {
        return Request.newIndicationRequest(serverCharacteristic, data != null ?
data.getValue() : null).setRequestHandler(this.requestHandler);
    }

    @NonNull
    protected WriteRequest sendIndication(@Nullable BluetoothGattCharacteristic
serverCharacteristic, @Nullable byte[] data) {
        return Request.newIndicationRequest(serverCharacteristic,
data).setRequestHandler(this.requestHandler);
    }

```

```

    @NonNull
    protected WriteRequest sendIndication(@Nullable BluetoothGattCharacteristic
serverCharacteristic, @Nullable byte[] data, int offset, int length) {
        return Request.newIndicationRequest(serverCharacteristic, data, offset,
length).setRequestHandler(this.requestHandler);
    }

    @NonNull
    protected RequestQueue beginAtomicRequestQueue() {
        return (new RequestQueue()).setRequestHandler(this.requestHandler);
    }

    @NonNull
    protected ReliableWriteRequest beginReliableWrite() {
        return
Request.newReliableWriteRequest().setRequestHandler(this.requestHandler);
    }

    protected final boolean isReliableWriteInProgress() {
        return this.requestHandler.isReliableWriteInProgress();
    }

    /** @deprecated */
    @Deprecated
    protected void readBatteryLevel() {

Request.newReadBatteryLevelRequest().setRequestHandler(this.requestHandler).with
(this.requestHandler.getBatteryLevelCallback()).enqueue();
    }

    /** @deprecated */
    @Deprecated
    protected void enableBatteryLevelNotifications() {

Request.newEnableBatteryLevelNotificationsRequest().setRequestHandler(this.reque
stHandler).before((device) -> {
        this.requestHandler.setBatteryLevelNotificationCallback();
    }).done((device) -> {
        this.log(4, "Battery Level notifications enabled");
    }).enqueue();
    }

    /** @deprecated */
    @Deprecated
    protected void disableBatteryLevelNotifications() {

Request.newDisableBatteryLevelNotificationsRequest().setRequestHandler(this.requ
estHandler).done((device) -> {
        this.log(4, "Battery Level notifications disabled");
    }).enqueue();
    }

    protected MtuRequest requestMtu(@IntRange(from = 23L,to = 517L) int mtu) {

```

```

        return Request.newMtuRequest(mtu).setRequestHandler(this.requestHandler);
    }

    @IntRange(
        from = 23L,
        to = 517L
    )
    protected int getMtu() {
        return this.requestHandler.getMtu();
    }

    protected void overrideMtu(@IntRange(from = 23L,to = 517L) int mtu) {
        this.requestHandler.overrideMtu(mtu);
    }

    @RequiresApi(
        api = 21
    )
    protected ConnectionPriorityRequest requestConnectionPriority(int priority) {
        return
Request.newConnectionPriorityRequest(priority).setRequestHandler(this.requestHan
dler);
    }

    @RequiresApi(
        api = 26
    )
    protected void setConnectionParametersListener(@Nullable
ConnectionParametersUpdatedCallback callback) {
        this.requestHandler.setConnectionParametersListener(callback);
    }

    protected PhyRequest setPreferredPhy(int txPhy, int rxPhy, int phyOptions) {
        return Request.newSetPreferredPhyRequest(txPhy, rxPhy,
phyOptions).setRequestHandler(this.requestHandler);
    }

    protected PhyRequest readPhy() {
        return Request.newReadPhyRequest().setRequestHandler(this.requestHandler);
    }

    protected ReadRssiRequest readRssi() {
        return
Request.newReadRssiRequest().setRequestHandler(this.requestHandler);
    }

    protected Request refreshDeviceCache() {
        return
Request.newRefreshCacheRequest().setRequestHandler(this.requestHandler);
    }

    protected SleepRequest sleep(@IntRange(from = 0L) long delay) {
        return
Request.newSleepRequest(delay).setRequestHandler(this.requestHandler);
    }

```

```

    }

    /** @deprecated */
    @Deprecated
    protected final void enqueue(@NonNull Request request) {
        this.requestHandler.enqueue(request);
    }

    protected final void cancelQueue() {
        this.requestHandler.cancelQueue();
    }

    protected abstract static class BleManagerGattCallback extends
BleManagerHandler {
    }
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: BleManagerCallbacks.java

package no.nordicsemi.android.ble;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;

/** @deprecated */
@Deprecated
public interface BleManagerCallbacks {
    /** @deprecated */
    @Deprecated
    void onDeviceConnecting(@NonNull BluetoothDevice var1);

    /** @deprecated */
    @Deprecated
    void onDeviceConnected(@NonNull BluetoothDevice var1);

    /** @deprecated */
    @Deprecated
    void onDeviceDisconnecting(@NonNull BluetoothDevice var1);

    /** @deprecated */
    @Deprecated
    void onDeviceDisconnected(@NonNull BluetoothDevice var1);

    /** @deprecated */
    @Deprecated
    void onLinkLossOccurred(@NonNull BluetoothDevice var1);

    /** @deprecated */
    @Deprecated
    void onServicesDiscovered(@NonNull BluetoothDevice var1, boolean var2);
}

```

```

    /** @deprecated */
    @Deprecated
    void onDeviceReady(@NonNull BluetoothDevice var1);

    /** @deprecated */
    @Deprecated
    default boolean shouldEnableBatteryLevelNotifications(@NonNull
BluetoothDevice device) {
        return false;
    }

    /** @deprecated */
    @Deprecated
    default void onBatteryValueReceived(@NonNull BluetoothDevice device,
@IntRange(from = 0L,to = 100L) int value) {
    }

    /** @deprecated */
    @Deprecated
    void onBondingRequired(@NonNull BluetoothDevice var1);

    /** @deprecated */
    @Deprecated
    void onBonded(@NonNull BluetoothDevice var1);

    /** @deprecated */
    @Deprecated
    void onBondingFailed(@NonNull BluetoothDevice var1);

    /** @deprecated */
    @Deprecated
    void onError(@NonNull BluetoothDevice var1, @NonNull String var2, int var3);

    /** @deprecated */
    @Deprecated
    void onDeviceNotSupported(@NonNull BluetoothDevice var1);
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: BleManagerHandler.java

```

```

package no.nordicsemi.android.ble;

import android.annotation.SuppressLint;
import android.annotation.TargetApi;
import android.bluetooth.BluetoothAdapter;
import android.bluetooth.BluetoothDevice;
import android.bluetooth.BluetoothGatt;
import android.bluetooth.BluetoothGattCallback;
import android.bluetooth.BluetoothGattCharacteristic;
import android.bluetooth.BluetoothGattDescriptor;

```

```

import android.bluetooth.BluetoothGattServer;
import android.bluetooth.BluetoothGattService;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.content.IntentFilter;
import android.os.Handler;
import android.os.SystemClock;
import android.os.Build.VERSION;
import android.util.Log;
import android.util.Pair;
import androidx.annotation.IntRange;
import androidx.annotation.Keep;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.annotation.RequiresApi;
import androidx.annotation.RequiresPermission;
import java.lang.reflect.Method;
import java.security.InvalidParameterException;
import java.util.Deque;
import java.util.HashMap;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.Map;
import java.util.concurrent.LinkedBlockingDeque;
import no.nordicsemi.android.ble.callback.ConnectionParametersUpdatedCallback;
import no.nordicsemi.android.ble.callback.DataReceivedCallback;
import no.nordicsemi.android.ble.data.DataProvider;
import no.nordicsemi.android.ble.error.GattError;
import no.nordicsemi.android.ble.observer.BondingObserver;
import no.nordicsemi.android.ble.observer.ConnectionObserver;
import no.nordicsemi.android.ble.utils.ParserUtils;

```

```

@SuppressLint("MissingPermission")

```

```

abstract class BleManagerHandler extends RequestHandler {
    private static final String TAG = "BleManager";
    private static final String ERROR_CONNECTION_STATE_CHANGE = "Error on
connection state change";
    private static final String ERROR_DISCOVERY_SERVICE = "Error on discovering
services";
    private static final String ERROR_AUTH_ERROR_WHILE_BONDED = "Phone has lost
bonding information";
    private static final String ERROR_READ_CHARACTERISTIC = "Error on reading
characteristic";
    private static final String ERROR_WRITE_CHARACTERISTIC = "Error on writing
characteristic";
    private static final String ERROR_READ_DESCRIPTOR = "Error on reading
descriptor";
    private static final String ERROR_WRITE_DESCRIPTOR = "Error on writing
descriptor";
    private static final String ERROR_MTU_REQUEST = "Error on mtu request";
    private static final String ERROR_CONNECTION_PRIORITY_REQUEST = "Error on
connection priority request";
    private static final String ERROR_READ_RSSI = "Error on RSSI read";

```



```

    private static final String ERROR_READ_PHY = "Error on PHY read";
    private static final String ERROR_PHY_UPDATE = "Error on PHY update";
    private static final String ERROR_RELIABLE_WRITE = "Error on Execute Reliable
Write";
    private static final String ERROR_NOTIFY = "Error on sending
notification/indication";
    private final Object LOCK = new Object();
    private BluetoothDevice bluetoothDevice;
    private BluetoothGatt bluetoothGatt;
    private BleManager manager;
    private BleServerManager serverManager;
    private Handler handler;
    private final Deque<Request> taskQueue = new LinkedBlockingDeque();
    private Deque<Request> initQueue;
    private boolean initialization;
    private static final long CONNECTION_TIMEOUT_THRESHOLD = 20000L;
    private boolean servicesDiscovered;
    private boolean deviceNotSupported;
    private boolean serviceDiscoveryRequested;
    private long connectionTime;
    private int connectionCount = 0;
    private boolean connected;
    private boolean ready;
    private boolean operationInProgress;
    private boolean userDisconnected;
    private boolean initialConnection;
    private int connectionState = 0;
    private boolean connectionPriorityOperationInProgress = false;
    private boolean reliableWriteInProgress;
    private int mtu = 23;
    private int interval;
    private int latency;
    private int timeout;
    /** @deprecated */
    @Deprecated
    @IntRange(
        from = -1L,
        to = 100L
    )
    private int batteryValue = -1;
    private Map<BluetoothGattCharacteristic, byte[]> characteristicValues;
    private Map<BluetoothGattDescriptor, byte[]> descriptorValues;
    private Deque<Pair<Object, byte[]>> preparedValues;
    private int prepareError;
    private ConnectRequest connectRequest;
    private Request request;
    private RequestQueue requestQueue;
    @NonNull
    private final HashMap<Object, ValueChangedCallback> valueChangedCallbacks =
new HashMap();
    @NonNull
    private final HashMap<Object, DataProvider> dataProviders = new HashMap();
    @Nullable
    private ConnectionParametersUpdatedCallback

```

```

connectionParametersUpdatedCallback;
/** @deprecated */
@Deprecated
@Nullable
private ValueChangedCallback batteryLevelNotificationCallback;
@Nullable
private AwaitingRequest<?> awaitingRequest;
private final BroadcastReceiver bluetoothStateBroadcastReceiver = new
BroadcastReceiver() {
    public void onReceive(Context context, Intent intent) {
        int state = intent.getIntExtra("android.bluetooth.adapter.extra.STATE",
10);
        int previousState =
intent.getIntExtra("android.bluetooth.adapter.extra.PREVIOUS_STATE", 10);
        BleManagerHandler.this.log(3, () -> {
            return "[Broadcast] Action received:
android.bluetooth.adapter.action.STATE_CHANGED, state changed to " +
this.state2String(state);
        });
        switch(state) {
            case 10:
            case 13:
                if (previousState != 13 && previousState != 10) {
                    BleManagerHandler.this.operationInProgress = true;
                    BleManagerHandler.this.taskQueue.clear();
                    BleManagerHandler.this.initQueue = null;
                    boolean wasConnected = BleManagerHandler.this.connected;
                    BleManagerHandler.this.connected = false;
                    BleManagerHandler.this.ready = false;
                    BleManagerHandler.this.connectionState = 0;
                    BluetoothDevice device = BleManagerHandler.this.bluetoothDevice;
                    if (device != null) {
                        if (BleManagerHandler.this.request != null &&
BleManagerHandler.this.request.type != Request.Type.DISCONNECT) {
                            BleManagerHandler.this.request.notifyFail(device, -100);
                            BleManagerHandler.this.request = null;
                        }

                        if (BleManagerHandler.this.awaitingRequest != null) {
                            BleManagerHandler.this.awaitingRequest.notifyFail(device,
-100);
                            BleManagerHandler.this.awaitingRequest = null;
                        }

                        if (BleManagerHandler.this.connectRequest != null) {
                            BleManagerHandler.this.connectRequest.notifyFail(device,
-100);
                            BleManagerHandler.this.connectRequest = null;
                        }
                    }

                    BleManagerHandler.this.userDisconnected = true;
                    BleManagerHandler.this.operationInProgress = false;
                    if (device != null) {

```

```

        BleManagerHandler.this.connected = wasConnected;
        BleManagerHandler.this.notifyDeviceDisconnected(device, 1);
    }
} else {
    BleManagerHandler.this.close();
}
default:
}
}

private String state2String(int state) {
    switch(state) {
        case 10:
            return "OFF";
        case 11:
            return "TURNING ON";
        case 12:
            return "ON";
        case 13:
            return "TURNING OFF";
        default:
            return "UNKNOWN (" + state + ")";
    }
}

};
private final BroadcastReceiver mBondingBroadcastReceiver = new
BroadcastReceiver() {
    public void onReceive(Context context, Intent intent) {
        BluetoothDevice device =
        (BluetoothDevice)intent.getParcelableExtra("android.bluetooth.device.extra.DEVIC
E");

        int bondState =
intent.getIntExtra("android.bluetooth.device.extra.BOND_STATE", -1);
        int previousBondState =
intent.getIntExtra("android.bluetooth.device.extra.PREVIOUS_BOND_STATE", -1);
        if (BleManagerHandler.this.bluetoothDevice != null && device != null &&
device.getAddress().equals(BleManagerHandler.this.bluetoothDevice.getAddress()))
        {
            BleManagerHandler.this.log(3, () -> {
                return "[Broadcast] Action received:
android.bluetooth.device.action.BOND_STATE_CHANGED, bond state changed to: " +
ParserUtils.bondStateToString(bondState) + " (" + bondState + ")";
            });
            switch(bondState) {
                case 10:
                    if (previousBondState == 11) {
                        BleManagerHandler.this.postCallback((c) -> {
                            c.onBondingFailed(device);
                        });
                        BleManagerHandler.this.postBondingStateChange((o) -> {
                            o.onBondingFailed(device);
                        });
                        BleManagerHandler.this.log(5, () -> {
                            return "Bonding failed";
                        });
                    }
                }
            }
        }
    }
}

```

```

    });
    if (BleManagerHandler.this.request != null &&
BleManagerHandler.this.request.type == Request.Type.CREATE_BOND) {
        BleManagerHandler.this.request.notifyFail(device, -4);
        BleManagerHandler.this.request = null;
    }

    if (!BleManagerHandler.this.servicesDiscovered &&
!BleManagerHandler.this.serviceDiscoveryRequested) {
        BleManagerHandler.this.post(() -> {
            BluetoothGatt bluetoothGatt =
BleManagerHandler.this.bluetoothGatt;
            if (!BleManagerHandler.this.servicesDiscovered &&
!BleManagerHandler.this.serviceDiscoveryRequested && bluetoothGatt != null) {
                BleManagerHandler.this.serviceDiscoveryRequested =
true;

                BleManagerHandler.this.log(2, () -> {
                    return "Discovering services...";
                });
                BleManagerHandler.this.log(3, () -> {
                    return "gatt.discoverServices()";
                });
                bluetoothGatt.discoverServices();
            }

        });
        return;
    }
} else if (previousBondState == 12) {
    BleManagerHandler.this.userDisconnected = true;
    if (BleManagerHandler.this.request != null &&
BleManagerHandler.this.request.type == Request.Type.REMOVE_BOND) {
        BleManagerHandler.this.log(4, () -> {
            return "Bond information removed";
        });
        BleManagerHandler.this.request.notifySuccess(device);
        BleManagerHandler.this.request = null;
    }

    if (!BleManagerHandler.this.isConnected()) {
        BleManagerHandler.this.close();
    }
}
break;
case 11:
    BleManagerHandler.this.postCallback((c) -> {
        c.onBondingRequired(device);
    });
    BleManagerHandler.this.postBondingStateChange((o) -> {
        o.onBondingRequired(device);
    });
    return;
case 12:
    BleManagerHandler.this.log(4, () -> {

```

```

        return "Device bonded";
    });
    BleManagerHandler.this.postCallback((c) -> {
        c.onBonded(device);
    });
    BleManagerHandler.this.postBondingStateChange((o) -> {
        o.onBonded(device);
    });
    if (BleManagerHandler.this.request != null &&
BleManagerHandler.this.request.type == Request.Type.CREATE_BOND) {
        BleManagerHandler.this.request.notifySuccess(device);
        BleManagerHandler.this.request = null;
    } else {
        if (!BleManagerHandler.this.servicesDiscovered &&
!BleManagerHandler.this.serviceDiscoveryRequested) {
            BleManagerHandler.this.post(() -> {
                BluetoothGatt bluetoothGatt =
BleManagerHandler.this.bluetoothGatt;
                if (!BleManagerHandler.this.servicesDiscovered &&
!BleManagerHandler.this.serviceDiscoveryRequested && bluetoothGatt != null) {
                    BleManagerHandler.this.serviceDiscoveryRequested =
true;

                    BleManagerHandler.this.log(2, () -> {
                        return "Discovering services...";
                    });
                    BleManagerHandler.this.log(3, () -> {
                        return "gatt.discoverServices()";
                    });
                    bluetoothGatt.discoverServices();
                }

            });
            return;
        }

        if (VERSION.SDK_INT >= 26 || BleManagerHandler.this.request ==
null) {
            return;
        }

        BleManagerHandler.this.enqueueFirst(BleManagerHandler.this.request);
    }

    BleManagerHandler.this.nextRequest(true);
}
};
private final BluetoothGattCallback gattCallback = new
BluetoothGattCallback() {
    public void onConnectionStateChange(@NonNull BluetoothGatt gatt, int
status, int newState) {
        BleManagerHandler.this.log(3, () -> {

```

```

        return "[Callback] Connection state changed with status: " + status
+ " and new state: " + newState + " (" + ParserUtils.stateToString(newState) +
+ ")";
    });
    if (status == 0 && newState == 2) {
        if (BleManagerHandler.this.bluetoothDevice == null) {
            Log.e("BleManager", "Device received notification after
disconnection.");
            BleManagerHandler.this.log(3, () -> {
                return "gatt.close()";
            });

            try {
                gatt.close();
            } catch (Throwable var11) {
            }

            return;
        }

        BleManagerHandler.this.log(4, () -> {
            return "Connected to " + gatt.getDevice().getAddress();
        });
        BleManagerHandler.this.connected = true;
        BleManagerHandler.this.connectionTime = 0L;
        BleManagerHandler.this.connectionState = 2;
        BleManagerHandler.this.postCallback((c) -> {
            c.onDeviceConnected(gatt.getDevice());
        });
        BleManagerHandler.this.postConnectionStateChange((o) -> {
            o.onDeviceConnected(gatt.getDevice());
        });
        if (!BleManagerHandler.this.serviceDiscoveryRequested) {
            boolean bonded = gatt.getDevice().getBondState() == 12;
            int delayx =
BleManagerHandler.this.manager.getServiceDiscoveryDelay(bonded);
            if (delayx > 0) {
                BleManagerHandler.this.log(3, () -> {
                    return "wait(" + delayx + ")";
                });
            }

            int connectionCount = ++BleManagerHandler.this.connectionCount;
            BleManagerHandler.this.postDelayed(() -> {
                if (connectionCount == BleManagerHandler.this.connectionCount)
{
                    if (BleManagerHandler.this.connected &&
!BleManagerHandler.this.servicesDiscovered &&
!BleManagerHandler.this.serviceDiscoveryRequested &&
gatt.getDevice().getBondState() != 11) {
                        BleManagerHandler.this.serviceDiscoveryRequested = true;
                        BleManagerHandler.this.log(2, () -> {
                            return "Discovering services...";
                        });
                    }
                }
            });
        }
    }
}

```

```

        BleManagerHandler.this.log(3, () -> {
            return "gatt.discoverServices()";
        });
        gatt.discoverServices();
    }

    }, (long)delayx);
}
} else {
    if (newState == 0) {
        long now = SystemClock.elapsedRealtime();
        boolean canTimeout = BleManagerHandler.this.connectionTime > 0L;
        boolean timeout = canTimeout && now >
BleManagerHandler.this.connectionTime + 20000L;
        if (status != 0) {
            BleManagerHandler.this.log(5, () -> {
                return "Error: (0x" + Integer.toHexString(status) + "): " +
GattError.parseConnectionError(status);
            });
        }

        if (status != 0 && canTimeout && !timeout &&
BleManagerHandler.this.connectRequest != null &&
BleManagerHandler.this.connectRequest.canRetry()) {
            int delay =
BleManagerHandler.this.connectRequest.getRetryDelay();
            if (delay > 0) {
                BleManagerHandler.this.log(3, () -> {
                    return "wait(" + delay + ")";
                });
            }

            BleManagerHandler.this.postDelayed(() -> {
                BleManagerHandler.this.internalConnect(gatt.getDevice(),
BleManagerHandler.this.connectRequest);
            }, (long)delay);
            return;
        }

        if (BleManagerHandler.this.connectRequest != null &&
BleManagerHandler.this.connectRequest.shouldAutoConnect() &&
BleManagerHandler.this.initialConnection && gatt.getDevice().getBondState() ==
12) {
            BleManagerHandler.this.log(3, () -> {
                return "autoConnect = false called failed; retrying with
autoConnect = true";
            });
            BleManagerHandler.this.post(() -> {
                BleManagerHandler.this.internalConnect(gatt.getDevice(),
BleManagerHandler.this.connectRequest);
            });
            return;
        }
    }
}

```

```

        BleManagerHandler.this.operationInProgress = true;
        BleManagerHandler.this.taskQueue.clear();
        BleManagerHandler.this.initQueue = null;
        BleManagerHandler.this.ready = false;
        boolean wasConnected = BleManagerHandler.this.connected;
        boolean notSupported = BleManagerHandler.this.deviceNotSupported;
        BleManagerHandler.this.notifyDeviceDisconnected(gatt.getDevice(),
timeout ? 10 : (notSupported ? 4 :
BleManagerHandler.this.mapDisconnectStatusToReason(status)));
        if (BleManagerHandler.this.request != null &&
BleManagerHandler.this.request.type != Request.Type.DISCONNECT &&
BleManagerHandler.this.request.type != Request.Type.REMOVE_BOND) {
            BleManagerHandler.this.request.notifyFail(gatt.getDevice(),
status == 0 ? -1 : status);
            BleManagerHandler.this.request = null;
        }

        if (BleManagerHandler.this.awaitingRequest != null) {
BleManagerHandler.this.awaitingRequest.notifyFail(gatt.getDevice(), -1);
            BleManagerHandler.this.awaitingRequest = null;
        }

        if (BleManagerHandler.this.connectRequest != null) {
            int reason;
            if (notSupported) {
                reason = -2;
            } else if (status == 0) {
                reason = -1;
            } else if (status == 133 && timeout) {
                reason = -5;
            } else {
                reason = status;
            }

BleManagerHandler.this.connectRequest.notifyFail(gatt.getDevice(), reason);
            BleManagerHandler.this.connectRequest = null;
        }

        BleManagerHandler.this.operationInProgress = false;
        if (wasConnected && BleManagerHandler.this.initialConnection) {
            BleManagerHandler.this.internalConnect(gatt.getDevice(),
(ConnectRequest)null);
        } else {
            BleManagerHandler.this.initialConnection = false;
            BleManagerHandler.this.nextRequest(false);
        }

        if (wasConnected || status == 0) {
            return;
        }
    } else if (status != 0) {

```



```

        BleManagerHandler.this.log(6, () -> {
            return "Error (0x" + Integer.toHexString(status) + "): " +
GattError.parseConnectionError(status);
        });
    }

    BleManagerHandler.this.postCallback((c) -> {
        c.onError(gatt.getDevice(), "Error on connection state change",
status);
    });
}

}

public void onServicesDiscovered(@NonNull BluetoothGatt gatt, int status)
{
    if (BleManagerHandler.this.serviceDiscoveryRequested) {
        BleManagerHandler.this.serviceDiscoveryRequested = false;
        if (status == 0) {
            BleManagerHandler.this.log(4, () -> {
                return "Services discovered";
            });
            BleManagerHandler.this.servicesDiscovered = true;
            if
(BleManagerHandler.this.manager.isRequiredServiceSupported(gatt)) {
                BleManagerHandler.this.log(2, () -> {
                    return "Primary service found";
                });
                BleManagerHandler.this.deviceNotSupported = false;
                boolean optionalServicesFound =
BleManagerHandler.this.manager.isOptionalServiceSupported(gatt);
                if (optionalServicesFound) {
                    BleManagerHandler.this.log(2, () -> {
                        return "Secondary service found";
                    });
                }

                BleManagerHandler.this.postCallback((c) -> {
                    c.onServicesDiscovered(gatt.getDevice(),
optionalServicesFound);
                });
                BleManagerHandler.this.initializeServerAttributes();
                BleManagerHandler.this.operationInProgress = true;
                BleManagerHandler.this.initialization = true;
                BleManagerHandler.this.initQueue =
BleManagerHandler.this.initGatt(gatt);
                boolean deprecatedApiUsed = BleManagerHandler.this.initQueue
!= null;

                Request request;
                if (deprecatedApiUsed) {
                    for(Iterator var5 =
BleManagerHandler.this.initQueue.iterator(); var5.hasNext(); request.enqueue =
true) {
                        request = (Request)var5.next();
                    }
                }
            }
        }
    }
}

```

```

        request.setRequestHandler(BleManagerHandler.this);
    }
}

if (BleManagerHandler.this.initQueue == null) {
    BleManagerHandler.this.initQueue = new
LinkedBlockingDeque();
}

if (VERSION.SDK_INT < 23 || VERSION.SDK_INT == 26 ||
VERSION.SDK_INT == 27 || VERSION.SDK_INT == 28) {

BleManagerHandler.this.enqueueFirst(Request.newEnableServiceChangedIndicationsRe
quest().setRequestHandler(BleManagerHandler.this));
    BleManagerHandler.this.operationInProgress = true;
}

if (deprecatedApiUsed) {
    BleManagerHandler.this.manager.readBatteryLevel();
    if (BleManagerHandler.this.manager.callbacks != null &&
BleManagerHandler.this.manager.callbacks.shouldEnableBatteryLevelNotifications(g
att.getDevice())) {
BleManagerHandler.this.manager.enableBatteryLevelNotifications();
    }
}

BleManagerHandler.this.manager.initialize();
BleManagerHandler.this.initialization = false;
BleManagerHandler.this.nextRequest(true);
} else {
    BleManagerHandler.this.log(5, () -> {
        return "Device is not supported";
    });
    BleManagerHandler.this.deviceNotSupported = true;
    BleManagerHandler.this.postCallback((c) -> {
        c.onDeviceNotSupported(gatt.getDevice());
    });
    BleManagerHandler.this.internalDisconnect(4);
}
} else {
    Log.e("BleManager", "onServicesDiscovered error " + status);
    BleManagerHandler.this.onError(gatt.getDevice(), "Error on
discovering services", status);
    if (BleManagerHandler.this.connectRequest != null) {
BleManagerHandler.this.connectRequest.notifyFail(gatt.getDevice(), -4);
        BleManagerHandler.this.connectRequest = null;
    }

    BleManagerHandler.this.internalDisconnect(-1);
}
}
}

```

```
}
```

```
@Keep
```

```
public void onServiceChanged(@NonNull BluetoothGatt gatt) {  
    BleManagerHandler.this.log(4, () -> {  
        return "Service changed, invalidating services";  
    });  
    BleManagerHandler.this.operationInProgress = true;  
    BleManagerHandler.this.manager.onServicesInvalidated();  
    BleManagerHandler.this.onDeviceDisconnected();  
    BleManagerHandler.this.taskQueue.clear();  
    BleManagerHandler.this.initQueue = null;  
    BleManagerHandler.this.serviceDiscoveryRequested = true;  
    BleManagerHandler.this.servicesDiscovered = false;  
    BleManagerHandler.this.log(2, () -> {  
        return "Discovering Services...";  
    });  
    BleManagerHandler.this.log(3, () -> {  
        return "gatt.discoverServices()";  
    });  
    gatt.discoverServices();  
}
```

```
    public void onCharacteristicRead(BluetoothGatt gatt,  
BluetoothGattCharacteristic characteristic, int status) {  
        this.onCharacteristicRead(gatt, characteristic,  
characteristic.getValue(), status);  
    }
```

```
    public void onCharacteristicRead(@NonNull BluetoothGatt gatt, @NonNull  
BluetoothGattCharacteristic characteristic, @NonNull byte[] data, int status) {  
        if (status == 0) {  
            BleManagerHandler.this.log(4, () -> {  
                return "Read Response received from " + characteristic.getUuid()  
+ ", value: " + ParserUtils.parse(data);  
            });  
            BleManagerHandler.this.onCharacteristicRead(gatt, characteristic);  
            if (BleManagerHandler.this.request instanceof ReadRequest) {  
                ReadRequest rr = (ReadRequest)BleManagerHandler.this.request;  
                boolean matches = rr.matches(data);  
                if (matches) {  
                    rr.notifyValueChanged(gatt.getDevice(), data);  
                }  
  
                if (matches && !rr.hasMore()) {  
                    rr.notifySuccess(gatt.getDevice());  
                } else {  
                    BleManagerHandler.this.enqueueFirst(rr);  
                }  
            }  
        } else {  
            if (status == 5 || status == 8 || status == 137) {  
                BleManagerHandler.this.log(5, () -> {  
                    return "Authentication required (" + status + ")";  
                });  
            }  
        }  
    }
```

```

    });
    if (gatt.getDevice().getBondState() != 10) {
        Log.w("BleManager", "Phone has lost bonding information");
        BleManagerHandler.this.postCallback((c) -> {
            c.onError(gatt.getDevice(), "Phone has lost bonding
information", status);
        });
    }

    return;
}

Log.e("BleManager", "onCharacteristicRead error " + status);
if (BleManagerHandler.this.request instanceof ReadRequest) {
    BleManagerHandler.this.request.notifyFail(gatt.getDevice(),
status);
}

BleManagerHandler.this.awaitingRequest = null;
BleManagerHandler.this.onError(gatt.getDevice(), "Error on reading
characteristic", status);
}

BleManagerHandler.this.checkCondition();
BleManagerHandler.this.nextRequest(true);
}

public void onCharacteristicWrite(BluetoothGatt gatt,
BluetoothGattCharacteristic characteristic, int status) {
    if (status == 0) {
        BleManagerHandler.this.log(4, () -> {
            return "Data written to " + characteristic.getUuid();
        });
        BleManagerHandler.this.onCharacteristicWrite(gatt, characteristic);
        if (BleManagerHandler.this.request instanceof WriteRequest) {
            WriteRequest wr = (WriteRequest)BleManagerHandler.this.request;
            boolean valid = wr.notifyPacketSent(gatt.getDevice(),
characteristic.getValue());
            if (!valid && BleManagerHandler.this.requestQueue instanceof
ReliableWriteRequest) {
                wr.notifyFail(gatt.getDevice(), -6);
                BleManagerHandler.this.requestQueue.cancelQueue();
            } else if (wr.hasMore()) {
                BleManagerHandler.this.enqueueFirst(wr);
            } else {
                wr.notifySuccess(gatt.getDevice());
            }
        }
    } else {
        if (status == 5 || status == 8 || status == 137) {
            BleManagerHandler.this.log(5, () -> {
                return "Authentication required (" + status + ")";
            });
            if (gatt.getDevice().getBondState() != 10) {

```

```

        Log.w("BleManager", "Phone has lost bonding information");
        BleManagerHandler.this.postCallback((c) -> {
            c.onError(gatt.getDevice(), "Phone has lost bonding
information", status);
        });
    }

    return;
}

Log.e("BleManager", "onCharacteristicWrite error " + status);
if (BleManagerHandler.this.request instanceof WriteRequest) {
    BleManagerHandler.this.request.notifyFail(gatt.getDevice(),
status);
    if (BleManagerHandler.this.requestQueue instanceof
ReliableWriteRequest) {
        BleManagerHandler.this.requestQueue.cancelQueue();
    }
}

BleManagerHandler.this.awaitingRequest = null;
BleManagerHandler.this.onError(gatt.getDevice(), "Error on writing
characteristic", status);
}

BleManagerHandler.this.checkCondition();
BleManagerHandler.this.nextRequest(true);
}

public void onReliableWriteCompleted(@NonNull BluetoothGatt gatt, int
status) {
    boolean execute = BleManagerHandler.this.request.type ==
Request.Type.EXECUTE_RELIABLE_WRITE;
    BleManagerHandler.this.reliableWriteInProgress = false;
    if (status == 0) {
        if (execute) {
            BleManagerHandler.this.log(4, () -> {
                return "Reliable Write executed";
            });
            BleManagerHandler.this.request.notifySuccess(gatt.getDevice());
        } else {
            BleManagerHandler.this.log(5, () -> {
                return "Reliable Write aborted";
            });
            BleManagerHandler.this.request.notifySuccess(gatt.getDevice());
            BleManagerHandler.this.requestQueue.notifyFail(gatt.getDevice(),
-4);
        }
    } else {
        Log.e("BleManager", "onReliableWriteCompleted execute " + execute +
", error " + status);
        BleManagerHandler.this.request.notifyFail(gatt.getDevice(), status);
        BleManagerHandler.this.onError(gatt.getDevice(), "Error on Execute
Reliable Write", status);
    }
}

```

```

    }

    BleManagerHandler.this.checkCondition();
    BleManagerHandler.this.nextRequest(true);
}

    public void onDescriptorRead(BluetoothGatt gatt, BluetoothGattDescriptor
descriptor, int status) {
        byte[] data = descriptor.getValue();
        if (status == 0) {
            BleManagerHandler.this.log(4, () -> {
                return "Read Response received from descr. " +
descriptor.getUuid() + ", value: " + ParserUtils.parse(data);
            });
            BleManagerHandler.this.onDescriptorRead(gatt, descriptor);
            if (BleManagerHandler.this.request instanceof ReadRequest) {
                ReadRequest request =
(ReadRequest)BleManagerHandler.this.request;
                request.notifyValueChanged(gatt.getDevice(), data);
                if (request.hasMore()) {
                    BleManagerHandler.this.enqueueFirst(request);
                } else {
                    request.notifySuccess(gatt.getDevice());
                }
            }
        } else {
            if (status == 5 || status == 8 || status == 137) {
                BleManagerHandler.this.log(5, () -> {
                    return "Authentication required (" + status + ")";
                });
                if (gatt.getDevice().getBondState() != 10) {
                    Log.w("BleManager", "Phone has lost bonding information");
                    BleManagerHandler.this.postCallback((c) -> {
                        c.onError(gatt.getDevice(), "Phone has lost bonding
information", status);
                    });
                }

                return;
            }

            Log.e("BleManager", "onDescriptorRead error " + status);
            if (BleManagerHandler.this.request instanceof ReadRequest) {
                BleManagerHandler.this.request.notifyFail(gatt.getDevice(),
status);
            }

            BleManagerHandler.this.awaitingRequest = null;
            BleManagerHandler.this.onError(gatt.getDevice(), "Error on reading
descriptor", status);
        }

        BleManagerHandler.this.checkCondition();
        BleManagerHandler.this.nextRequest(true);
}

```

```

    }

    public void onDescriptorWrite(BluetoothGatt gatt, BluetoothGattDescriptor
descriptor, int status) {
        byte[] data = descriptor.getValue();
        if (status == 0) {
            BleManagerHandler.this.log(4, () -> {
                return "Data written to descr. " + descriptor.getUuid();
            });
            if (BleManagerHandler.this.isServiceChangedCCCD(descriptor)) {
                BleManagerHandler.this.log(4, () -> {
                    return "Service Changed notifications enabled";
                });
            } else if (BleManagerHandler.this.isCCCD(descriptor)) {
                if (data != null && data.length == 2 && data[1] == 0) {
                    switch(data[0]) {
                        case 0:
                            BleManagerHandler.this.log(4, () -> {
                                return "Notifications and indications disabled";
                            });
                            break;
                        case 1:
                            BleManagerHandler.this.log(4, () -> {
                                return "Notifications enabled";
                            });
                            break;
                        case 2:
                            BleManagerHandler.this.log(4, () -> {
                                return "Indications enabled";
                            });
                    }

                    BleManagerHandler.this.onDescriptorWrite(gatt, descriptor);
                }
            } else {
                BleManagerHandler.this.onDescriptorWrite(gatt, descriptor);
            }

            if (BleManagerHandler.this.request instanceof WriteRequest) {
                WriteRequest wr = (WriteRequest)BleManagerHandler.this.request;
                boolean valid = wr.notifyPacketSent(gatt.getDevice(), data);
                if (!valid && BleManagerHandler.this.requestQueue instanceof
ReliableWriteRequest) {
                    wr.notifyFail(gatt.getDevice(), -6);
                    BleManagerHandler.this.requestQueue.cancelQueue();
                } else if (wr.hasMore()) {
                    BleManagerHandler.this.enqueueFirst(wr);
                } else {
                    wr.notifySuccess(gatt.getDevice());
                }
            }
        } else {
            if (status == 5 || status == 8 || status == 137) {
                BleManagerHandler.this.log(5, () -> {

```

```

        return "Authentication required (" + status + ")";
    });
    if (gatt.getDevice().getBondState() != 10) {
        Log.w("BleManager", "Phone has lost bonding information");
        BleManagerHandler.this.postCallback((c) -> {
            c.onError(gatt.getDevice(), "Phone has lost bonding
information", status);
        });
    }

    return;
}

Log.e("BleManager", "onDescriptorWrite error " + status);
if (BleManagerHandler.this.request instanceof WriteRequest) {
    BleManagerHandler.this.request.notifyFail(gatt.getDevice(),
status);
    if (BleManagerHandler.this.requestQueue instanceof
ReliableWriteRequest) {
        BleManagerHandler.this.requestQueue.cancelQueue();
    }
}

BleManagerHandler.this.awaitingRequest = null;
BleManagerHandler.this.onError(gatt.getDevice(), "Error on writing
descriptor", status);
}

BleManagerHandler.this.checkCondition();
BleManagerHandler.this.nextRequest(true);
}

public void onCharacteristicChanged(BluetoothGatt gatt,
BluetoothGattCharacteristic characteristic) {
    this.onCharacteristicChanged(gatt, characteristic,
characteristic.getValue());
}

public void onCharacteristicChanged(@NonNull BluetoothGatt gatt, @NonNull
BluetoothGattCharacteristic characteristic, @NonNull byte[] data) {
    if
(BleManagerHandler.this.isServiceChangedCharacteristic(characteristic)) {
        if (VERSION.SDK_INT <= 30) {
            BleManagerHandler.this.log(4, () -> {
                return "Service Changed indication received";
            });
            BleManagerHandler.this.operationInProgress = true;
            BleManagerHandler.this.manager.onServicesInvalidated();
            BleManagerHandler.this.onDeviceDisconnected();
            BleManagerHandler.this.taskQueue.clear();
            BleManagerHandler.this.initQueue = null;
            BleManagerHandler.this.serviceDiscoveryRequested = true;
            BleManagerHandler.this.log(2, () -> {
                return "Discovering Services...";
            });
        }
    }
}

```



```

    });
    BleManagerHandler.this.log(3, () -> {
        return "gatt.discoverServices()";
    });
    gatt.discoverServices();
}

} else {
    BluetoothGattDescriptor cccd =
characteristic.getDescriptor(BleManager.CLIENT_CHARACTERISTIC_CONFIG_DESCRIPTOR_
UUID);
    boolean notifications = cccd == null || cccd.getValue() == null ||
cccd.getValue().length != 2 || cccd.getValue()[0] == 1;
    if (notifications) {
        BleManagerHandler.this.log(4, () -> {
            return "Notification received from " +
characteristic.getUuid() + ", value: " + ParserUtils.parse(data);
        });
        BleManagerHandler.this.onCharacteristicNotified(gatt,
characteristic);
    } else {
        BleManagerHandler.this.log(4, () -> {
            return "Indication received from " + characteristic.getUuid()
+ ", value: " + ParserUtils.parse(data);
        });
        BleManagerHandler.this.onCharacteristicIndicated(gatt,
characteristic);
    }

    if (BleManagerHandler.this.batteryLevelNotificationCallback != null
&& BleManagerHandler.this.isBatteryLevelCharacteristic(characteristic)) {

BleManagerHandler.this.batteryLevelNotificationCallback.notifyValueChanged(gatt.
getDevice(), data);
    }

    ValueChangedCallback request =
(ValueChangedCallback)BleManagerHandler.this.valueChangedCallbacks.get(character
istic);
    if (request != null && request.matches(data)) {
        request.notifyValueChanged(gatt.getDevice(), data);
    }

    if (BleManagerHandler.this.awaitingRequest instanceof
WaitForValueChangedRequest &&
BleManagerHandler.this.awaitingRequest.characteristic == characteristic &&
!BleManagerHandler.this.awaitingRequest.isTriggerPending()) {
        WaitForValueChangedRequest valueChangedRequest =
(WaitForValueChangedRequest)BleManagerHandler.this.awaitingRequest;
        if (valueChangedRequest.matches(data)) {
            valueChangedRequest.notifyValueChanged(gatt.getDevice(),
data);
        }
        if (valueChangedRequest.isComplete()) {
            BleManagerHandler.this.log(4, () -> {

```

```

        return "Wait for value changed complete";
    });
    valueChangedRequest.notifySuccess(gatt.getDevice());
    BleManagerHandler.this.awaitingRequest = null;
    if (valueChangedRequest.isTriggerCompleteOrNull()) {
        BleManagerHandler.this.nextRequest(true);
    }
}

}

}

if (BleManagerHandler.this.checkCondition()) {
    BleManagerHandler.this.nextRequest(true);
}

}

}

@RequiresApi(
    api = 21
)
public void onMtuChanged(@NonNull BluetoothGatt gatt, @IntRange(from =
23L,to = 517L) int mtu, int status) {
    if (status == 0) {
        BleManagerHandler.this.log(4, () -> {
            return "MTU changed to: " + mtu;
        });
        BleManagerHandler.this.mtu = mtu;
        BleManagerHandler.this.onMtuChanged(gatt, mtu);
        if (BleManagerHandler.this.request instanceof MtuRequest) {
            ((MtuRequest)BleManagerHandler.this.request).notifyMtuChanged(gatt.getDevice(),
mtu);
            BleManagerHandler.this.request.notifySuccess(gatt.getDevice());
        }
    } else {
        Log.e("BleManager", "onMtuChanged error: " + status + ", mtu: " +
mtu);
        if (BleManagerHandler.this.request instanceof MtuRequest) {
            BleManagerHandler.this.request.notifyFail(gatt.getDevice(),
status);
            BleManagerHandler.this.awaitingRequest = null;
        }

        BleManagerHandler.this.onError(gatt.getDevice(), "Error on mtu
request", status);
    }

    BleManagerHandler.this.checkCondition();
    if (BleManagerHandler.this.servicesDiscovered) {
        BleManagerHandler.this.nextRequest(true);
    }
}
}

```

```

    @RequiresApi(
        api = 26
    )
    @Keep
    public void onConnectionUpdated(@NonNull BluetoothGatt gatt,
        @IntRange(from = 6L,to = 3200L) int interval, @IntRange(from = 0L,to = 499L) int
        latency, @IntRange(from = 10L,to = 3200L) int timeout, int status) {
        if (status == 0) {
            BleManagerHandler.this.log(4, () -> {
                return "Connection parameters updated (interval: " +
                    (double)interval * 1.25D + "ms, latency: " + latency + ", timeout: " + timeout *
                    10 + "ms)";
            });
            BleManagerHandler.this.interval = interval;
            BleManagerHandler.this.latency = latency;
            BleManagerHandler.this.timeout = timeout;
            BleManagerHandler.this.onConnectionUpdated(gatt, interval, latency,
            timeout);
            ConnectionParametersUpdatedCallback cpuc =
            BleManagerHandler.this.connectionParametersUpdatedCallback;
            if (cpuc != null) {
                cpuc.onConnectionUpdated(gatt.getDevice(), interval, latency,
            timeout);
            }

            if (BleManagerHandler.this.request instanceof
            ConnectionPriorityRequest) {
                ((ConnectionPriorityRequest)BleManagerHandler.this.request).notifyConnectionPrio
            rityChanged(gatt.getDevice(), interval, latency, timeout);
                BleManagerHandler.this.request.notifySuccess(gatt.getDevice());
            }
            } else if (status == 59) {
                Log.e("BleManager", "onConnectionUpdated received status:
            Unacceptable connection interval, interval: " + interval + ", latency: " +
            latency + ", timeout: " + timeout);
                BleManagerHandler.this.log(5, () -> {
                    return "Connection parameters update failed with status: UNACCEPT
            CONN INTERVAL (0x3b) (interval: " + (double)interval * 1.25D + "ms, latency: " +
            latency + ", timeout: " + timeout * 10 + "ms)";
                });
                if (BleManagerHandler.this.request instanceof
            ConnectionPriorityRequest) {
                    BleManagerHandler.this.request.notifyFail(gatt.getDevice(),
            status);
                    BleManagerHandler.this.awaitingRequest = null;
                }
            } else {
                Log.e("BleManager", "onConnectionUpdated received status: " + status
            + ", interval: " + interval + ", latency: " + latency + ", timeout: " +
            timeout);
                BleManagerHandler.this.log(5, () -> {
                    return "Connection parameters update failed with status " +

```

```

status + " (interval: " + (double)interval * 1.25D + "ms, latency: " + latency +
", timeout: " + timeout * 10 + "ms)";
    });
    if (BleManagerHandler.this.request instanceof
ConnectionPriorityRequest) {
        BleManagerHandler.this.request.notifyFail(gatt.getDevice(),
status);
        BleManagerHandler.this.awaitingRequest = null;
    }

    BleManagerHandler.this.postCallback((c) -> {
        c.onError(gatt.getDevice(), "Error on connection priority
request", status);
    });
}

    if (BleManagerHandler.this.connectionPriorityOperationInProgress) {
        BleManagerHandler.this.connectionPriorityOperationInProgress =
false;
        BleManagerHandler.this.checkCondition();
        BleManagerHandler.this.nextRequest(true);
    }

}

@RequiresApi(
    api = 26
)
public void onPhyUpdate(@NonNull BluetoothGatt gatt, int txPhy, int rxPhy,
int status) {
    if (status == 0) {
        BleManagerHandler.this.log(4, () -> {
            return "PHY updated (TX: " + ParserUtils.phyToString(txPhy) + ",
RX: " + ParserUtils.phyToString(rxPhy) + ")";
        });
        if (BleManagerHandler.this.request instanceof PhyRequest) {
            ((PhyRequest)BleManagerHandler.this.request).notifyPhyChanged(gatt.getDevice(),
txPhy, rxPhy);
            BleManagerHandler.this.request.notifySuccess(gatt.getDevice());
        }
    } else {
        BleManagerHandler.this.log(5, () -> {
            return "PHY updated failed with status " + status;
        });
        if (BleManagerHandler.this.request instanceof PhyRequest) {
            BleManagerHandler.this.request.notifyFail(gatt.getDevice(),
status);
            BleManagerHandler.this.awaitingRequest = null;
        }

        BleManagerHandler.this.postCallback((c) -> {
            c.onError(gatt.getDevice(), "Error on PHY update", status);
        });
    }
}

```

```

    }

    if (BleManagerHandler.this.checkCondition() ||
BleManagerHandler.this.request instanceof PhyRequest) {
        BleManagerHandler.this.nextRequest(true);
    }

}

@RequiresApi(
    api = 26
)
public void onPhyRead(@NonNull BluetoothGatt gatt, int txPhy, int rxPhy,
int status) {
    if (status == 0) {
        BleManagerHandler.this.log(4, () -> {
            return "PHY read (TX: " + ParserUtils.phyToString(txPhy) + ", RX:
" + ParserUtils.phyToString(rxPhy) + ")";
        });
        if (BleManagerHandler.this.request instanceof PhyRequest) {
((PhyRequest)BleManagerHandler.this.request).notifyPhyChanged(gatt.getDevice(),
txPhy, rxPhy);
            BleManagerHandler.this.request.notifySuccess(gatt.getDevice());
        }
    } else {
        BleManagerHandler.this.log(5, () -> {
            return "PHY read failed with status " + status;
        });
        if (BleManagerHandler.this.request instanceof PhyRequest) {
            BleManagerHandler.this.request.notifyFail(gatt.getDevice(),
status);
        }

        BleManagerHandler.this.awaitingRequest = null;
        BleManagerHandler.this.postCallback((c) -> {
            c.onError(gatt.getDevice(), "Error on PHY read", status);
        });
    }

    BleManagerHandler.this.checkCondition();
    BleManagerHandler.this.nextRequest(true);
}

public void onReadRemoteRssi(@NonNull BluetoothGatt gatt, @IntRange(from =
-128L,to = 20L) int rssi, int status) {
    if (status == 0) {
        BleManagerHandler.this.log(4, () -> {
            return "Remote RSSI received: " + rssi + " dBm";
        });
        if (BleManagerHandler.this.request instanceof ReadRssiRequest) {
((ReadRssiRequest)BleManagerHandler.this.request).notifyRssiRead(gatt.getDevice(
), rssi);
    }
}

```

```

        BleManagerHandler.this.request.notifySuccess(gatt.getDevice());
    }
    } else {
        BleManagerHandler.this.log(5, () -> {
            return "Reading remote RSSI failed with status " + status;
        });
        if (BleManagerHandler.this.request instanceof ReadRssiRequest) {
            BleManagerHandler.this.request.notifyFail(gatt.getDevice(),
status);
        }

        BleManagerHandler.this.awaitingRequest = null;
        BleManagerHandler.this.postCallback((c) -> {
            c.onError(gatt.getDevice(), "Error on RSSI read", status);
        });
    }

    BleManagerHandler.this.checkCondition();
    BleManagerHandler.this.nextRequest(true);
}
};

void init(@NonNull BleManager manager, @NonNull Handler handler) {
    this.manager = manager;
    this.handler = handler;
}

void useServer(@Nullable BleServerManager server) {
    this.serverManager = server;
}

void attachClientConnection(BluetoothDevice clientDevice) {
    if (this.bluetoothDevice != null) {
        this.log(6, () -> {
            return "attachClientConnection called on existing connection, call
ignored";
        });
    } else {
        this.bluetoothDevice = clientDevice;
        this.initializeServerAttributes();
        this.manager.initialize();
    }
}

private void initializeServerAttributes() {
    if (this.serverManager != null) {
        BluetoothGattServer server = this.serverManager.getServer();
        if (server != null) {
            Iterator var2 = server.getServices().iterator();

            while(var2.hasNext()) {
                BluetoothGattService service = (BluetoothGattService)var2.next();
                Iterator var4 = service.getCharacteristics().iterator();

```

```

        while(var4.hasNext()) {
            BluetoothGattCharacteristic characteristic =
(BluetoothGattCharacteristic)var4.next();
            if (!this.serverManager.isShared(characteristic)) {
                if (this.characteristicValues == null) {
                    this.characteristicValues = new HashMap();
                }

                this.characteristicValues.put(characteristic,
characteristic.getValue());
            }

            Iterator var6 = characteristic.getDescriptors().iterator();

            while(var6.hasNext()) {
                BluetoothGattDescriptor descriptor =
(BluetoothGattDescriptor)var6.next();
                if (!this.serverManager.isShared(descriptor)) {
                    if (this.descriptorValues == null) {
                        this.descriptorValues = new HashMap();
                    }

                    this.descriptorValues.put(descriptor,
descriptor.getValue());
                }
            }
        }

        this.manager.onServerReady(server);
    }
}

void close() {
    try {
        Context context = this.manager.getContext();
        context.unregisterReceiver(this.bluetoothStateBroadcastReceiver);
        context.unregisterReceiver(this.mBondingBroadcastReceiver);
    } catch (Exception var6) {
    }

    synchronized(this.LOCK) {
        if (this.bluetoothGatt != null) {
            if (this.manager.shouldClearCacheWhenDisconnected()) {
                if (this.internalRefreshDeviceCache()) {
                    this.log(4, () -> {
                        return "Cache refreshed";
                    });
                } else {
                    this.log(5, () -> {
                        return "Refreshing failed";
                    });
                }
            }
        }
    }
}

```

```

        });
    }
}

this.log(3, () -> {
    return "gatt.close()";
});

try {
    this.bluetoothGatt.close();
} catch (Throwable var4) {
}

this.bluetoothGatt = null;
}

this.reliableWriteInProgress = false;
this.initialConnection = false;
this.taskQueue.clear();
this.initQueue = null;
this.initialization = false;
this.bluetoothDevice = null;
this.connected = false;
}
}

public BluetoothDevice getBluetoothDevice() {
    return this.bluetoothDevice;
}

@Nullable
public final byte[] getCharacteristicValue(@NonNull
BluetoothGattCharacteristic serverCharacteristic) {
    return this.characteristicValues != null &&
this.characteristicValues.containsKey(serverCharacteristic) ?
(byte[])this.characteristicValues.get(serverCharacteristic) :
serverCharacteristic.getValue();
}

@Nullable
public final byte[] getDescriptorValue(@NonNull BluetoothGattDescriptor
serverDescriptor) {
    return this.descriptorValues != null &&
this.descriptorValues.containsKey(serverDescriptor) ?
(byte[])this.descriptorValues.get(serverDescriptor) :
serverDescriptor.getValue();
}

private boolean internalConnect(@NonNull BluetoothDevice device, @Nullable
ConnectRequest connectRequest) {
    boolean bluetoothEnabled =
BluetoothAdapter.getDefaultAdapter().isEnabled();
    if (!this.connected && bluetoothEnabled) {
        Context context = this.manager.getContext();

```



```

synchronized(this.LOCK) {
    if (this.bluetoothGatt != null) {
        if (this.initialConnection) {
            this.initialConnection = false;
            this.connectionTime = 0L;
            this.connectionState = 1;
            this.log(2, () -> {
                return "Connecting...";
            });
            this.postCallback((c) -> {
                c.onDeviceConnecting(device);
            });
            this.postConnectionStateChange((o) -> {
                o.onDeviceConnecting(device);
            });
            this.log(3, () -> {
                return "gatt.connect()";
            });
            this.bluetoothGatt.connect();
            return true;
        }

        this.log(3, () -> {
            return "gatt.close()";
        });

        try {
            this.bluetoothGatt.close();
        } catch (Throwable var9) {
        }

        this.bluetoothGatt = null;

        try {
            this.log(3, () -> {
                return "wait(200)";
            });
            Thread.sleep(200L);
        } catch (InterruptedException var8) {
        }
    } else if (connectRequest != null) {
        context.registerReceiver(this.bluetoothStateBroadcastReceiver,
            new IntentFilter("android.bluetooth.adapter.action.STATE_CHANGED"));
        context.registerReceiver(this.mBondingBroadcastReceiver, new
            IntentFilter("android.bluetooth.device.action.BOND_STATE_CHANGED"));
    }
}

if (connectRequest == null) {
    return false;
} else {
    boolean shouldAutoConnect = connectRequest.shouldAutoConnect();
    this.userDisconnected = !shouldAutoConnect;
    if (shouldAutoConnect) {

```

```

        this.initialConnection = true;
    }

    this.bluetoothDevice = device;
    this.log(2, () -> {
        return connectRequest.isFirstAttempt() ? "Connecting..." :
"Retrying...";
    });
    this.connectionState = 1;
    this.postCallback((c) -> {
        c.onDeviceConnecting(device);
    });
    this.postConnectionStateChange((o) -> {
        o.onDeviceConnecting(device);
    });
    this.connectionTime = SystemClock.elapsedRealtime();
    int preferredPhy;
    if (VERSION.SDK_INT > 26) {
        preferredPhy = connectRequest.getPreferredPhy();
        this.log(3, () -> {
            return "gatt = device.connectGatt(autoConnect = false,
TRANSPORT_LE, " + ParserUtils.phyMaskToString(preferredPhy) + ")";
        });
        this.bluetoothGatt = device.connectGatt(context, false,
this.gattCallback, 2, preferredPhy, this.handler);
    } else if (VERSION.SDK_INT == 26) {
        preferredPhy = connectRequest.getPreferredPhy();
        this.log(3, () -> {
            return "gatt = device.connectGatt(autoConnect = false,
TRANSPORT_LE, " + ParserUtils.phyMaskToString(preferredPhy) + ")";
        });
        this.bluetoothGatt = device.connectGatt(context, false,
this.gattCallback, 2, preferredPhy);
    } else if (VERSION.SDK_INT >= 23) {
        this.log(3, () -> {
            return "gatt = device.connectGatt(autoConnect = false,
TRANSPORT_LE)";
        });
        this.bluetoothGatt = device.connectGatt(context, false,
this.gattCallback, 2);
    } else {
        this.log(3, () -> {
            return "gatt = device.connectGatt(autoConnect = false)";
        });
        this.bluetoothGatt = device.connectGatt(context, false,
this.gattCallback);
    }

    return true;
}
} else {
    BluetoothDevice currentDevice = this.bluetoothDevice;
    if (bluetoothEnabled && currentDevice != null &&
currentDevice.equals(device)) {

```

```

        if (this.connectRequest != null) {
            this.connectRequest.notifySuccess(device);
        }
    } else if (this.connectRequest != null) {
        this.connectRequest.notifyFail(device, bluetoothEnabled ? -4 :
-100);
    }

    this.connectRequest = null;
    this.nextRequest(true);
    return true;
}
}

```

```

private boolean internalDisconnect(int reason) {
    this.userDisconnected = true;
    this.initialConnection = false;
    this.ready = false;
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null) {
        boolean wasConnected = this.connected;
        this.connectionState = 3;
        this.log(2, () -> {
            return wasConnected ? "Disconnecting..." : "Cancelling
connection...";
        });
        BluetoothDevice device = gatt.getDevice();
        if (wasConnected) {
            this.postCallback((c) -> {
                c.onDeviceDisconnecting(device);
            });
            this.postConnectionStateChange((o) -> {
                o.onDeviceDisconnecting(device);
            });
        }

        this.log(3, () -> {
            return "gatt.disconnect()";
        });
        gatt.disconnect();
        if (wasConnected) {
            return true;
        }

        this.connectionState = 0;
        this.log(4, () -> {
            return "Disconnected";
        });
        this.close();
        this.postCallback((c) -> {
            c.onDeviceDisconnected(device);
        });
        this.postConnectionStateChange((o) -> {
            o.onDeviceDisconnected(device, reason);
        });
    }
}

```

```

    });
}

Request r = this.request;
if (r != null && r.type == Request.Type.DISCONNECT) {
    if (this.bluetoothDevice == null && gatt == null) {
        r.notifyInvalidRequest();
    } else {
        r.notifySuccess(this.bluetoothDevice != null ? this.bluetoothDevice
: gatt.getDevice());
    }
}

this.nextRequest(true);
return true;
}

@RequiresPermission("android.permission.BLUETOOTH_ADMIN")
private boolean internalCreateBond(boolean ensure) {
    BluetoothDevice device = this.bluetoothDevice;
    if (device == null) {
        return false;
    } else {
        if (ensure) {
            this.log(2, () -> {
                return "Ensuring bonding...";
            });
        } else {
            this.log(2, () -> {
                return "Starting bonding...";
            });
        }
    }

    if (!ensure && device.getBondState() == 12) {
        this.log(5, () -> {
            return "Bond information present on client, skipping bonding";
        });
        this.request.notifySuccess(device);
        this.nextRequest(true);
        return true;
    } else {
        boolean result = this.createBond(device);
        if (ensure && !result) {
            Request bond = Request.createBond().setRequestHandler(this);
            bond.successCallback = this.request.successCallback;
            bond.invalidRequestCallback =
this.request.invalidRequestCallback;
            bond.failCallback = this.request.failCallback;
            bond.internalSuccessCallback =
this.request.internalSuccessCallback;
            bond.internalFailCallback = this.request.internalFailCallback;
            this.request.successCallback = null;
            this.request.invalidRequestCallback = null;
            this.request.failCallback = null;
        }
    }
}

```

```

        this.request.internalSuccessCallback = null;
        this.request.internalFailCallback = null;
        this.enqueueFirst(bond);
        this.enqueueFirst(Request.removeBond().setRequestHandler(this));
        this.nextRequest(true);
        return true;
    } else {
        return result;
    }
}
}
}
}
}

```

```

@RequiresPermission("android.permission.BLUETOOTH_ADMIN")
private boolean createBond(@NonNull BluetoothDevice device) {
    if (VERSION.SDK_INT >= 19) {
        this.log(3, () -> {
            return "device.createBond()";
        });
        return device.createBond();
    } else {
        try {
            Method createBond = device.getClass().getMethod("createBond");
            this.log(3, () -> {
                return "device.createBond() (hidden)";
            });
            return createBond.invoke(device) == Boolean.TRUE;
        } catch (Exception var3) {
            Log.w("BleManager", "An exception occurred while creating bond",
var3);
            return false;
        }
    }
}
}
}

```

```

@RequiresPermission("android.permission.BLUETOOTH_ADMIN")
private boolean internalRemoveBond() {
    BluetoothDevice device = this.bluetoothDevice;
    if (device == null) {
        return false;
    } else {
        this.log(2, () -> {
            return "Removing bond information...";
        });
        if (device.getBondState() == 10) {
            this.log(5, () -> {
                return "Device is not bonded";
            });
            this.request.notifySuccess(device);
            this.nextRequest(true);
            return true;
        } else {
            try {
                Method removeBond = device.getClass().getMethod("removeBond");

```

```

        this.log(3, () -> {
            return "device.removeBond() (hidden)";
        });
        this.userDisconnected = true;
        return removeBond.invoke(device) == Boolean.TRUE;
    } catch (Exception var3) {
        Log.w("BleManager", "An exception occurred while removing bond",
var3);
        return false;
    }
}
}
}
}

```

```

private boolean ensureServiceChangedEnabled() {
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null && this.connected) {
        BluetoothDevice device = gatt.getDevice();
        if (device.getBondState() != 12) {
            return false;
        } else {
            BluetoothGattService gaService =
gatt.getService(BleManager.GENERIC_ATTRIBUTE_SERVICE);
            if (gaService == null) {
                return false;
            } else {
                BluetoothGattCharacteristic scCharacteristic =
gaService.getCharacteristic(BleManager.SERVICE_CHANGED_CHARACTERISTIC);
                if (scCharacteristic == null) {
                    return false;
                } else {
                    this.log(4, () -> {
device";
                        return "Service Changed characteristic found on a bonded
device";
                    });
                    return this.internalEnableIndications(scCharacteristic);
                }
            }
        }
    } else {
        return false;
    }
}
}

```

```

private boolean internalEnableNotifications(@Nullable
BluetoothGattCharacteristic characteristic) {
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null && characteristic != null && this.connected) {
        BluetoothGattDescriptor descriptor = getCccd(characteristic, 16);
        if (descriptor != null) {
            this.log(3, () -> {
                return "gatt.setCharacteristicNotification(" +
characteristic.getUuid() + ", true)";
            });
        }
    }
}

```

```

        gatt.setCharacteristicNotification(characteristic, true);
        this.log(2, () -> {
            return "Enabling notifications for " + characteristic.getUuid();
        });
        if (VERSION.SDK_INT >= 33) {
            this.log(3, () -> {
                return
"Gatt.writeDescriptor(00002902-0000-1000-8000-00805f9b34fb, value=0x01-00)";
            });
            return gatt.writeDescriptor(descriptor,
BluetoothGattDescriptor.ENABLE_NOTIFICATION_VALUE) == 0;
        } else {
            this.log(3, () -> {
                return "descriptor.setValue(0x01-00)";
            });

            descriptor.setValue(BluetoothGattDescriptor.ENABLE_NOTIFICATION_VALUE);
            this.log(3, () -> {
                return
"Gatt.writeDescriptor(00002902-0000-1000-8000-00805f9b34fb)";
            });
            return VERSION.SDK_INT >= 24 ? gatt.writeDescriptor(descriptor) :
this.internalWriteDescriptorWorkaround(descriptor);
        }
    } else {
        return false;
    }
} else {
    return false;
}
}

private boolean internalDisableNotifications(@Nullable
BluetoothGattCharacteristic characteristic) {
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null && characteristic != null && this.connected) {
        BluetoothGattDescriptor descriptor = getCccd(characteristic, 48);
        if (descriptor != null) {
            this.log(3, () -> {
                return "Gatt.setCharacteristicNotification(" +
characteristic.getUuid() + ", false)";
            });
            gatt.setCharacteristicNotification(characteristic, false);
            this.log(2, () -> {
                return "Disabling notifications and indications for " +
characteristic.getUuid();
            });
            if (VERSION.SDK_INT >= 33) {
                this.log(3, () -> {
                    return
"Gatt.writeDescriptor(00002902-0000-1000-8000-00805f9b34fb, value=0x00-00)";
                });
                return gatt.writeDescriptor(descriptor,
BluetoothGattDescriptor.ENABLE_NOTIFICATION_VALUE) == 0;
            }
        }
    }
}

```

```

        } else {
            this.log(3, () -> {
                return "descriptor.setValue(0x00-00)";
            });
        }

        descriptor.setValue(BluetoothGattDescriptor.DISABLE_NOTIFICATION_VALUE);
        this.log(3, () -> {
            return
            "gatt.writeDescriptor(00002902-0000-1000-8000-00805f9b34fb)";
        });
        return VERSION.SDK_INT >= 24 ? gatt.writeDescriptor(descriptor) :
        this.internalWriteDescriptorWorkaround(descriptor);
    }
    } else {
        return false;
    }
    } else {
        return false;
    }
}

private boolean internalEnableIndications(@Nullable
BluetoothGattCharacteristic characteristic) {
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null && characteristic != null && this.connected) {
        BluetoothGattDescriptor descriptor = getCccd(characteristic, 32);
        if (descriptor != null) {
            this.log(3, () -> {
                return "gatt.setCharacteristicNotification(" +
characteristic.getUuid() + ", true)";
            });
            gatt.setCharacteristicNotification(characteristic, true);
            this.log(2, () -> {
                return "Enabling indications for " + characteristic.getUuid();
            });
            if (VERSION.SDK_INT >= 33) {
                this.log(3, () -> {
                    return
                    "gatt.writeDescriptor(00002902-0000-1000-8000-00805f9b34fb, value=0x02-00)";
                });
                return gatt.writeDescriptor(descriptor,
BluetoothGattDescriptor.ENABLE_INDICATION_VALUE) == 0;
            } else {
                this.log(3, () -> {
                    return "descriptor.setValue(0x02-00)";
                });
            }

            descriptor.setValue(BluetoothGattDescriptor.ENABLE_INDICATION_VALUE);
            this.log(3, () -> {
                return
                "gatt.writeDescriptor(00002902-0000-1000-8000-00805f9b34fb)";
            });
            return VERSION.SDK_INT >= 24 ? gatt.writeDescriptor(descriptor) :
            this.internalWriteDescriptorWorkaround(descriptor);
        }
    }
}

```



```

        }
    } else {
        return false;
    }
} else {
    return false;
}
}

private boolean internalDisableIndications(@Nullable
BluetoothGattCharacteristic characteristic) {
    return this.internalDisableNotifications(characteristic);
}

private boolean internalSendNotification(@Nullable
BluetoothGattCharacteristic serverCharacteristic, boolean confirm, @Nullable
byte[] data) {
    if (this.serverManager != null && this.serverManager.getServer() != null
    && serverCharacteristic != null) {
        int requiredProperty = confirm ? 32 : 16;
        if ((serverCharacteristic.getProperties() & requiredProperty) == 0) {
            return false;
        } else {
            BluetoothGattDescriptor cccd =
serverCharacteristic.getDescriptor(BleManager.CLIENT_CHARACTERISTIC_CONFIG_DESCRIPTOR_UUID);
            if (cccd == null) {
                return false;
            } else {
                byte[] value = this.descriptorValues != null &&
this.descriptorValues.containsKey(cccd) ?
(byte[])this.descriptorValues.get(cccd) : cccd.getValue();
                if (value != null && value.length == 2 && value[0] != 0) {
                    this.log(2, () -> {
                        return "[Server] Sending " + (confirm ? "indication" :
"notification") + " to " + serverCharacteristic.getUuid();
                    });
                    if (VERSION.SDK_INT >= 33) {
                        this.log(3, () -> {
                            return "[Server]
gattServer.notifyCharacteristicChanged(" + serverCharacteristic.getUuid() + ",
confirm=" + confirm + ", value=" + ParserUtils.parseDebug(data) + ")";
                        });
                        return
this.serverManager.getServer().notifyCharacteristicChanged(this.bluetoothDevice,
serverCharacteristic, confirm, data) == 0;
                    } else {
                        this.log(3, () -> {
                            return "[Server] characteristic.setValue(" +
ParserUtils.parseDebug(data) + ")";
                        });
                        serverCharacteristic.setValue(data);
                        this.log(3, () -> {
                            return "[Server]

```

```

gattServer.notifyCharacteristicChanged(" + serverCharacteristic.getUuid() + ",
confirm=" + confirm + ")";
    });
    boolean result =
this.serverManager.getServer().notifyCharacteristicChanged(this.bluetoothDevice,
serverCharacteristic, confirm);
    if (result && VERSION.SDK_INT < 21) {
        this.post(() -> {
            this.notifyNotificationSent(this.bluetoothDevice);
            this.nextRequest(true);
        });
    }

    return result;
}
} else {
    this.nextRequest(true);
    return true;
}
}
} else {
    return false;
}
}
}

```

```

private static BluetoothGattDescriptor getCccd(@Nullable
BluetoothGattCharacteristic characteristic, int requiredProperty) {
    if (characteristic == null) {
        return null;
    } else {
        int properties = characteristic.getProperties();
        return (properties & requiredProperty) == 0 ? null :
characteristic.getDescriptor(BleManager.CLIENT_CHARACTERISTIC_CONFIG_DESCRIPTOR_
UUID);
    }
}

```

```

private boolean internalReadCharacteristic(@Nullable
BluetoothGattCharacteristic characteristic) {
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null && characteristic != null && this.connected) {
        int properties = characteristic.getProperties();
        if ((properties & 2) == 0) {
            return false;
        } else {
            this.log(2, () -> {
                return "Reading characteristic " + characteristic.getUuid();
            });
            this.log(3, () -> {
                return "gatt.readCharacteristic(" + characteristic.getUuid() +
");";
            });
            return gatt.readCharacteristic(characteristic);
        }
    }
}

```

```

    }
    } else {
        return false;
    }
}

private boolean internalWriteCharacteristic(@Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] data, int
writeType) {
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null && characteristic != null && this.connected) {
        int properties = characteristic.getProperties();
        if ((properties & 12) == 0) {
            return false;
        } else if (VERSION.SDK_INT >= 33) {
            this.log(2, () -> {
                return "Writing characteristic " + characteristic.getUuid() + "
(" + ParserUtils.writeTypeToString(writeType) + ")";
            });
            this.log(3, () -> {
                return "gatt.writeCharacteristic(" + characteristic.getUuid() +
", value=" + ParserUtils.parseDebug(data) + ", " +
ParserUtils.writeTypeToString(writeType) + ")";
            });
            return gatt.writeCharacteristic(characteristic, data, writeType) ==
0;
        } else {
            this.log(2, () -> {
                return "Writing characteristic " + characteristic.getUuid() + "
(" + ParserUtils.writeTypeToString(writeType) + ")";
            });
            this.log(3, () -> {
                return "characteristic.setValue(" + ParserUtils.parseDebug(data)
+ ")";
            });
            characteristic.setValue(data);
            this.log(3, () -> {
                return "characteristic.setWriteType(" +
ParserUtils.writeTypeToString(writeType) + ")";
            });
            characteristic.setWriteType(writeType);
            this.log(3, () -> {
                return "gatt.writeCharacteristic(" + characteristic.getUuid() +
")";
            });
            return gatt.writeCharacteristic(characteristic);
        }
    } else {
        return false;
    }
}

private boolean internalReadDescriptor(@Nullable BluetoothGattDescriptor
descriptor) {

```

```

BluetoothGatt gatt = this.bluetoothGatt;
if (gatt != null && descriptor != null && this.connected) {
    this.log(2, () -> {
        return "Reading descriptor " + descriptor.getUuid();
    });
    this.log(3, () -> {
        return "gatt.readDescriptor(" + descriptor.getUuid() + ")";
    });
    return gatt.readDescriptor(descriptor);
} else {
    return false;
}
}

```

```

private boolean internalWriteDescriptor(@Nullable BluetoothGattDescriptor
descriptor, @Nullable byte[] data) {
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null && descriptor != null && this.connected) {
        this.log(2, () -> {
            return "Writing descriptor " + descriptor.getUuid();
        });
        if (VERSION.SDK_INT >= 33) {
            this.log(3, () -> {
                return "gatt.writeDescriptor(" + descriptor.getUuid() + ",
value=" + ParserUtils.parseDebug(data) + ")";
            });
            return gatt.writeDescriptor(descriptor, data) == 0;
        } else {
            this.log(3, () -> {
                return "descriptor.setValue(" + descriptor.getUuid() + ")";
            });
            descriptor.setValue(data);
            this.log(3, () -> {
                return "gatt.writeDescriptor(" + descriptor.getUuid() + ")";
            });
            return VERSION.SDK_INT >= 24 ?
this.internalWriteDescriptorWorkaround(descriptor) :
gatt.writeDescriptor(descriptor);
        }
    } else {
        return false;
    }
}

```

```

private boolean internalWriteDescriptorWorkaround(@Nullable
BluetoothGattDescriptor descriptor) {
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null && descriptor != null && this.connected) {
        BluetoothGattCharacteristic parentCharacteristic =
descriptor.getCharacteristic();
        int originalWriteType = parentCharacteristic.getWriteType();
        parentCharacteristic.setWriteType(2);
        boolean result = gatt.writeDescriptor(descriptor);
        parentCharacteristic.setWriteType(originalWriteType);
    }
}

```

```

        return result;
    } else {
        return false;
    }
}

private boolean internalBeginReliableWrite() {
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null && this.connected) {
        if (this.reliableWriteInProgress) {
            return true;
        } else {
            this.log(2, () -> {
                return "Beginning reliable write...";
            });
            this.log(3, () -> {
                return "gatt.beginReliableWrite()";
            });
            return this.reliableWriteInProgress = gatt.beginReliableWrite();
        }
    } else {
        return false;
    }
}

```

```

private boolean internalExecuteReliableWrite() {
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null && this.connected) {
        if (!this.reliableWriteInProgress) {
            return false;
        } else {
            this.log(2, () -> {
                return "Executing reliable write...";
            });
            this.log(3, () -> {
                return "gatt.executeReliableWrite()";
            });
            return gatt.executeReliableWrite();
        }
    } else {
        return false;
    }
}

```

```

private boolean internalAbortReliableWrite() {
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null && this.connected) {
        if (!this.reliableWriteInProgress) {
            return false;
        } else {
            this.log(2, () -> {
                return "Aborting reliable write...";
            });
            if (VERSION.SDK_INT >= 19) {

```

```

        this.log(3, () -> {
            return "gatt.abortReliableWrite()";
        });
        gatt.abortReliableWrite();
    } else {
        this.log(3, () -> {
            return "gatt.abortReliableWrite(device)";
        });
        gatt.abortReliableWrite(gatt.getDevice());
    }

    return true;
}
} else {
    return false;
}
}

/** @deprecated */
@Deprecated
private boolean internalReadBatteryLevel() {
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null && this.connected) {
        BluetoothGattService batteryService =
gatt.getService(BleManager.BATTERY_SERVICE);
        if (batteryService == null) {
            return false;
        } else {
            BluetoothGattCharacteristic batteryLevelCharacteristic =
batteryService.getCharacteristic(BleManager.BATTERY_LEVEL_CHARACTERISTIC);
            return this.internalReadCharacteristic(batteryLevelCharacteristic);
        }
    } else {
        return false;
    }
}

/** @deprecated */
@Deprecated
private boolean internalSetBatteryNotifications(boolean enable) {
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null && this.connected) {
        BluetoothGattService batteryService =
gatt.getService(BleManager.BATTERY_SERVICE);
        if (batteryService == null) {
            return false;
        } else {
            BluetoothGattCharacteristic batteryLevelCharacteristic =
batteryService.getCharacteristic(BleManager.BATTERY_LEVEL_CHARACTERISTIC);
            return enable ?
this.internalEnableNotifications(batteryLevelCharacteristic) :
this.internalDisableNotifications(batteryLevelCharacteristic);
        }
    } else {

```

```

        return false;
    }
}

@RequiresApi(
    api = 21
)
private boolean internalRequestMtu(@IntRange(from = 23L,to = 517L) int mtu) {
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null && this.connected) {
        this.log(2, () -> {
            return "Requesting new MTU...";
        });
        this.log(3, () -> {
            return "gatt.requestMtu(" + mtu + ")";
        });
        return gatt.requestMtu(mtu);
    } else {
        return false;
    }
}

@RequiresApi(
    api = 21
)
private boolean internalRequestConnectionPriority(int priority) {
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null && this.connected) {
        int supervisionTimeout = VERSION.SDK_INT >= 26 ? 5 : 20;
        this.log(2, () -> {
            String text;
            switch(priority) {
                case 0:
                default:
                    text = "BALANCED (30-50ms, 0, " + supervisionTimeout + "s)";
                    break;
                case 1:
                    text = VERSION.SDK_INT >= 23 ? "HIGH (11.25-15ms, 0, " +
supervisionTimeout + "s)" : "HIGH (7.5-10ms, 0, " + supervisionTimeout + "s)";
                    break;
                case 2:
                    text = "LOW POWER (100-125ms, 2, " + supervisionTimeout + "s)";
            }

            return "Requesting connection priority: " + text + "...";
        });
        this.log(3, () -> {
            String text;
            switch(priority) {
                case 0:
                default:
                    text = "BALANCED";
                    break;
                case 1:

```

```

        text = "HIGH";
        break;
    case 2:
        text = "LOW POWER";
    }

    return "gatt.requestConnectionPriority(" + text + ")";
});
return gatt.requestConnectionPriority(priority);
} else {
    return false;
}
}

@RequiresApi(
    api = 26
)
private boolean internalSetPreferredPhy(int txPhy, int rxPhy, int phyOptions)
{
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null && this.connected) {
        this.log(2, () -> {
            return "Requesting preferred PHYs...";
        });
        this.log(3, () -> {
            return "gatt.setPreferredPhy(" + ParserUtils.phyMaskToString(txPhy)
+ ", " + ParserUtils.phyMaskToString(rxPhy) + ", coding option = " +
ParserUtils.phyCodedOptionToString(phyOptions) + ")";
        });
        gatt.setPreferredPhy(txPhy, rxPhy, phyOptions);
        return true;
    } else {
        return false;
    }
}

@RequiresApi(
    api = 26
)
private boolean internalReadPhy() {
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null && this.connected) {
        this.log(2, () -> {
            return "Reading PHY...";
        });
        this.log(3, () -> {
            return "gatt.readPhy()";
        });
        gatt.readPhy();
        return true;
    } else {
        return false;
    }
}
}

```



```

private boolean internalReadRssi() {
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt != null && this.connected) {
        this.log(2, () -> {
            return "Reading remote RSSI...";
        });
        this.log(3, () -> {
            return "gatt.readRemoteRssi()";
        });
        return gatt.readRemoteRssi();
    } else {
        return false;
    }
}

@NonNull
ValueChangedCallback getValueChangedCallback(@Nullable Object attribute) {
    ValueChangedCallback callback =
(ValueChangedCallback)this.valueChangedCallbacks.get(attribute);
    if (callback == null) {
        callback = new ValueChangedCallback(this);
        if (attribute != null) {
            this.valueChangedCallbacks.put(attribute, callback);
        }
    } else if (this.bluetoothDevice != null) {
        callback.notifyClosed();
    }

    return callback;
}

void removeValueChangedCallback(@Nullable Object attribute) {
    ValueChangedCallback callback =
(ValueChangedCallback)this.valueChangedCallbacks.remove(attribute);
    if (callback != null) {
        callback.notifyClosed();
    }
}

void setCharacteristicValue(@Nullable BluetoothGattCharacteristic
serverCharacteristic, @Nullable DataProvider dataProvider) {
    if (serverCharacteristic != null) {
        if (dataProvider == null) {
            this.dataProviders.remove(serverCharacteristic);
        } else {
            this.dataProviders.put(serverCharacteristic, dataProvider);
        }
    }
}

void setDescriptorValue(@Nullable BluetoothGattDescriptor serverDescriptor,

```

```

@Nullable DataProvider dataProvider) {
    if (serverDescriptor != null) {
        if (dataProvider == null) {
            this.dataProviders.remove(serverDescriptor);
        } else {
            this.dataProviders.put(serverDescriptor, dataProvider);
        }
    }
}

@RequiresApi(
    api = 26
)
void setConnectionParametersListener(@Nullable
ConnectionParametersUpdatedCallback callback) {
    this.connectionParametersUpdatedCallback = callback;
    if (callback != null && this.bluetoothDevice != null && this.interval > 0)
{
        callback.onConnectionUpdated(this.bluetoothDevice, this.interval,
this.latency, this.timeout);
    }

}

/** @deprecated */
@Deprecated
DataReceivedCallback getBatteryLevelCallback() {
    return (device, data) -> {
        if (data.size() == 1) {
            int batteryLevel = data.getIntValue(17, 0);
            this.log(4, () -> {
                return "Battery Level received: " + batteryLevel + "%";
            });
            this.batteryValue = batteryLevel;
            this.onBatteryValueReceived(this.bluetoothGatt, batteryLevel);
            this.postCallback((c) -> {
                c.onBatteryValueReceived(device, batteryLevel);
            });
        }
    };
}

/** @deprecated */
@Deprecated
void setBatteryLevelNotificationCallback() {
    if (this.batteryLevelNotificationCallback == null) {
        this.batteryLevelNotificationCallback = (new
ValueChangedCallback(this)).with((device, data) -> {
            if (data.size() == 1) {
                int batteryLevel = data.getIntValue(17, 0);
                this.batteryValue = batteryLevel;
                this.onBatteryValueReceived(this.bluetoothGatt, batteryLevel);
            }
        });
    }
}

```

```

        this.postCallback((c) -> {
            c.onBatteryValueReceived(device, batteryLevel);
        });
    }

    });
}

}

private boolean internalRefreshDeviceCache() {
    BluetoothGatt gatt = this.bluetoothGatt;
    if (gatt == null) {
        return false;
    } else {
        this.log(2, () -> {
            return "Refreshing device cache...";
        });
        this.log(3, () -> {
            return "gatt.refresh() (hidden)";
        });

        try {
            Method refresh = gatt.getClass().getMethod("refresh");
            return refresh.invoke(gatt) == Boolean.TRUE;
        } catch (Exception var3) {
            Log.w("BleManager", "An exception occurred while refreshing device",
var3);

            this.log(5, () -> {
                return "gatt.refresh() method not found";
            });
            return false;
        }
    }
}

private void enqueueFirst(@NonNull Request request) {
    RequestQueue rq = this.requestQueue;
    if (rq == null) {
        Deque<Request> queue = this.initialization && this.initQueue != null ?
this.initQueue : this.taskQueue;
        queue.addFirst(request);
    } else {
        rq.addFirst(request);
    }

    request.enqueue = true;
    this.operationInProgress = false;
}

final void enqueue(@NonNull Request request) {
    if (!request.enqueue) {
        Deque<Request> queue = this.initialization && this.initQueue != null ?
this.initQueue : this.taskQueue;

```

```

        queue.add(request);
        request.enqueued = true;
    }

    this.nextRequest(false);
}

final void cancelQueue() {
    this.taskQueue.clear();
    this.initQueue = null;
    this.initialization = false;
    BluetoothDevice device = this.bluetoothDevice;
    if (device != null) {
        if (this.operationInProgress) {
            this.cancelCurrent();
        }

        if (this.connectRequest != null) {
            this.connectRequest.notifyFail(device, -7);
            this.connectRequest = null;
            this.internalDisconnect(5);
        }
    }
}

final void cancelCurrent() {
    BluetoothDevice device = this.bluetoothDevice;
    if (device != null) {
        this.log(5, () -> {
            return "Request cancelled";
        });
        if (this.request instanceof TimeoutableRequest) {
            this.request.notifyFail(device, -7);
        }

        if (this.awaitingRequest != null) {
            this.awaitingRequest.notifyFail(device, -7);
            this.awaitingRequest = null;
        }

        if (this.requestQueue instanceof ReliableWriteRequest) {
            this.requestQueue.cancelQueue();
        } else if (this.requestQueue != null) {
            this.requestQueue.notifyFail(device, -7);
            this.requestQueue = null;
        }

        this.nextRequest(this.request == null || this.request.finished);
    }
}

final void onRequestTimeout(@NonNull BluetoothDevice device, @NonNull
TimeoutableRequest tr) {

```

```

        if (tr instanceof SleepRequest) {
            tr.notifySuccess(device);
        } else {
            this.log(5, () -> {
                return "Request timed out";
            });
        }

        if (this.request instanceof TimeoutableRequest) {
            this.request.notifyFail(device, -5);
        }

        if (this.awaitingRequest != null) {
            this.awaitingRequest.notifyFail(device, -5);
            this.awaitingRequest = null;
        }

        tr.notifyFail(device, -5);
        if (tr.type == Request.Type.CONNECT) {
            this.connectRequest = null;
            this.internalDisconnect(10);
        } else if (tr.type == Request.Type.DISCONNECT) {
            this.close();
        } else {
            this.nextRequest(this.request == null || this.request.finished);
        }
    }

    public void post(@NonNull Runnable r) {
        this.handler.post(r);
    }

    public void postDelayed(@NonNull Runnable r, long delayMillis) {
        this.handler.postDelayed(r, delayMillis);
    }

    public void removeCallbacks(@NonNull Runnable r) {
        this.handler.removeCallbacks(r);
    }

    /** @deprecated */
    @Deprecated
    private void postCallback(@NonNull BleManagerHandler.CallbackRunnable r) {
        BleManagerCallbacks callbacks = this.manager.callbacks;
        if (callbacks != null) {
            this.post(() -> {
                r.run(callbacks);
            });
        }
    }

    private void postBondingStateChange(@NonNull
    BleManagerHandler.BondingObserverRunnable r) {

```

```

        BondingObserver observer = this.manager.bondingObserver;
        if (observer != null) {
            this.post(() -> {
                r.run(observer);
            });
        }
    }

    private void postConnectionStateChange(@NonNull
BleManagerHandler.ConnectionObserverRunnable r) {
        ConnectionObserver observer = this.manager.connectionObserver;
        if (observer != null) {
            this.post(() -> {
                r.run(observer);
            });
        }
    }

    final int getConnectionState() {
        return this.connectionState;
    }

    final boolean isConnected() {
        return this.connected;
    }

    /** @deprecated */
    @Deprecated
    final int getBatteryValue() {
        return this.batteryValue;
    }

    final boolean isReady() {
        return this.ready;
    }

    final boolean isReliableWriteInProgress() {
        return this.reliableWriteInProgress;
    }

    final int getMtu() {
        return this.mtu;
    }

    final void overrideMtu(@IntRange(from = 23L,to = 517L) int mtu) {
        if (VERSION.SDK_INT >= 21) {
            this.mtu = mtu;
        }
    }

    /** @deprecated */

```

```

    protected abstract boolean isRequiredServiceSupported(@NonNull BluetoothGatt
var1);

    /** @deprecated */
    @Deprecated
    protected boolean isOptionalServiceSupported(@NonNull BluetoothGatt gatt) {
        return false;
    }

    /** @deprecated */
    @Deprecated
    protected Deque<Request> initGatt(@NonNull BluetoothGatt gatt) {
        return null;
    }

    /** @deprecated */
    @Deprecated
    protected void initialize() {
    }

    /** @deprecated */
    @Deprecated
    protected void onServerReady(@NonNull BluetoothGattServer server) {
    }

    /** @deprecated */
    @Deprecated
    protected void onDeviceReady() {
    }

    /** @deprecated */
    @Deprecated
    protected void onManagerReady() {
    }

    /** @deprecated */
    @Deprecated
    protected void onDeviceDisconnected() {
    }

    /** @deprecated */
    @Deprecated
    protected abstract void onServicesInvalidated();

    private void notifyDeviceDisconnected(@NonNull BluetoothDevice device, int
status) {
        boolean wasConnected = this.connected;
        this.connected = false;
        this.ready = false;
        this.servicesDiscovered = false;
        this.serviceDiscoveryRequested = false;
        this.deviceNotSupported = false;
        this.mtu = 23;
        this.interval = this.latency = this.timeout = 0;
    }

```

```

this.connectionState = 0;
this.checkCondition();
if (!wasConnected) {
    this.log(5, () -> {
        return "Connection attempt timed out";
    });
    this.close();
    this.postCallback((c) -> {
        c.onDeviceDisconnected(device);
    });
    this.postConnectionStateChange((o) -> {
        o.onDeviceFailedToConnect(device, status);
    });
} else if (this.userDisconnected) {
    this.log(4, () -> {
        return "Disconnected";
    });
    Request request = this.request;
    if (request == null || request.type != Request.Type.REMOVE_BOND) {
        this.close();
    }

    this.postCallback((c) -> {
        c.onDeviceDisconnected(device);
    });
    this.postConnectionStateChange((o) -> {
        o.onDeviceDisconnected(device, status);
    });
    if (request != null && request.type == Request.Type.DISCONNECT) {
        request.notifySuccess(device);
        this.request = null;
    }
} else {
    this.log(5, () -> {
        return "Connection lost";
    });
    this.postCallback((c) -> {
        c.onLinkLossOccurred(device);
    });
    int reason = status == 2 ? 2 : 3;
    this.postConnectionStateChange((o) -> {
        o.onDeviceDisconnected(device, reason);
    });
}

Iterator var7 = this.valueChangedCallbacks.values().iterator();

while(var7.hasNext()) {
    ValueChangedCallback callback = (ValueChangedCallback)var7.next();
    callback.notifyClosed();
}

this.valueChangedCallbacks.clear();
this.dataProviders.clear();

```



```

        this.batteryLevelNotificationCallback = null;
        this.batteryValue = -1;
        this.manager.onServicesInvalidated();
        this.onDeviceDisconnected();
    }

    /** @deprecated */
    @Deprecated
    protected void onCharacteristicRead(@NonNull BluetoothGatt gatt, @NonNull
BluetoothGattCharacteristic characteristic) {
    }

    /** @deprecated */
    @Deprecated
    protected void onCharacteristicWrite(@NonNull BluetoothGatt gatt, @NonNull
BluetoothGattCharacteristic characteristic) {
    }

    /** @deprecated */
    @Deprecated
    protected void onDescriptorRead(@NonNull BluetoothGatt gatt, @NonNull
BluetoothGattDescriptor descriptor) {
    }

    /** @deprecated */
    @Deprecated
    protected void onDescriptorWrite(@NonNull BluetoothGatt gatt, @NonNull
BluetoothGattDescriptor descriptor) {
    }

    /** @deprecated */
    @Deprecated
    protected void onBatteryValueReceived(@NonNull BluetoothGatt gatt,
@IntRange(from = 0L,to = 100L) int value) {
    }

    /** @deprecated */
    @Deprecated
    protected void onCharacteristicNotified(@NonNull BluetoothGatt gatt, @NonNull
BluetoothGattCharacteristic characteristic) {
    }

    /** @deprecated */
    @Deprecated
    protected void onCharacteristicIndicated(@NonNull BluetoothGatt gatt,
@NonNull BluetoothGattCharacteristic characteristic) {
    }

    /** @deprecated */
    @Deprecated
    protected void onMtuChanged(@NonNull BluetoothGatt gatt, @IntRange(from =
23L,to = 517L) int mtu) {
    }

```

```

/** @deprecated */
@Deprecated
@TargetApi(26)
protected void onConnectionUpdated(@NonNull BluetoothGatt gatt,
@IntRange(from = 6L,to = 3200L) int interval, @IntRange(from = 0L,to = 499L) int
latency, @IntRange(from = 10L,to = 3200L) int timeout) {
}

private void onError(BluetoothDevice device, String message, int errorCode) {
    this.log(6, () -> {
        return "Error (0x" + Integer.toHexString(errorCode) + "): " +
GattError.parse(errorCode);
    });
    this.postCallback((c) -> {
        c.onError(device, message, errorCode);
    });
}

private int mapDisconnectStatusToReason(int status) {
    switch(status) {
        case 0:
            return 0;
        case 8:
            return 10;
        case 19:
            return 2;
        case 22:
            return 1;
        default:
            return -1;
    }
}

final void onCharacteristicReadRequest(@NonNull BluetoothGattServer server,
@NonNull BluetoothDevice device, int requestId, int offset, @NonNull
BluetoothGattCharacteristic characteristic) {
    this.log(3, () -> {
        return "[Server callback] Read request for characteristic " +
characteristic.getUuid() + " (requestId=" + requestId + ", offset: " + offset +
")";
    });
    if (offset == 0) {
        this.log(4, () -> {
            return "[Server] READ request for characteristic " +
characteristic.getUuid() + " received";
        });
    }

    DataProvider dataProvider =
(DataProvider)this.dataProviders.get(characteristic);
    byte[] data = offset == 0 && dataProvider != null ?
dataProvider.getData(device) : null;
    if (data != null) {
        this.assign(characteristic, data);
    }
}

```

```

    } else {
        data = this.characteristicValues != null &&
this.characteristicValues.containsKey(characteristic) ?
(byte[])this.characteristicValues.get(characteristic) :
characteristic.getValue();
    }

    WaitForReadRequest waitForReadRequest = null;
    if (this.awaitingRequest instanceof WaitForReadRequest &&
this.awaitingRequest.characteristic == characteristic &&
!this.awaitingRequest.isTriggerPending()) {
        waitForReadRequest = (WaitForReadRequest)this.awaitingRequest;
        waitForReadRequest.setDataIfNull(data);
        data = waitForReadRequest.getData(this.mtu);
    }

    if (data != null && data.length > this.mtu - 1) {
        data = Bytes.copy(data, offset, this.mtu - 1);
    }

    this.sendResponse(server, device, 0, requestId, offset, data);
    if (waitForReadRequest != null) {
        waitForReadRequest.notifyPacketRead(device, data);
        if (!waitForReadRequest.hasMore() && (data == null || data.length <
this.mtu - 1)) {
            this.log(4, () -> {
                return "Wait for read complete";
            });
            waitForReadRequest.notifySuccess(device);
            this.awaitingRequest = null;
            this.nextRequest(true);
        }
    } else if (this.checkCondition()) {
        this.nextRequest(true);
    }
}
}

```

```

final void onCharacteristicWriteRequest(@NonNull BluetoothGattServer server,
@NonNull BluetoothDevice device, int requestId, @NonNull
BluetoothGattCharacteristic characteristic, boolean preparedWrite, boolean
responseNeeded, int offset, @NonNull byte[] value) {
    this.log(3, () -> {
        return "[Server callback] Write " + (responseNeeded ? "request" :
"command") + " to characteristic " + characteristic.getUuid() + " (requestId=" +
requestId + ", prepareWrite=" + preparedWrite + ", responseNeeded=" +
responseNeeded + ", offset: " + offset + ", value=" +
ParserUtils.parseDebug(value) + ")";
    });
    if (offset == 0) {
        this.log(4, () -> {
            String type = responseNeeded ? "WRITE REQUEST" : "WRITE COMMAND";
            String option = preparedWrite ? "Prepare " : "";
            return "[Server] " + option + type + " for characteristic " +

```

```

characteristic.getUuid() + " received, value: " + ParserUtils.parse(value);
    });
}

if (responseNeeded) {
    this.sendResponse(server, device, 0, requestId, offset, value);
}

if (preparedWrite) {
    if (this.preparedValues == null) {
        this.preparedValues = new LinkedList();
    }

    if (offset == 0) {
        this.preparedValues.offer(new Pair(characteristic, value));
    } else {
        Pair<Object, byte[]> last = (Pair)this.preparedValues.peekLast();
        if (last != null && characteristic.equals(last.first)) {
            this.preparedValues.pollLast();
            this.preparedValues.offer(new Pair(characteristic,
Bytes.concat((byte[])last.second, value, offset)));
        } else {
            this.prepareError = 7;
        }
    }
} else if (this.assignAndNotify(device, characteristic, value) ||
this.checkCondition()) {
    this.nextRequest(true);
}

}

final void onDescriptorReadRequest(@NonNull BluetoothGattServer server,
@NonNull BluetoothDevice device, int requestId, int offset, @NonNull
BluetoothGattDescriptor descriptor) {
    this.log(3, () -> {
        return "[Server callback] Read request for descriptor " +
descriptor.getUuid() + " (requestId=" + requestId + ", offset: " + offset + ")";
    });
    if (offset == 0) {
        this.log(4, () -> {
            return "[Server] READ request for descriptor " +
descriptor.getUuid() + " received";
        });
    }

    DataProvider dataProvider =
(DataProvider)this.dataProviders.get(descriptor);
    byte[] data = offset == 0 && dataProvider != null ?
dataProvider.getData(device) : null;
    if (data != null) {
        this.assign(descriptor, data);
    } else {
        data = this.descriptorValues != null &&

```

```

this.descriptorValues.containsKey(descriptor) ?
(byte[])this.descriptorValues.get(descriptor) : descriptor.getValue();
    }

    WaitForReadRequest waitForReadRequest = null;
    if (this.awaitingRequest instanceof WaitForReadRequest &&
this.awaitingRequest.descriptor == descriptor &&
!this.awaitingRequest.isTriggerPending()) {
        waitForReadRequest = (WaitForReadRequest)this.awaitingRequest;
        waitForReadRequest.setDataIfNull(data);
        data = waitForReadRequest.getData(this.mtu);
    }

    if (data != null && data.length > this.mtu - 1) {
        data = Bytes.copy(data, offset, this.mtu - 1);
    }

    this.sendResponse(server, device, 0, requestId, offset, data);
    if (waitForReadRequest != null) {
        waitForReadRequest.notifyPacketRead(device, data);
        if (!waitForReadRequest.hasMore() && (data == null || data.length <
this.mtu - 1)) {
            waitForReadRequest.notifySuccess(device);
            this.awaitingRequest = null;
            this.nextRequest(true);
        }
    } else if (this.checkCondition()) {
        this.nextRequest(true);
    }
}

}

final void onDescriptorWriteRequest(@NonNull BluetoothGattServer server,
@NonNull BluetoothDevice device, int requestId, @NonNull BluetoothGattDescriptor
descriptor, boolean preparedWrite, boolean responseNeeded, int offset, @NonNull
byte[] value) {
    this.log(3, () -> {
        return "[Server callback] Write " + (responseNeeded ? "request" :
"command") + " to descriptor " + descriptor.getUuid() + " (requestId=" +
requestId + ", prepareWrite=" + preparedWrite + ", responseNeeded=" +
responseNeeded + ", offset: " + offset + ", value=" +
ParserUtils.parseDebug(value) + ")";
    });
    if (offset == 0) {
        this.log(4, () -> {
            String type = responseNeeded ? "WRITE REQUEST" : "WRITE COMMAND";
            String option = preparedWrite ? "Prepare " : "";
            return "[Server] " + option + type + " request for descriptor " +
descriptor.getUuid() + " received, value: " + ParserUtils.parse(value);
        });
    }

    if (responseNeeded) {
        this.sendResponse(server, device, 0, requestId, offset, value);
    }
}

```

```

    }

    if (preparedWrite) {
        if (this.preparedValues == null) {
            this.preparedValues = new LinkedList();
        }

        if (offset == 0) {
            this.preparedValues.offer(new Pair(descriptor, value));
        } else {
            Pair<Object, byte[]> last = (Pair)this.preparedValues.peekLast();
            if (last != null && descriptor.equals(last.first)) {
                this.preparedValues.pollLast();
                this.preparedValues.offer(new Pair(descriptor,
Bytes.concat((byte[])last.second, value, offset)));
            } else {
                this.prepareError = 7;
            }
        }
    } else if (this.assignAndNotify(device, descriptor, value) ||
this.checkCondition()) {
        this.nextRequest(true);
    }
}

final void onExecuteWrite(@NonNull BluetoothGattServer server, @NonNull
BluetoothDevice device, int requestId, boolean execute) {
    this.log(3, () -> {
        return "[Server callback] Execute write request (requestId=" +
requestId + ", execute=" + execute + ")";
    });
    if (execute) {
        Deque<Pair<Object, byte[]>> values = this.preparedValues;
        this.log(4, () -> {
            return "[Server] Execute write request received";
        });
        this.preparedValues = null;
        if (this.prepareError != 0) {
            this.sendResponse(server, device, this.prepareError, requestId, 0,
(byte[])null);
            this.prepareError = 0;
            return;
        }

        this.sendResponse(server, device, 0, requestId, 0, (byte[])null);
        if (values == null || values.isEmpty()) {
            return;
        }

        boolean startNextRequest = false;
        Iterator var7 = values.iterator();

        while(true) {

```

```

        while(var7.hasNext()) {
            Pair<Object, byte[]> value = (Pair)var7.next();
            if (value.first instanceof BluetoothGattCharacteristic) {
                BluetoothGattCharacteristic characteristic =
                (BluetoothGattCharacteristic)value.first;
                startNextRequest = this.assignAndNotify(device,
                characteristic, (byte[])value.second) || startNextRequest;
            } else if (value.first instanceof BluetoothGattDescriptor) {
                BluetoothGattDescriptor descriptor =
                (BluetoothGattDescriptor)value.first;
                startNextRequest = this.assignAndNotify(device, descriptor,
                (byte[])value.second) || startNextRequest;
            }
        }

        if (this.checkCondition() || startNextRequest) {
            this.nextRequest(true);
        }
        break;
    }
} else {
    this.log(4, () -> {
        return "[Server] Cancel write request received";
    });
    this.preparedValues = null;
    this.sendResponse(server, device, 0, requestId, 0, (byte[])null);
}
}
}

```

```

final void onNotificationSent(@NonNull BluetoothGattServer server, @NonNull
BluetoothDevice device, int status) {
    this.log(3, () -> {
        return "[Server callback] Notification sent (status=" + status + ")";
    });
    if (status == 0) {
        this.notifyNotificationSent(device);
    } else {
        Log.e("BleManager", "onNotificationSent error " + status);
        if (this.request instanceof WriteRequest) {
            this.request.notifyFail(device, status);
        }

        this.awaitingRequest = null;
        this.onError(device, "Error on sending notification/indication",
status);
    }

    this.checkCondition();
    this.nextRequest(true);
}

```

```

@RequiresApi(
    api = 22

```

```

    )
    final void onMtuChanged(@NonNull BluetoothGattServer server, @NonNull
BluetoothDevice device, int mtu) {
        this.log(4, () -> {
            return "[Server] MTU changed to: " + mtu;
        });
        this.mtu = mtu;
        this.checkCondition();
        this.nextRequest(false);
    }

    private void notifyNotificationSent(@NonNull BluetoothDevice device) {
        if (this.request instanceof WriteRequest) {
            WriteRequest wr = (WriteRequest)this.request;
            switch(wr.type) {
                case NOTIFY:
                    this.log(4, () -> {
                        return "[Server] Notification sent";
                    });
                    break;
                case INDICATE:
                    this.log(4, () -> {
                        return "[Server] Indication sent";
                    });
            }

            wr.notifyPacketSent(device, wr.characteristic.getValue());
            if (wr.hasMore()) {
                this.enqueueFirst(wr);
            } else {
                wr.notifySuccess(device);
            }
        }
    }

    private void assign(@NonNull BluetoothGattCharacteristic characteristic,
@NonNull byte[] value) {
        boolean isShared = this.characteristicValues == null ||
!this.characteristicValues.containsKey(characteristic);
        if (isShared) {
            characteristic.setValue(value);
        } else {
            this.characteristicValues.put(characteristic, value);
        }
    }

    private boolean assignAndNotify(@NonNull BluetoothDevice device, @NonNull
BluetoothGattCharacteristic characteristic, @NonNull byte[] value) {
        this.assign(characteristic, value);
        ValueChangedCallback callback;
        if ((callback =
(ValueChangedCallback)this.valueChangedCallbacks.get(characteristic)) != null) {

```



```

        callback.notifyValueChanged(device, value);
    }

    if (this.awaitingRequest instanceof WaitForValueChangedRequest &&
this.awaitingRequest.characteristic == characteristic &&
!this.awaitingRequest.isTriggerPending()) {
        WaitForValueChangedRequest waitForWrite =
(WaitForValueChangedRequest)this.awaitingRequest;
        if (waitForWrite.matches(value)) {
            waitForWrite.notifyValueChanged(device, value);
            if (waitForWrite.isComplete()) {
                waitForWrite.notifySuccess(device);
                this.awaitingRequest = null;
                return waitForWrite.isTriggerCompleteOrNull();
            }
        }
    }

    return false;
}

private void assign(@NonNull BluetoothGattDescriptor descriptor, @NonNull
byte[] value) {
    boolean isShared = this.descriptorValues == null ||
!this.descriptorValues.containsKey(descriptor);
    if (isShared) {
        descriptor.setValue(value);
    } else {
        this.descriptorValues.put(descriptor, value);
    }
}

private boolean assignAndNotify(@NonNull BluetoothDevice device, @NonNull
BluetoothGattDescriptor descriptor, @NonNull byte[] value) {
    this.assign(descriptor, value);
    ValueChangedCallback callback;
    if ((callback =
(ValueChangedCallback)this.valueChangedCallbacks.get(descriptor)) != null) {
        callback.notifyValueChanged(device, value);
    }

    if (this.awaitingRequest instanceof WaitForValueChangedRequest &&
this.awaitingRequest.descriptor == descriptor &&
!this.awaitingRequest.isTriggerPending()) {
        WaitForValueChangedRequest waitForWrite =
(WaitForValueChangedRequest)this.awaitingRequest;
        if (waitForWrite.matches(value)) {
            waitForWrite.notifyValueChanged(device, value);
            if (waitForWrite.isComplete()) {
                waitForWrite.notifySuccess(device);
                this.awaitingRequest = null;
                return waitForWrite.isTriggerCompleteOrNull();
            }
        }
    }
}

```

```

        }
    }

    return false;
}

private void sendResponse(@NonNull BluetoothGattServer server, @NonNull
BluetoothDevice device, int status, int requestId, int offset, @Nullable byte[]
response) {
    String msg;
    switch(status) {
        case 0:
            msg = "GATT_SUCCESS";
            break;
        case 6:
            msg = "GATT_REQUEST_NOT_SUPPORTED";
            break;
        case 7:
            msg = "GATT_INVALID_OFFSET";
            break;
        default:
            throw new InvalidParameterException();
    }

    this.log(3, () -> {
        return "server.sendResponse(" + msg + ", offset=" + offset + ", value="
+ ParserUtils.parseDebug(response) + ")";
    });
    server.sendResponse(device, requestId, status, offset, response);
    this.log(2, () -> {
        return "[Server] Response sent";
    });
}

private boolean checkCondition() {
    if (this.awaitingRequest instanceof ConditionalWaitRequest) {
        ConditionalWaitRequest<?> cwr =
(ConditionalWaitRequest)this.awaitingRequest;
        if (cwr.isFulfilled()) {
            this.log(4, () -> {
                return "Condition fulfilled";
            });
            cwr.notifySuccess(this.bluetoothDevice);
            this.awaitingRequest = null;
            return true;
        }
    }

    return false;
}

@SuppressWarnings({"MissingPermission"})
private synchronized void nextRequest(boolean force) {
    if (force && this.operationInProgress) {

```

```

        this.operationInProgress = this.awaitingRequest != null;
    }

    if (!this.operationInProgress) {
        BluetoothDevice bluetoothDevice = this.bluetoothDevice;
        Request request = null;

        try {
            if (this.requestQueue != null) {
                if (this.requestQueue.hasMore()) {
                    request = this.requestQueue.getNext().setRequestHandler(this);
                } else {
                    if (this.requestQueue instanceof ReliableWriteRequest) {
                        ReliableWriteRequest rwr =
(ReliableWriteRequest)this.requestQueue;
                        if (rwr.isCancelled()) {
                            this.requestQueue.notifyFail(bluetoothDevice, -7);
                        }
                    }

                    this.requestQueue.notifySuccess(bluetoothDevice);
                    this.requestQueue = null;
                }
            }

            if (request == null) {
                request = this.initQueue != null ? (Request)this.initQueue.poll()
: null;
            }
        } catch (Exception var9) {
            request = null;
        }

        if (request == null) {
            if (this.initQueue != null) {
                this.initQueue = null;
                this.operationInProgress = true;
                this.ready = true;
                this.manager.onDeviceReady();
                if (bluetoothDevice != null) {
                    this.postCallback((c) -> {
                        c.onDeviceReady(bluetoothDevice);
                    });
                    this.postConnectionStateChange((o) -> {
                        o.onDeviceReady(bluetoothDevice);
                    });
                }

                if (this.connectRequest != null) {
                    this.connectRequest.notifySuccess(this.connectRequest.getDevice());
                    this.connectRequest = null;
                }
            }
        }
    }

```

```

    try {
        request = (Request)this.taskQueue.remove();
    } catch (Exception var8) {
        this.operationInProgress = false;
        this.request = null;
        this.manager.onManagerReady();
        return;
    }
}

if (request.finished) {
    this.nextRequest(false);
} else {
    boolean result = false;
    this.operationInProgress = true;
    this.request = request;
    if (request instanceof AwaitingRequest) {
        AwaitingRequest<?> r = (AwaitingRequest)request;
        int requiredProperty = 0;
        switch(request.type) {
            case WAIT_FOR_NOTIFICATION:
                requiredProperty = 16;
                break;
            case WAIT_FOR_INDICATION:
                requiredProperty = 32;
                break;
            case WAIT_FOR_READ:
                requiredProperty = 2;
                break;
            case WAIT_FOR_WRITE:
                requiredProperty = 76;
        }

        result = this.connected && bluetoothDevice != null &&
(r.characteristic == null || (r.characteristic.getProperties() &
requiredProperty) != 0);
        if (result) {
            if (r instanceof ConditionalWaitRequest) {
                ConditionalWaitRequest<?> cwr = (ConditionalWaitRequest)r;
                this.log(2, () -> {
                    return "Waiting for fulfillment of condition...";
                });
                if (cwr.isFulfilled()) {
                    cwr.notifyStarted(bluetoothDevice);
                    this.log(4, () -> {
                        return "Condition fulfilled";
                    });
                    cwr.notifySuccess(bluetoothDevice);
                    this.nextRequest(true);
                    return;
                }
            }
        }
    }
}

```

```

        if (r instanceof WaitForReadRequest) {
            this.log(2, () -> {
                return "Waiting for read request...";
            });
        }

        if (r instanceof WaitForValueChangedRequest) {
            this.log(2, () -> {
                return "Waiting for value change...";
            });
        }

        this.awaitingRequest = r;
        if (r.getTrigger() != null) {
            r.notifyStarted(bluetoothDevice);
            this.request = request = r.getTrigger();
        }
    }

    ConnectRequest cr;
    if (request.type == Request.Type.CONNECT) {
        cr = (ConnectRequest)request;
        cr.notifyStarted(cr.getDevice());
    } else {
        if (bluetoothDevice == null) {
            request.notifyInvalidRequest();
            this.awaitingRequest = null;
            this.nextRequest(true);
            return;
        }

        request.notifyStarted(bluetoothDevice);
    }

    PhyRequest pr;
    SetValueRequest svr;
    WriteRequest wr;
    switch(request.type) {
        case NOTIFY:
        case INDICATE:
            wr = (WriteRequest)request;
            byte[] data = wr.getData(this.mtu);
            if (wr.characteristic != null) {
                wr.characteristic.setValue(data);
                if (this.characteristicValues != null &&
this.characteristicValues.containsKey(wr.characteristic)) {
                    this.characteristicValues.put(wr.characteristic, data);
                }
            }

            result = this.internalSendNotification(wr.characteristic,
request.type == Request.Type.INDICATE, data);
            case WAIT_FOR_NOTIFICATION:

```

```

case WAIT_FOR_INDICATION:
case WAIT_FOR_READ:
case WAIT_FOR_WRITE:
default:
    break;
case CONNECT:
    cr = (ConnectRequest)request;
    this.connectRequest = cr;
    this.request = null;
    result = this.internalConnect(cr.getDevice(), cr);
    break;
case DISCONNECT:
    result = this.internalDisconnect(0);
    break;
case ENSURE_BOND:
    result = this.internalCreateBond(true);
    break;
case CREATE_BOND:
    result = this.internalCreateBond(false);
    break;
case REMOVE_BOND:
    result = this.internalRemoveBond();
    break;
case SET:
    this.requestQueue = (RequestQueue)request;
    this.nextRequest(true);
    return;
case READ:
    result = this.internalReadCharacteristic(request.characteristic);
    break;
case WRITE:
    wr = (WriteRequest)request;
    result = this.internalWriteCharacteristic(wr.characteristic,
wr.getData(this.mtu), wr.getWriteType());
    break;
case READ_DESCRIPTOR:
    result = this.internalReadDescriptor(request.descriptor);
    break;
case WRITE_DESCRIPTOR:
    wr = (WriteRequest)request;
    result = this.internalWriteDescriptor(wr.descriptor,
wr.getData(this.mtu));
    break;
case SET_VALUE:
    svr = (SetValueRequest)request;
    if (svr.characteristic != null) {
        if (this.characteristicValues != null &&
this.characteristicValues.containsKey(svr.characteristic)) {
            this.characteristicValues.put(svr.characteristic,
svr.getData(this.mtu));
        } else {
            svr.characteristic.setValue(svr.getData(this.mtu));
        }
    }

```

```

        result = true;
        svr.notifySuccess(bluetoothDevice);
        this.nextRequest(true);
    }
    break;
case SET_DESCRIPTOR_VALUE:
    svr = (SetValueRequest)request;
    if (svr.descriptor != null) {
        if (this.descriptorValues != null &&
this.descriptorValues.containsKey(svr.descriptor)) {
            this.descriptorValues.put(svr.descriptor,
svr.getData(this.mtu));
        } else {
            svr.descriptor.setValue(svr.getData(this.mtu));
        }

        result = true;
        svr.notifySuccess(bluetoothDevice);
        this.nextRequest(true);
    }
    break;
case BEGIN_RELIABLE_WRITE:
    result = this.internalBeginReliableWrite();
    if (result) {
        this.request.notifySuccess(bluetoothDevice);
        this.nextRequest(true);
        return;
    }
    break;
case EXECUTE_RELIABLE_WRITE:
    result = this.internalExecuteReliableWrite();
    break;
case ABORT_RELIABLE_WRITE:
    result = this.internalAbortReliableWrite();
    break;
case ENABLE_NOTIFICATIONS:
    result =
this.internalEnableNotifications(request.characteristic);
    break;
case ENABLE_INDICATIONS:
    result = this.internalEnableIndications(request.characteristic);
    break;
case DISABLE_NOTIFICATIONS:
    result =
this.internalDisableNotifications(request.characteristic);
    break;
case DISABLE_INDICATIONS:
    result = this.internalDisableIndications(request.characteristic);
    break;
case READ_BATTERY_LEVEL:
    result = this.internalReadBatteryLevel();
    break;
case ENABLE_BATTERY_LEVEL_NOTIFICATIONS:
    result = this.internalSetBatteryNotifications(true);

```

```

        break;
    case DISABLE_BATTERY_LEVEL_NOTIFICATIONS:
        result = this.internalSetBatteryNotifications(false);
        break;
    case ENABLE_SERVICE_CHANGED_INDICATIONS:
        result = this.ensureServiceChangedEnabled();
        break;
    case REQUEST_MTU:
        MtuRequest mr = (MtuRequest)request;
        if (this.mtu != mr.getRequiredMtu() && VERSION.SDK_INT >= 21) {
            result = this.internalRequestMtu(mr.getRequiredMtu());
        } else {
            result = this.connected;
            if (result) {
                mr.notifyMtuChanged(bluetoothDevice, this.mtu);
                mr.notifySuccess(bluetoothDevice);
                this.nextRequest(true);
                return;
            }
        }
        break;
    case REQUEST_CONNECTION_PRIORITY:
        ConnectionPriorityRequest cpr =
        (ConnectionPriorityRequest)request;
        this.connectionPriorityOperationInProgress = VERSION.SDK_INT >=
        26;

        if (VERSION.SDK_INT >= 21) {
            result =
this.internalRequestConnectionPriority(cpr.getRequiredPriority());
            if (result) {
                this.postDelayed(() -> {
                    if (cpr.notifySuccess(bluetoothDevice)) {
                        this.connectionPriorityOperationInProgress = false;
                        this.nextRequest(true);
                    }
                }, 200L);
            } else {
                this.connectionPriorityOperationInProgress = false;
            }
        }
        break;
    case SET_PREFERRED_PHY:
        pr = (PhyRequest)request;
        if (VERSION.SDK_INT >= 26) {
            result = this.internalSetPreferredPhy(pr.getPreferredTxPhy(),
pr.getPreferredRxPhy(), pr.getPreferredPhyOptions());
            if (VERSION.SDK_INT == 33) {
                this.handler.postDelayed(() -> {
                    if (!pr.finished) {
                        this.log(5, () -> {
                            return "Callback not received in 1000 ms";
                        });
                    }
                }, 1000L);
            }
        }
        this.internalReadPhy();

```



```

        }

        }, 1000L);
    }
} else {
    result = this.connected;
    if (result) {
        pr.notifyLegacyPhy(bluetoothDevice);
        pr.notifySuccess(bluetoothDevice);
        this.nextRequest(true);
        return;
    }
}
break;
case READ_PHY:
    pr = (PhyRequest)request;
    if (VERSION.SDK_INT >= 26) {
        result = this.internalReadPhy();
    } else {
        result = this.connected;
        if (result) {
            pr.notifyLegacyPhy(bluetoothDevice);
            pr.notifySuccess(bluetoothDevice);
            this.nextRequest(true);
            return;
        }
    }
    break;
case READ_RSSI:
    result = this.internalReadRssi();
    if (result) {
        this.postDelayed(() -> {
            if (this.request == request) {
                request.notifyFail(bluetoothDevice, -5);
                this.nextRequest(true);
            }
        }, 1000L);
    }
    break;
case REFRESH_CACHE:
    result = this.internalRefreshDeviceCache();
    if (result) {
        this.postDelayed(() -> {
            this.log(4, () -> {
                return "Cache refreshed";
            });
            request.notifySuccess(bluetoothDevice);
            this.request = null;
            if (this.awaitingRequest != null) {
                this.awaitingRequest.notifyFail(bluetoothDevice, -3);
                this.awaitingRequest = null;
            }
        });
    }
}

```

```

        this.taskQueue.clear();
        this.initQueue = null;
        BluetoothGatt bluetoothGatt = this.bluetoothGatt;
        if (this.connected && bluetoothGatt != null) {
            this.manager.onServicesInvalidated();
            this.onDeviceDisconnected();
            this.serviceDiscoveryRequested = true;
            this.servicesDiscovered = false;
            this.log(2, () -> {
                return "Discovering Services...";
            });
            this.log(3, () -> {
                return "gatt.discoverServices()";
            });
            bluetoothGatt.discoverServices();
        }

        }, 200L);
    }
    break;
case SLEEP:
    SleepRequest sr = (SleepRequest)request;
    this.log(3, () -> {
        return "sleep(" + sr.timeout + ")";
    });
    result = true;
}

    if (!result && bluetoothDevice != null) {
        request.notifyFail(bluetoothDevice, this.connected ? -3 :
(BluetoothAdapter.getDefaultAdapter().isEnabled() ? -1 : -100));
        this.awaitingRequest = null;
        this.connectionPriorityOperationInProgress = false;
        this.nextRequest(true);
    }

    }
}

    private boolean isServiceChangedCCCD(@Nullable BluetoothGattDescriptor
descriptor) {
        return descriptor != null &&
BleManager.SERVICE_CHANGED_CHARACTERISTIC.equals(descriptor.getCharacteristic().
getUuid());
    }

    private boolean isServiceChangedCharacteristic(@Nullable
BluetoothGattCharacteristic characteristic) {
        return characteristic != null &&
BleManager.SERVICE_CHANGED_CHARACTERISTIC.equals(characteristic.getUuid());
    }

    /** @deprecated */

```

```

    @Deprecated
    private boolean isBatteryLevelCharacteristic(@Nullable
BluetoothGattCharacteristic characteristic) {
        return characteristic != null &&
BleManager.BATTERY_LEVEL_CHARACTERISTIC.equals(characteristic.getUuid());
    }

    private boolean isCCCD(@Nullable BluetoothGattDescriptor descriptor) {
        return descriptor != null &&
BleManager.CLIENT_CHARACTERISTIC_CONFIG_DESCRIPTOR_UUID.equals(descriptor.getUi
d());
    }

    private void log(int priority, @NonNull BleManagerHandler.Loggable message) {
        if (priority >= this.manager.getMinLogPriority()) {
            this.manager.log(priority, message.log());
        }
    }

    @FunctionalInterface
    private interface Loggable {
        String log();
    }

    private interface ConnectionObserverRunnable {
        void run(@NonNull ConnectionObserver var1);
    }

    private interface BondingObserverRunnable {
        void run(@NonNull BondingObserver var1);
    }

    /** @deprecated */
    @Deprecated
    private interface CallbackRunnable {
        void run(@NonNull BleManagerCallbacks var1);
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: BleServerManager.java

```

```

package no.nordicsemi.android.ble;

import android.annotation.SuppressLint;
import android.bluetooth.BluetoothDevice;
import android.bluetooth.BluetoothGattCharacteristic;
import android.bluetooth.BluetoothGattDescriptor;
import android.bluetooth.BluetoothGattServer;
import android.bluetooth.BluetoothGattServerCallback;
import android.bluetooth.BluetoothGattService;

```

```

import android.bluetooth.BluetoothManager;
import android.content.Context;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.annotation.RequiresApi;
import androidx.annotation.StringRes;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;
import java.util.NoSuchElementException;
import java.util.Queue;
import java.util.UUID;
import no.nordicsemi.android.ble.data.Data;
import no.nordicsemi.android.ble.observer.ServerObserver;
import no.nordicsemi.android.ble.utils.ILogger;

public abstract class BleServerManager implements ILogger {
    private static final UUID CHARACTERISTIC_EXTENDED_PROPERTIES_DESCRIPTOR_UUID
= UUID.fromString("00002900-0000-1000-8000-00805f9b34fb");
    private static final UUID CLIENT_USER_DESCRIPTION_DESCRIPTOR_UUID =
UUID.fromString("00002901-0000-1000-8000-00805f9b34fb");
    private static final UUID CLIENT_CHARACTERISTIC_CONFIG_DESCRIPTOR_UUID =
UUID.fromString("00002902-0000-1000-8000-00805f9b34fb");
    private BluetoothGattServer server;
    private final List<BleManager> managers = new ArrayList();
    private final Context context;
    private ServerObserver serverObserver;
    private Queue<BluetoothGattService> serverServices;
    private List<BluetoothGattCharacteristic> sharedCharacteristics;
    private List<BluetoothGattDescriptor> sharedDescriptors;
    private final BluetoothGattServerCallback gattServerCallback = new
BluetoothGattServerCallback() {
        @SuppressWarnings("MissingPermission")
        public void onServiceAdded(int status, @NonNull BluetoothGattService
service) {
            if (status == 0) {
                try {
                    BluetoothGattService nextService =
(BluetoothGattService)BleServerManager.this.serverServices.remove();
                    BleServerManager.this.server.addService(nextService);
                } catch (Exception var4) {
                    BleServerManager.this.log(4, "[Server] All services added
successfully");
                    if (BleServerManager.this.serverObserver != null) {
                        BleServerManager.this.serverObserver.onServerReady();
                    }

                    BleServerManager.this.serverServices = null;
                }
            } else {
                BleServerManager.this.log(6, "[Server] Adding service failed with
error " + status);
            }
        }
    }

```

```

    }

    public void onConnectionStateChange(@NonNull BluetoothDevice device, int
status, int newState) {
        if (status == 0 && newState == 2) {
            BleServerManager.this.log(4, "[Server] " + device.getAddress() + "
is now connected");
            if (BleServerManager.this.serverObserver != null) {
                BleServerManager.this.serverObserver.onDeviceConnectedToServer(device);
            }
        } else {
            if (status == 0) {
                BleServerManager.this.log(4, "[Server] " + device.getAddress() +
" is disconnected");
            } else {
                BleServerManager.this.log(5, "[Server] " + device.getAddress() +
" has disconnected connected with status: " + status);
            }

            if (BleServerManager.this.serverObserver != null) {
                BleServerManager.this.serverObserver.onDeviceDisconnectedFromServer(device);
            }
        }
    }

    public void onCharacteristicReadRequest(@NonNull BluetoothDevice device,
int requestId, int offset, @NonNull BluetoothGattCharacteristic characteristic)
{
        BleManagerHandler handler =
BleServerManager.this.getRequestHandler(device);
        if (handler != null) {
            handler.onCharacteristicReadRequest(BleServerManager.this.server,
device, requestId, offset, characteristic);
        }
    }

    public void onCharacteristicWriteRequest(@NonNull BluetoothDevice device,
int requestId, @NonNull BluetoothGattCharacteristic characteristic, boolean
preparedWrite, boolean responseNeeded, int offset, @NonNull byte[] value) {
        BleManagerHandler handler =
BleServerManager.this.getRequestHandler(device);
        if (handler != null) {
            handler.onCharacteristicWriteRequest(BleServerManager.this.server,
device, requestId, characteristic, preparedWrite, responseNeeded, offset,
value);
        }
    }
}

```

```

        public void onDescriptorReadRequest(@NonNull BluetoothDevice device, int
requestId, int offset, @NonNull BluetoothGattDescriptor descriptor) {
            BleManagerHandler handler =
BleServerManager.this.getRequestHandler(device);
            if (handler != null) {
                handler.onDescriptorReadRequest(BleServerManager.this.server,
device, requestId, offset, descriptor);
            }

        }

        public void onDescriptorWriteRequest(@NonNull BluetoothDevice device, int
requestId, @NonNull BluetoothGattDescriptor descriptor, boolean preparedWrite,
boolean responseNeeded, int offset, @NonNull byte[] value) {
            BleManagerHandler handler =
BleServerManager.this.getRequestHandler(device);
            if (handler != null) {
                handler.onDescriptorWriteRequest(BleServerManager.this.server,
device, requestId, descriptor, preparedWrite, responseNeeded, offset, value);
            }

        }

        public void onExecuteWrite(@NonNull BluetoothDevice device, int requestId,
boolean execute) {
            BleManagerHandler handler =
BleServerManager.this.getRequestHandler(device);
            if (handler != null) {
                handler.onExecuteWrite(BleServerManager.this.server, device,
requestId, execute);
            }

        }

        @RequiresApi(
            api = 21
        )
        public void onNotificationSent(@NonNull BluetoothDevice device, int
status) {
            BleManagerHandler handler =
BleServerManager.this.getRequestHandler(device);
            if (handler != null) {
                handler.onNotificationSent(BleServerManager.this.server, device,
status);
            }

        }

        @RequiresApi(
            api = 22
        )
        public void onMtuChanged(@NonNull BluetoothDevice device, int mtu) {
            BleManagerHandler handler =
BleServerManager.this.getRequestHandler(device);

```

```

        if (handler != null) {
            handler.onMtuChanged(BleServerManager.this.server, device, mtu);
        }
    }
};

public BleServerManager(@NonNull Context context) {
    this.context = context;
}

@SuppressLint("MissingPermission")
public final boolean open() {
    if (this.server != null) {
        return true;
    } else {
        this.serverServices = new LinkedList(this.initializeServer());
        BluetoothManager bm =
(BluetoothManager)this.context.getSystemService("bluetooth");
        if (bm != null) {
            this.server = bm.openGattServer(this.context,
this.gattServerCallback);
        }

        if (this.server != null) {
            this.log(4, "[Server] Server started successfully");

            try {
                BluetoothGattService service =
(BluetoothGattService)this.serverServices.remove();
                this.server.addService(service);
            } catch (NoSuchElementException var3) {
                if (this.serverObserver != null) {
                    this.serverObserver.onServerReady();
                }
            } catch (Exception var4) {
                this.close();
                return false;
            }

            return true;
        } else {
            this.log(5, "GATT server initialization failed");
            this.serverServices = null;
            return false;
        }
    }
}

@SuppressLint("MissingPermission")
public final void close() {
    if (this.server != null) {
        this.server.close();
        this.server = null;
    }
}

```

```

    }

    this.serverServices = null;
    Iterator var1 = this.managers.iterator();

    while(var1.hasNext()) {
        BleManager manager = (BleManager)var1.next();
        manager.closeServer();
        manager.close();
    }

    this.managers.clear();
}

public final void setServerObserver(@Nullable ServerObserver observer) {
    this.serverObserver = observer;
}

@Nullable
final BluetoothGattServer getServer() {
    return this.server;
}

final void addManager(@NonNull BleManager manager) {
    if (!this.managers.contains(manager)) {
        this.managers.add(manager);
    }
}

final void removeManager(@NonNull BleManager manager) {
    this.managers.remove(manager);
}

final boolean isShared(@NonNull BluetoothGattCharacteristic characteristic) {
    return this.sharedCharacteristics != null &&
this.sharedCharacteristics.contains(characteristic);
}

final boolean isShared(@NonNull BluetoothGattDescriptor descriptor) {
    return this.sharedDescriptors != null &&
this.sharedDescriptors.contains(descriptor);
}

@Nullable
private BleManagerHandler getRequestHandler(@NonNull BluetoothDevice device)
{
    Iterator var2 = this.managers.iterator();

    BleManager manager;
    do {
        if (!var2.hasNext()) {
            return null;
        }
    }
}

```



```

        manager = (BleManager)var2.next();
    } while(!device.equals(manager.getBluetoothDevice()));

    return manager.requestHandler;
}

public int getMinLogPriority() {
    return 4;
}

public void log(int priority, @NonNull String message) {
}

public void log(int priority, @StringRes int messageRes, @Nullable Object...
params) {
    String message = this.context.getString(messageRes, params);
    this.log(priority, message);
}

@NonNull
protected abstract List<BluetoothGattService> initializeServer();

@NonNull
protected final BluetoothGattService service(@NonNull UUID uuid,
BluetoothGattCharacteristic... characteristics) {
    BluetoothGattService service = new BluetoothGattService(uuid, 0);
    BluetoothGattCharacteristic[] var4 = characteristics;
    int var5 = characteristics.length;

    for(int var6 = 0; var6 < var5; ++var6) {
        BluetoothGattCharacteristic characteristic = var4[var6];
        service.addCharacteristic(characteristic);
    }

    return service;
}

@NonNull
protected final BluetoothGattCharacteristic characteristic(@NonNull UUID
uuid, int properties, int permissions, @Nullable byte[] initialValue,
BluetoothGattDescriptor... descriptors) {
    boolean writableAuxiliaries = false;
    boolean cccdFound = false;
    boolean cepdFound = false;
    BluetoothGattDescriptor cepd = null;
    BluetoothGattDescriptor[] var10 = descriptors;
    int var11 = descriptors.length;

    for(int var12 = 0; var12 < var11; ++var12) {
        BluetoothGattDescriptor descriptor = var10[var12];
        if
(CLIENT_CHARACTERISTIC_CONFIG_DESCRIPTOR_UUID.equals(descriptor.getUuid())) {
            cccdFound = true;

```

```

        } else if
(CLIENT_USER_DESCRIPTION_DESCRIPTOR_UUID.equals(descriptor.getUuid()) && 0 !=
(descriptor.getPermissions() & 112)) {
            writableAuxiliaries = true;
        } else if
(CHARACTERISTIC_EXTENDED_PROPERTIES_DESCRIPTOR_UUID.equals(descriptor.getUuid())
) {
            cepd = descriptor;
            cepdFound = true;
        }
    }

    if (writableAuxiliaries) {
        if (cepd == null) {
            cepd = new
BluetoothGattDescriptor(CHARACTERISTIC_EXTENDED_PROPERTIES_DESCRIPTOR_UUID, 1);
            cepd.setValue(new byte[]{2, 0});
        } else if (cepd.getValue() != null && cepd.getValue().length == 2) {
            byte[] var10000 = cepd.getValue();
            var10000[0] = (byte)(var10000[0] | 2);
        } else {
            cepd.setValue(new byte[]{2, 0});
        }
    }

    boolean cccdRequired = (properties & 48) != 0;
    boolean reliableWrite = cepd != null && cepd.getValue() != null &&
cepd.getValue().length == 2 && (cepd.getValue()[0] & 1) != 0;
    if (writableAuxiliaries || reliableWrite) {
        properties |= 128;
    }

    if ((properties & 128) != 0 && cepd == null) {
        cepd = new
BluetoothGattDescriptor(CHARACTERISTIC_EXTENDED_PROPERTIES_DESCRIPTOR_UUID, 1);
        cepd.setValue(new byte[]{0, 0});
    }

    BluetoothGattCharacteristic characteristic = new
BluetoothGattCharacteristic(uuid, properties, permissions);
    if (cccdRequired && !cccdFound) {
        characteristic.addDescriptor(this.cccd());
    }

    BluetoothGattDescriptor[] var20 = descriptors;
    int var14 = descriptors.length;

    for(int var15 = 0; var15 < var14; ++var15) {
        BluetoothGattDescriptor descriptor = var20[var15];
        characteristic.addDescriptor(descriptor);
    }

    if (cepd != null && !cepdFound) {
        characteristic.addDescriptor(cepd);
    }

```

```

    }

    characteristic.setValue(initialValue);
    return characteristic;
}

@NonNull
protected final BluetoothGattCharacteristic characteristic(@NonNull UUID
uuid, int properties, int permissions, @Nullable Data initialValue,
BluetoothGattDescriptor... descriptors) {
    return this.characteristic(uuid, properties, permissions, initialValue !=
null ? initialValue.getValue() : null, descriptors);
}

@NonNull
protected final BluetoothGattCharacteristic characteristic(@NonNull UUID
uuid, int properties, int permissions, BluetoothGattDescriptor... descriptors) {
    return this.characteristic(uuid, properties, permissions, (byte[])null,
descriptors);
}

@NonNull
protected final BluetoothGattCharacteristic sharedCharacteristic(@NonNull
UUID uuid, int properties, int permissions, @Nullable byte[] initialValue,
BluetoothGattDescriptor... descriptors) {
    BluetoothGattCharacteristic characteristic = this.characteristic(uuid,
properties, permissions, initialValue, descriptors);
    if (this.sharedCharacteristics == null) {
        this.sharedCharacteristics = new ArrayList();
    }

    this.sharedCharacteristics.add(characteristic);
    return characteristic;
}

@NonNull
protected final BluetoothGattCharacteristic sharedCharacteristic(@NonNull
UUID uuid, int properties, int permissions, @Nullable Data initialValue,
BluetoothGattDescriptor... descriptors) {
    return this.sharedCharacteristic(uuid, properties, permissions,
initialValue != null ? initialValue.getValue() : null, descriptors);
}

@NonNull
protected final BluetoothGattCharacteristic sharedCharacteristic(@NonNull
UUID uuid, int properties, int permissions, BluetoothGattDescriptor...
descriptors) {
    return this.sharedCharacteristic(uuid, properties, permissions,
(byte[])null, descriptors);
}

@NonNull
protected final BluetoothGattDescriptor descriptor(@NonNull UUID uuid, int
permissions, @Nullable byte[] initialValue) {

```

```

        BluetoothGattDescriptor descriptor = new BluetoothGattDescriptor(uuid,
permissions);
        descriptor.setValue(initialValue);
        return descriptor;
    }

    @NonNull
    protected final BluetoothGattDescriptor descriptor(@NonNull UUID uuid, int
permissions, @Nullable Data initialValue) {
        return this.descriptor(uuid, permissions, initialValue != null ?
initialValue.getValue() : null);
    }

    @NonNull
    protected final BluetoothGattDescriptor sharedDescriptor(@NonNull UUID uuid,
int permissions, @Nullable byte[] initialValue) {
        BluetoothGattDescriptor descriptor = this.descriptor(uuid, permissions,
initialValue);
        if (this.sharedDescriptors == null) {
            this.sharedDescriptors = new ArrayList();
        }

        this.sharedDescriptors.add(descriptor);
        return descriptor;
    }

    @NonNull
    protected final BluetoothGattDescriptor sharedDescriptor(@NonNull UUID uuid,
int permissions, @Nullable Data initialValue) {
        return this.sharedDescriptor(uuid, permissions, initialValue != null ?
initialValue.getValue() : null);
    }

    @NonNull
    protected final BluetoothGattDescriptor cccd() {
        return this.descriptor(CLIENT_CHARACTERISTIC_CONFIG_DESCRIPTOR_UUID, 17,
(byte[])(new byte[]{0, 0}));
    }

    @NonNull
    protected final BluetoothGattDescriptor sharedCccd() {
        return this.sharedDescriptor(CLIENT_CHARACTERISTIC_CONFIG_DESCRIPTOR_UUID,
17, (byte[])(new byte[]{0, 0}));
    }

    @NonNull
    protected final BluetoothGattDescriptor reliableWrite() {
        return
this.sharedDescriptor(CHARACTERISTIC_EXTENDED_PROPERTIES_DESCRIPTOR_UUID, 1,
(byte[])(new byte[]{1, 0}));
    }

    @NonNull
    protected final BluetoothGattDescriptor description(@Nullable String

```

```

description, boolean writableAuxiliaries) {
    BluetoothGattDescriptor cud =
this.descriptor(CLIENT_USER_DESCRIPTION_DESCRIPTOR_UUID, 1 |
(writableAuxiliaries ? 16 : 0), description != null ? description.getBytes() :
null);
    if (!writableAuxiliaries) {
        if (this.sharedDescriptors == null) {
            this.sharedDescriptors = new ArrayList();
        }

        this.sharedDescriptors.add(cud);
    }

    return cud;
}
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\no\nordicsemi\android\ble
// File: BuildConfig.java

```

```

package no.nordicsemi.android.ble;

public final class BuildConfig {
    public static final boolean DEBUG = false;
    public static final String LIBRARY_PACKAGE_NAME =
"no.nordicsemi.android.ble";
    public static final String BUILD_TYPE = "release";
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\no\nordicsemi\android\ble
// File: Bytes.java

```

```

package no.nordicsemi.android.ble;

import androidx.annotation.IntRange;
import androidx.annotation.Nullable;

final class Bytes {
    static byte[] copy(@Nullable byte[] value, @IntRange(from = 0L) int offset,
@IntRange(from = 0L) int length) {
        if (value != null && offset <= value.length) {
            int maxLength = Math.min(value.length - offset, length);
            byte[] copy = new byte[maxLength];
            System.arraycopy(value, offset, copy, 0, maxLength);
            return copy;
        } else {
            return null;
        }
    }
}

```

```

        static byte[] concat(@Nullable byte[] left, @Nullable byte[] right,
@IntRange(from = 0L) int offset) {
            int length = offset + (right != null ? right.length : 0);
            byte[] result = new byte[length];
            if (left != null) {
                System.arraycopy(left, 0, result, 0, left.length);
            }

            if (right != null) {
                System.arraycopy(right, 0, result, offset, right.length);
            }

            return result;
        }
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: CallbackHandler.java

```

```

package no.nordicsemi.android.ble;

import androidx.annotation.NonNull;

interface CallbackHandler {
    void post(@NonNull Runnable var1);

    void postDelayed(@NonNull Runnable var1, long var2);

    void removeCallbacks(@NonNull Runnable var1);
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: ConditionalWaitRequest.java

```

```

package no.nordicsemi.android.ble;

import android.os.Handler;
import android.util.Log;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.AfterCallback;
import no.nordicsemi.android.ble.callback.BeforeCallback;
import no.nordicsemi.android.ble.callback.FailCallback;
import no.nordicsemi.android.ble.callback.InvalidRequestCallback;
import no.nordicsemi.android.ble.callback.SuccessCallback;

public final class ConditionalWaitRequest<T> extends AwaitingRequest<T>
implements Operation {
    @NonNull
    private final ConditionalWaitRequest.Condition<T> condition;
}

```

```

@Nullable
private final T parameter;
private boolean expected = false;

ConditionalWaitRequest(@NonNull Request.Type type, @NonNull
ConditionalWaitRequest.Condition<T> condition, @Nullable T parameter) {
    super(type);
    this.condition = condition;
    this.parameter = parameter;
}

@NonNull
ConditionalWaitRequest<T> setRequestHandler(@NonNull RequestHandler
requestHandler) {
    super.setRequestHandler(requestHandler);
    return this;
}

@NonNull
public ConditionalWaitRequest<T> setHandler(@Nullable Handler handler) {
    super.setHandler(handler);
    return this;
}

@NonNull
public ConditionalWaitRequest<T> done(@NonNull SuccessCallback callback) {
    super.done(callback);
    return this;
}

@NonNull
public ConditionalWaitRequest<T> fail(@NonNull FailCallback callback) {
    super.fail(callback);
    return this;
}

@NonNull
public ConditionalWaitRequest<T> invalid(@NonNull InvalidRequestCallback
callback) {
    super.invalid(callback);
    return this;
}

@NonNull
public ConditionalWaitRequest<T> before(@NonNull BeforeCallback callback) {
    super.before(callback);
    return this;
}

@NonNull
public ConditionalWaitRequest<T> then(@NonNull AfterCallback callback) {
    super.then(callback);
    return this;
}

```

```

@NonNull
public ConditionalWaitRequest<T> negate() {
    this.expected = true;
    return this;
}

boolean isFulfilled() {
    try {
        return this.condition.predicate(this.parameter) == this.expected;
    } catch (Exception var2) {
        Log.e("ConditionalWaitRequest", "Error while checking predicate",
var2);
        return true;
    }
}

public interface Condition<T> {
    boolean predicate(@Nullable T var1);
}
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: ConnectionPriorityRequest.java

```

```

package no.nordicsemi.android.ble;

import android.bluetooth.BluetoothDevice;
import android.os.Handler;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.annotation.RequiresApi;
import no.nordicsemi.android.ble.callback.AfterCallback;
import no.nordicsemi.android.ble.callback.BeforeCallback;
import no.nordicsemi.android.ble.callback.ConnectionParametersUpdatedCallback;
import no.nordicsemi.android.ble.callback.ConnectionPriorityCallback;
import no.nordicsemi.android.ble.callback.FailCallback;
import no.nordicsemi.android.ble.callback.InvalidRequestCallback;
import no.nordicsemi.android.ble.callback.SuccessCallback;
import no.nordicsemi.android.ble.exception.BluetoothDisabledException;
import no.nordicsemi.android.ble.exception.DeviceDisconnectedException;
import no.nordicsemi.android.ble.exception.InvalidRequestException;
import no.nordicsemi.android.ble.exception.RequestFailedException;

public final class ConnectionPriorityRequest extends
SimpleValueRequest<ConnectionParametersUpdatedCallback> implements Operation {
    public static final int CONNECTION_PRIORITY_BALANCED = 0;
    public static final int CONNECTION_PRIORITY_HIGH = 1;
    public static final int CONNECTION_PRIORITY_LOW_POWER = 2;
    private final int value;
}

```



```

ConnectionPriorityRequest(@NonNull Request.Type type, int priority) {
    super(type);
    if (priority < 0 || priority > 2) {
        priority = 0;
    }

    this.value = priority;
}

@NonNull
ConnectionPriorityRequest setRequestHandler(@NonNull RequestHandler
requestHandler) {
    super.setRequestHandler(requestHandler);
    return this;
}

@NonNull
public ConnectionPriorityRequest setHandler(@Nullable Handler handler) {
    super.setHandler(handler);
    return this;
}

@NonNull
public ConnectionPriorityRequest done(@NonNull SuccessCallback callback) {
    super.done(callback);
    return this;
}

@NonNull
public ConnectionPriorityRequest fail(@NonNull FailCallback callback) {
    super.fail(callback);
    return this;
}

@NonNull
public ConnectionPriorityRequest invalid(@NonNull InvalidRequestCallback
callback) {
    super.invalid(callback);
    return this;
}

@NonNull
public ConnectionPriorityRequest before(@NonNull BeforeCallback callback) {
    super.before(callback);
    return this;
}

@NonNull
public ConnectionPriorityRequest then(@NonNull AfterCallback callback) {
    super.then(callback);
    return this;
}

/** @deprecated */

```

```

    @Deprecated
    @RequiresApi(26)
    @NonNull
    public ConnectionPriorityRequest with(@NonNull ConnectionPriorityCallback
callback) {
        super.with(callback);
        return this;
    }

    @RequiresApi(26)
    @NonNull
    public ConnectionPriorityRequest with(@NonNull
ConnectionParametersUpdatedCallback callback) {
        super.with(callback);
        return this;
    }

    @RequiresApi(26)
    @NonNull
    public <E extends ConnectionParametersUpdatedCallback> E await(@NonNull
Class<E> responseClass) throws RequestFailedException,
DeviceDisconnectedException, BluetoothDisabledException, InvalidRequestException
{
        return (ConnectionParametersUpdatedCallback)super.await(responseClass);
    }

    @RequiresApi(26)
    @NonNull
    public <E extends ConnectionParametersUpdatedCallback> E await(@NonNull E
response) throws RequestFailedException, DeviceDisconnectedException,
BluetoothDisabledException, InvalidRequestException {
        return (ConnectionParametersUpdatedCallback)super.await((Object)response);
    }

    @RequiresApi(
        api = 26
    )
    void notifyConnectionPriorityChanged(@NonNull BluetoothDevice device,
@IntRange(from = 6L,to = 3200L) int interval, @IntRange(from = 0L,to = 499L) int
latency, @IntRange(from = 10L,to = 3200L) int timeout) {
        if (this.valueCallback != null) {

            ((ConnectionParametersUpdatedCallback)this.valueCallback).onConnectionUpdated(de
vice, interval, latency, timeout);
        }

    }

    int getRequiredPriority() {
        return this.value;
    }
}

```

```
// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: ConnectRequest.java
```

```
package no.nordicsemi.android.ble;
```

```
import android.bluetooth.BluetoothDevice;
import android.os.Handler;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.AfterCallback;
import no.nordicsemi.android.ble.callback.BeforeCallback;
import no.nordicsemi.android.ble.callback.FailCallback;
import no.nordicsemi.android.ble.callback.InvalidRequestCallback;
import no.nordicsemi.android.ble.callback.SuccessCallback;
```

```
public class ConnectRequest extends TimeoutableRequest {
```

```
    @NonNull
    private final BluetoothDevice device;
    private int preferredPhy;
    @IntRange(
        from = 0L
    )
    private int attempt = 0;
    @IntRange(
        from = 0L
    )
    private int retries = 0;
    @IntRange(
        from = 0L
    )
    private int delay = 0;
    private boolean autoConnect = false;
```

```
    ConnectRequest(@NonNull Request.Type type, @NonNull BluetoothDevice device) {
        super(type);
        this.device = device;
        this.preferredPhy = 1;
    }
```

```
    @NonNull
    ConnectRequest setRequestHandler(@NonNull RequestHandler requestHandler) {
        super.setRequestHandler(requestHandler);
        return this;
    }
```

```
    @NonNull
    public ConnectRequest setHandler(@Nullable Handler handler) {
        super.setHandler(handler);
        return this;
    }
```

```
    @NonNull
```

```

public ConnectRequest timeout(@IntRange(from = 0L) long timeout) {
    super.timeout(timeout);
    return this;
}

@NonNull
public ConnectRequest done(@NonNull SuccessCallback callback) {
    super.done(callback);
    return this;
}

@NonNull
public ConnectRequest fail(@NonNull FailCallback callback) {
    super.fail(callback);
    return this;
}

@NonNull
public ConnectRequest invalid(@NonNull InvalidRequestCallback callback) {
    super.invalid(callback);
    return this;
}

@NonNull
public ConnectRequest before(@NonNull BeforeCallback callback) {
    super.before(callback);
    return this;
}

@NonNull
public ConnectRequest then(@NonNull AfterCallback callback) {
    super.then(callback);
    return this;
}

public ConnectRequest retry(@IntRange(from = 0L) int count) {
    this.retries = count;
    this.delay = 0;
    return this;
}

public ConnectRequest retry(@IntRange(from = 0L) int count, @IntRange(from =
0L) int delay) {
    this.retries = count;
    this.delay = delay;
    return this;
}

public ConnectRequest useAutoConnect(boolean autoConnect) {
    this.autoConnect = autoConnect;
    return this;
}

public ConnectRequest usePreferredPhy(int phy) {

```

```

        this.preferredPhy = phy;
        return this;
    }

    public void cancelPendingConnection() {
        this.cancel();
    }

    public void cancel() {
        if (!this.started) {
            this.cancelled = true;
            this.finished = true;
        } else if (!this.finished) {
            this.cancelled = true;
            this.requestHandler.cancelQueue();
        }
    }

    }

    @NonNull
    public BluetoothDevice getDevice() {
        return this.device;
    }

    int getPreferredPhy() {
        return this.preferredPhy;
    }

    boolean canRetry() {
        if (this.retries > 0) {
            --this.retries;
            return true;
        } else {
            return false;
        }
    }

    boolean isFirstAttempt() {
        return this.attempt++ == 0;
    }

    @IntRange(
        from = 0L
    )
    int getRetryDelay() {
        return this.delay;
    }

    boolean shouldAutoConnect() {
        return this.autoConnect;
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: DisconnectRequest.java

package no.nordicsemi.android.ble;

import android.os.Handler;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.AfterCallback;
import no.nordicsemi.android.ble.callback.BeforeCallback;
import no.nordicsemi.android.ble.callback.FailCallback;
import no.nordicsemi.android.ble.callback.InvalidRequestCallback;
import no.nordicsemi.android.ble.callback.SuccessCallback;

public class DisconnectRequest extends TimeoutableRequest {
    DisconnectRequest(@NonNull Request.Type type) {
        super(type);
    }

    @NonNull
    DisconnectRequest setRequestHandler(@NonNull RequestHandler requestHandler) {
        super.setRequestHandler(requestHandler);
        return this;
    }

    @NonNull
    public DisconnectRequest setHandler(@Nullable Handler handler) {
        super.setHandler(handler);
        return this;
    }

    @NonNull
    public DisconnectRequest timeout(@IntRange(from = 0L) long timeout) {
        super.timeout(timeout);
        return this;
    }

    @NonNull
    public DisconnectRequest done(@NonNull SuccessCallback callback) {
        super.done(callback);
        return this;
    }

    @NonNull
    public DisconnectRequest fail(@NonNull FailCallback callback) {
        super.fail(callback);
        return this;
    }

    @NonNull
    public DisconnectRequest invalid(@NonNull InvalidRequestCallback callback) {
        super.invalid(callback);
    }
}

```

```

        return this;
    }

    @NonNull
    public DisconnectRequest before(@NonNull BeforeCallback callback) {
        super.before(callback);
        return this;
    }

    @NonNull
    public DisconnectRequest then(@NonNull AfterCallback callback) {
        super.then(callback);
        return this;
    }
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: LegacyBleManager.java

package no.nordicsemi.android.ble;

import android.content.Context;
import android.os.Handler;
import androidx.annotation.NonNull;

/** @deprecated */
@Deprecated
public abstract class LegacyBleManager<E extends BleManagerCallbacks> extends
BleManager {
    protected E mCallbacks;

    public LegacyBleManager(@NonNull Context context) {
        super(context);
    }

    public LegacyBleManager(@NonNull Context context, @NonNull Handler handler) {
        super(context, handler);
    }

    public void setGattCallbacks(@NonNull BleManagerCallbacks callbacks) {
        super.setGattCallbacks(callbacks);
        this.mCallbacks = callbacks;
    }
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: MtuRequest.java

package no.nordicsemi.android.ble;

```

```

import android.bluetooth.BluetoothDevice;
import android.os.Handler;
import android.util.Log;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.AfterCallback;
import no.nordicsemi.android.ble.callback.BeforeCallback;
import no.nordicsemi.android.ble.callback.FailCallback;
import no.nordicsemi.android.ble.callback.InvalidRequestCallback;
import no.nordicsemi.android.ble.callback.MtuCallback;
import no.nordicsemi.android.ble.callback.SuccessCallback;

public final class MtuRequest extends SimpleValueRequest<MtuCallback> implements
Operation {
    private final int value;

    MtuRequest(@NonNull Request.Type type, @IntRange(from = 23L,to = 517L) int
mtu) {
        super(type);
        if (mtu < 23) {
            mtu = 23;
        }

        if (mtu > 517) {
            mtu = 517;
        }

        this.value = mtu;
    }

    @NonNull
    MtuRequest setRequestHandler(@NonNull RequestHandler requestHandler) {
        super.setRequestHandler(requestHandler);
        return this;
    }

    @NonNull
    public MtuRequest setHandler(@Nullable Handler handler) {
        super.setHandler(handler);
        return this;
    }

    @NonNull
    public MtuRequest done(@NonNull SuccessCallback callback) {
        super.done(callback);
        return this;
    }

    @NonNull
    public MtuRequest fail(@NonNull FailCallback callback) {
        super.fail(callback);
        return this;
    }
}

```



```

@NonNull
public MtuRequest invalid(@NonNull InvalidRequestCallback callback) {
    super.invalid(callback);
    return this;
}

@NonNull
public MtuRequest before(@NonNull BeforeCallback callback) {
    super.before(callback);
    return this;
}

@NonNull
public MtuRequest then(@NonNull AfterCallback callback) {
    super.then(callback);
    return this;
}

@NonNull
public MtuRequest with(@NonNull MtuCallback callback) {
    super.with(callback);
    return this;
}

void notifyMtuChanged(@NonNull BluetoothDevice device, @IntRange(from =
23L,to = 517L) int mtu) {
    this.handler.post(() -> {
        if (this.valueCallback != null) {
            try {
                ((MtuCallback)this.valueCallback).onMtuChanged(device, mtu);
            } catch (Throwable var4) {
                Log.e(TAG, "Exception in Value callback", var4);
            }
        }
    });
}

int getRequiredMtu() {
    return this.value;
}
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: Operation.java

```

```

package no.nordicsemi.android.ble;

public interface Operation {
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: PhyRequest.java

package no.nordicsemi.android.ble;

import android.bluetooth.BluetoothDevice;
import android.os.Handler;
import android.util.Log;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.AfterCallback;
import no.nordicsemi.android.ble.callback.BeforeCallback;
import no.nordicsemi.android.ble.callback.FailCallback;
import no.nordicsemi.android.ble.callback.InvalidRequestCallback;
import no.nordicsemi.android.ble.callback.PhyCallback;
import no.nordicsemi.android.ble.callback.SuccessCallback;

public final class PhyRequest extends SimpleValueRequest<PhyCallback> implements
Operation {
    public static final int PHY_LE_1M_MASK = 1;
    public static final int PHY_LE_2M_MASK = 2;
    public static final int PHY_LE_CODED_MASK = 4;
    public static final int PHY_OPTION_NO_PREFERRED = 0;
    public static final int PHY_OPTION_S2 = 1;
    public static final int PHY_OPTION_S8 = 2;
    private final int txPhy;
    private final int rxPhy;
    private final int phyOptions;

    PhyRequest(@NonNull Request.Type type) {
        super(type);
        this.txPhy = 0;
        this.rxPhy = 0;
        this.phyOptions = 0;
    }

    PhyRequest(@NonNull Request.Type type, int txPhy, int rxPhy, int phyOptions)
    {
        super(type);
        if ((txPhy & -8) > 0) {
            txPhy = 1;
        }

        if ((rxPhy & -8) > 0) {
            rxPhy = 1;
        }

        if (phyOptions < 0 || phyOptions > 2) {
            phyOptions = 0;
        }

        this.txPhy = txPhy;
    }

```

```

        this.rxPhy = rxPhy;
        this.phyOptions = phyOptions;
    }

    @NonNull
    PhyRequest setRequestHandler(@NonNull RequestHandler requestHandler) {
        super.setRequestHandler(requestHandler);
        return this;
    }

    @NonNull
    public PhyRequest setHandler(@Nullable Handler handler) {
        super.setHandler(handler);
        return this;
    }

    @NonNull
    public PhyRequest done(@NonNull SuccessCallback callback) {
        super.done(callback);
        return this;
    }

    @NonNull
    public PhyRequest fail(@NonNull FailCallback callback) {
        super.fail(callback);
        return this;
    }

    @NonNull
    public PhyRequest invalid(@NonNull InvalidRequestCallback callback) {
        super.invalid(callback);
        return this;
    }

    @NonNull
    public PhyRequest before(@NonNull BeforeCallback callback) {
        super.before(callback);
        return this;
    }

    @NonNull
    public PhyRequest then(@NonNull AfterCallback callback) {
        super.then(callback);
        return this;
    }

    @NonNull
    public PhyRequest with(@NonNull PhyCallback callback) {
        super.with(callback);
        return this;
    }

    void notifyPhyChanged(@NonNull BluetoothDevice device, int txPhy, int rxPhy)
{

```

```

        this.handler.post(() -> {
            if (this.valueCallback != null) {
                try {
                    ((PhyCallback)this.valueCallback).onPhyChanged(device, txPhy,
rxPhy);
                } catch (Throwable var5) {
                    Log.e(TAG, "Exception in Value callback", var5);
                }
            }
        });
    }

    void notifyLegacyPhy(@NonNull BluetoothDevice device) {
        this.handler.post(() -> {
            if (this.valueCallback != null) {
                try {
                    ((PhyCallback)this.valueCallback).onPhyChanged(device, 1, 1);
                } catch (Throwable var3) {
                    Log.e(TAG, "Exception in Value callback", var3);
                }
            }
        });
    }

    int getPreferredTxPhy() {
        return this.txPhy;
    }

    int getPreferredRxPhy() {
        return this.rxPhy;
    }

    int getPreferredPhyOptions() {
        return this.phyOptions;
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: ReadRequest.java

```

```

package no.nordicsemi.android.ble;

import android.bluetooth.BluetoothDevice;
import android.bluetooth.BluetoothGattCharacteristic;
import android.bluetooth.BluetoothGattDescriptor;
import android.os.Handler;
import android.util.Log;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.AfterCallback;

```

```

import no.nordicsemi.android.ble.callback.BeforeCallback;
import no.nordicsemi.android.ble.callback.DataReceivedCallback;
import no.nordicsemi.android.ble.callback.FailCallback;
import no.nordicsemi.android.ble.callback.InvalidRequestCallback;
import no.nordicsemi.android.ble.callback.ReadProgressCallback;
import no.nordicsemi.android.ble.callback.SuccessCallback;
import no.nordicsemi.android.ble.callback.profile.ProfileReadResponse;
import no.nordicsemi.android.ble.data.Data;
import no.nordicsemi.android.ble.data.DataFilter;
import no.nordicsemi.android.ble.data.DataMerger;
import no.nordicsemi.android.ble.data.DataStream;
import no.nordicsemi.android.ble.data.PacketFilter;
import no.nordicsemi.android.ble.exceptionBluetoothDisabledException;
import no.nordicsemi.android.ble.exception.DeviceDisconnectedException;
import no.nordicsemi.android.ble.exception.InvalidDataException;
import no.nordicsemi.android.ble.exception.InvalidRequestException;
import no.nordicsemi.android.ble.exception.RequestFailedException;

public final class ReadRequest extends SimpleValueRequest<DataReceivedCallback>
implements Operation {
    private ReadProgressCallback progressCallback;
    private DataMerger dataMerger;
    private DataStream buffer;
    private DataFilter filter;
    private PacketFilter packetFilter;
    private int count = 0;
    private boolean complete = false;

    ReadRequest(@NonNull Request.Type type) {
        super(type);
    }

    ReadRequest(@NonNull Request.Type type, @Nullable BluetoothGattCharacteristic
characteristic) {
        super(type, characteristic);
    }

    ReadRequest(@NonNull Request.Type type, @Nullable BluetoothGattDescriptor
descriptor) {
        super(type, descriptor);
    }

    @NonNull
    ReadRequest setRequestHandler(@NonNull RequestHandler requestHandler) {
        super.setRequestHandler(requestHandler);
        return this;
    }

    @NonNull
    public ReadRequest setHandler(@Nullable Handler handler) {
        super.setHandler(handler);
        return this;
    }
}

```

```

@NonNull
public ReadRequest done(@NonNull SuccessCallback callback) {
    super.done(callback);
    return this;
}

@NonNull
public ReadRequest fail(@NonNull FailCallback callback) {
    super.fail(callback);
    return this;
}

@NonNull
public ReadRequest invalid(@NonNull InvalidRequestCallback callback) {
    super.invalid(callback);
    return this;
}

@NonNull
public ReadRequest before(@NonNull BeforeCallback callback) {
    super.before(callback);
    return this;
}

@NonNull
public ReadRequest then(@NonNull AfterCallback callback) {
    super.then(callback);
    return this;
}

@NonNull
public ReadRequest with(@NonNull DataReceivedCallback callback) {
    super.with(callback);
    return this;
}

@NonNull
public ReadRequest filter(@NonNull DataFilter filter) {
    this.filter = filter;
    return this;
}

@NonNull
public ReadRequest filterPacket(@NonNull PacketFilter filter) {
    this.packetFilter = filter;
    return this;
}

@NonNull
public ReadRequest merge(@NonNull DataMerger merger) {
    this.dataMerger = merger;
    this.progressCallback = null;
    return this;
}

```

```

    @NonNull
    public ReadRequest merge(@NonNull DataMerger merger, @NonNull
ReadProgressCallback callback) {
        this.dataMerger = merger;
        this.progressCallback = callback;
        return this;
    }

    @NonNull
    public <E extends ProfileReadResponse> E awaitValid(@NonNull Class<E>
responseClass) throws RequestFailedException, InvalidDataException,
DeviceDisconnectedException, BluetoothDisabledException, InvalidRequestException
{
        E response = (ProfileReadResponse)this.await(responseClass);
        if (!response.isValid()) {
            throw new InvalidDataException(response);
        } else {
            return response;
        }
    }

    @NonNull
    public <E extends ProfileReadResponse> E awaitValid(@NonNull E response)
throws RequestFailedException, InvalidDataException,
DeviceDisconnectedException, BluetoothDisabledException, InvalidRequestException
{
        this.await(response);
        if (!response.isValid()) {
            throw new InvalidDataException(response);
        } else {
            return response;
        }
    }

    boolean matches(byte[] packet) {
        return this.filter == null || this.filter.filter(packet);
    }

    void notifyValueChanged(@NonNull BluetoothDevice device, @Nullable byte[]
value) {
        DataReceivedCallback valueCallback =
(DataReceivedCallback)this.valueCallback;
        if (valueCallback == null) {
            if (this.packetFilter == null || this.packetFilter.filter(value)) {
                this.complete = true;
            }
        } else {
            if (this.dataMerger == null) {
                this.complete = true;
                Data data = new Data(value);
                this.handler.post(() -> {
                    try {

```

```

        valueCallback.onDataReceived(device, data);
    } catch (Throwable var4) {
        Log.e(TAG, "Exception in Value callback", var4);
    }
});
} else {
    this.handler.post(() -> {
        if (this.progressCallback != null) {
            try {
                this.progressCallback.onPacketReceived(device, value,
this.count);
            } catch (Throwable var4) {
                Log.e(TAG, "Exception in Progress callback", var4);
            }
        }
    });
    if (this.buffer == null) {
        this.buffer = new DataStream();
    }

    if (this.dataMerger.merge(this.buffer, value, this.count++)) {
        byte[] merged = this.buffer.toByteArray();
        if (this.packetFilter == null ||
this.packetFilter.filter(merged)) {
            this.complete = true;
            Data data = new Data(merged);
            this.handler.post(() -> {
                try {
                    valueCallback.onDataReceived(device, data);
                } catch (Throwable var4) {
                    Log.e(TAG, "Exception in Value callback", var4);
                }
            });
        }

        this.buffer = null;
        this.count = 0;
    }
}

}

}

boolean hasMore() {
    return !this.complete;
}
}

```

// Directory: app

// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble


```
// File: ReadRssiRequest.java
```

```
package no.nordicsemi.android.ble;
```

```
import android.bluetooth.BluetoothDevice;
import android.os.Handler;
import android.util.Log;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.AfterCallback;
import no.nordicsemi.android.ble.callback.BeforeCallback;
import no.nordicsemi.android.ble.callback.FailCallback;
import no.nordicsemi.android.ble.callback.InvalidRequestCallback;
import no.nordicsemi.android.ble.callback.RssiCallback;
import no.nordicsemi.android.ble.callback.SuccessCallback;
```

```
public final class ReadRssiRequest extends SimpleValueRequest<RssiCallback>
implements Operation {
```

```
    ReadRssiRequest(@NonNull Request.Type type) {
        super(type);
    }
```

```
    @NonNull
    ReadRssiRequest setRequestHandler(@NonNull RequestHandler requestHandler) {
        super.setRequestHandler(requestHandler);
        return this;
    }
```

```
    @NonNull
    public ReadRssiRequest setHandler(@Nullable Handler handler) {
        super.setHandler(handler);
        return this;
    }
```

```
    @NonNull
    public ReadRssiRequest done(@NonNull SuccessCallback callback) {
        super.done(callback);
        return this;
    }
```

```
    @NonNull
    public ReadRssiRequest fail(@NonNull FailCallback callback) {
        super.fail(callback);
        return this;
    }
```

```
    @NonNull
    public ReadRssiRequest invalid(@NonNull InvalidRequestCallback callback) {
        super.invalid(callback);
        return this;
    }
```

```
    @NonNull
```

```

    public ReadRssiRequest before(@NonNull BeforeCallback callback) {
        super.before(callback);
        return this;
    }

    @NonNull
    public ReadRssiRequest then(@NonNull AfterCallback callback) {
        super.then(callback);
        return this;
    }

    @NonNull
    public ReadRssiRequest with(@NonNull RssiCallback callback) {
        super.with(callback);
        return this;
    }

    void notifyRssiRead(@NonNull BluetoothDevice device, @IntRange(from =
-128L,to = 20L) int rssi) {
        this.handler.post(() -> {
            if (this.valueCallback != null) {
                try {
                    ((RssiCallback)this.valueCallback).onRssiRead(device, rssi);
                } catch (Throwable var4) {
                    Log.e(TAG, "Exception in Value callback", var4);
                }
            }
        });
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: ReliableWriteRequest.java

```

```

package no.nordicsemi.android.ble;

import android.os.Handler;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.AfterCallback;
import no.nordicsemi.android.ble.callback.BeforeCallback;
import no.nordicsemi.android.ble.callback.FailCallback;
import no.nordicsemi.android.ble.callback.InvalidRequestCallback;
import no.nordicsemi.android.ble.callback.SuccessCallback;

public final class ReliableWriteRequest extends RequestQueue {
    private boolean initialized;
    private boolean closed;

    @NonNull

```

```

    ReliableWriteRequest setRequestHandler(@NonNull RequestHandler
requestHandler) {
    super.setRequestHandler(requestHandler);
    return this;
}

@NonNull
public ReliableWriteRequest setHandler(@Nullable Handler handler) {
    super.setHandler(handler);
    return this;
}

@NonNull
public ReliableWriteRequest done(@NonNull SuccessCallback callback) {
    super.done(callback);
    return this;
}

@NonNull
public ReliableWriteRequest fail(@NonNull FailCallback callback) {
    super.fail(callback);
    return this;
}

@NonNull
public ReliableWriteRequest invalid(@NonNull InvalidRequestCallback callback)
{
    super.invalid(callback);
    return this;
}

@NonNull
public ReliableWriteRequest before(@NonNull BeforeCallback callback) {
    super.before(callback);
    return this;
}

@NonNull
public ReliableWriteRequest then(@NonNull AfterCallback callback) {
    super.then(callback);
    return this;
}

@NonNull
public ReliableWriteRequest timeout(@IntRange(from = 0L) long timeout) {
    super.timeout(timeout);
    return this;
}

@NonNull
public ReliableWriteRequest add(@NonNull Operation operation) {
    super.add(operation);
    if (operation instanceof WriteRequest) {
        ((WriteRequest)operation).forceSplit();
    }
}

```

```

    }

    return this;
}

public void abort() {
    this.cancel();
}

public int size() {
    int size = super.size();
    if (!this.initialized) {
        ++size;
    }

    if (!this.closed) {
        ++size;
    }

    return size;
}

Request getNext() {
    if (!this.initialized) {
        this.initialized = true;
        return newBeginReliableWriteRequest();
    } else if (super.isEmpty()) {
        this.closed = true;
        return this.cancelled ? newAbortReliableWriteRequest() :
newExecuteReliableWriteRequest();
    } else {
        return super.getNext();
    }
}

boolean hasMore() {
    if (!this.initialized) {
        return super.hasMore();
    } else {
        return !this.closed;
    }
}

void cancelQueue() {
    this.cancelled = true;
    super.cancelQueue();
}
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: Request.java

```

```

package no.nordicsemi.android.ble;

import android.bluetooth.BluetoothDevice;
import android.bluetooth.BluetoothGattCharacteristic;
import android.bluetooth.BluetoothGattDescriptor;
import android.os.ConditionVariable;
import android.os.Handler;
import android.os.Looper;
import android.util.Log;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.AfterCallback;
import no.nordicsemi.android.ble.callback.BeforeCallback;
import no.nordicsemi.android.ble.callback.FailCallback;
import no.nordicsemi.android.ble.callback.InvalidRequestCallback;
import no.nordicsemi.android.ble.callback.SuccessCallback;

public abstract class Request {
    protected static final String TAG = Request.class.getSimpleName();
    protected RequestHandler requestHandler;
    protected CallbackHandler handler;
    final ConditionVariable syncLock;
    final Request.Type type;
    final BluetoothGattCharacteristic characteristic;
    final BluetoothGattDescriptor descriptor;
    BeforeCallback beforeCallback;
    AfterCallback afterCallback;
    SuccessCallback successCallback;
    FailCallback failCallback;
    InvalidRequestCallback invalidRequestCallback;
    BeforeCallback internalBeforeCallback;
    SuccessCallback internalSuccessCallback;
    FailCallback internalFailCallback;
    boolean enqueued;
    boolean started;
    boolean finished;

    Request(@NonNull Request.Type type) {
        this.type = type;
        this.characteristic = null;
        this.descriptor = null;
        this.syncLock = new ConditionVariable(true);
    }

    Request(@NonNull Request.Type type, @Nullable BluetoothGattCharacteristic
characteristic) {
        this.type = type;
        this.characteristic = characteristic;
        this.descriptor = null;
        this.syncLock = new ConditionVariable(true);
    }

    Request(@NonNull Request.Type type, @Nullable BluetoothGattDescriptor

```

```

descriptor) {
    this.type = type;
    this.characteristic = null;
    this.descriptor = descriptor;
    this.syncLock = new ConditionVariable(true);
}

@NonNull
Request setRequestHandler(@NonNull RequestHandler requestHandler) {
    this.requestHandler = requestHandler;
    if (this.handler == null) {
        this.handler = requestHandler;
    }

    return this;
}

@NonNull
public Request setHandler(@Nullable final Handler handler) {
    this.handler = new CallbackHandler() {
        public void post(@NonNull Runnable r) {
            if (handler != null) {
                handler.post(r);
            } else {
                r.run();
            }
        }
    };

    public void postDelayed(@NonNull Runnable r, long delayMillis) {
        if (handler != null) {
            handler.postDelayed(r, delayMillis);
        } else {
            Request.this.requestHandler.postDelayed(r, delayMillis);
        }
    }

    public void removeCallbacks(@NonNull Runnable r) {
        if (handler != null) {
            handler.removeCallbacks(r);
        } else {
            Request.this.requestHandler.removeCallbacks(r);
        }
    }

    };
    return this;
}

@NonNull
static ConnectRequest connect(@NonNull BluetoothDevice device) {
    return new ConnectRequest(Request.Type.CONNECT, device);
}

```

```

@NonNull
static DisconnectRequest disconnect() {
    return new DisconnectRequest(Request.Type.DISCONNECT);
}

/** @deprecated */
@Deprecated
@NonNull
public static SimpleRequest createBond() {
    return new SimpleRequest(Request.Type.CREATE_BOND);
}

@NonNull
static SimpleRequest ensureBond() {
    return new SimpleRequest(Request.Type.ENSURE_BOND);
}

/** @deprecated */
@Deprecated
@NonNull
public static SimpleRequest removeBond() {
    return new SimpleRequest(Request.Type.REMOVE_BOND);
}

/** @deprecated */
@Deprecated
@NonNull
public static ReadRequest newReadRequest(@Nullable
BluetoothGattCharacteristic characteristic) {
    return new ReadRequest(Request.Type.READ, characteristic);
}

/** @deprecated */
@Deprecated
@NonNull
public static WriteRequest newWriteRequest(@Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] value) {
    return new WriteRequest(Request.Type.WRITE, characteristic, value, 0,
value != null ? value.length : 0, characteristic != null ?
characteristic.getWriteType() : 2);
}

/** @deprecated */
@Deprecated
@NonNull
public static WriteRequest newWriteRequest(@Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] value, int
writeType) {
    return new WriteRequest(Request.Type.WRITE, characteristic, value, 0,
value != null ? value.length : 0, writeType);
}

/** @deprecated */

```

```

    @Deprecated
    @NonNull
    public static WriteRequest newWriteRequest(@Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] value,
@IntRange(from = 0L) int offset, @IntRange(from = 0L) int length) {
        return new WriteRequest(Request.Type.WRITE, characteristic, value, offset,
length, characteristic != null ? characteristic.getWriteType() : 2);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    public static WriteRequest newWriteRequest(@Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] value,
@IntRange(from = 0L) int offset, @IntRange(from = 0L) int length, int writeType)
{
        return new WriteRequest(Request.Type.WRITE, characteristic, value, offset,
length, writeType);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    public static ReadRequest newReadRequest(@Nullable BluetoothGattDescriptor
descriptor) {
        return new ReadRequest(Request.Type.READ_DESCRIPTOR, descriptor);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    public static WriteRequest newWriteRequest(@Nullable BluetoothGattDescriptor
descriptor, @Nullable byte[] value) {
        return new WriteRequest(Request.Type.WRITE_DESCRIPTOR, descriptor, value,
0, value != null ? value.length : 0);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    public static WriteRequest newWriteRequest(@Nullable BluetoothGattDescriptor
descriptor, @Nullable byte[] value, @IntRange(from = 0L) int offset,
@IntRange(from = 0L) int length) {
        return new WriteRequest(Request.Type.WRITE_DESCRIPTOR, descriptor, value,
offset, length);
    }

    @NonNull
    static ReliableWriteRequest newReliableWriteRequest() {
        return new ReliableWriteRequest();
    }

    @NonNull
    static SimpleRequest newBeginReliableWriteRequest() {

```



```

        return new SimpleRequest(Request.Type.BEGIN_RELIABLE_WRITE);
    }

    @NonNull
    static SimpleRequest newExecuteReliableWriteRequest() {
        return new SimpleRequest(Request.Type.EXECUTE_RELIABLE_WRITE);
    }

    @NonNull
    static SimpleRequest newAbortReliableWriteRequest() {
        return new SimpleRequest(Request.Type.ABORT_RELIABLE_WRITE);
    }

    @NonNull
    static WriteRequest newNotificationRequest(@Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] value) {
        return new WriteRequest(Request.Type.NOTIFY, characteristic, value, 0,
value != null ? value.length : 0);
    }

    @NonNull
    static WriteRequest newNotificationRequest(@Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] value,
@IntRange(from = 0L) int offset, @IntRange(from = 0L) int length) {
        return new WriteRequest(Request.Type.NOTIFY, characteristic, value,
offset, length);
    }

    @NonNull
    static WriteRequest newIndicationRequest(@Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] value) {
        return new WriteRequest(Request.Type.INDICATE, characteristic, value, 0,
value != null ? value.length : 0);
    }

    @NonNull
    static WriteRequest newIndicationRequest(@Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] value,
@IntRange(from = 0L) int offset, @IntRange(from = 0L) int length) {
        return new WriteRequest(Request.Type.INDICATE, characteristic, value,
offset, length);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    public static WriteRequest newEnableNotificationsRequest(@Nullable
BluetoothGattCharacteristic characteristic) {
        return new WriteRequest(Request.Type.ENABLE_NOTIFICATIONS,
characteristic);
    }

    /** @deprecated */
    @Deprecated

```

```

    @NonNull
    public static WriteRequest newDisableNotificationsRequest(@Nullable
BluetoothGattCharacteristic characteristic) {
        return new WriteRequest(Request.Type.DISABLE_NOTIFICATIONS,
characteristic);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    public static WriteRequest newEnableIndicationsRequest(@Nullable
BluetoothGattCharacteristic characteristic) {
        return new WriteRequest(Request.Type.ENABLE_INDICATIONS, characteristic);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    public static WriteRequest newDisableIndicationsRequest(@Nullable
BluetoothGattCharacteristic characteristic) {
        return new WriteRequest(Request.Type.DISABLE_INDICATIONS, characteristic);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    public static WaitForValueChangedRequest
newWaitForNotificationRequest(@Nullable BluetoothGattCharacteristic
characteristic) {
        return new WaitForValueChangedRequest(Request.Type.WAIT_FOR_NOTIFICATION,
characteristic);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    public static WaitForValueChangedRequest
newWaitForIndicationRequest(@Nullable BluetoothGattCharacteristic
characteristic) {
        return new WaitForValueChangedRequest(Request.Type.WAIT_FOR_INDICATION,
characteristic);
    }

    @NonNull
    static WaitForValueChangedRequest newWaitForWriteRequest(@Nullable
BluetoothGattCharacteristic characteristic) {
        return new WaitForValueChangedRequest(Request.Type.WAIT_FOR_WRITE,
characteristic);
    }

    @NonNull
    static WaitForValueChangedRequest newWaitForWriteRequest(@Nullable
BluetoothGattDescriptor descriptor) {
        return new WaitForValueChangedRequest(Request.Type.WAIT_FOR_WRITE,

```

```

descriptor);
    }

    @NonNull
    static WaitForReadRequest newWaitForReadRequest(@Nullable
BluetoothGattCharacteristic characteristic) {
        return new WaitForReadRequest(Request.Type.WAIT_FOR_READ, characteristic);
    }

    @NonNull
    static WaitForReadRequest newWaitForReadRequest(@Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] value) {
        return new WaitForReadRequest(Request.Type.WAIT_FOR_READ, characteristic,
value, 0, value != null ? value.length : 0);
    }

    @NonNull
    static WaitForReadRequest newWaitForReadRequest(@Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] value,
@IntRange(from = 0L) int offset, @IntRange(from = 0L) int length) {
        return new WaitForReadRequest(Request.Type.WAIT_FOR_READ, characteristic,
value, offset, length);
    }

    @NonNull
    static WaitForReadRequest newWaitForReadRequest(@Nullable
BluetoothGattDescriptor descriptor) {
        return new WaitForReadRequest(Request.Type.WAIT_FOR_READ, descriptor);
    }

    @NonNull
    static WaitForReadRequest newWaitForReadRequest(@Nullable
BluetoothGattDescriptor descriptor, @Nullable byte[] value) {
        return new WaitForReadRequest(Request.Type.WAIT_FOR_READ, descriptor,
value, 0, value != null ? value.length : 0);
    }

    @NonNull
    static WaitForReadRequest newWaitForReadRequest(@Nullable
BluetoothGattDescriptor descriptor, @Nullable byte[] value, @IntRange(from = 0L)
int offset, @IntRange(from = 0L) int length) {
        return new WaitForReadRequest(Request.Type.WAIT_FOR_READ, descriptor,
value, offset, length);
    }

    @NonNull
    static <T> ConditionalWaitRequest<T> newConditionalWaitRequest(@NonNull
ConditionalWaitRequest.Condition<T> condition, @Nullable T parameter) {
        return new ConditionalWaitRequest(Request.Type.WAIT_FOR_CONDITION,
condition, parameter);
    }

    @NonNull
    static SetValueRequest newSetValueRequest(@Nullable

```

```

BluetoothGattCharacteristic characteristic, @Nullable byte[] value) {
    return new SetValueRequest(Request.Type.SET_VALUE, characteristic, value,
0, value != null ? value.length : 0);
}

@NonNull
static SetValueRequest newSetValueRequest(@Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] value,
@IntRange(from = 0L) int offset, @IntRange(from = 0L) int length) {
    return new SetValueRequest(Request.Type.SET_VALUE, characteristic, value,
offset, length);
}

@NonNull
static SetValueRequest newSetValueRequest(@Nullable BluetoothGattDescriptor
descriptor, @Nullable byte[] value) {
    return new SetValueRequest(Request.Type.SET_DESCRIPTOR_VALUE, descriptor,
value, 0, value != null ? value.length : 0);
}

@NonNull
static SetValueRequest newSetValueRequest(@Nullable BluetoothGattDescriptor
descriptor, @Nullable byte[] value, @IntRange(from = 0L) int offset,
@IntRange(from = 0L) int length) {
    return new SetValueRequest(Request.Type.SET_DESCRIPTOR_VALUE, descriptor,
value, offset, length);
}

/** @deprecated */
@Deprecated
@NonNull
public static ReadRequest newReadBatteryLevelRequest() {
    return new ReadRequest(Request.Type.READ_BATTERY_LEVEL);
}

/** @deprecated */
@Deprecated
@NonNull
public static WriteRequest newEnableBatteryLevelNotificationsRequest() {
    return new WriteRequest(Request.Type.ENABLE_BATTERY_LEVEL_NOTIFICATIONS);
}

/** @deprecated */
@Deprecated
@NonNull
public static WriteRequest newDisableBatteryLevelNotificationsRequest() {
    return new WriteRequest(Request.Type.DISABLE_BATTERY_LEVEL_NOTIFICATIONS);
}

@NonNull
static WriteRequest newEnableServiceChangedIndicationsRequest() {
    return new WriteRequest(Request.Type.ENABLE_SERVICE_CHANGED_INDICATIONS);
}

```

```

    /** @deprecated */
    @Deprecated
    @NonNull
    public static MtuRequest newMtuRequest(@IntRange(from = 23L,to = 517L) int
mtu) {
        return new MtuRequest(Request.Type.REQUEST_MTU, mtu);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    public static ConnectionPriorityRequest newConnectionPriorityRequest(int
priority) {
        return new
ConnectionPriorityRequest(Request.Type.REQUEST_CONNECTION_PRIORITY, priority);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    public static PhyRequest newSetPreferredPhyRequest(int txPhy, int rxPhy, int
phyOptions) {
        return new PhyRequest(Request.Type.SET_PREFERRED_PHY, txPhy, rxPhy,
phyOptions);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    public static PhyRequest newReadPhyRequest() {
        return new PhyRequest(Request.Type.READ_PHY);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    public static ReadRssiRequest newReadRssiRequest() {
        return new ReadRssiRequest(Request.Type.READ_RSSI);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    public static SimpleRequest newRefreshCacheRequest() {
        return new SimpleRequest(Request.Type.REFRESH_CACHE);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    public static SleepRequest newSleepRequest(@IntRange(from = 0L) long delay) {
        return new SleepRequest(Request.Type.SLEEP, delay);
    }

```

```

@NonNull
public Request done(@NonNull SuccessCallback callback) {
    this.successCallback = callback;
    return this;
}

@NonNull
public Request fail(@NonNull FailCallback callback) {
    this.failCallback = callback;
    return this;
}

void internalBefore(@NonNull BeforeCallback callback) {
    this.internalBeforeCallback = callback;
}

void internalSuccess(@NonNull SuccessCallback callback) {
    this.internalSuccessCallback = callback;
}

void internalFail(@NonNull FailCallback callback) {
    this.internalFailCallback = callback;
}

@NonNull
public Request invalid(@NonNull InvalidRequestCallback callback) {
    this.invalidRequestCallback = callback;
    return this;
}

@NonNull
public Request before(@NonNull BeforeCallback callback) {
    this.beforeCallback = callback;
    return this;
}

@NonNull
public Request then(@NonNull AfterCallback callback) {
    this.afterCallback = callback;
    return this;
}

public void enqueue() {
    this.requestHandler.enqueue(this);
}

void notifyStarted(@NonNull BluetoothDevice device) {
    if (!this.started) {
        this.started = true;
        if (this.internalBeforeCallback != null) {
            this.internalBeforeCallback.onRequestStarted(device);
        }

        this.handler.post(() -> {

```

```

        if (this.beforeCallback != null) {
            try {
                this.beforeCallback.onRequestStarted(device);
            } catch (Throwable var3) {
                Log.e(TAG, "Exception in Before callback", var3);
            }
        }

    });
}

boolean notifySuccess(@NonNull BluetoothDevice device) {
    if (!this.finished) {
        this.finished = true;
        if (this.internalSuccessCallback != null) {
            this.internalSuccessCallback.onRequestCompleted(device);
        }

        this.handler.post(() -> {
            if (this.successCallback != null) {
                try {
                    this.successCallback.onRequestCompleted(device);
                } catch (Throwable var4) {
                    Log.e(TAG, "Exception in Success callback", var4);
                }
            }

            if (this.afterCallback != null) {
                try {
                    this.afterCallback.onRequestFinished(device);
                } catch (Throwable var3) {
                    Log.e(TAG, "Exception in After callback", var3);
                }
            }

        });
        return true;
    } else {
        return false;
    }
}

void notifyFail(@NonNull BluetoothDevice device, int status) {
    if (!this.finished) {
        this.finished = true;
        if (this.internalFailCallback != null) {
            this.internalFailCallback.onRequestFailed(device, status);
        }

        this.handler.post(() -> {
            if (this.failCallback != null) {
                try {

```

```

        this.failCallback.onRequestFailed(device, status);
    } catch (Throwable var5) {
        Log.e(TAG, "Exception in Fail callback", var5);
    }
}

if (this.afterCallback != null) {
    try {
        this.afterCallback.onRequestFinished(device);
    } catch (Throwable var4) {
        Log.e(TAG, "Exception in After callback", var4);
    }
}

});
}

}

void notifyInvalidRequest() {
    if (!this.finished) {
        this.finished = true;
        this.handler.post(() -> {
            if (this.invalidRequestCallback != null) {
                try {
                    this.invalidRequestCallback.onInvalidRequest();
                } catch (Throwable var2) {
                    Log.e(TAG, "Exception in Invalid Request callback", var2);
                }
            }
        });
    }
}

}

static void assertNotMainThread() throws IllegalStateException {
    if (Looper.myLooper() == Looper.getMainLooper()) {
        throw new IllegalStateException("Cannot execute synchronous operation
from the UI thread.");
    }
}

final class RequestCallback implements SuccessCallback, FailCallback,
InvalidRequestCallback {
    static final int REASON_REQUEST_INVALID = -1000000;
    int status = 0;

    public void onRequestCompleted(@NonNull BluetoothDevice device) {
        Request.this.syncLock.open();
    }

    public void onRequestFailed(@NonNull BluetoothDevice device, int status) {
        this.status = status;
    }
}

```



```

        Request.this.syncLock.open();
    }

    public void onInvalidRequest() {
        this.status = -1000000;
        Request.this.syncLock.open();
    }

    boolean isSuccess() {
        return this.status == 0;
    }
}

static enum Type {
    SET,
    CONNECT,
    DISCONNECT,
    CREATE_BOND,
    ENSURE_BOND,
    REMOVE_BOND,
    WRITE,
    NOTIFY,
    INDICATE,
    READ,
    WRITE_DESCRIPTOR,
    READ_DESCRIPTOR,
    BEGIN_RELIABLE_WRITE,
    EXECUTE_RELIABLE_WRITE,
    ABORT_RELIABLE_WRITE,
    ENABLE_NOTIFICATIONS,
    ENABLE_INDICATIONS,
    DISABLE_NOTIFICATIONS,
    DISABLE_INDICATIONS,
    WAIT_FOR_NOTIFICATION,
    WAIT_FOR_INDICATION,
    WAIT_FOR_READ,
    WAIT_FOR_WRITE,
    WAIT_FOR_CONDITION,
    SET_VALUE,
    SET_DESCRIPTOR_VALUE,
    /** @deprecated */
    @Deprecated
    READ_BATTERY_LEVEL,
    /** @deprecated */
    @Deprecated
    ENABLE_BATTERY_LEVEL_NOTIFICATIONS,
    /** @deprecated */
    @Deprecated
    DISABLE_BATTERY_LEVEL_NOTIFICATIONS,
    ENABLE_SERVICE_CHANGED_INDICATIONS,
    REQUEST_MTU,
    REQUEST_CONNECTION_PRIORITY,
    SET_PREFERRED_PHY,
    READ_PHY,

```

```

        READ_RSSI,
        REFRESH_CACHE,
        SLEEP;
    }
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: RequestHandler.java

package no.nordicsemi.android.ble;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;

abstract class RequestHandler implements CallbackHandler {
    abstract void enqueue(@NonNull Request var1);

    abstract void cancelQueue();

    abstract void cancelCurrent();

    abstract void onRequestTimeout(@NonNull BluetoothDevice var1, @NonNull
TimeoutableRequest var2);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: RequestQueue.java

package no.nordicsemi.android.ble;

import android.os.Handler;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import java.util.Deque;
import java.util.LinkedList;
import no.nordicsemi.android.ble.callback.AfterCallback;
import no.nordicsemi.android.ble.callback.BeforeCallback;
import no.nordicsemi.android.ble.callback.FailCallback;
import no.nordicsemi.android.ble.callback.InvalidRequestCallback;
import no.nordicsemi.android.ble.callback.SuccessCallback;

public class RequestQueue extends TimeoutableRequest {
    @NonNull
    private final Deque<Request> requests = new LinkedList();

    RequestQueue() {
        super(Request.Type.SET);
    }
}

```

```
@NonNull
RequestQueue setRequestHandler(@NonNull RequestHandler requestHandler) {
    super.setRequestHandler(requestHandler);
    return this;
}
```

```
@NonNull
public RequestQueue setHandler(@Nullable Handler handler) {
    super.setHandler(handler);
    return this;
}
```

```
@NonNull
public RequestQueue done(@NonNull SuccessCallback callback) {
    super.done(callback);
    return this;
}
```

```
@NonNull
public RequestQueue fail(@NonNull FailCallback callback) {
    super.fail(callback);
    return this;
}
```

```
@NonNull
public RequestQueue invalid(@NonNull InvalidRequestCallback callback) {
    super.invalid(callback);
    return this;
}
```

```
@NonNull
public RequestQueue before(@NonNull BeforeCallback callback) {
    super.before(callback);
    return this;
}
```

```
@NonNull
public RequestQueue then(@NonNull AfterCallback callback) {
    super.then(callback);
    return this;
}
```

```
@NonNull
public RequestQueue timeout(@IntRange(from = 0L) long timeout) {
    super.timeout(timeout);
    return this;
}
```

```
@NonNull
public RequestQueue add(@NonNull Operation operation) {
    if (operation instanceof Request) {
        Request request = (Request)operation;
        if (request.enqueue() {
            throw new IllegalStateException("Request already enqueued");
        }
    }
}
```

```

        } else {
            request.internalFail(this::notifyFail);
            this.requests.add(request);
            request.enqueued = true;
            return this;
        }
    } else {
        throw new IllegalArgumentException("Operation does not extend
Request");
    }
}

void addFirst(@NonNull Request request) {
    this.requests.addFirst(request);
}

@IntRange(
    from = 0L
)
public int size() {
    return this.requests.size();
}

public boolean isEmpty() {
    return this.requests.isEmpty();
}

public void cancel() {
    this.cancelQueue();
    super.cancel();
}

@Nullable
Request getNext() {
    try {
        return (Request)this.requests.remove();
    } catch (Exception var2) {
        return null;
    }
}

boolean hasMore() {
    return !this.finished && !this.requests.isEmpty();
}

void cancelQueue() {
    this.requests.clear();
}
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: SetValueRequest.java

```

```

package no.nordicsemi.android.ble;

import android.bluetooth.BluetoothGattCharacteristic;
import android.bluetooth.BluetoothGattDescriptor;
import android.os.Handler;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.AfterCallback;
import no.nordicsemi.android.ble.callback.BeforeCallback;
import no.nordicsemi.android.ble.callback.FailCallback;
import no.nordicsemi.android.ble.callback.InvalidRequestCallback;
import no.nordicsemi.android.ble.callback.SuccessCallback;

public final class SetValueRequest extends SimpleRequest {
    private final byte[] data;
    private boolean longReadSupported = true;

    SetValueRequest(@NonNull Request.Type type, @Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] data,
@IntRange(from = 0L) int offset, @IntRange(from = 0L) int length) {
        super(type, characteristic);
        this.data = Bytes.copy(data, offset, length);
    }

    SetValueRequest(@NonNull Request.Type type, @Nullable BluetoothGattDescriptor
descriptor, @Nullable byte[] data, @IntRange(from = 0L) int offset,
@IntRange(from = 0L) int length) {
        super(type, descriptor);
        this.data = Bytes.copy(data, offset, length);
    }

    @NonNull
    SetValueRequest setRequestHandler(@NonNull RequestHandler requestHandler) {
        super.setRequestHandler(requestHandler);
        return this;
    }

    @NonNull
    public SetValueRequest setHandler(@Nullable Handler handler) {
        super.setHandler(handler);
        return this;
    }

    @NonNull
    public SetValueRequest done(@NonNull SuccessCallback callback) {
        super.done(callback);
        return this;
    }

    @NonNull
    public SetValueRequest fail(@NonNull FailCallback callback) {
        super.fail(callback);
    }

```

```

        return this;
    }

    @NonNull
    public SetValueRequest invalid(@NonNull InvalidRequestCallback callback) {
        super.invalid(callback);
        return this;
    }

    @NonNull
    public SetValueRequest before(@NonNull BeforeCallback callback) {
        super.before(callback);
        return this;
    }

    @NonNull
    public SetValueRequest then(@NonNull AfterCallback callback) {
        super.then(callback);
        return this;
    }

    @NonNull
    public SetValueRequest allowLongRead(boolean longReadSupported) {
        this.longReadSupported = longReadSupported;
        return this;
    }

    byte[] getData(@IntRange(from = 23L,to = 517L) int mtu) {
        int maxLength = this.longReadSupported ? 512 : mtu - 3;
        return this.data.length < maxLength ? this.data : Bytes.copy(this.data, 0,
maxLength);
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: SimpleRequest.java

```

```

package no.nordicsemi.android.ble;

```

```

import android.bluetooth.BluetoothGattCharacteristic;
import android.bluetooth.BluetoothGattDescriptor;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.BeforeCallback;
import no.nordicsemi.android.ble.callback.FailCallback;
import no.nordicsemi.android.ble.callback.SuccessCallback;
import no.nordicsemi.android.ble.exception.BluetoothDisabledException;
import no.nordicsemi.android.ble.exception.DeviceDisconnectedException;
import no.nordicsemi.android.ble.exception.InvalidRequestException;
import no.nordicsemi.android.ble.exception.RequestFailedException;

```

```

public class SimpleRequest extends Request {

```

```

SimpleRequest(@NonNull Request.Type type) {
    super(type);
}

SimpleRequest(@NonNull Request.Type type, @Nullable
BluetoothGattCharacteristic characteristic) {
    super(type, characteristic);
}

SimpleRequest(@NonNull Request.Type type, @Nullable BluetoothGattDescriptor
descriptor) {
    super(type, descriptor);
}

public final void await() throws RequestFailedException,
DeviceDisconnectedException, BluetoothDisabledException, InvalidRequestException
{
    assertNotMainThread();
    BeforeCallback bc = this.beforeCallback;
    SuccessCallback sc = this.successCallback;
    FailCallback fc = this.failCallback;

    try {
        if (!this.finished && !this.enqueued) {
            this.syncLock.close();
            Request.RequestCallback callback = new Request.RequestCallback();
            this.beforeCallback = null;
            this.done(callback).fail(callback).invalid(callback).enqueue();
            this.syncLock.block();
            if (!callback.isSuccess()) {
                if (callback.status == -1) {
                    throw new DeviceDisconnectedException();
                } else if (callback.status == -100) {
                    throw new BluetoothDisabledException();
                } else if (callback.status == -1000000) {
                    throw new InvalidRequestException(this);
                } else {
                    throw new RequestFailedException(this, callback.status);
                }
            }
        } else {
            throw new IllegalStateException();
        }
    } finally {
        this.beforeCallback = bc;
        this.successCallback = sc;
        this.failCallback = fc;
    }
}
}

```

// Directory: app

// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble

```
// File: SimpleValueRequest.java

package no.nordicsemi.android.ble;

import android.bluetooth.BluetoothGattCharacteristic;
import android.bluetooth.BluetoothGattDescriptor;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.exception.BluetoothDisabledException;
import no.nordicsemi.android.ble.exception.DeviceDisconnectedException;
import no.nordicsemi.android.ble.exception.InvalidRequestException;
import no.nordicsemi.android.ble.exception.RequestFailedException;

public abstract class SimpleValueRequest<T> extends SimpleRequest {
    T valueCallback;

    SimpleValueRequest(@NonNull Request.Type type) {
        super(type);
    }

    SimpleValueRequest(@NonNull Request.Type type, @Nullable
BluetoothGattCharacteristic characteristic) {
        super(type, characteristic);
    }

    SimpleValueRequest(@NonNull Request.Type type, @Nullable
BluetoothGattDescriptor descriptor) {
        super(type, descriptor);
    }

    @NonNull
    public SimpleValueRequest<T> with(@NonNull T callback) {
        this.valueCallback = callback;
        return this;
    }

    @NonNull
    public <E extends T> E await(@NonNull E response) throws
RequestFailedException, DeviceDisconnectedException, BluetoothDisabledException,
InvalidRequestException {
        assertNotMainThread();
        Object vc = this.valueCallback;

        Object var3;
        try {
            this.with(response).await();
            var3 = response;
        } finally {
            this.valueCallback = vc;
        }

        return var3;
    }
}
```



```

    @NonNull
    public <E extends T> E await(@NonNull Class<E> responseClass) throws
    RequestFailedException, DeviceDisconnectedException, BluetoothDisabledException,
    InvalidRequestException {
        assertNotMainThread();

        try {
            E response = responseClass.newInstance();
            return this.await(response);
        } catch (IllegalAccessException var3) {
            throw new IllegalArgumentException("Couldn't instantiate " +
            responseClass.getCanonicalName() + " class. Is the default constructor
            accessible?");
        } catch (InstantiationException var4) {
            throw new IllegalArgumentException("Couldn't instantiate " +
            responseClass.getCanonicalName() + " class. Does it have a default constructor
            with no arguments?");
        }
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: SleepRequest.java

```

```

package no.nordicsemi.android.ble;

```

```

import android.os.Handler;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.AfterCallback;
import no.nordicsemi.android.ble.callback.BeforeCallback;
import no.nordicsemi.android.ble.callback.FailCallback;
import no.nordicsemi.android.ble.callback.InvalidRequestCallback;
import no.nordicsemi.android.ble.callback.SuccessCallback;

```

```

public final class SleepRequest extends TimeoutableRequest implements Operation
{

```

```

    SleepRequest(@NonNull Request.Type type, @IntRange(from = 0L) long delay) {
        super(type);
        this.timeout = delay;
    }

```

```

    @NonNull
    SleepRequest setRequestHandler(@NonNull RequestHandler requestHandler) {
        super.setRequestHandler(requestHandler);
        return this;
    }

```

```

    @NonNull
    public SleepRequest setHandler(@Nullable Handler handler) {
        super.setHandler(handler);
    }

```

```

        return this;
    }

    @NonNull
    public SleepRequest done(@NonNull SuccessCallback callback) {
        super.done(callback);
        return this;
    }

    @NonNull
    public SleepRequest fail(@NonNull FailCallback callback) {
        super.fail(callback);
        return this;
    }

    @NonNull
    public SleepRequest invalid(@NonNull InvalidRequestCallback callback) {
        super.invalid(callback);
        return this;
    }

    @NonNull
    public SleepRequest before(@NonNull BeforeCallback callback) {
        super.before(callback);
        return this;
    }

    @NonNull
    public SleepRequest then(@NonNull AfterCallback callback) {
        super.then(callback);
        return this;
    }

    @NonNull
    public SleepRequest timeout(@IntRange(from = 0L) long timeout) {
        super.timeout(timeout);
        return this;
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: TimeoutableRequest.java

```

```

package no.nordicsemi.android.ble;

import android.bluetooth.BluetoothDevice;
import android.bluetooth.BluetoothGattCharacteristic;
import android.bluetooth.BluetoothGattDescriptor;
import android.os.Handler;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;

```

```

import java.util.concurrent.CancellationException;
import no.nordicsemi.android.ble.callback.FailCallback;
import no.nordicsemi.android.ble.callback.SuccessCallback;
import no.nordicsemi.android.ble.exceptionBluetoothDisabledException;
import no.nordicsemi.android.ble.exception.DeviceDisconnectedException;
import no.nordicsemi.android.ble.exception.InvalidRequestException;
import no.nordicsemi.android.ble.exception.RequestFailedException;

public abstract class TimeoutableRequest extends Request {
    @Nullable
    private Runnable timeoutCallback;
    protected boolean cancelled;
    protected long timeout;

    TimeoutableRequest(@NonNull Request.Type type) {
        super(type);
    }

    TimeoutableRequest(@NonNull Request.Type type, @Nullable
BluetoothGattCharacteristic characteristic) {
        super(type, characteristic);
    }

    TimeoutableRequest(@NonNull Request.Type type, @Nullable
BluetoothGattDescriptor descriptor) {
        super(type, descriptor);
    }

    @NonNull
    TimeoutableRequest setRequestHandler(@NonNull RequestHandler requestHandler)
{
        super.setRequestHandler(requestHandler);
        return this;
    }

    @NonNull
    public TimeoutableRequest setHandler(@Nullable Handler handler) {
        super.setHandler(handler);
        return this;
    }

    @NonNull
    public TimeoutableRequest timeout(@IntRange(from = 0L) long timeout) {
        if (this.timeoutCallback != null) {
            throw new IllegalStateException("Request already started");
        } else {
            this.timeout = timeout;
            return this;
        }
    }

    public void cancel() {
        if (!this.started) {
            this.cancelled = true;

```

```

        this.finished = true;
    } else if (!this.finished) {
        this.cancelled = true;
        this.requestHandler.cancelCurrent();
    }
}

public final void enqueue() {
    super.enqueue();
}

/** @deprecated */
@Deprecated
public final void enqueue(@IntRange(from = 0L) long timeout) {
    this.timeout(timeout).enqueue();
}

public final void await() throws RequestFailedException,
DeviceDisconnectedException, BluetoothDisabledException,
InvalidRequestException, InterruptedException {
    assertNotMainThread();
    if (this.cancelled) {
        throw new CancellationException();
    } else if (!this.finished && !this.enqueue()) {
        SuccessCallback sc = this.successCallback;
        FailCallback fc = this.failCallback;

        try {
            this.syncLock.close();
            Request.RequestCallback callback = new Request.RequestCallback();
            this.done(callback).fail(callback).invalid(callback).enqueue();
            if (!this.syncLock.block(this.timeout)) {
                throw new InterruptedException();
            }

            if (!callback.isSuccess()) {
                if (callback.status == -7) {
                    throw new CancellationException();
                }

                if (callback.status == -1) {
                    throw new DeviceDisconnectedException();
                }

                if (callback.status == -100) {
                    throw new BluetoothDisabledException();
                }

                if (callback.status == -1000000) {
                    throw new InvalidRequestException(this);
                }

                throw new RequestFailedException(this, callback.status);
            }

```

```

        }
    } finally {
        this.successCallback = sc;
        this.failCallback = fc;
    }

    } else {
        throw new IllegalStateException();
    }
}

/** @deprecated */
@Deprecated
public final void await(@IntRange(from = 0L) long timeout) throws
RequestFailedException, InterruptedException, DeviceDisconnectedException,
BluetoothDisabledException, InvalidRequestException, CancellationException {
    this.timeout(timeout).await();
}

void notifyStarted(@NonNull BluetoothDevice device) {
    if (this.timeout > 0L) {
        this.timeoutCallback = () -> {
            this.timeoutCallback = null;
            if (!this.finished) {
                this.requestHandler.onRequestTimeout(device, this);
            }
        };
        this.handler.postDelayed(this.timeoutCallback, this.timeout);
    }

    super.notifyStarted(device);
}

boolean notifySuccess(@NonNull BluetoothDevice device) {
    if (this.timeoutCallback != null) {
        this.handler.removeCallbacks(this.timeoutCallback);
        this.timeoutCallback = null;
    }

    return super.notifySuccess(device);
}

void notifyFail(@NonNull BluetoothDevice device, int status) {
    if (this.timeoutCallback != null) {
        this.handler.removeCallbacks(this.timeoutCallback);
        this.timeoutCallback = null;
    }

    super.notifyFail(device, status);
}

void notifyInvalidRequest() {
    if (this.timeoutCallback != null) {

```

```

        this.handler.removeCallbacks(this.timeoutCallback);
        this.timeoutCallback = null;
    }

    super.notifyInvalidRequest();
}

public final boolean isCancelled() {
    return this.cancelled;
}
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: TimeoutableValueRequest.java

package no.nordicsemi.android.ble;

import android.bluetooth.BluetoothGattCharacteristic;
import android.bluetooth.BluetoothGattDescriptor;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import java.util.concurrent.CancellationException;
import no.nordicsemi.android.ble.exception.BluetoothDisabledException;
import no.nordicsemi.android.ble.exception.DeviceDisconnectedException;
import no.nordicsemi.android.ble.exception.InvalidRequestException;
import no.nordicsemi.android.ble.exception.RequestFailedException;

public abstract class TimeoutableValueRequest<T> extends TimeoutableRequest {
    T valueCallback;

    TimeoutableValueRequest(@NonNull Request.Type type) {
        super(type);
    }

    TimeoutableValueRequest(@NonNull Request.Type type, @Nullable
BluetoothGattCharacteristic characteristic) {
        super(type, characteristic);
    }

    TimeoutableValueRequest(@NonNull Request.Type type, @Nullable
BluetoothGattDescriptor descriptor) {
        super(type, descriptor);
    }

    @NonNull
    public TimeoutableValueRequest<T> timeout(@IntRange(from = 0L) long timeout)
{
        super.timeout(timeout);
        return this;
    }
}

```

```

@NonNull
public TimeoutableValueRequest<T> with(@NonNull T callback) {
    this.valueCallback = callback;
    return this;
}

```

```

@NonNull
public <E extends T> E await(@NonNull E response) throws
RequestFailedException, DeviceDisconnectedException, BluetoothDisabledException,
InvalidRequestException, InterruptedException, CancellationException {
    assertNotMainThread();
    Object vc = this.valueCallback;

    Object var3;
    try {
        this.with(response).await();
        var3 = response;
    } finally {
        this.valueCallback = vc;
    }

    return var3;
}

```

```

@NonNull
public <E extends T> E await(@NonNull Class<E> responseClass) throws
RequestFailedException, DeviceDisconnectedException, BluetoothDisabledException,
InvalidRequestException, InterruptedException, CancellationException {
    assertNotMainThread();

    try {
        E response = responseClass.newInstance();
        return this.await(response);
    } catch (IllegalAccessException var3) {
        throw new IllegalArgumentException("Couldn't instantiate " +
responseClass.getCanonicalName() + " class. Is the default constructor
accessible?");
    } catch (InstantiationException var4) {
        throw new IllegalArgumentException("Couldn't instantiate " +
responseClass.getCanonicalName() + " class. Does it have a default constructor
with no arguments?");
    }
}

```

```

/** @deprecated */
@Deprecated
@NonNull
public <E extends T> E await(@NonNull Class<E> responseClass, @IntRange(from
= 0L) long timeout) throws RequestFailedException, InterruptedException,
DeviceDisconnectedException, BluetoothDisabledException,
InvalidRequestException, CancellationException {
    return this.timeout(timeout).await(responseClass);
}

```

```

    /** @deprecated */
    @Deprecated
    @NonNull
    public <E extends T> E await(@NonNull E response, @IntRange(from = 0L) long
timeout) throws RequestFailedException, InterruptedException,
DeviceDisconnectedException, BluetoothDisabledException,
InvalidRequestException, CancellationException {
        return this.timeout(timeout).await(response);
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: ValueChangedCallback.java

```

```

package no.nordicsemi.android.ble;

```

```

import android.bluetooth.BluetoothDevice;
import android.os.Handler;
import android.util.Log;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.ClosedCallback;
import no.nordicsemi.android.ble.callback.DataReceivedCallback;
import no.nordicsemi.android.ble.callback.ReadProgressCallback;
import no.nordicsemi.android.ble.data.Data;
import no.nordicsemi.android.ble.data.DataFilter;
import no.nordicsemi.android.ble.data.DataMerger;
import no.nordicsemi.android.ble.data.DataStream;
import no.nordicsemi.android.ble.data.PacketFilter;

```

```

public class ValueChangedCallback {
    private static final String TAG = ValueChangedCallback.class.getSimpleName();
    private ClosedCallback closedCallback;
    private ReadProgressCallback progressCallback;
    private DataReceivedCallback valueCallback;
    private DataMerger dataMerger;
    private DataStream buffer;
    private DataFilter filter;
    private PacketFilter packetFilter;
    private CallbackHandler handler;
    private int count = 0;

```

```

    ValueChangedCallback(CallbackHandler handler) {
        this.handler = handler;
    }

```

```

    @NonNull
    public ValueChangedCallback setHandler(@Nullable final Handler handler) {
        this.handler = new CallbackHandler() {
            public void post(@NonNull Runnable r) {
                if (handler != null) {
                    handler.post(r);
                }
            }
        };
    }

```



```

        } else {
            r.run();
        }

    }

    public void postDelayed(@NonNull Runnable r, long delayMillis) {
    }

    public void removeCallbacks(@NonNull Runnable r) {
    }
};
return this;
}

@NonNull
public ValueChangedCallback with(@NonNull DataReceivedCallback callback) {
    this.valueCallback = callback;
    return this;
}

@NonNull
public ValueChangedCallback filter(@NonNull DataFilter filter) {
    this.filter = filter;
    return this;
}

@NonNull
public ValueChangedCallback filterPacket(@NonNull PacketFilter filter) {
    this.packetFilter = filter;
    return this;
}

@NonNull
public ValueChangedCallback merge(@NonNull DataMerger merger) {
    this.dataMerger = merger;
    this.progressCallback = null;
    return this;
}

@NonNull
public ValueChangedCallback merge(@NonNull DataMerger merger, @NonNull
ReadProgressCallback callback) {
    this.dataMerger = merger;
    this.progressCallback = callback;
    return this;
}

@NonNull
public ValueChangedCallback then(@NonNull ClosedCallback callback) {
    this.closedCallback = callback;
    return this;
}

```

```

boolean matches(byte[] packet) {
    return this.filter == null || this.filter.filter(packet);
}

void notifyValueChanged(@NonNull BluetoothDevice device, @Nullable byte[]
value) {
    DataReceivedCallback valueCallback = this.valueCallback;
    if (valueCallback != null) {
        if (this.dataMerger != null || this.packetFilter != null &&
!this.packetFilter.filter(value)) {
            this.handler.post(() -> {
                if (this.progressCallback != null) {
                    try {
                        this.progressCallback.onPacketReceived(device, value,
this.count);
                    } catch (Throwable var4) {
                        Log.e(TAG, "Exception in Progress callback", var4);
                    }
                }
            });
        }
        if (this.buffer == null) {
            this.buffer = new DataStream();
        }

        if (this.dataMerger.merge(this.buffer, value, this.count++)) {
            byte[] merged = this.buffer.toByteArray();
            if (this.packetFilter == null ||
this.packetFilter.filter(merged)) {
                Data data = new Data(merged);
                this.handler.post(() -> {
                    try {
                        valueCallback.onDataReceived(device, data);
                    } catch (Throwable var4) {
                        Log.e(TAG, "Exception in Value callback", var4);
                    }
                });
            }

            this.buffer = null;
            this.count = 0;
        }
    } else {
        Data data = new Data(value);
        this.handler.post(() -> {
            try {
                valueCallback.onDataReceived(device, data);
            } catch (Throwable var4) {
                Log.e(TAG, "Exception in Value callback", var4);
            }
        });
    }
}

```

```

    }
}

void notifyClosed() {
    if (this.closedCallback != null) {
        try {
            this.closedCallback.onClosed();
        } catch (Throwable var2) {
            Log.e(TAG, "Exception in Closed callback", var2);
        }
    }

    this.free();
}

private void free() {
    this.closedCallback = null;
    this.valueCallback = null;
    this.dataMerger = null;
    this.progressCallback = null;
    this.filter = null;
    this.packetFilter = null;
    this.buffer = null;
    this.count = 0;
}
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: WaitForReadRequest.java

package no.nordicsemi.android.ble;

import android.bluetooth.BluetoothDevice;
import android.bluetooth.BluetoothGattCharacteristic;
import android.bluetooth.BluetoothGattDescriptor;
import android.os.Handler;
import android.util.Log;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.AfterCallback;
import no.nordicsemi.android.ble.callback.BeforeCallback;
import no.nordicsemi.android.ble.callback.DataSentCallback;
import no.nordicsemi.android.ble.callback.FailCallback;
import no.nordicsemi.android.ble.callback.InvalidRequestCallback;
import no.nordicsemi.android.ble.callback.SuccessCallback;
import no.nordicsemi.android.ble.callback.WriteProgressCallback;
import no.nordicsemi.android.ble.data.Data;
import no.nordicsemi.android.ble.data.DataSplitter;
import no.nordicsemi.android.ble.data.DefaultMtuSplitter;

```

```

public final class WaitForReadRequest extends AwaitingRequest<DataSentCallback>
implements Operation {
    private static final DataSplitter MTU_SPLITTER = new DefaultMtuSplitter();
    private WriteProgressCallback progressCallback;
    private DataSplitter dataSplitter;
    private byte[] data;
    private byte[] nextChunk;
    private int count = 0;
    private boolean complete = false;

    WaitForReadRequest(@NonNull Request.Type type, @Nullable
BluetoothGattCharacteristic characteristic) {
        super(type, characteristic);
        this.data = null;
        this.complete = true;
    }

    WaitForReadRequest(@NonNull Request.Type type, @Nullable
BluetoothGattDescriptor descriptor) {
        super(type, descriptor);
        this.data = null;
        this.complete = true;
    }

    WaitForReadRequest(@NonNull Request.Type type, @Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] data,
@IntRange(from = 0L) int offset, @IntRange(from = 0L) int length) {
        super(type, characteristic);
        this.data = Bytes.copy(data, offset, length);
    }

    WaitForReadRequest(@NonNull Request.Type type, @Nullable
BluetoothGattDescriptor descriptor, @Nullable byte[] data, @IntRange(from = 0L)
int offset, @IntRange(from = 0L) int length) {
        super(type, descriptor);
        this.data = Bytes.copy(data, offset, length);
    }

    void setDataIfNull(@Nullable byte[] data) {
        if (this.data == null) {
            this.data = data;
        }
    }

    @NonNull
    WaitForReadRequest setRequestHandler(@NonNull RequestHandler requestHandler)
{
        super.setRequestHandler(requestHandler);
        return this;
    }

    @NonNull
    public WaitForReadRequest setHandler(@Nullable Handler handler) {

```

```
    super.setHandler(handler);  
    return this;  
}
```

```
@NonNull  
public WaitForReadRequest done(@NonNull SuccessCallback callback) {  
    super.done(callback);  
    return this;  
}
```

```
@NonNull  
public WaitForReadRequest fail(@NonNull FailCallback callback) {  
    super.fail(callback);  
    return this;  
}
```

```
@NonNull  
public WaitForReadRequest invalid(@NonNull InvalidRequestCallback callback) {  
    super.invalid(callback);  
    return this;  
}
```

```
@NonNull  
public WaitForReadRequest before(@NonNull BeforeCallback callback) {  
    super.before(callback);  
    return this;  
}
```

```
@NonNull  
public WaitForReadRequest then(@NonNull AfterCallback callback) {  
    super.then(callback);  
    return this;  
}
```

```
@NonNull  
public WaitForReadRequest with(@NonNull DataSentCallback callback) {  
    super.with(callback);  
    return this;  
}
```

```
@NonNull  
public WaitForReadRequest trigger(@NonNull Operation trigger) {  
    super.trigger(trigger);  
    return this;  
}
```

```
@NonNull  
public WaitForReadRequest split(@NonNull DataSplitter splitter) {  
    this.dataSplitter = splitter;  
    this.progressCallback = null;  
    return this;  
}
```

```
@NonNull
```

```

    public WaitForReadRequest split(@NonNull DataSplitter splitter, @NonNull
WriteProgressCallback callback) {
        this.dataSplitter = splitter;
        this.progressCallback = callback;
        return this;
    }

    @NonNull
    public WaitForReadRequest split() {
        this.dataSplitter = MTU_SPLITTER;
        this.progressCallback = null;
        return this;
    }

    @NonNull
    public WaitForReadRequest split(@NonNull WriteProgressCallback callback) {
        this.dataSplitter = MTU_SPLITTER;
        this.progressCallback = callback;
        return this;
    }

    @NonNull
    byte[] getData(@IntRange(from = 23L,to = 517L) int mtu) {
        if (this.dataSplitter != null && this.data != null) {
            int maxLength = mtu - 1;
            byte[] chunk = this.nextChunk;
            if (chunk == null) {
                chunk = this.dataSplitter.chunk(this.data, this.count, maxLength);
            }

            if (chunk != null) {
                this.nextChunk = this.dataSplitter.chunk(this.data, this.count + 1,
maxLength);
            }

            if (this.nextChunk == null) {
                this.complete = true;
            }

            return chunk != null ? chunk : new byte[0];
        } else {
            this.complete = true;
            return this.data != null ? this.data : new byte[0];
        }
    }

    void notifyPacketRead(@NonNull BluetoothDevice device, @Nullable byte[] data)
{
    this.handler.post(() -> {
        if (this.progressCallback != null) {
            try {
                this.progressCallback.onPacketSent(device, data, this.count);
            } catch (Throwable var4) {
                Log.e(TAG, "Exception in Progress callback", var4);
            }
        }
    });
}

```

```

        }
    }

    });
    ++this.count;
}

boolean notifySuccess(@NonNull BluetoothDevice device) {
    this.handler.post(() -> {
        if (this.valueCallback != null) {
            try {
                ((DataSentCallback)this.valueCallback).onDataSent(device, new
Data(this.data));
            } catch (Throwable var3) {
                Log.e(TAG, "Exception in Value callback", var3);
            }
        }
    });
    return super.notifySuccess(device);
}

boolean hasMore() {
    return !this.complete;
}
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: WaitForValueChangedRequest.java

```

```

package no.nordicsemi.android.ble;

import android.bluetooth.BluetoothDevice;
import android.bluetooth.BluetoothGattCharacteristic;
import android.bluetooth.BluetoothGattDescriptor;
import android.os.Handler;
import android.util.Log;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import java.util.concurrent.CancellationException;
import no.nordicsemi.android.ble.callback.AfterCallback;
import no.nordicsemi.android.ble.callback.BeforeCallback;
import no.nordicsemi.android.ble.callback.DataReceivedCallback;
import no.nordicsemi.android.ble.callback.FailCallback;
import no.nordicsemi.android.ble.callback.InvalidRequestCallback;
import no.nordicsemi.android.ble.callback.ReadProgressCallback;
import no.nordicsemi.android.ble.callback.SuccessCallback;
import no.nordicsemi.android.ble.callback.profile.ProfileReadResponse;
import no.nordicsemi.android.ble.data.Data;
import no.nordicsemi.android.ble.data.DataFilter;
import no.nordicsemi.android.ble.data.DataMerger;

```

```

import no.nordicsemi.android.ble.data.DataStream;
import no.nordicsemi.android.ble.data.PacketFilter;
import no.nordicsemi.android.ble.exceptionBluetoothDisabledException;
import no.nordicsemi.android.ble.exception.DeviceDisconnectedException;
import no.nordicsemi.android.ble.exception.InvalidDataException;
import no.nordicsemi.android.ble.exception.InvalidRequestException;
import no.nordicsemi.android.ble.exception.RequestFailedException;

public final class WaitForValueChangedRequest extends
AwaitingRequest<DataReceivedCallback> implements Operation {
    private ReadProgressCallback progressCallback;
    private DataMerger dataMerger;
    private DataStream buffer;
    private DataFilter filter;
    private PacketFilter packetFilter;
    private boolean deviceDisconnected;
    private boolean bluetoothDisabled;
    private int count = 0;
    private boolean complete = false;

    WaitForValueChangedRequest(@NonNull Request.Type type, @Nullable
BluetoothGattCharacteristic characteristic) {
        super(type, characteristic);
    }

    WaitForValueChangedRequest(@NonNull Request.Type type, @Nullable
BluetoothGattDescriptor descriptor) {
        super(type, descriptor);
    }

    @NonNull
    WaitForValueChangedRequest setRequestHandler(@NonNull RequestHandler
requestHandler) {
        super.setRequestHandler(requestHandler);
        return this;
    }

    @NonNull
    public WaitForValueChangedRequest setHandler(@Nullable Handler handler) {
        super.setHandler(handler);
        return this;
    }

    @NonNull
    public WaitForValueChangedRequest timeout(@IntRange(from = 0L) long timeout)
{
        super.timeout(timeout);
        return this;
    }

    @NonNull
    public WaitForValueChangedRequest done(@NonNull SuccessCallback callback) {
        super.done(callback);
        return this;
    }

```



```

    }

    @NonNull
    public WaitForValueChangedRequest fail(@NonNull FailCallback callback) {
        super.fail(callback);
        return this;
    }

    @NonNull
    public WaitForValueChangedRequest invalid(@NonNull InvalidRequestCallback
callback) {
        super.invalid(callback);
        return this;
    }

    @NonNull
    public WaitForValueChangedRequest before(@NonNull BeforeCallback callback) {
        super.before(callback);
        return this;
    }

    @NonNull
    public WaitForValueChangedRequest then(@NonNull AfterCallback callback) {
        super.then(callback);
        return this;
    }

    @NonNull
    public WaitForValueChangedRequest with(@NonNull DataReceivedCallback
callback) {
        super.with(callback);
        return this;
    }

    @NonNull
    public WaitForValueChangedRequest trigger(@NonNull Operation trigger) {
        super.trigger(trigger);
        return this;
    }

    @NonNull
    public WaitForValueChangedRequest filter(@NonNull DataFilter filter) {
        this.filter = filter;
        return this;
    }

    @NonNull
    public WaitForValueChangedRequest filterPacket(@NonNull PacketFilter filter)
{
        this.packetFilter = filter;
        return this;
    }

    @NonNull

```

```

    public WaitForValueChangedRequest merge(@NonNull DataMerger merger) {
        this.dataMerger = merger;
        this.progressCallback = null;
        return this;
    }

    @NonNull
    public WaitForValueChangedRequest merge(@NonNull DataMerger merger, @NonNull
ReadProgressCallback callback) {
        this.dataMerger = merger;
        this.progressCallback = callback;
        return this;
    }

    @NonNull
    public <E extends ProfileReadResponse> E awaitValid(@NonNull E response)
throws RequestFailedException, InvalidDataException,
DeviceDisconnectedException, BluetoothDisabledException,
InvalidRequestException, InterruptedException, CancellationException {
        E result = (ProfileReadResponse)this.await(response);
        if (result != null && !result.isValid()) {
            throw new InvalidDataException(result);
        } else {
            return result;
        }
    }

    @NonNull
    public <E extends ProfileReadResponse> E awaitValid(@NonNull Class<E>
responseClass) throws RequestFailedException, InvalidDataException,
DeviceDisconnectedException, BluetoothDisabledException,
InvalidRequestException, InterruptedException, CancellationException {
        E response = (ProfileReadResponse)this.await(responseClass);
        if (response != null && !response.isValid()) {
            throw new InvalidDataException(response);
        } else {
            return response;
        }
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    public <E extends ProfileReadResponse> E awaitValid(@NonNull Class<E>
responseClass, @IntRange(from = 0L) long timeout) throws InterruptedException,
InvalidDataException, RequestFailedException, DeviceDisconnectedException,
BluetoothDisabledException, InvalidRequestException, CancellationException {
        return this.timeout(timeout).awaitValid(responseClass);
    }

    /** @deprecated */
    @Deprecated
    @NonNull
    public <E extends ProfileReadResponse> E awaitValid(@NonNull E response,

```

```

@IntRange(from = 0L) long timeout) throws InterruptedException,
InvalidDataException, DeviceDisconnectedException, RequestFailedException,
BluetoothDisabledException, InvalidRequestException, CancellationException {
    return this.timeout(timeout).awaitValid(response);
}

boolean matches(byte[] packet) {
    return this.filter == null || this.filter.filter(packet);
}

void notifyValueChanged(BluetoothDevice device, byte[] value) {
    DataReceivedCallback valueCallback =
(DataReceivedCallback)this.valueCallback;
    if (valueCallback == null) {
        if (this.packetFilter == null || this.packetFilter.filter(value)) {
            this.complete = true;
        }

        } else {
            if (this.dataMerger == null && (this.packetFilter == null ||
this.packetFilter.filter(value))) {
                this.complete = true;
                Data data = new Data(value);
                this.handler.post(() -> {
                    try {
                        valueCallback.onDataReceived(device, data);
                    } catch (Throwable var4) {
                        Log.e(TAG, "Exception in Value callback", var4);
                    }
                });
            } else {
                int c = this.count;
                this.handler.post(() -> {
                    if (this.progressCallback != null) {
                        try {
                            this.progressCallback.onPacketReceived(device, value, c);
                        } catch (Throwable var5) {
                            Log.e(TAG, "Exception in Progress callback", var5);
                        }
                    }
                });
            }
            if (this.buffer == null) {
                this.buffer = new DataStream();
            }

            if (this.dataMerger.merge(this.buffer, value, this.count++)) {
                byte[] merged = this.buffer.toByteArray();
                if (this.packetFilter == null ||
this.packetFilter.filter(merged)) {
                    this.complete = true;
                    Data data = new Data(merged);
                    this.handler.post(() -> {

```

```

        try {
            valueCallback.onDataReceived(device, data);
        } catch (Throwable var4) {
            Log.e(TAG, "Exception in Value callback", var4);
        }

    });
}

this.buffer = null;
this.count = 0;
}

}

}

boolean isComplete() {
    return this.complete;
}
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble
// File: WriteRequest.java

```

```

package no.nordicsemi.android.ble;

```

```

import android.bluetooth.BluetoothDevice;
import android.bluetooth.BluetoothGattCharacteristic;
import android.bluetooth.BluetoothGattDescriptor;
import android.os.Handler;
import android.util.Log;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import java.util.Arrays;
import no.nordicsemi.android.ble.callback.AfterCallback;
import no.nordicsemi.android.ble.callback.BeforeCallback;
import no.nordicsemi.android.ble.callback.DataSentCallback;
import no.nordicsemi.android.ble.callback.FailCallback;
import no.nordicsemi.android.ble.callback.InvalidRequestCallback;
import no.nordicsemi.android.ble.callback.SuccessCallback;
import no.nordicsemi.android.ble.callback.WriteProgressCallback;
import no.nordicsemi.android.ble.data.Data;
import no.nordicsemi.android.ble.data.DataSplitter;
import no.nordicsemi.android.ble.data.DefaultMtuSplitter;

```

```

public final class WriteRequest extends SimpleValueRequest<DataSentCallback>
implements Operation {
    private static final DataSplitter MTU_SPLITTER = new DefaultMtuSplitter();
    private WriteProgressCallback progressCallback;
    private DataSplitter dataSplitter;

```

```

private final byte[] data;
private final int writeType;
private byte[] currentChunk;
private byte[] nextChunk;
private int count;
private boolean complete;

WriteRequest(@NonNull Request.Type type) {
    this(type, (BluetoothGattCharacteristic)null);
}

WriteRequest(@NonNull Request.Type type, @Nullable
BluetoothGattCharacteristic characteristic) {
    super(type, characteristic);
    this.count = 0;
    this.complete = false;
    this.data = null;
    this.writeType = 0;
    this.complete = true;
}

WriteRequest(@NonNull Request.Type type, @Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] data,
@IntRange(from = 0L) int offset, @IntRange(from = 0L) int length, int writeType)
{
    super(type, characteristic);
    this.count = 0;
    this.complete = false;
    this.data = Bytes.copy(data, offset, length);
    this.writeType = writeType;
}

WriteRequest(@NonNull Request.Type type, @Nullable
BluetoothGattCharacteristic characteristic, @Nullable byte[] data,
@IntRange(from = 0L) int offset, @IntRange(from = 0L) int length) {
    super(type, characteristic);
    this.count = 0;
    this.complete = false;
    this.data = Bytes.copy(data, offset, length);
    this.writeType = 0;
}

WriteRequest(@NonNull Request.Type type, @Nullable BluetoothGattDescriptor
descriptor, @Nullable byte[] data, @IntRange(from = 0L) int offset,
@IntRange(from = 0L) int length) {
    super(type, descriptor);
    this.count = 0;
    this.complete = false;
    this.data = Bytes.copy(data, offset, length);
    this.writeType = 2;
}

@NonNull
WriteRequest setRequestHandler(@NonNull RequestHandler requestHandler) {

```

```

        super.setRequestHandler(requestHandler);
        return this;
    }

    @NonNull
    public WriteRequest setHandler(@Nullable Handler handler) {
        super.setHandler(handler);
        return this;
    }

    @NonNull
    public WriteRequest done(@NonNull SuccessCallback callback) {
        super.done(callback);
        return this;
    }

    @NonNull
    public WriteRequest fail(@NonNull FailCallback callback) {
        super.fail(callback);
        return this;
    }

    @NonNull
    public WriteRequest invalid(@NonNull InvalidRequestCallback callback) {
        super.invalid(callback);
        return this;
    }

    @NonNull
    public WriteRequest before(@NonNull BeforeCallback callback) {
        super.before(callback);
        return this;
    }

    @NonNull
    public WriteRequest then(@NonNull AfterCallback callback) {
        super.then(callback);
        return this;
    }

    @NonNull
    public WriteRequest with(@NonNull DataSentCallback callback) {
        super.with(callback);
        return this;
    }

    @NonNull
    public WriteRequest split(@NonNull DataSplitter splitter) {
        this.dataSplitter = splitter;
        this.progressCallback = null;
        return this;
    }

    @NonNull

```

```

    public WriteRequest split(@NonNull DataSplitter splitter, @NonNull
WriteProgressCallback callback) {
        this.dataSplitter = splitter;
        this.progressCallback = callback;
        return this;
    }

    @NonNull
    public WriteRequest split() {
        this.dataSplitter = MTU_SPLITTER;
        this.progressCallback = null;
        return this;
    }

    @NonNull
    public WriteRequest split(@NonNull WriteProgressCallback callback) {
        this.dataSplitter = MTU_SPLITTER;
        this.progressCallback = callback;
        return this;
    }

    void forceSplit() {
        if (this.dataSplitter == null) {
            this.split();
        }
    }

    @NonNull
    byte[] getData(@IntRange(from = 23L,to = 517L) int mtu) {
        if (this.dataSplitter != null && this.data != null) {
            int maxLength = this.writeType != 4 ? mtu - 3 : mtu - 12;
            byte[] chunk = this.nextChunk;
            if (chunk == null) {
                chunk = this.dataSplitter.chunk(this.data, this.count, maxLength);
            }

            if (chunk != null) {
                this.nextChunk = this.dataSplitter.chunk(this.data, this.count + 1,
maxLength);
            }

            if (this.nextChunk == null) {
                this.complete = true;
            }

            this.currentChunk = chunk;
            return chunk != null ? chunk : new byte[0];
        } else {
            this.complete = true;
            this.currentChunk = this.data;
            return this.data != null ? this.data : new byte[0];
        }
    }
}

```

```

        boolean notifyPacketSent(@NonNull BluetoothDevice device, @Nullable byte[]
data) {
            this.handler.post(() -> {
                if (this.progressCallback != null) {
                    try {
                        this.progressCallback.onPacketSent(device, this.currentChunk,
this.count);
                    } catch (Throwable var3) {
                        Log.e(TAG, "Exception in Progress callback", var3);
                    }
                }
            });
            ++this.count;
            if (this.complete) {
                this.handler.post(() -> {
                    if (this.valueCallback != null) {
                        try {
                            ((DataSentCallback)this.valueCallback).onDataSent(device, new
Data(this.data));
                        } catch (Throwable var3) {
                            Log.e(TAG, "Exception in Value callback", var3);
                        }
                    }
                });
            }
        }

        return this.writeType == 2 ? Arrays.equals(data, this.currentChunk) :
true;
    }

    boolean hasMore() {
        return !this.complete;
    }

    int getWriteType() {
        return this.writeType;
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\annotation
// File: BondState.java

```

```

package no.nordicsemi.android.ble.annotation;

```

```

import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;

```

```

@Retention(RetentionPolicy.SOURCE)
public @interface BondState {

```



```
}
```

```
// Directory: app  
// Subdirectory: classes.jar_Decompile.com\no\nordicsemi\android\ble\annotation  
// File: CharacteristicPermissions.java
```

```
package no.nordicsemi.android.ble.annotation;
```

```
import java.lang.annotation.Retention;  
import java.lang.annotation.RetentionPolicy;
```

```
@Retention(RetentionPolicy.SOURCE)  
public @interface CharacteristicPermissions {  
}
```

```
// Directory: app  
// Subdirectory: classes.jar_Decompile.com\no\nordicsemi\android\ble\annotation  
// File: CharacteristicProperties.java
```

```
package no.nordicsemi.android.ble.annotation;
```

```
import java.lang.annotation.Retention;  
import java.lang.annotation.RetentionPolicy;
```

```
@Retention(RetentionPolicy.SOURCE)  
public @interface CharacteristicProperties {  
}
```

```
// Directory: app  
// Subdirectory: classes.jar_Decompile.com\no\nordicsemi\android\ble\annotation  
// File: ConnectionPriority.java
```

```
package no.nordicsemi.android.ble.annotation;
```

```
import java.lang.annotation.Retention;  
import java.lang.annotation.RetentionPolicy;
```

```
@Retention(RetentionPolicy.SOURCE)  
public @interface ConnectionPriority {  
}
```

```
// Directory: app  
// Subdirectory: classes.jar_Decompile.com\no\nordicsemi\android\ble\annotation  
// File: ConnectionState.java
```

```
package no.nordicsemi.android.ble.annotation;
```

```
import java.lang.annotation.Retention;  
import java.lang.annotation.RetentionPolicy;
```

```
@Retention(RetentionPolicy.SOURCE)
public @interface ConnectionState {
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\annotation
// File: DescriptorPermissions.java
```

```
package no.nordicsemi.android.ble.annotation;
```

```
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
```

```
@Retention(RetentionPolicy.SOURCE)
public @interface DescriptorPermissions {
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\annotation
// File: DisconnectionReason.java
```

```
package no.nordicsemi.android.ble.annotation;
```

```
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
```

```
@Retention(RetentionPolicy.SOURCE)
public @interface DisconnectionReason {
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\annotation
// File: LogPriority.java
```

```
package no.nordicsemi.android.ble.annotation;
```

```
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
```

```
@Retention(RetentionPolicy.SOURCE)
public @interface LogPriority {
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\annotation
// File: PairingVariant.java
```

```
package no.nordicsemi.android.ble.annotation;
```

```
import java.lang.annotation.Retention;
```

```

import java.lang.annotation.RetentionPolicy;

@Retention(RetentionPolicy.SOURCE)
public @interface PairingVariant {
}

// Directory: app
// Subdirectory: classes.jar_Decompiler.com\no\nordicsemi\android\ble\annotation
// File: PhyMask.java

package no.nordicsemi.android.ble.annotation;

import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;

@Retention(RetentionPolicy.SOURCE)
public @interface PhyMask {
}

// Directory: app
// Subdirectory: classes.jar_Decompiler.com\no\nordicsemi\android\ble\annotation
// File: PhyOption.java

package no.nordicsemi.android.ble.annotation;

import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;

@Retention(RetentionPolicy.SOURCE)
public @interface PhyOption {
}

// Directory: app
// Subdirectory: classes.jar_Decompiler.com\no\nordicsemi\android\ble\annotation
// File: PhyValue.java

package no.nordicsemi.android.ble.annotation;

import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;

@Retention(RetentionPolicy.SOURCE)
public @interface PhyValue {
}

// Directory: app
// Subdirectory: classes.jar_Decompiler.com\no\nordicsemi\android\ble\annotation
// File: WriteType.java

package no.nordicsemi.android.ble.annotation;

```

```

import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;

@Retention(RetentionPolicy.SOURCE)
public @interface WriteType {
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\callback
// File: AfterCallback.java

package no.nordicsemi.android.ble.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;

@FunctionalInterface
public interface AfterCallback {
    void onRequestFinished(@NonNull BluetoothDevice var1);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\callback
// File: BeforeCallback.java

package no.nordicsemi.android.ble.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;

@FunctionalInterface
public interface BeforeCallback {
    void onRequestStarted(@NonNull BluetoothDevice var1);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\callback
// File: ClosedCallback.java

package no.nordicsemi.android.ble.callback;

@FunctionalInterface
public interface ClosedCallback {
    void onClose();
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\callback
// File: ConnectionParametersUpdatedCallback.java

```

```

package no.nordicsemi.android.ble.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;

@FunctionalInterface
public interface ConnectionParametersUpdatedCallback {
    void onConnectionUpdated(@NonNull BluetoothDevice var1, @IntRange(from =
6L,to = 3200L) int var2, @IntRange(from = 0L,to = 499L) int var3, @IntRange(from
= 10L,to = 3200L) int var4);
}

// Directory: app
// Subdirectory: classes.jar_Decompiler.com\no\nordicsemi\android\ble\callback
// File: ConnectionPriorityCallback.java

package no.nordicsemi.android.ble.callback;

/** @deprecated */
@Deprecated
@FunctionalInterface
public interface ConnectionPriorityCallback extends
ConnectionParametersUpdatedCallback {
}

// Directory: app
// Subdirectory: classes.jar_Decompiler.com\no\nordicsemi\android\ble\callback
// File: DataReceivedCallback.java

package no.nordicsemi.android.ble.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;
import no.nordicsemi.android.ble.data.Data;

@FunctionalInterface
public interface DataReceivedCallback {
    void onDataReceived(@NonNull BluetoothDevice var1, @NonNull Data var2);
}

// Directory: app
// Subdirectory: classes.jar_Decompiler.com\no\nordicsemi\android\ble\callback
// File: DataSentCallback.java

package no.nordicsemi.android.ble.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;
import no.nordicsemi.android.ble.data.Data;

```

```
@FunctionalInterface
public interface DataSentCallback {
    void onDataSent(@NonNull BluetoothDevice var1, @NonNull Data var2);
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\callback
// File: FailCallback.java
```

```
package no.nordicsemi.android.ble.callback;
```

```
import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;
```

```
@FunctionalInterface
public interface FailCallback {
    int REASON_DEVICE_DISCONNECTED = -1;
    int REASON_DEVICE_NOT_SUPPORTED = -2;
    int REASON_NULL_ATTRIBUTE = -3;
    int REASON_REQUEST_FAILED = -4;
    int REASON_TIMEOUT = -5;
    int REASON_VALIDATION = -6;
    int REASON_CANCELLED = -7;
    int REASON_BLUETOOTH_DISABLED = -100;

    void onRequestFailed(@NonNull BluetoothDevice var1, int var2);
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\callback
// File: InvalidRequestCallback.java
```

```
package no.nordicsemi.android.ble.callback;
```

```
@FunctionalInterface
public interface InvalidRequestCallback {
    void onInvalidRequest();
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\callback
// File: MtuCallback.java
```

```
package no.nordicsemi.android.ble.callback;
```

```
import android.bluetooth.BluetoothDevice;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
```

```
@FunctionalInterface
```

```
public interface MtuCallback {
    void onMtuChanged(@NonNull BluetoothDevice var1, @IntRange(from = 23L,to = 517L) int var2);
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\callback
// File: PhyCallback.java
```

```
package no.nordicsemi.android.ble.callback;
```

```
import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;
```

```
@FunctionalInterface
public interface PhyCallback {
    int PHY_LE_1M = 1;
    int PHY_LE_2M = 2;
    int PHY_LE_CODED = 3;

    void onPhyChanged(@NonNull BluetoothDevice var1, int var2, int var3);
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\callback
// File: ReadProgressCallback.java
```

```
package no.nordicsemi.android.ble.callback;
```

```
import android.bluetooth.BluetoothDevice;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
```

```
@FunctionalInterface
public interface ReadProgressCallback {
    void onPacketReceived(@NonNull BluetoothDevice var1, @Nullable byte[] var2,
@IntRange(from = 0L) int var3);
}
```

```
// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\callback
// File: RssiCallback.java
```

```
package no.nordicsemi.android.ble.callback;
```

```
import android.bluetooth.BluetoothDevice;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
```

```
@FunctionalInterface
```

```

public interface RssiCallback {
    void onRssiRead(@NonNull BluetoothDevice var1, @IntRange(from = -128L,to =
20L) int var2);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\no\nordicsemi\android\ble\callback
// File: SuccessCallback.java

package no.nordicsemi.android.ble.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;

@FunctionalInterface
public interface SuccessCallback {
    void onRequestCompleted(@NonNull BluetoothDevice var1);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\no\nordicsemi\android\ble\callback
// File: WriteProgressCallback.java

package no.nordicsemi.android.ble.callback;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;

@FunctionalInterface
public interface WriteProgressCallback {
    void onPacketSent(@NonNull BluetoothDevice var1, @Nullable byte[] var2,
@IntRange(from = 0L) int var3);
}

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\no\nordicsemi\android\ble\callback\profile
// File: ProfileDataCallback.java

package no.nordicsemi.android.ble.callback.profile;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;
import no.nordicsemi.android.ble.callback.DataReceivedCallback;
import no.nordicsemi.android.ble.data.Data;

public interface ProfileDataCallback extends DataReceivedCallback {
    default void onInvalidDataReceived(@NonNull BluetoothDevice device, @NonNull
Data data) {

```



```
    }  
}
```

```
// Directory: app  
// Subdirectory:  
classes.jar_Decompile.com\no\nordicsemi\android\ble\callback\profile  
// File: ProfileReadResponse.java
```

```
package no.nordicsemi.android.ble.callback.profile;
```

```
import android.bluetooth.BluetoothDevice;  
import android.os.Parcel;  
import android.os.Parcelable;  
import android.os.Parcelable.Creator;  
import androidx.annotation.NonNull;  
import no.nordicsemi.android.ble.data.Data;  
import no.nordicsemi.android.ble.response.ReadResponse;
```

```
public class ProfileReadResponse extends ReadResponse implements  
ProfileDataCallback, Parcelable {
```

```
    private boolean valid = true;
```

```
    public static final Creator<ProfileReadResponse> CREATOR = new  
Creator<ProfileReadResponse>() {
```

```
        public ProfileReadResponse createFromParcel(Parcel in) {  
            return new ProfileReadResponse(in);  
        }  
    }
```

```
        public ProfileReadResponse[] newArray(int size) {  
            return new ProfileReadResponse[size];  
        }  
    }
```

```
};
```

```
public ProfileReadResponse() {  
}
```

```
    public void onInvalidDataReceived(@NonNull BluetoothDevice device, @NonNull  
Data data) {  
        this.valid = false;  
    }
```

```
    public boolean isValid() {  
        return this.valid;  
    }
```

```
    protected ProfileReadResponse(Parcel in) {  
        super(in);  
        this.valid = in.readByte() != 0;  
    }
```

```
    public void writeToParcel(Parcel dest, int flags) {  
        super.writeToParcel(dest, flags);  
        dest.writeByte((byte)(this.valid ? 1 : 0));  
    }
```

```
}
```

```
// Directory: app  
// Subdirectory: classes.jar_Decompile.com\no\nordicsemi\android\ble\data  
// File: Data.java
```

```
package no.nordicsemi.android.ble.data;
```

```
import android.annotation.SuppressLint;  
import android.bluetooth.BluetoothGattCharacteristic;  
import android.bluetooth.BluetoothGattDescriptor;  
import android.os.Parcel;  
import android.os.Parcelable;  
import android.os.Parcelable.Creator;  
import androidx.annotation.IntRange;  
import androidx.annotation.NonNull;  
import androidx.annotation.Nullable;  
import java.lang.annotation.Retention;  
import java.lang.annotation.RetentionPolicy;
```

```
public class Data implements Parcelable {  
    private static final char[] HEX_ARRAY = "0123456789ABCDEF".toCharArray();  
    public static final int FORMAT_UINT8 = 17;  
    /** @deprecated */  
    @Deprecated  
    public static final int FORMAT_UINT16 = 18;  
    public static final int FORMAT_UINT16_LE = 18;  
    public static final int FORMAT_UINT16_BE = 274;  
    /** @deprecated */  
    @Deprecated  
    public static final int FORMAT_UINT24 = 19;  
    public static final int FORMAT_UINT24_LE = 19;  
    public static final int FORMAT_UINT24_BE = 275;  
    /** @deprecated */  
    @Deprecated  
    public static final int FORMAT_UINT32 = 20;  
    public static final int FORMAT_UINT32_LE = 20;  
    public static final int FORMAT_UINT32_BE = 276;  
    public static final int FORMAT_SINT8 = 33;  
    /** @deprecated */  
    @Deprecated  
    public static final int FORMAT_SINT16 = 34;  
    public static final int FORMAT_SINT16_LE = 34;  
    public static final int FORMAT_SINT16_BE = 290;  
    /** @deprecated */  
    @Deprecated  
    public static final int FORMAT_SINT24 = 35;  
    public static final int FORMAT_SINT24_LE = 35;  
    public static final int FORMAT_SINT24_BE = 291;  
    /** @deprecated */  
    @Deprecated  
    public static final int FORMAT_SINT32 = 36;  
    public static final int FORMAT_SINT32_LE = 36;
```

```

public static final int FORMAT_SINT32_BE = 292;
public static final int FORMAT_SFLOAT = 50;
public static final int FORMAT_FLOAT = 52;
protected byte[] mValue;
public static final Creator<Data> CREATOR = new Creator<Data>() {
    public Data createFromParcel(Parcel in) {
        return new Data(in);
    }

    public Data[] newArray(int size) {
        return new Data[size];
    }
};

public Data() {
    this.mValue = null;
}

public Data(@Nullable byte[] value) {
    this.mValue = value;
}

public static Data from(@NonNull String value) {
    return new Data(value.getBytes());
}

public static Data from(@NonNull BluetoothGattCharacteristic characteristic)
{
    return new Data(characteristic.getValue());
}

public static Data from(@NonNull BluetoothGattDescriptor descriptor) {
    return new Data(descriptor.getValue());
}

public static Data opCode(byte opCode) {
    return new Data(new byte[]{opCode});
}

public static Data opCode(byte opCode, byte parameter) {
    return new Data(new byte[]{opCode, parameter});
}

@Nullable
public byte[] getValue() {
    return this.mValue;
}

@Nullable
public String getStringValue(@IntRange(from = 0L) int offset) {
    if (this.mValue != null && offset <= this.mValue.length) {
        byte[] strBytes = new byte[this.mValue.length - offset];

        for(int i = 0; i != this.mValue.length - offset; ++i) {

```

```

        strBytes[i] = this.mValue[offset + i];
    }

    return new String(strBytes);
} else {
    return null;
}
}

public int size() {
    return this.mValue != null ? this.mValue.length : 0;
}

@NonNull
public String toString() {
    if (this.size() == 0) {
        return "";
    } else {
        char[] out = new char[this.mValue.length * 3 - 1];

        for(int j = 0; j < this.mValue.length; ++j) {
            int v = this.mValue[j] & 255;
            out[j * 3] = HEX_ARRAY[v >>> 4];
            out[j * 3 + 1] = HEX_ARRAY[v & 15];
            if (j != this.mValue.length - 1) {
                out[j * 3 + 2] = '-';
            }
        }

        return "(0x) " + new String(out);
    }
}

@Nullable
public Byte getByte(@IntRange(from = 0L) int offset) {
    return offset + 1 > this.size() ? null : this.mValue[offset];
}

@Nullable
public Integer getIntValue(int formatType, @IntRange(from = 0L) int offset) {
    if (offset + getTypeLen(formatType) > this.size()) {
        return null;
    } else {
        switch(formatType) {
            case 17:
                return unsignedByteToInt(this.mValue[offset]);
            case 18:
                return unsignedBytesToInt(this.mValue[offset], this.mValue[offset +
1]);
            case 19:
                return unsignedBytesToInt(this.mValue[offset], this.mValue[offset +
1], this.mValue[offset + 2], (byte)0);
            case 20:
                return unsignedBytesToInt(this.mValue[offset], this.mValue[offset +

```

```

1], this.mValue[offset + 2], this.mValue[offset + 3]);
    case 33:
        return unsignedToSigned(unsignedByteToInt(this.mValue[offset]), 8);
    case 34:
        return unsignedToSigned(unsignedBytesToInt(this.mValue[offset],
this.mValue[offset + 1]), 16);
    case 35:
        return unsignedToSigned(unsignedBytesToInt(this.mValue[offset],
this.mValue[offset + 1], this.mValue[offset + 2], (byte)0), 24);
    case 36:
        return unsignedToSigned(unsignedBytesToInt(this.mValue[offset],
this.mValue[offset + 1], this.mValue[offset + 2], this.mValue[offset + 3]), 32);
    case 274:
        return unsignedBytesToInt(this.mValue[offset + 1],
this.mValue[offset]);
    case 275:
        return unsignedBytesToInt(this.mValue[offset + 2],
this.mValue[offset + 1], this.mValue[offset], (byte)0);
    case 276:
        return unsignedBytesToInt(this.mValue[offset + 3],
this.mValue[offset + 2], this.mValue[offset + 1], this.mValue[offset]);
    case 290:
        return unsignedToSigned(unsignedBytesToInt(this.mValue[offset + 1],
this.mValue[offset]), 16);
    case 291:
        return unsignedToSigned(unsignedBytesToInt((byte)0,
this.mValue[offset + 2], this.mValue[offset + 1], this.mValue[offset]), 24);
    case 292:
        return unsignedToSigned(unsignedBytesToInt(this.mValue[offset + 3],
this.mValue[offset + 2], this.mValue[offset + 1], this.mValue[offset]), 32);
    default:
        return null;
    }
}
}

```

@Nullable

```

public Long getLongValue(int formatType, @IntRange(from = 0L) int offset) {
    if (offset + getTypeLen(formatType) > this.size()) {
        return null;
    } else {
        switch(formatType) {
            case 20:
                return unsignedBytesToLong(this.mValue[offset], this.mValue[offset +
1], this.mValue[offset + 2], this.mValue[offset + 3]);
            case 36:
                return unsignedToSigned(unsignedBytesToLong(this.mValue[offset],
this.mValue[offset + 1], this.mValue[offset + 2], this.mValue[offset + 3]), 32);
            case 276:
                return unsignedBytesToLong(this.mValue[offset + 3],
this.mValue[offset + 2], this.mValue[offset + 1], this.mValue[offset]);
            case 292:
                return unsignedToSigned(unsignedBytesToLong(this.mValue[offset + 3],
this.mValue[offset + 2], this.mValue[offset + 1], this.mValue[offset]), 32);

```

```

        default:
            return null;
        }
    }
}

@Nullable
public Float getFloatValue(int formatType, @IntRange(from = 0L) int offset) {
    if (offset + getTypeLen(formatType) > this.size()) {
        return null;
    } else {
        switch(formatType) {
            case 50:
                if (this.mValue[offset + 1] == 7 && this.mValue[offset] == -2) {
                    return Float.POSITIVE_INFINITY;
                } else if (this.mValue[offset + 1] == 7 && this.mValue[offset] == -1
|| this.mValue[offset + 1] == 8 && this.mValue[offset] == 0 ||
this.mValue[offset + 1] == 8 && this.mValue[offset] == 1) {
                    return Float.NaN;
                } else {
                    if (this.mValue[offset + 1] == 8 && this.mValue[offset] == 2) {
                        return Float.NEGATIVE_INFINITY;
                    }

                    return bytesToFloat(this.mValue[offset], this.mValue[offset +
1]);
                }
            case 52:
                if (this.mValue[offset + 3] == 0) {
                    if (this.mValue[offset + 2] == 127 && this.mValue[offset + 1] ==
-1) {
                        if (this.mValue[offset] == -2) {
                            return Float.POSITIVE_INFINITY;
                        }

                        if (this.mValue[offset] == -1) {
                            return Float.NaN;
                        }
                    } else if (this.mValue[offset + 2] == -128 && this.mValue[offset
+ 1] == 0) {
                        if (this.mValue[offset] == 0 || this.mValue[offset] == 1) {
                            return Float.NaN;
                        }

                        if (this.mValue[offset] == 2) {
                            return Float.NEGATIVE_INFINITY;
                        }
                    }
                }
            }

            return bytesToFloat(this.mValue[offset], this.mValue[offset + 1],
this.mValue[offset + 2], this.mValue[offset + 3]);
        default:
            return null;
    }
}

```

```

    }
}

public static int getTypeLen(int formatType) {
    return formatType & 15;
}

private static int unsignedByteToInt(byte b) {
    return b & 255;
}

private static long unsignedByteToLong(byte b) {
    return (long)b & 255L;
}

private static int unsignedBytesToInt(byte b0, byte b1) {
    return unsignedByteToInt(b0) + (unsignedByteToInt(b1) << 8);
}

private static int unsignedBytesToInt(byte b0, byte b1, byte b2, byte b3) {
    return unsignedByteToInt(b0) + (unsignedByteToInt(b1) << 8) +
(unsignedByteToInt(b2) << 16) + (unsignedByteToInt(b3) << 24);
}

private static long unsignedBytesToLong(byte b0, byte b1, byte b2, byte b3) {
    return unsignedByteToLong(b0) + (unsignedByteToLong(b1) << 8) +
(unsignedByteToLong(b2) << 16) + (unsignedByteToLong(b3) << 24);
}

private static float bytesToFloat(byte b0, byte b1) {
    int mantissa = unsignedToSigned(unsignedByteToInt(b0) +
((unsignedByteToInt(b1) & 15) << 8), 12);
    int exponent = unsignedToSigned(unsignedByteToInt(b1) >> 4, 4);
    return (float)((double)mantissa * Math.pow(10.0D, (double)exponent));
}

private static float bytesToFloat(byte b0, byte b1, byte b2, byte b3) {
    int mantissa = unsignedToSigned(unsignedByteToInt(b0) +
(unsignedByteToInt(b1) << 8) + (unsignedByteToInt(b2) << 16), 24);
    return (float)((double)mantissa * Math.pow(10.0D, (double)b3));
}

private static int unsignedToSigned(int unsigned, int size) {
    if ((unsigned & 1 << size - 1) != 0) {
        unsigned = -1 * ((1 << size - 1) - (unsigned & (1 << size - 1) - 1));
    }

    return unsigned;
}

private static long unsignedToSigned(long unsigned, int size) {
    if ((unsigned & 1L << size - 1) != 0L) {
        unsigned = -1L * ((1L << size - 1) - (unsigned & (1L << size - 1) -

```

```

1L));
    }

    return unsigned;
}

protected Data(Parcel in) {
    this.mValue = in.createByteArray();
}

public void writeToParcel(Parcel dest, int flags) {
    dest.writeByteArray(this.mValue);
}

public int describeContents() {
    return 0;
}

@Retention(RetentionPolicy.SOURCE)
public @interface FloatFormat {
}

@Retention(RetentionPolicy.SOURCE)
@SuppressLint({"UniqueConstants"})
public @interface LongFormat {
}

@Retention(RetentionPolicy.SOURCE)
@SuppressLint({"UniqueConstants"})
public @interface IntFormat {
}

@Retention(RetentionPolicy.SOURCE)
@SuppressLint({"UniqueConstants"})
public @interface ValueFormat {
}
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\data
// File: DataFilter.java

package no.nordicsemi.android.ble.data;

import androidx.annotation.Nullable;

public interface DataFilter {
    boolean filter(@Nullable byte[] var1);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\data

```



```

// File: DataMerger.java

package no.nordicsemi.android.ble.data;

import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;

public interface DataMerger {
    boolean merge(@NonNull DataStream var1, @Nullable byte[] var2, @IntRange(from
= 0L) int var3);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\no\nordicsemi\android\ble\data
// File: DataProvider.java

package no.nordicsemi.android.ble.data;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;

public interface DataProvider {
    @Nullable
    byte[] getData(@NonNull BluetoothDevice var1);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\no\nordicsemi\android\ble\data
// File: DataSplitter.java

package no.nordicsemi.android.ble.data;

import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;

public interface DataSplitter {
    @Nullable
    byte[] chunk(@NonNull byte[] var1, @IntRange(from = 0L) int var2,
@IntRange(from = 20L) int var3);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\no\nordicsemi\android\ble\data
// File: DataStream.java

package no.nordicsemi.android.ble.data;

import androidx.annotation.IntRange;

```

```

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import java.io.ByteArrayOutputStream;

public class DataStream {
    private final ByteArrayOutputStream buffer = new ByteArrayOutputStream();

    public boolean write(@Nullable byte[] data) {
        return data == null ? false : this.write(data, 0, data.length);
    }

    public boolean write(@Nullable byte[] data, @IntRange(from = 0L) int offset,
@IntRange(from = 0L) int length) {
        if (data != null && data.length >= offset) {
            int len = Math.min(data.length - offset, length);
            this.buffer.write(data, offset, len);
            return true;
        } else {
            return false;
        }
    }

    public boolean write(@Nullable Data data) {
        return data != null && this.write(data.getValue());
    }

    @IntRange(
        from = 0L
    )
    public int size() {
        return this.buffer.size();
    }

    @NonNull
    public byte[] toByteArray() {
        return this.buffer.toByteArray();
    }

    @NonNull
    public Data toData() {
        return new Data(this.buffer.toByteArray());
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\data
// File: DefaultMtuSplitter.java

```

```

package no.nordicsemi.android.ble.data;

```

```

import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;

```

```

public final class DefaultMtuSplitter implements DataSplitter {
    @Nullable
    public byte[] chunk(@NonNull byte[] message, @IntRange(from = 0L) int index,
@IntRange(from = 20L) int maxLength) {
        int offset = index * maxLength;
        int length = Math.min(maxLength, message.length - offset);
        if (length <= 0) {
            return null;
        } else {
            byte[] data = new byte[length];
            System.arraycopy(message, offset, data, 0, length);
            return data;
        }
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\data
// File: JsonMerger.java

```

```

package no.nordicsemi.android.ble.data;

```

```

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import org.json.JSONArray;
import org.json.JSONException;
import org.json.JSONObject;

```

```

public class JsonMerger implements DataMerger {
    private String buffer = "";

    public boolean merge(@NonNull DataStream output, @Nullable byte[] lastPacket,
int index) {
        output.write(lastPacket);
        this.buffer = this.buffer + new String(lastPacket);

        try {
            new JSONObject(this.buffer);
        } catch (JSONException var7) {
            try {
                new JSONArray(this.buffer);
            } catch (JSONException var6) {
                return false;
            }
        }

        this.reset();
        return true;
    }

    public void reset() {
        this.buffer = "";
    }
}

```

```
}  
}
```

```
// Directory: app  
// Subdirectory: classes.jar_Decompile.com\no\nordicsemi\android\ble\data  
// File: MutableData.java
```

```
package no.nordicsemi.android.ble.data;
```

```
import android.bluetooth.BluetoothGattCharacteristic;  
import android.bluetooth.BluetoothGattDescriptor;  
import androidx.annotation.IntRange;  
import androidx.annotation.NonNull;  
import androidx.annotation.Nullable;
```

```
public class MutableData extends Data {  
    private static final int SFLOAT_POSITIVE_INFINITY = 2046;  
    private static final int SFLOAT_NAN = 2047;  
    private static final int SFLOAT_NEGATIVE_INFINITY = 2050;  
    private static final int SFLOAT_MANTISSA_MAX = 2045;  
    private static final int SFLOAT_EXPONENT_MAX = 7;  
    private static final int SFLOAT_EXPONENT_MIN = -8;  
    private static final float SFLOAT_MAX = 2.04500009E10F;  
    private static final float SFLOAT_MIN = -2.04500009E10F;  
    private static final int SFLOAT_PRECISION = 10000;  
    private static final int FLOAT_POSITIVE_INFINITY = 8388606;  
    private static final int FLOAT_NAN = 8388607;  
    private static final int FLOAT_NEGATIVE_INFINITY = 8388610;  
    private static final int FLOAT_MANTISSA_MAX = 8388605;  
    private static final int FLOAT_EXPONENT_MAX = 127;  
    private static final int FLOAT_EXPONENT_MIN = -128;  
    private static final int FLOAT_PRECISION = 10000000;  
  
    public MutableData() {  
    }  
  
    public MutableData(@Nullable byte[] data) {  
        super(data);  
    }  
  
    public static MutableData from(@NonNull BluetoothGattCharacteristic  
characteristic) {  
        return new MutableData(characteristic.getValue());  
    }  
  
    public static MutableData from(@NonNull BluetoothGattDescriptor descriptor) {  
        return new MutableData(descriptor.getValue());  
    }  
  
    public boolean setValue(@Nullable byte[] value) {  
        this.mValue = value;  
        return true;  
    }  
}
```

```

public boolean setByte(int value, @IntRange(from = 0L) int offset) {
    int len = offset + 1;
    if (this.mValue == null) {
        this.mValue = new byte[len];
    }

    if (len > this.mValue.length) {
        return false;
    } else {
        this.mValue[offset] = (byte)value;
        return true;
    }
}

public boolean setValue(int value, int formatType, @IntRange(from = 0L) int
offset) {
    int len = offset + getTypeLen(formatType);
    if (this.mValue == null) {
        this.mValue = new byte[len];
    }

    if (len > this.mValue.length) {
        return false;
    } else {
        switch(formatType) {
            case 33:
                value = intToSignedBits(value, 8);
            case 17:
                this.mValue[offset] = (byte)(value & 255);
                break;
            case 34:
                value = intToSignedBits(value, 16);
            case 18:
                this.mValue[offset++] = (byte)(value & 255);
                this.mValue[offset] = (byte)(value >> 8 & 255);
                break;
            case 35:
                value = intToSignedBits(value, 24);
            case 19:
                this.mValue[offset++] = (byte)(value & 255);
                this.mValue[offset++] = (byte)(value >> 8 & 255);
                this.mValue[offset] = (byte)(value >> 16 & 255);
                break;
            case 36:
                value = intToSignedBits(value, 32);
            case 20:
                this.mValue[offset++] = (byte)(value & 255);
                this.mValue[offset++] = (byte)(value >> 8 & 255);
                this.mValue[offset++] = (byte)(value >> 16 & 255);
                this.mValue[offset] = (byte)(value >> 24 & 255);
                break;
            case 290:
                value = intToSignedBits(value, 16);

```

```

    case 274:
        this.mValue[offset++] = (byte)(value >> 8 & 255);
        this.mValue[offset] = (byte)(value & 255);
        break;
    case 291:
        value = intToSignedBits(value, 24);
    case 275:
        this.mValue[offset++] = (byte)(value >> 16 & 255);
        this.mValue[offset++] = (byte)(value >> 8 & 255);
        this.mValue[offset] = (byte)(value & 255);
        break;
    case 292:
        value = intToSignedBits(value, 32);
    case 276:
        this.mValue[offset++] = (byte)(value >> 24 & 255);
        this.mValue[offset++] = (byte)(value >> 16 & 255);
        this.mValue[offset++] = (byte)(value >> 8 & 255);
        this.mValue[offset] = (byte)(value & 255);
        break;
    default:
        return false;
    }

    return true;
}
}

```

```

    public boolean setValue(int mantissa, int exponent, int formatType,
@IntRange(from = 0L) int offset) {
        int len = offset + getTypeLen(formatType);
        if (this.mValue == null) {
            this.mValue = new byte[len];
        }

        if (len > this.mValue.length) {
            return false;
        } else {
            byte[] var10000;
            switch(formatType) {
                case 50:
                    mantissa = intToSignedBits(mantissa, 12);
                    exponent = intToSignedBits(exponent, 4);
                    this.mValue[offset++] = (byte)(mantissa & 255);
                    this.mValue[offset] = (byte)(mantissa >> 8 & 15);
                    var10000 = this.mValue;
                    var10000[offset] += (byte)((exponent & 15) << 4);
                    break;
                case 52:
                    mantissa = intToSignedBits(mantissa, 24);
                    exponent = intToSignedBits(exponent, 8);
                    this.mValue[offset++] = (byte)(mantissa & 255);
                    this.mValue[offset++] = (byte)(mantissa >> 8 & 255);
                    this.mValue[offset++] = (byte)(mantissa >> 16 & 255);
                    var10000 = this.mValue;

```

```

        var10000[offset] += (byte)(exponent & 255);
        break;
    default:
        return false;
    }

    return true;
}
}

```

```

    public boolean setValue(long value, int formatType, @IntRange(from = 0L) int
offset) {
        int len = offset + getTypeLen(formatType);
        if (this.mValue == null) {
            this.mValue = new byte[len];
        }

        if (len > this.mValue.length) {
            return false;
        } else {
            switch(formatType) {
                case 36:
                    value = longToSignedBits(value, 32);
                case 20:
                    this.mValue[offset++] = (byte)((int)(value & 255L));
                    this.mValue[offset++] = (byte)((int)(value >> 8 & 255L));
                    this.mValue[offset++] = (byte)((int)(value >> 16 & 255L));
                    this.mValue[offset] = (byte)((int)(value >> 24 & 255L));
                    break;
                case 292:
                    value = longToSignedBits(value, 32);
                case 276:
                    this.mValue[offset++] = (byte)((int)(value >> 24 & 255L));
                    this.mValue[offset++] = (byte)((int)(value >> 16 & 255L));
                    this.mValue[offset++] = (byte)((int)(value >> 8 & 255L));
                    this.mValue[offset] = (byte)((int)(value & 255L));
                    break;
                default:
                    return false;
            }

            return true;
        }
    }
}

```

```

    public boolean setValue(float value, int formatType, @IntRange(from = 0L) int
offset) {
        int len = offset + getTypeLen(formatType);
        if (this.mValue == null) {
            this.mValue = new byte[len];
        }

        if (len > this.mValue.length) {
            return false;
        }
    }
}

```

```

    } else {
        switch(formatType) {
            case 50:
                int sfloatAsInt = sfloatToInt(value);
                this.mValue[offset++] = (byte)(sfloatAsInt & 255);
                this.mValue[offset] = (byte)(sfloatAsInt >> 8 & 255);
                break;
            case 52:
                int floatAsInt = floatToInt(value);
                this.mValue[offset++] = (byte)(floatAsInt & 255);
                this.mValue[offset++] = (byte)(floatAsInt >> 8 & 255);
                this.mValue[offset++] = (byte)(floatAsInt >> 16 & 255);
                byte[] var10000 = this.mValue;
                var10000[offset] += (byte)(floatAsInt >> 24 & 255);
                break;
            default:
                return false;
        }

        return true;
    }
}

```

```

private static int sfloatToInt(float value) {
    if (Float.isNaN(value)) {
        return 2047;
    } else if (value > 2.04500009E10F) {
        return 2046;
    } else if (value < -2.04500009E10F) {
        return 2050;
    } else {
        int sign = value >= 0.0F ? 1 : -1;
        float mantissa = Math.abs(value);
        int exponent = 0;

        while(mantissa > 2045.0F) {
            mantissa /= 10.0F;
            ++exponent;
            if (exponent > 7) {
                if (sign > 0) {
                    return 2046;
                }

                return 2050;
            }
        }

        while(mantissa < 1.0F) {
            mantissa *= 10.0F;
            --exponent;
            if (exponent < -8) {
                return 0;
            }
        }
    }
}

```



```

        double smantissa = (double)Math.round(mantissa * 10000.0F);
        double rmantissa = (double)(Math.round(mantissa) * 10000);

        for(double mdiff = Math.abs(smantissa - rmantissa); mdiff > 0.5D &&
exponent > -8 && mantissa * 10.0F <= 2045.0F; mdiff = Math.abs(smantissa -
rmantissa)) {
            mantissa *= 10.0F;
            --exponent;
            smantissa = (double)Math.round(mantissa * 10000.0F);
            rmantissa = (double)(Math.round(mantissa) * 10000);
        }

        int int_mantissa = Math.round((float)sign * mantissa);
        return (exponent & 15) << 12 | int_mantissa & 4095;
    }
}

private static int floatToInt(float value) {
    if (Float.isNaN(value)) {
        return 8388607;
    } else if (value == Float.POSITIVE_INFINITY) {
        return 8388606;
    } else if (value == Float.NEGATIVE_INFINITY) {
        return 8388610;
    } else {
        int sign = value >= 0.0F ? 1 : -1;
        float mantissa = Math.abs(value);
        int exponent = 0;

        while(mantissa > 8388605.0F) {
            mantissa /= 10.0F;
            ++exponent;
            if (exponent > 127) {
                if (sign > 0) {
                    return 8388606;
                }

                return 8388610;
            }
        }

        while(mantissa < 1.0F) {
            mantissa *= 10.0F;
            --exponent;
            if (exponent < -128) {
                return 0;
            }
        }

        double smantissa = (double)Math.round(mantissa * 1.0E7F);
        double rmantissa = (double)(Math.round(mantissa) * 10000000);

        for(double mdiff = Math.abs(smantissa - rmantissa); mdiff > 0.5D &&

```

```

exponent > -128 && mantissa * 10.0F <= 8388605.0F; mdiff = Math.abs(smantissa -
rmantissa)) {
    mantissa *= 10.0F;
    --exponent;
    smantissa = (double)Math.round(mantissa * 1.0E7F);
    rmantissa = (double)(Math.round(mantissa) * 10000000);
}

int int_mantissa = Math.round((float)sign * mantissa);
return exponent << 24 | int_mantissa & 16777215;
}
}

private static int intToSignedBits(int i, int size) {
    if (i < 0) {
        i = (1 << size - 1) + (i & (1 << size - 1) - 1);
    }

    return i;
}

private static long longToSignedBits(long i, int size) {
    if (i < 0L) {
        i = (1L << size - 1) + (i & (1L << size - 1) - 1L);
    }

    return i;
}
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\data
// File: PacketFilter.java

```

```

package no.nordicsemi.android.ble.data;

import androidx.annotation.Nullable;

public interface PacketFilter {
    boolean filter(@Nullable byte[] var1);
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\error
// File: GattError.java

```

```

package no.nordicsemi.android.ble.error;

public class GattError {
    public static final int GATT_SUCCESS = 0;
    public static final int GATT_CONN_L2C_FAILURE = 1;
    public static final int GATT_CONN_TIMEOUT = 8;
}

```

```

public static final int GATT_CONN_TERMINATE_PEER_USER = 19;
public static final int GATT_CONN_TERMINATE_LOCAL_HOST = 22;
public static final int GATT_CONN_FAIL_ESTABLISH = 62;
public static final int GATT_CONN_LMP_TIMEOUT = 34;
public static final int GATT_CONN_CANCEL = 256;
public static final int GATT_ERROR = 133;
public static final int GATT_INVALID_HANDLE = 1;
public static final int GATT_READ_NOT_PERMIT = 2;
public static final int GATT_WRITE_NOT_PERMIT = 3;
public static final int GATT_INVALID_PDU = 4;
public static final int GATT_INSUF_AUTHENTICATION = 5;
public static final int GATT_REQ_NOT_SUPPORTED = 6;
public static final int GATT_INVALID_OFFSET = 7;
public static final int GATT_INSUF_AUTHORIZATION = 8;
public static final int GATT_PREPARE_Q_FULL = 9;
public static final int GATT_NOT_FOUND = 10;
public static final int GATT_NOT_LONG = 11;
public static final int GATT_INSUF_KEY_SIZE = 12;
public static final int GATT_INVALID_ATTR_LEN = 13;
public static final int GATT_ERR_UNLIKELY = 14;
public static final int GATT_INSUF_ENCRYPTION = 15;
public static final int GATT_UNSUPPORT_GRP_TYPE = 16;
public static final int GATT_INSUF_RESOURCE = 17;
public static final int GATT_CONTROLLER_BUSY = 58;
public static final int GATT_UNACCEPT_CONN_INTERVAL = 59;
public static final int GATT_ILLEGAL_PARAMETER = 135;
public static final int GATT_NO_RESOURCES = 128;
public static final int GATT_INTERNAL_ERROR = 129;
public static final int GATT_WRONG_STATE = 130;
public static final int GATT_DB_FULL = 131;
public static final int GATT_BUSY = 132;
public static final int GATT_CMD_STARTED = 134;
public static final int GATT_PENDING = 136;
public static final int GATT_AUTH_FAIL = 137;
public static final int GATT_MORE = 138;
public static final int GATT_INVALID_CFG = 139;
public static final int GATT_SERVICE_STARTED = 140;
public static final int GATT_ENCRYPTED_NO_MITM = 141;
public static final int GATT_NOT_ENCRYPTED = 142;
public static final int GATT_CONGESTED = 143;
public static final int GATT_CCCD_CFG_ERROR = 253;
public static final int GATT_PROCEDURE_IN_PROGRESS = 254;
public static final int GATT_VALUE_OUT_OF_RANGE = 255;
public static final int TOO_MANY_OPEN_CONNECTIONS = 257;

public static String parseConnectionError(int error) {
    switch(error) {
        case 0:
            return "SUCCESS";
        case 1:
            return "GATT CONN L2C FAILURE";
        case 8:
            return "GATT CONN TIMEOUT";
        case 19:

```

```

        return "GATT CONN TERMINATE PEER USER";
case 22:
    return "GATT CONN TERMINATE LOCAL HOST";
case 34:
    return "GATT CONN LMP TIMEOUT";
case 62:
    return "GATT CONN FAIL ESTABLISH";
case 133:
    return "GATT ERROR";
case 256:
    return "GATT CONN CANCEL ";
default:
    return "UNKNOWN (" + error + ")";
}
}

```

```

public static String parse(int error) {
    switch(error) {
case 1:
    return "GATT INVALID HANDLE";
case 2:
    return "GATT READ NOT PERMIT";
case 3:
    return "GATT WRITE NOT PERMIT";
case 4:
    return "GATT INVALID PDU";
case 5:
    return "GATT INSUF AUTHENTICATION";
case 6:
    return "GATT REQ NOT SUPPORTED";
case 7:
    return "GATT INVALID OFFSET";
case 8:
    return "GATT INSUF AUTHORIZATION";
case 9:
    return "GATT PREPARE Q FULL";
case 10:
    return "GATT NOT FOUND";
case 11:
    return "GATT NOT LONG";
case 12:
    return "GATT INSUF KEY SIZE";
case 13:
    return "GATT INVALID ATTR LEN";
case 14:
    return "GATT ERR UNLIKELY";
case 15:
    return "GATT INSUF ENCRYPTION";
case 16:
    return "GATT UNSUPPORT GRP TYPE";
case 17:
    return "GATT INSUF RESOURCE";
case 34:
    return "GATT CONN LMP TIMEOUT";

```

```

        case 58:
            return "GATT CONTROLLER BUSY";
        case 59:
            return "GATT UNACCEPT CONN INTERVAL";
        case 128:
            return "GATT NO RESOURCES";
        case 129:
            return "GATT INTERNAL ERROR";
        case 130:
            return "GATT WRONG STATE";
        case 131:
            return "GATT DB FULL";
        case 132:
            return "GATT BUSY";
        case 133:
            return "GATT ERROR";
        case 134:
            return "GATT CMD STARTED";
        case 135:
            return "GATT ILLEGAL PARAMETER";
        case 136:
            return "GATT PENDING";
        case 137:
            return "GATT AUTH FAIL";
        case 138:
            return "GATT MORE";
        case 139:
            return "GATT INVALID CFG";
        case 140:
            return "GATT SERVICE STARTED";
        case 141:
            return "GATT ENCRYPTED NO MITM";
        case 142:
            return "GATT NOT ENCRYPTED";
        case 143:
            return "GATT CONGESTED";
        case 253:
            return "GATT CCCD CFG ERROR";
        case 254:
            return "GATT PROCEDURE IN PROGRESS";
        case 255:
            return "GATT VALUE OUT OF RANGE";
        case 257:
            return "TOO MANY OPEN CONNECTIONS";
        default:
            return "UNKNOWN (" + error + ")";
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar Decompiler.com\nordicsemi\android\ble\exception
// File: BluetoothDisabledException.java

```

```

package no.nordicsemi.android.ble.exception;

public class BluetoothDisabledException extends ConnectionException {
}

// Directory: app
// Subdirectory: classes.jar_Decompiler.com\no\nordicsemi\android\ble\exception
// File: ConnectionException.java

package no.nordicsemi.android.ble.exception;

public class ConnectionException extends Exception {
}

// Directory: app
// Subdirectory: classes.jar_Decompiler.com\no\nordicsemi\android\ble\exception
// File: DeviceDisconnectedException.java

package no.nordicsemi.android.ble.exception;

public class DeviceDisconnectedException extends ConnectionException {
}

// Directory: app
// Subdirectory: classes.jar_Decompiler.com\no\nordicsemi\android\ble\exception
// File: InvalidDataException.java

package no.nordicsemi.android.ble.exception;

import androidx.annotation.NonNull;
import no.nordicsemi.android.ble.callback.profile.ProfileReadResponse;

public final class InvalidDataException extends Exception {
    private final ProfileReadResponse response;

    public InvalidDataException(@NonNull ProfileReadResponse response) {
        this.response = response;
    }

    public ProfileReadResponse getResponse() {
        return this.response;
    }
}

// Directory: app
// Subdirectory: classes.jar_Decompiler.com\no\nordicsemi\android\ble\exception
// File: InvalidRequestException.java

package no.nordicsemi.android.ble.exception;

```

```

import no.nordicsemi.android.ble.Request;

public final class InvalidRequestException extends Exception {
    private final Request request;

    public InvalidRequestException(Request request) {
        super("Invalid request");
        this.request = request;
    }

    public Request getRequest() {
        return this.request;
    }
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\no\nordicsemi\android\ble\exception
// File: RequestFailedException.java

package no.nordicsemi.android.ble.exception;

import no.nordicsemi.android.ble.Request;

public final class RequestFailedException extends Exception {
    private final Request request;
    private final int status;

    public RequestFailedException(Request request, int status) {
        super("Request failed with status " + status);
        this.request = request;
        this.status = status;
    }

    public int getStatus() {
        return this.status;
    }

    public Request getRequest() {
        return this.request;
    }
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\no\nordicsemi\android\ble\observer
// File: BondingObserver.java

package no.nordicsemi.android.ble.observer;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;

```

```

public interface BondingObserver {
    void onBondingRequired(@NonNull BluetoothDevice var1);

    void onBonded(@NonNull BluetoothDevice var1);

    void onBondingFailed(@NonNull BluetoothDevice var1);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\observer
// File: ConnectionObserver.java

package no.nordicsemi.android.ble.observer;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;

public interface ConnectionObserver {
    int REASON_UNKNOWN = -1;
    int REASON_SUCCESS = 0;
    int REASON_TERMINATE_LOCAL_HOST = 1;
    int REASON_TERMINATE_PEER_USER = 2;
    int REASON_LINK_LOSS = 3;
    int REASON_NOT_SUPPORTED = 4;
    int REASON_CANCELLED = 5;
    int REASON_TIMEOUT = 10;

    void onDeviceConnecting(@NonNull BluetoothDevice var1);

    void onDeviceConnected(@NonNull BluetoothDevice var1);

    void onDeviceFailedToConnect(@NonNull BluetoothDevice var1, int var2);

    void onDeviceReady(@NonNull BluetoothDevice var1);

    void onDeviceDisconnecting(@NonNull BluetoothDevice var1);

    void onDeviceDisconnected(@NonNull BluetoothDevice var1, int var2);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\observer
// File: ServerObserver.java

package no.nordicsemi.android.ble.observer;

import android.bluetooth.BluetoothDevice;
import androidx.annotation.NonNull;

public interface ServerObserver {
    void onServerReady();
}

```



```

    void onDeviceConnectedToServer(@NonNull BluetoothDevice var1);

    void onDeviceDisconnectedFromServer(@NonNull BluetoothDevice var1);
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\response
// File: ConnectionPriorityResponse.java

package no.nordicsemi.android.ble.response;

import android.bluetooth.BluetoothDevice;
import android.os.Parcel;
import android.os.Parcelable;
import android.os.Parcelable.Creator;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.ConnectionParametersUpdatedCallback;

public class ConnectionPriorityResponse implements
ConnectionParametersUpdatedCallback, Parcelable {
    private BluetoothDevice device;
    @IntRange(
        from = 6L,
        to = 3200L
    )
    private int interval;
    @IntRange(
        from = 0L,
        to = 499L
    )
    private int latency;
    @IntRange(
        from = 10L,
        to = 3200L
    )
    private int supervisionTimeout;
    public static final Creator<ConnectionPriorityResponse> CREATOR = new
Creator<ConnectionPriorityResponse>() {
        public ConnectionPriorityResponse createFromParcel(Parcel in) {
            return new ConnectionPriorityResponse(in);
        }

        public ConnectionPriorityResponse[] newArray(int size) {
            return new ConnectionPriorityResponse[size];
        }
    };

    public ConnectionPriorityResponse() {
    }

    public void onConnectionUpdated(@NonNull BluetoothDevice device,

```

```

@IntRange(from = 6L,to = 3200L) int interval, @IntRange(from = 0L,to = 499L) int
latency, @IntRange(from = 10L,to = 3200L) int timeout) {
    this.device = device;
    this.interval = interval;
    this.latency = latency;
    this.supervisionTimeout = timeout;
}

@Nullable
public BluetoothDevice getBluetoothDevice() {
    return this.device;
}

@IntRange(
    from = 6L,
    to = 3200L
)
public int getConnectionInterval() {
    return this.interval;
}

@IntRange(
    from = 0L,
    to = 499L
)
public int getSlaveLatency() {
    return this.latency;
}

@IntRange(
    from = 10L,
    to = 3200L
)
public int getSupervisionTimeout() {
    return this.supervisionTimeout;
}

protected ConnectionPriorityResponse(Parcel in) {
    this.device =
(BluetoothDevice)in.readParcelable(BluetoothDevice.class.getClassLoader());
    this.interval = in.readInt();
    this.latency = in.readInt();
    this.supervisionTimeout = in.readInt();
}

public void writeToParcel(Parcel dest, int flags) {
    dest.writeParcelable(this.device, flags);
    dest.writeInt(this.interval);
    dest.writeInt(this.latency);
    dest.writeInt(this.supervisionTimeout);
}

public int describeContents() {
    return 0;
}

```

```
    }  
}
```

```
// Directory: app  
// Subdirectory: classes.jar_Decompile.com\no\nordicsemi\android\ble\response  
// File: MtuResult.java
```

```
package no.nordicsemi.android.ble.response;
```

```
import android.bluetooth.BluetoothDevice;  
import android.os.Parcel;  
import android.os.Parcelable;  
import android.os.Parcelable.Creator;  
import androidx.annotation.IntRange;  
import androidx.annotation.NonNull;  
import androidx.annotation.Nullable;  
import no.nordicsemi.android.ble.callback.MtuCallback;
```

```
public class MtuResult implements MtuCallback, Parcelable {  
    private BluetoothDevice device;  
    @IntRange(  
        from = 23L,  
        to = 517L  
    )  
    private int mtu;  
    public static final Creator<MtuResult> CREATOR = new Creator<MtuResult>() {  
        public MtuResult createFromParcel(Parcel in) {  
            return new MtuResult(in);  
        }  
  
        public MtuResult[] newArray(int size) {  
            return new MtuResult[size];  
        }  
    };  
  
    public MtuResult() {  
    }  
  
    public void onMtuChanged(@NonNull BluetoothDevice device, @IntRange(from =  
23L,to = 517L) int mtu) {  
        this.device = device;  
        this.mtu = mtu;  
    }  
  
    @Nullable  
    public BluetoothDevice getBluetoothDevice() {  
        return this.device;  
    }  
  
    @IntRange(  
        from = 23L,  
        to = 517L  
    )  
}
```

```

    public int getMtu() {
        return this.mtu;
    }

    protected MtuResult(Parcel in) {
        this.device =
        (BluetoothDevice)in.readParcelable(BluetoothDevice.class.getClassLoader());
        this.mtu = in.readInt();
    }

    public void writeToParcel(Parcel dest, int flags) {
        dest.writeParcelable(this.device, flags);
        dest.writeInt(this.mtu);
    }

    public int describeContents() {
        return 0;
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\response
// File: PhyResult.java

```

```

package no.nordicsemi.android.ble.response;

import android.bluetooth.BluetoothDevice;
import android.os.Parcel;
import android.os.Parcelable;
import android.os.Parcelable.Creator;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.PhyCallback;

public class PhyResult implements PhyCallback, Parcelable {
    private BluetoothDevice device;
    private int txPhy;
    private int rxPhy;
    public static final Creator<PhyResult> CREATOR = new Creator<PhyResult>() {
        public PhyResult createFromParcel(Parcel in) {
            return new PhyResult(in);
        }

        public PhyResult[] newArray(int size) {
            return new PhyResult[size];
        }
    };

    public PhyResult() {
    }

    public void onPhyChanged(@NonNull BluetoothDevice device, int txPhy, int
rxPhy) {

```

```

        this.device = device;
        this.txPhy = txPhy;
        this.rxPhy = rxPhy;
    }

    @Nullable
    public BluetoothDevice getBluetoothDevice() {
        return this.device;
    }

    public int getTxPhy() {
        return this.txPhy;
    }

    public int getRxPhy() {
        return this.rxPhy;
    }

    protected PhyResult(Parcel in) {
        this.device =
        (BluetoothDevice)in.readParcelable(BluetoothDevice.class.getClassLoader());
        this.txPhy = in.readInt();
        this.rxPhy = in.readInt();
    }

    public void writeToParcel(Parcel dest, int flags) {
        dest.writeParcelable(this.device, flags);
        dest.writeInt(this.txPhy);
        dest.writeInt(this.rxPhy);
    }

    public int describeContents() {
        return 0;
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\response
// File: ReadResponse.java

```

```

package no.nordicsemi.android.ble.response;

```

```

import android.bluetooth.BluetoothDevice;
import android.os.Parcel;
import android.os.Parcelable;
import android.os.Parcelable.Creator;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.DataReceivedCallback;
import no.nordicsemi.android.ble.data.Data;

```

```

public class ReadResponse implements DataReceivedCallback, Parcelable {
    private BluetoothDevice device;

```

```

    private Data data;
    public static final Creator<ReadResponse> CREATOR = new
Creator<ReadResponse>() {
    public ReadResponse createFromParcel(Parcel in) {
        return new ReadResponse(in);
    }

    public ReadResponse[] newArray(int size) {
        return new ReadResponse[size];
    }
};

    public ReadResponse() {
    }

    public void onDataReceived(@NonNull BluetoothDevice device, @NonNull Data
data) {
        this.device = device;
        this.data = data;
    }

    @Nullable
    public BluetoothDevice getBluetoothDevice() {
        return this.device;
    }

    @Nullable
    public Data getRawData() {
        return this.data;
    }

    protected ReadResponse(Parcel in) {
        this.device =
(BluetoothDevice)in.readParcelable(BluetoothDevice.class.getClassLoader());
        this.data = (Data)in.readParcelable(Data.class.getClassLoader());
    }

    public void writeToParcel(Parcel dest, int flags) {
        dest.writeParcelable(this.device, flags);
        dest.writeParcelable(this.data, flags);
    }

    public int describeContents() {
        return 0;
    }
}

// Directory: app
// Subdirectory: classes.jar_Decompile.com\no\nordicsemi\android\ble\response
// File: RssiResult.java

package no.nordicsemi.android.ble.response;

```

```

import android.bluetooth.BluetoothDevice;
import android.os.Parcel;
import android.os.Parcelable;
import android.os.Parcelable.Creator;
import androidx.annotation.IntRange;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.RssiCallback;

public class RssiResult implements RssiCallback, Parcelable {
    private BluetoothDevice device;
    @IntRange(
        from = -128L,
        to = 20L
    )
    private int rssi;
    public static final Creator<RssiResult> CREATOR = new Creator<RssiResult>() {
        public RssiResult createFromParcel(Parcel in) {
            return new RssiResult(in);
        }

        public RssiResult[] newArray(int size) {
            return new RssiResult[size];
        }
    };

    public RssiResult() {
    }

    public void onRssiRead(@NonNull BluetoothDevice device, @IntRange(from =
-128L,to = 20L) int rssi) {
        this.device = device;
        this.rssi = rssi;
    }

    @Nullable
    public BluetoothDevice getBluetoothDevice() {
        return this.device;
    }

    @IntRange(
        from = -128L,
        to = 20L
    )
    public int getRssi() {
        return this.rssi;
    }

    protected RssiResult(Parcel in) {
        this.device =
(BluetoothDevice)in.readParcelable(BluetoothDevice.class.getClassLoader());
        this.rssi = in.readInt();
    }

```

```

    public void writeToParcel(Parcel dest, int flags) {
        dest.writeParcelable(this.device, flags);
        dest.writeInt(this.rssi);
    }

    public int describeContents() {
        return 0;
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompiler.com\nordicsemi\android\ble\response
// File: WriteResponse.java

```

```

package no.nordicsemi.android.ble.response;

```

```

import android.bluetooth.BluetoothDevice;
import android.os.Parcel;
import android.os.Parcelable;
import android.os.Parcelable.Creator;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import no.nordicsemi.android.ble.callback.DataSentCallback;
import no.nordicsemi.android.ble.data.Data;

```

```

public class WriteResponse implements DataSentCallback, Parcelable {
    private BluetoothDevice device;
    private Data data;
    public static final Creator<WriteResponse> CREATOR = new
Creator<WriteResponse>() {
        public WriteResponse createFromParcel(Parcel in) {
            return new WriteResponse(in);
        }

        public WriteResponse[] newArray(int size) {
            return new WriteResponse[size];
        }
    };
}

```

```

    public WriteResponse() {
    }

```

```

    public void onDataSent(@NonNull BluetoothDevice device, @NonNull Data data) {
        this.device = device;
        this.data = data;
    }

```

```

    @Nullable
    public BluetoothDevice getBluetoothDevice() {
        return this.device;
    }

```

```

    @Nullable

```



```

    public Data getRawData() {
        return this.data;
    }

    protected WriteResponse(Parcel in) {
        this.device =
        (BluetoothDevice)in.readParcelable(BluetoothDevice.class.getClassLoader());
        this.data = (Data)in.readParcelable(Data.class.getClassLoader());
    }

    public void writeToParcel(Parcel dest, int flags) {
        dest.writeParcelable(this.device, flags);
        dest.writeParcelable(this.data, flags);
    }

    public int describeContents() {
        return 0;
    }
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\utils
// File: ILogger.java

```

```

package no.nordicsemi.android.ble.utils;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.annotation.StringRes;

public interface ILogger {
    int getMinLogPriority();

    void log(int var1, @NonNull String var2);

    void log(int var1, @StringRes int var2, @Nullable Object... var3);
}

```

```

// Directory: app
// Subdirectory: classes.jar_Decompile.com\nordicsemi\android\ble\utils
// File: ParserUtils.java

```

```

package no.nordicsemi.android.ble.utils;

import android.bluetooth.BluetoothGattCharacteristic;
import android.bluetooth.BluetoothGattDescriptor;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;

public class ParserUtils {
    protected static final char[] HEX_ARRAY = "0123456789ABCDEF".toCharArray();
}

```

```

    public static String parse(@NonNull BluetoothGattCharacteristic
characteristic) {
        return parse(characteristic.getValue());
    }

    public static String parse(@NonNull BluetoothGattDescriptor descriptor) {
        return parse(descriptor.getValue());
    }

    public static String parse(@Nullable byte[] data) {
        if (data != null && data.length != 0) {
            char[] out = new char[data.length * 3 - 1];

            for(int j = 0; j < data.length; ++j) {
                int v = data[j] & 255;
                out[j * 3] = HEX_ARRAY[v >>> 4];
                out[j * 3 + 1] = HEX_ARRAY[v & 15];
                if (j != data.length - 1) {
                    out[j * 3 + 2] = '-';
                }
            }

            return "(0x) " + new String(out);
        } else {
            return "";
        }
    }

    public static String parseDebug(@Nullable byte[] data) {
        if (data != null && data.length != 0) {
            char[] out = new char[data.length * 2];

            for(int j = 0; j < data.length; ++j) {
                int v = data[j] & 255;
                out[j * 2] = HEX_ARRAY[v >>> 4];
                out[j * 2 + 1] = HEX_ARRAY[v & 15];
            }

            return "0x" + new String(out);
        } else {
            return "null";
        }
    }

    @NonNull
    public static String pairingVariantToString(int variant) {
        switch(variant) {
            case 0:
                return "PAIRING_VARIANT_PIN";
            case 1:
                return "PAIRING_VARIANT_PASSKEY";
            case 2:
                return "PAIRING_VARIANT_PASSKEY_CONFIRMATION";
            case 3:

```

```

        return "PAIRING_VARIANT_CONSENT";
    case 4:
        return "PAIRING_VARIANT_DISPLAY_PASSKEY";
    case 5:
        return "PAIRING_VARIANT_DISPLAY_PIN";
    case 6:
        return "PAIRING_VARIANT_OOB_CONSENT";
    default:
        return "UNKNOWN (" + variant + ")";
    }
}

@NonNull
public static String bondStateToString(int state) {
    switch(state) {
    case 10:
        return "BOND_NONE";
    case 11:
        return "BOND_BONDING";
    case 12:
        return "BOND_BONDED";
    default:
        return "UNKNOWN (" + state + ")";
    }
}

@NonNull
public static String writeTypeToString(int type) {
    switch(type) {
    case 1:
        return "WRITE_COMMAND";
    case 2:
        return "WRITE_REQUEST";
    case 3:
    default:
        return "UNKNOWN (" + type + ")";
    case 4:
        return "WRITE_SIGNED";
    }
}

@NonNull
public static String stateToString(int state) {
    switch(state) {
    case 0:
        return "DISCONNECTED";
    case 1:
        return "CONNECTING";
    case 2:
        return "CONNECTED";
    case 3:
        return "DISCONNECTING";
    default:
        return "UNKNOWN (" + state + ")";
    }
}

```

```
    }  
}
```

```
@NonNull  
public static String phyToString(int phy) {  
    switch(phy) {  
        case 1:  
            return "LE 1M";  
        case 2:  
            return "LE 2M";  
        case 3:  
            return "LE Coded";  
        default:  
            return "UNKNOWN (" + phy + ")";  
    }  
}
```

```
@NonNull  
public static String phyMaskToString(int mask) {  
    switch(mask) {  
        case 1:  
            return "LE 1M";  
        case 2:  
            return "LE 2M";  
        case 3:  
            return "LE 1M or LE 2M";  
        case 4:  
            return "LE Coded";  
        case 5:  
            return "LE 1M or LE Coded";  
        case 6:  
            return "LE 2M or LE Coded";  
        case 7:  
            return "LE 1M, LE 2M or LE Coded";  
        default:  
            return "UNKNOWN (" + mask + ")";  
    }  
}
```

```
@NonNull  
public static String phyCodedOptionToString(int option) {  
    switch(option) {  
        case 0:  
            return "No preferred";  
        case 1:  
            return "S2";  
        case 2:  
            return "S8";  
        default:  
            return "UNKNOWN (" + option + ")";  
    }  
}  
}
```

```
// Directory: app
// Subdirectory:
classes.jar_Decompile.com\no\nordicsemi\android\support\v18\scanner
// File: BluetoothLeScannerCompat.java
```

```
package no.nordicsemi.android.support.v18.scanner;
```

```
import android.app.PendingIntent;
import android.content.Context;
import android.os.Handler;
import android.os.Looper;
import android.os.SystemClock;
import android.os.Build.VERSION;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.Set;
```

```
public abstract class BluetoothLeScannerCompat {
    public static final String EXTRA_LIST_SCAN_RESULT =
"android.bluetooth.le.extra.LIST_SCAN_RESULT";
    public static final String EXTRA_ERROR_CODE =
"android.bluetooth.le.extra.ERROR_CODE";
    public static final String EXTRA_CALLBACK_TYPE =
"android.bluetooth.le.extra.CALLBACK_TYPE";
    private static BluetoothLeScannerCompat instance;
```

```
@NonNull
```

```
public static synchronized BluetoothLeScannerCompat getScanner() {
    if (instance != null) {
        return instance;
    } else if (VERSION.SDK_INT >= 26) {
        return instance = new BluetoothLeScannerImplOreo();
    } else if (VERSION.SDK_INT >= 23) {
        return instance = new BluetoothLeScannerImplMarshmallow();
    } else {
        return VERSION.SDK_INT >= 21 ? (instance = new
BluetoothLeScannerImplLollipop()) : (instance = new BluetoothLeScannerImplJB());
    }
}
```

```
BluetoothLeScannerCompat() {
}
```

```
public final void startScan(@NonNull ScanCallback callback) {
    if (callback == null) {
        throw new IllegalArgumentException("callback is null");
    }
}
```

```

        } else {
            Handler handler = new Handler(Looper.getMainLooper());
            this.startScanInternal(Collections.emptyList(), (new
ScanSettings.Builder()).build(), callback, handler);
        }
    }

    public final void startScan(@Nullable List<ScanFilter> filters, @Nullable
ScanSettings settings, @NonNull ScanCallback callback) {
        if (callback == null) {
            throw new IllegalArgumentException("callback is null");
        } else {
            Handler handler = new Handler(Looper.getMainLooper());
            this.startScanInternal(filters != null ? filters :
Collections.emptyList(), settings != null ? settings : (new
ScanSettings.Builder()).build(), callback, handler);
        }
    }

    public final void startScan(@Nullable List<ScanFilter> filters, @Nullable
ScanSettings settings, @NonNull ScanCallback callback, @Nullable Handler
handler) {
        if (callback == null) {
            throw new IllegalArgumentException("callback is null");
        } else {
            this.startScanInternal(filters != null ? filters :
Collections.emptyList(), settings != null ? settings : (new
ScanSettings.Builder()).build(), callback, handler != null ? handler : new
Handler(Looper.getMainLooper()));
        }
    }

    public final void stopScan(@NonNull ScanCallback callback) {
        if (callback == null) {
            throw new IllegalArgumentException("callback is null");
        } else {
            this.stopScanInternal(callback);
        }
    }

    abstract void startScanInternal(@NonNull List<ScanFilter> var1, @NonNull
ScanSettings var2, @NonNull ScanCallback var3, @NonNull Handler var4);

    abstract void stopScanInternal(@NonNull ScanCallback var1);

    public final void startScan(@Nullable List<ScanFilter> filters, @Nullable
ScanSettings settings, @NonNull Context context, @NonNull PendingIntent
callbackIntent, int requestCode) {
        if (callbackIntent == null) {
            throw new IllegalArgumentException("callbackIntent is null");
        } else if (context == null) {
            throw new IllegalArgumentException("context is null");
        } else {
            this.startScanInternal(filters != null ? filters :

```

```

Collections.emptyList(), settings != null ? settings : (new
ScanSettings.Builder()).build(), context, callbackIntent, requestCode);
    }
}

    public final void startScan(@Nullable List<ScanFilter> filters, @Nullable
ScanSettings settings, @NonNull Context context, @NonNull PendingIntent
callbackIntent) {
        this.startScan(filters, settings, context, callbackIntent, 0);
    }

    public final void stopScan(@NonNull Context context, @NonNull PendingIntent
callbackIntent, int requestCode) {
        if (callbackIntent == null) {
            throw new IllegalArgumentException("callbackIntent is null");
        } else if (context == null) {
            throw new IllegalArgumentException("context is null");
        } else {
            this.stopScanInternal(context, callbackIntent, requestCode);
        }
    }

    public final void stopScan(@NonNull Context context, @NonNull PendingIntent
callbackIntent) {
        this.stopScanInternal(context, callbackIntent, 0);
    }

    abstract void startScanInternal(@NonNull List<ScanFilter> var1, @NonNull
ScanSettings var2, @NonNull Context var3, @NonNull PendingIntent var4, int
var5);

    abstract void stopScanInternal(@NonNull Context var1, @NonNull PendingIntent
var2, int var3);

    public abstract void flushPendingScanResults(@NonNull ScanCallback var1);

    static class ScanCallbackWrapper {
        @NonNull
        private final Object LOCK = new Object();
        private final boolean emulateFiltering;
        private final boolean emulateBatching;
        private final boolean emulateFoundOrLostCallbackType;
        private boolean scanningStopped;
        @NonNull
        final List<ScanFilter> filters;
        @NonNull
        final ScanSettings scanSettings;
        @NonNull
        final ScanCallback scanCallback;
        @NonNull
        final Handler handler;
        @NonNull
        private final List<ScanResult> scanResults = new ArrayList();
        @NonNull

```

```

private final Set<String> devicesInBatch = new HashSet();
@NonNull
private final Map<String, ScanResult> devicesInRange = new HashMap();
@NonNull
private final Runnable matchLostNotifierTask = new Runnable() {
    public void run() {
        long now = SystemClock.elapsedRealtimeNanos();
        synchronized(ScanCallbackWrapper.this.LOCK) {
            Iterator iterator =
ScanCallbackWrapper.this.devicesInRange.values().iterator();

            while(iterator.hasNext()) {
                ScanResult result = (ScanResult)iterator.next();
                if (result.getTimestampNanos() < now -
ScanCallbackWrapper.this.scanSettings.getMatchLostDeviceTimeout()) {
                    iterator.remove();
                    ScanCallbackWrapper.this.handler.post(() -> {
                        ScanCallbackWrapper.this.scanCallback.onScanResult(4,
result);
                    });
                }
            }

            if (!ScanCallbackWrapper.this.devicesInRange.isEmpty()) {
                ScanCallbackWrapper.this.handler.postDelayed(this,
ScanCallbackWrapper.this.scanSettings.getMatchLostTaskInterval());
            }
        }
    }
};

```

```

ScanCallbackWrapper(boolean offloadedBatchingSupported, boolean
offloadedFilteringSupported, @NonNull List<ScanFilter> filters, @NonNull
ScanSettings settings, @NonNull ScanCallback callback, @NonNull final Handler
handler) {
    this.filters = Collections.unmodifiableList(filters);
    this.scanSettings = settings;
    this.scanCallback = callback;
    this.handler = handler;
    this.scanningStopped = false;
    boolean callbackTypesSupported = VERSION.SDK_INT >= 23;
    this.emulateFoundOrLostCallbackType = settings.getCallbackType() != 1
&& (!callbackTypesSupported ||
!settings.getUseHardwareCallbackTypesIfSupported());
    this.emulateFiltering = !filters.isEmpty() &&
(!offloadedFilteringSupported ||
!settings.getUseHardwareFilteringIfSupported());
    long delay = settings.getReportDelayMillis();
    this.emulateBatching = delay > 0L && (!offloadedBatchingSupported ||
!settings.getUseHardwareBatchingIfSupported());
    if (this.emulateBatching) {
        Runnable flushPendingScanResultsTask = new Runnable() {
            public void run() {

```



```

        if (!ScanCallbackWrapper.this.scanningStopped) {
            ScanCallbackWrapper.this.flushPendingScanResults();
            handler.postDelayed(this,
ScanCallbackWrapper.this.scanSettings.getReportDelayMillis());
        }

    }
    };
    handler.postDelayed(flushPendingScanResultsTask, delay);
}

}

void close() {
    this.scanningStopped = true;
    this.handler.removeCallbacksAndMessages((Object)null);
    synchronized(this.LOCK) {
        this.devicesInRange.clear();
        this.devicesInBatch.clear();
        this.scanResults.clear();
    }
}

void flushPendingScanResults() {
    if (this.emulateBatching && !this.scanningStopped) {
        synchronized(this.LOCK) {
            this.scanCallback.onBatchScanResults(new
ArrayList(this.scanResults));
            this.scanResults.clear();
            this.devicesInBatch.clear();
        }
    }
}

void handleScanResult(int callbackType, @NonNull ScanResult scanResult) {
    if (!this.scanningStopped && (this.filters.isEmpty() ||
this.matches(scanResult))) {
        String deviceAddress = scanResult.getDevice().getAddress();
        if (this.emulateFoundOrLostCallbackType) {
            ScanResult previousResult;
            boolean firstResult;
            synchronized(this.devicesInRange) {
                firstResult = this.devicesInRange.isEmpty();
                previousResult =
(ScanResult)this.devicesInRange.put(deviceAddress, scanResult);
            }

            if (previousResult == null &&
(this.scanSettings.getCallbackType() & 2) > 0) {
                this.scanCallback.onScanResult(2, scanResult);
            }

            if (firstResult && (this.scanSettings.getCallbackType() & 4) > 0)

```

```

{
    this.handler.removeCallbacks(this.matchLostNotifierTask);
    this.handler.postDelayed(this.matchLostNotifierTask,
this.scanSettings.getMatchLostTaskInterval());
    }
    } else {
        if (this.emulateBatching) {
            synchronized(this.LOCK) {
                if (!this.devicesInBatch.contains(deviceAddress)) {
                    this.scanResults.add(scanResult);
                    this.devicesInBatch.add(deviceAddress);
                }

                return;
            }
        }

        this.scanCallback.onScanResult(callbackType, scanResult);
    }
}
}

```

```

void handleScanResults(@NonNull List<ScanResult> results) {
    if (!this.scanningStopped) {
        List<ScanResult> filteredResults = results;
        if (this.emulateFiltering) {
            filteredResults = new ArrayList();
            Iterator var3 = results.iterator();

            while(var3.hasNext()) {
                ScanResult result = (ScanResult)var3.next();
                if (this.matches(result)) {
                    ((List)filteredResults).add(result);
                }
            }
        }

        this.scanCallback.onBatchScanResults((List)filteredResults);
    }
}

```

```

void handleScanError(int errorCode) {
    this.scanCallback.onScanFailed(errorCode);
}

```

```

private boolean matches(@NonNull ScanResult result) {
    Iterator var2 = this.filters.iterator();

    ScanFilter filter;
    do {
        if (!var2.hasNext()) {
            return false;
        }
    }
}

```

```

        filter = (ScanFilter)var2.next();
    } while(!filter.matches(result));

    return true;
}
}
}

```

```

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\no\nordicsemi\android\support\v18\scanner
// File: BluetoothLeScannerImplJB.java

```

```

package no.nordicsemi.android.support.v18.scanner;

```

```

import android.app.PendingIntent;
import android.bluetooth.BluetoothAdapter;
import android.bluetooth.BluetoothAdapter.LeScanCallback;
import android.content.Context;
import android.content.Intent;
import android.os.Handler;
import android.os.HandlerThread;
import android.os.SystemClock;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Iterator;
import java.util.List;

```

```

class BluetoothLeScannerImplJB extends BluetoothLeScannerCompat {
    @NonNull
    private final
ScanCallbackWrapperSet<BluetoothLeScannerCompat.ScanCallbackWrapper> wrappers =
new ScanCallbackWrapperSet();
    @Nullable
    private HandlerThread handlerThread;
    @Nullable
    private Handler powerSaveHandler;
    private long powerSaveRestInterval;
    private long powerSaveScanInterval;
    private final Runnable powerSaveSleepTask = new Runnable() {
        public void run() {
            BluetoothAdapter adapter = BluetoothAdapter.getDefaultAdapter();
            if (adapter != null &&
BluetoothLeScannerImplJB.this.powerSaveRestInterval > 0L &&
BluetoothLeScannerImplJB.this.powerSaveScanInterval > 0L) {
                adapter.stopLeScan(BluetoothLeScannerImplJB.this.scanCallback);

BluetoothLeScannerImplJB.this.powerSaveHandler.postDelayed(BluetoothLeScannerImp
lJB.this.powerSaveScanTask,
BluetoothLeScannerImplJB.this.powerSaveRestInterval);

```

```

        }

    }
};
private final Runnable powerSaveScanTask = new Runnable() {
    public void run() {
        BluetoothAdapter adapter = BluetoothAdapter.getDefaultAdapter();
        if (adapter != null &&
BluetoothLeScannerImplJB.this.powerSaveRestInterval > 0L &&
BluetoothLeScannerImplJB.this.powerSaveScanInterval > 0L) {
            adapter.startLeScan(BluetoothLeScannerImplJB.this.scanCallback);

BluetoothLeScannerImplJB.this.powerSaveHandler.postDelayed(BluetoothLeScannerImp
lJB.this.powerSaveSleepTask,
BluetoothLeScannerImplJB.this.powerSaveScanInterval);
        }

    }
};
private final LeScanCallback scanCallback = (device, rssi, scanRecord) -> {
    ScanResult scanResult = new ScanResult(device,
ScanRecord.parseFromBytes(scanRecord), rssi,
SystemClock.elapsedRealtimeNanos());
    synchronized(this.wrappers) {
        Collection<BluetoothLeScannerCompat.ScanCallbackWrapper>
scanCallbackWrappers = this.wrappers.values();
        Iterator var7 = scanCallbackWrappers.iterator();

        while(var7.hasNext()) {
            BluetoothLeScannerCompat.ScanCallbackWrapper wrapper =
(BluetoothLeScannerCompat.ScanCallbackWrapper)var7.next();
            wrapper.handler.post(() -> {
                wrapper.handleScanResult(1, scanResult);
            });
        }

    }
};

void startScanInternal(@NonNull List<ScanFilter> filters, @NonNull
ScanSettings settings, @NonNull ScanCallback callback, @NonNull Handler handler)
{
    BluetoothAdapter adapter = BluetoothAdapter.getDefaultAdapter();
    boolean shouldStart;
    synchronized(this.wrappers) {
        if (this.wrappers.contains(callback)) {
            throw new IllegalArgumentException("scanner already started with
given scanCallback");
        }

        UserScanCallbackWrapper callbackWrapper = new
UserScanCallbackWrapper(callback);
        BluetoothLeScannerCompat.ScanCallbackWrapper wrapper = new
BluetoothLeScannerCompat.ScanCallbackWrapper(false, false, filters, settings,

```

```

callbackWrapper, handler);
    shouldStart = this.wrappers.isEmpty();
    this.wrappers.add(wrapper);
}

if (this.handlerThread == null) {
    this.handlerThread = new
HandlerThread(BluetoothLeScannerImplJB.class.getName());
    this.handlerThread.start();
    this.powerSaveHandler = new Handler(this.handlerThread.getLooper());
}

this.setPowerSaveSettings();
if (shouldStart) {
    adapter.startLeScan(this.scanCallback);
}

}

void stopScanInternal(@NonNull ScanCallback callback) {
    BluetoothAdapter adapter = BluetoothAdapter.getDefaultAdapter();
    boolean shouldStop;
    BluetoothLeScannerCompat.ScanCallbackWrapper wrapper;
    synchronized(this.wrappers) {
        wrapper = this.wrappers.remove(callback);
        shouldStop = this.wrappers.isEmpty();
    }

    if (wrapper != null) {
        wrapper.close();
        this.setPowerSaveSettings();
        if (shouldStop) {
            adapter.stopLeScan(this.scanCallback);
            if (this.powerSaveHandler != null) {
                this.powerSaveHandler.removeCallbacksAndMessages((Object)null);
            }

            if (this.handlerThread != null) {
                this.handlerThread.quitSafely();
                this.handlerThread = null;
            }
        }
    }
}

}

void startScanInternal(@NonNull List<ScanFilter> filters, @NonNull
ScanSettings settings, @NonNull Context context, @NonNull PendingIntent
callbackIntent, int requestCode) {
    Intent service = new Intent(context, ScannerService.class);

    service.putParcelableArrayListExtra("no.nordicsemi.android.support.v18.EXTRA_FIL
TERS", new ArrayList(filters));
    service.putExtra("no.nordicsemi.android.support.v18.EXTRA_SETTINGS",

```

```

settings);
    service.putExtra("no.nordicsemi.android.support.v18.EXTRA_PENDING_INTENT",
callbackIntent);
    service.putExtra("no.nordicsemi.android.support.v18.REQUEST_CODE",
requestCode);
    service.putExtra("no.nordicsemi.android.support.v18.EXTRA_START", true);
    context.startService(service);
}

    void stopScanInternal(@NonNull Context context, @NonNull PendingIntent
callbackIntent, int requestCode) {
        Intent service = new Intent(context, ScannerService.class);
        service.putExtra("no.nordicsemi.android.support.v18.EXTRA_PENDING_INTENT",
callbackIntent);
        service.putExtra("no.nordicsemi.android.support.v18.REQUEST_CODE",
requestCode);
        service.putExtra("no.nordicsemi.android.support.v18.EXTRA_START", false);
        context.startService(service);
    }

    public void flushPendingScanResults(@NonNull ScanCallback callback) {
        if (callback == null) {
            throw new IllegalArgumentException("callback cannot be null!");
        } else {
            BluetoothLeScannerCompat.ScanCallbackWrapper wrapper;
            synchronized(this.wrappers) {
                wrapper = this.wrappers.get(callback);
            }

            if (wrapper == null) {
                throw new IllegalArgumentException("callback not registered!");
            } else {
                wrapper.flushPendingScanResults();
            }
        }
    }

    private void setPowerSaveSettings() {
        long minRest = Long.MAX_VALUE;
        long minScan = Long.MAX_VALUE;
        synchronized(this.wrappers) {
            Iterator var6 = this.wrappers.values().iterator();

            while(var6.hasNext()) {
                BluetoothLeScannerCompat.ScanCallbackWrapper wrapper =
(BluetoothLeScannerCompat.ScanCallbackWrapper)var6.next();
                ScanSettings settings = wrapper.scanSettings;
                if (settings.hasPowerSaveMode()) {
                    if (minRest > settings.getPowerSaveRest()) {
                        minRest = settings.getPowerSaveRest();
                    }

                    if (minScan > settings.getPowerSaveScan()) {
                        minScan = settings.getPowerSaveScan();
                    }
                }
            }
        }
    }
}

```

```

        }
    }
}

if (minRest < Long.MAX_VALUE && minScan < Long.MAX_VALUE) {
    this.powerSaveRestInterval = minRest;
    this.powerSaveScanInterval = minScan;
    if (this.powerSaveHandler != null) {
        this.powerSaveHandler.removeCallbacks(this.powerSaveScanTask);
        this.powerSaveHandler.removeCallbacks(this.powerSaveSleepTask);
        this.powerSaveHandler.postDelayed(this.powerSaveSleepTask,
this.powerSaveScanInterval);
    }
} else {
    this.powerSaveRestInterval = this.powerSaveScanInterval = 0L;
    if (this.powerSaveHandler != null) {
        this.powerSaveHandler.removeCallbacks(this.powerSaveScanTask);
        this.powerSaveHandler.removeCallbacks(this.powerSaveSleepTask);
    }
}
}
}

```

```

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\no\nordicsemi\android\support\v18\scanner
// File: BluetoothLeScannerImplLollipop.java

```

```

package no.nordicsemi.android.support.v18.scanner;

import android.annotation.TargetApi;
import android.app.PendingIntent;
import android.bluetooth.BluetoothAdapter;
import android.bluetooth.le.BluetoothLeScanner;
import android.bluetooth.le.ScanSettings.Builder;
import android.content.Context;
import android.content.Intent;
import android.os.Handler;
import android.os.SystemClock;
import androidx.annotation.NonNull;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

@TargetApi(21)
class BluetoothLeScannerImplLollipop extends BluetoothLeScannerCompat {
    @NonNull
    private final
ScanCallbackWrapperSet<BluetoothLeScannerImplLollipop.ScanCallbackWrapperLollipop>
    wrappers = new ScanCallbackWrapperSet();
}

```

```

void startScanInternal(@NonNull List<ScanFilter> filters, @NonNull
ScanSettings settings, @NonNull ScanCallback callback, @NonNull Handler handler)
{
    BluetoothAdapter adapter = BluetoothAdapter.getDefaultAdapter();
    BluetoothLeScanner scanner = adapter.getBluetoothLeScanner();
    if (scanner == null) {
        throw new IllegalStateException("BT le scanner not available");
    } else {
        boolean offloadedBatchingSupported =
adapter.isOffloadedScanBatchingSupported();
        boolean offloadedFilteringSupported =
adapter.isOffloadedFilteringSupported();
        BluetoothLeScannerImplLollipop.ScanCallbackWrapperLollipop wrapper;
        synchronized(this.wrappers) {
            if (this.wrappers.contains(callback)) {
                throw new IllegalArgumentException("scanner already started with
given callback");
            }

            UserScanCallbackWrapper callbackWrapper = new
UserScanCallbackWrapper(callback);
            wrapper = new
BluetoothLeScannerImplLollipop.ScanCallbackWrapperLollipop(offloadedBatchingSupp
orted, offloadedFilteringSupported, filters, settings, callbackWrapper,
handler);
            this.wrappers.add(wrapper);
        }

        android.bluetooth.le.ScanSettings nativeScanSettings =
this.toNativeScanSettings(adapter, settings, false);
        List<android.bluetooth.le.ScanFilter> nativeScanFilters = null;
        if (!filters.isEmpty() && offloadedFilteringSupported &&
settings.getUseHardwareFilteringIfSupported()) {
            nativeScanFilters = this.toNativeScanFilters(filters);
        }

        scanner.startScan(nativeScanFilters, nativeScanSettings,
wrapper.nativeCallback);
    }
}

void stopScanInternal(@NonNull ScanCallback callback) {
    BluetoothLeScannerImplLollipop.ScanCallbackWrapperLollipop wrapper;
    synchronized(this.wrappers) {
        wrapper =
(BluetoothLeScannerImplLollipop.ScanCallbackWrapperLollipop)this.wrappers.remove
(callback);
    }

    if (wrapper != null) {
        wrapper.close();
        BluetoothAdapter adapter = BluetoothAdapter.getDefaultAdapter();
        if (adapter != null) {
            BluetoothLeScanner scanner = adapter.getBluetoothLeScanner();

```



```

        if (scanner != null) {
            scanner.stopScan(wrapper.nativeCallback);
        }
    }
}

void startScanInternal(@NonNull List<ScanFilter> filters, @NonNull
ScanSettings settings, @NonNull Context context, @NonNull PendingIntent
callbackIntent, int requestCode) {
    BluetoothAdapter adapter = BluetoothAdapter.getDefaultAdapter();
    BluetoothLeScanner scanner = adapter.getBluetoothLeScanner();
    if (scanner == null) {
        throw new IllegalStateException("BT le scanner not available");
    } else {
        Intent service = new Intent(context, ScannerService.class);

service.putParcelableArrayListExtra("no.nordicsemi.android.support.v18.EXTRA_FIL
TERS", new ArrayList(filters));
        service.putExtra("no.nordicsemi.android.support.v18.EXTRA_SETTINGS",
settings);

service.putExtra("no.nordicsemi.android.support.v18.EXTRA_PENDING_INTENT",
callbackIntent);
        service.putExtra("no.nordicsemi.android.support.v18.REQUEST_CODE",
requestCode);
        service.putExtra("no.nordicsemi.android.support.v18.EXTRA_START",
true);
        context.startService(service);
    }
}

void stopScanInternal(@NonNull Context context, @NonNull PendingIntent
callbackIntent, int requestCode) {
    Intent service = new Intent(context, ScannerService.class);
    service.putExtra("no.nordicsemi.android.support.v18.EXTRA_PENDING_INTENT",
callbackIntent);
    service.putExtra("no.nordicsemi.android.support.v18.REQUEST_CODE",
requestCode);
    service.putExtra("no.nordicsemi.android.support.v18.EXTRA_START", false);
    context.startService(service);
}

public void flushPendingScanResults(@NonNull ScanCallback callback) {
    BluetoothAdapter adapter = BluetoothAdapter.getDefaultAdapter();
    if (callback == null) {
        throw new IllegalArgumentException("callback cannot be null!");
    } else {
        BluetoothLeScannerImplLollipop.ScanCallbackWrapperLollipop wrapper;
        synchronized(this.wrappers) {
            wrapper =
(BluetoothLeScannerImplLollipop.ScanCallbackWrapperLollipop)this.wrappers.get(ca
llback);

```

```

    }

    if (wrapper == null) {
        throw new IllegalArgumentException("callback not registered!");
    } else {
        ScanSettings settings = wrapper.scanSettings;
        if (adapter.isOffloadedScanBatchingSupported() &&
settings.getUseHardwareBatchingIfSupported()) {
            BluetoothLeScanner scanner = adapter.getBluetoothLeScanner();
            if (scanner == null) {
                return;
            }

            scanner.flushPendingScanResults(wrapper.nativeCallback);
        } else {
            wrapper.flushPendingScanResults();
        }
    }
}

}

}

@NonNull
android.bluetooth.le.ScanSettings toNativeScanSettings(@NonNull
BluetoothAdapter adapter, @NonNull ScanSettings settings, boolean exactCopy) {
    Builder builder = new Builder();
    if (exactCopy || adapter.isOffloadedScanBatchingSupported() &&
settings.getUseHardwareBatchingIfSupported()) {
        builder.setReportDelay(settings.getReportDelayMillis());
    }

    if (settings.getScanMode() != -1) {
        builder.setScanMode(settings.getScanMode());
    } else {
        builder.setScanMode(0);
    }

    settings.disableUseHardwareCallbackTypes();
    return builder.build();
}

@NonNull
ArrayList<android.bluetooth.le.ScanFilter> toNativeScanFilters(@NonNull
List<ScanFilter> filters) {
    ArrayList<android.bluetooth.le.ScanFilter> nativeScanFilters = new
ArrayList();
    Iterator var3 = filters.iterator();

    while(var3.hasNext()) {
        ScanFilter filter = (ScanFilter)var3.next();
        nativeScanFilters.add(this.toNativeScanFilter(filter));
    }

    return nativeScanFilters;
}

```

```

    }

    @NonNull
    android.bluetooth.le.ScanFilter toNativeScanFilter(@NonNull ScanFilter
filter) {
        android.bluetooth.le.ScanFilter.Builder builder = new
android.bluetooth.le.ScanFilter.Builder();
        builder.setServiceUuid(filter.getServiceUuid(),
filter.getServiceUuidMask()).setManufacturerData(filter.getManufacturerId(),
filter.getManufacturerData(), filter.getManufacturerDataMask());
        if (filter.getAddress() != null) {
            builder.setDeviceAddress(filter.getAddress());
        }

        if (filter.getDeviceName() != null) {
            builder.setDeviceName(filter.getDeviceName());
        }

        if (filter.getServiceDataUuid() != null) {
            builder.setServiceData(filter.getServiceDataUuid(),
filter.getServiceData(), filter.getServiceDataMask());
        }

        return builder.build();
    }

    @NonNull
    ScanResult fromNativeScanResult(@NonNull android.bluetooth.le.ScanResult
nativeScanResult) {
        byte[] data = nativeScanResult.getScanRecord() != null ?
nativeScanResult.getScanRecord().getBytes() : null;
        return new ScanResult(nativeScanResult.getDevice(),
ScanRecord.parseFromBytes(data), nativeScanResult.getRssi(),
nativeScanResult.getTimestampNanos());
    }

    @NonNull
    ArrayList<ScanResult> fromNativeScanResults(@NonNull
List<android.bluetooth.le.ScanResult> nativeScanResults) {
        ArrayList<ScanResult> results = new ArrayList();
        Iterator var3 = nativeScanResults.iterator();

        while(var3.hasNext()) {
            android.bluetooth.le.ScanResult nativeScanResult =
(android.bluetooth.le.ScanResult)var3.next();
            ScanResult result = this.fromNativeScanResult(nativeScanResult);
            results.add(result);
        }

        return results;
    }

    static class ScanCallbackWrapperLollipop extends
BluetoothLeScannerCompat.ScanCallbackWrapper {

```

```

@NonNull
private final android.bluetooth.le.ScanCallback nativeCallback;

private ScanCallbackWrapperLollipop(boolean offloadedBatchingSupported,
boolean offloadedFilteringSupported, @NonNull List<ScanFilter> filters, @NonNull
ScanSettings settings, @NonNull ScanCallback callback, @NonNull Handler handler)
{
    super(offloadedBatchingSupported, offloadedFilteringSupported, filters,
settings, callback, handler);
    this.nativeCallback = new android.bluetooth.le.ScanCallback() {
        private long lastBatchTimestamp;

        public void onScanResult(int callbackType,
android.bluetooth.le.ScanResult nativeScanResult) {
            ScanCallbackWrapperLollipop.this.handler.post(() -> {
                BluetoothLeScannerImplLollipop scannerImpl =
                (BluetoothLeScannerImplLollipop)BluetoothLeScannerCompat.getScanner();
                ScanResult result =
                scannerImpl.fromNativeScanResult(nativeScanResult);

                ScanCallbackWrapperLollipop.this.handleScanResult(callbackType, result);
            });
        }

        public void onBatchScanResults(List<android.bluetooth.le.ScanResult>
nativeScanResults) {
            ScanCallbackWrapperLollipop.this.handler.post(() -> {
                long now = SystemClock.elapsedRealtime();
                if (this.lastBatchTimestamp <= now -
                ScanCallbackWrapperLollipop.this.scanSettings.getReportDelayMillis() + 5L) {
                    this.lastBatchTimestamp = now;
                    BluetoothLeScannerImplLollipop scannerImpl =
                    (BluetoothLeScannerImplLollipop)BluetoothLeScannerCompat.getScanner();
                    List<ScanResult> results =
                    scannerImpl.fromNativeScanResults(nativeScanResults);

                    ScanCallbackWrapperLollipop.this.handleScanResults(results);
                }
            });
        }

        public void onScanFailed(int errorCode) {
            ScanCallbackWrapperLollipop.this.handler.post(() -> {
                if
                (ScanCallbackWrapperLollipop.this.scanSettings.getUseHardwareCallbackTypesIfSupp
orted() && ScanCallbackWrapperLollipop.this.scanSettings.getCallbackType() != 1)
                {
                    ScanCallbackWrapperLollipop.this.scanSettings.disableUseHardwareCallbackTypes();
                    BluetoothLeScannerCompat scanner =
                    BluetoothLeScannerCompat.getScanner();

                    try {

```

```

scanner.stopScan(ScanCallbackWrapperLollipop.this.scanCallback);
        } catch (Exception var5) {
        }

        try {

scanner.startScanInternal(ScanCallbackWrapperLollipop.this.filters,
ScanCallbackWrapperLollipop.this.scanSettings,
ScanCallbackWrapperLollipop.this.scanCallback,
ScanCallbackWrapperLollipop.this.handler);
        } catch (Exception var4) {
        }

        } else {

ScanCallbackWrapperLollipop.this.handleScanError(errorCode);
        }
    });
}
};
}

// $FF: synthetic method
ScanCallbackWrapperLollipop(boolean x0, boolean x1, List x2, ScanSettings
x3, ScanCallback x4, Handler x5, Object x6) {
    this(x0, x1, x2, x3, x4, x5);
}
}
}

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\no\nordicsemi\android\support\v18\scanner
// File: BluetoothLeScannerImplMarshmallow.java

package no.nordicsemi.android.support.v18.scanner;

import android.annotation.TargetApi;
import android.bluetooth.BluetoothAdapter;
import android.bluetooth.le.ScanSettings.Builder;
import androidx.annotation.NonNull;

@TargetApi(23)
class BluetoothLeScannerImplMarshmallow extends BluetoothLeScannerImplLollipop {
    @NonNull
    android.bluetooth.le.ScanSettings toNativeScanSettings(@NonNull
BluetoothAdapter adapter, @NonNull ScanSettings settings, boolean exactCopy) {
        Builder builder = new Builder();
        if (exactCopy || adapter.isOffloadedScanBatchingSupported() &&
settings.getUseHardwareBatchingIfSupported()) {
            builder.setReportDelay(settings.getReportDelayMillis());
        }
    }
}

```

```

        if (exactCopy || settings.getUseHardwareCallbackTypesIfSupported()) {

builder.setCallbackType(settings.getCallbackType()).setMatchMode(settings.getMatchMode()).setNumOfMatches(settings.getNumOfMatches());
        }

        builder.setScanMode(settings.getScanMode());
        return builder.build();
    }
}

```

```

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\no\nordicsemi\android\support\v18\scanner
// File: BluetoothLeScannerImplOreo.java

```

```

package no.nordicsemi.android.support.v18.scanner;

```

```

import android.annotation.TargetApi;
import android.app.PendingIntent;
import android.bluetooth.BluetoothAdapter;
import android.bluetooth.le.BluetoothLeScanner;
import android.bluetooth.le.ScanSettings.Builder;
import android.content.Context;
import android.content.Intent;
import android.os.Handler;
import android.os.Build.VERSION;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;

```

```

@TargetApi(26)
class BluetoothLeScannerImplOreo extends BluetoothLeScannerImplMarshmallow {
    @NonNull
    private final HashMap<PendingIntent,
BluetoothLeScannerImplOreo.PendingIntentExecutorWrapper> wrappers = new
HashMap();

    @Nullable
    BluetoothLeScannerImplOreo.PendingIntentExecutorWrapper getWrapper(@NonNull
PendingIntent callbackIntent) {
        synchronized(this.wrappers) {
            if (this.wrappers.containsKey(callbackIntent)) {
                BluetoothLeScannerImplOreo.PendingIntentExecutorWrapper wrapper =
(BluetoothLeScannerImplOreo.PendingIntentExecutorWrapper)this.wrappers.get(callbackIntent);
                if (wrapper == null) {
                    throw new IllegalStateException("Scanning has been stopped");
                } else {

```

```

        return wrapper;
    }
    } else {
        return null;
    }
}

void addWrapper(@NonNull PendingIntent callbackIntent, @NonNull
BluetoothLeScannerImplOreo.PendingIntentExecutorWrapper wrapper) {
    synchronized(this.wrappers) {
        this.wrappers.put(callbackIntent, wrapper);
    }
}

void startScanInternal(@Nullable List<ScanFilter> filters, @Nullable
ScanSettings settings, @NonNull Context context, @NonNull PendingIntent
callbackIntent, int requestCode) {
    BluetoothAdapter adapter = BluetoothAdapter.getDefaultAdapter();
    BluetoothLeScanner scanner = adapter.getBluetoothLeScanner();
    if (scanner == null) {
        throw new IllegalStateException("BT le scanner not available");
    } else {
        ScanSettings nonNullSettings = settings != null ? settings : (new
ScanSettings.Builder()).build();
        List<ScanFilter> nonNullFilters = filters != null ? filters :
Collections.emptyList();
        android.bluetooth.le.ScanSettings nativeSettings =
this.toNativeScanSettings(adapter, nonNullSettings, false);
        List<android.bluetooth.le.ScanFilter> nativeFilters = null;
        if (filters != null && adapter.isOffloadedFilteringSupported() &&
nonNullSettings.getUseHardwareFilteringIfSupported()) {
            nativeFilters = this.toNativeScanFilters(filters);
        }

        synchronized(this.wrappers) {
            this.wrappers.remove(callbackIntent);
        }

        PendingIntent pendingIntent =
this.createStartingPendingIntent(nonNullFilters, nonNullSettings, context,
callbackIntent, requestCode);
        scanner.startScan(nativeFilters, nativeSettings, pendingIntent);
    }
}

void stopScanInternal(@NonNull Context context, @NonNull PendingIntent
callbackIntent, int requestCode) {
    BluetoothAdapter adapter = BluetoothAdapter.getDefaultAdapter();
    BluetoothLeScanner scanner = adapter.getBluetoothLeScanner();
    if (scanner == null) {
        throw new IllegalStateException("BT le scanner not available");
    } else {
        PendingIntent pendingIntent = this.createStoppingPendingIntent(context,

```

```

requestCode);
        scanner.stopScan(pendingIntent);
        synchronized(this.wrappers) {
            this.wrappers.put(callbackIntent, (Object)null);
        }
    }
}

@NonNull
private PendingIntent createStartingPendingIntent(@NonNull List<ScanFilter>
filters, @NonNull ScanSettings settings, @NonNull Context context, @NonNull
PendingIntent callbackIntent, int requestCode) {
    Intent intent = new Intent(context, PendingIntentReceiver.class);
    intent.setAction("no.nordicsemi.android.support.v18.ACTION_FOUND");
    BluetoothAdapter adapter = BluetoothAdapter.getDefaultAdapter();
    intent.putExtra("no.nordicsemi.android.support.v18.EXTRA_PENDING_INTENT",
callbackIntent);

    intent.putParcelableArrayListExtra("no.nordicsemi.android.support.v18.EXTRA_FILT
ERS", this.toNativeScanFilters(filters));
    intent.putExtra("no.nordicsemi.android.support.v18.EXTRA_SETTINGS",
this.toNativeScanSettings(adapter, settings, true));

    intent.putExtra("no.nordicsemi.android.support.v18.EXTRA_USE_HARDWARE_BATCHING",
settings.getUseHardwareBatchingIfSupported());

    intent.putExtra("no.nordicsemi.android.support.v18.EXTRA_USE_HARDWARE_FILTERING"
, settings.getUseHardwareFilteringIfSupported());

    intent.putExtra("no.nordicsemi.android.support.v18.EXTRA_USE_HARDWARE_CALLBACK_T
YPES", settings.getUseHardwareCallbackTypesIfSupported());
    intent.putExtra("no.nordicsemi.android.support.v18.EXTRA_MATCH_MODE",
settings.getMatchMode());
    intent.putExtra("no.nordicsemi.android.support.v18.EXTRA_NUM_OF_MATCHES",
settings.getNumOfMatches());
    int flags = 134217728;
    if (VERSION.SDK_INT >= 31) {
        flags |= 33554432;
    }

    return PendingIntent.getBroadcast(context, requestCode, intent, flags);
}

@NonNull
private PendingIntent createStoppingPendingIntent(@NonNull Context context,
int requestCode) {
    Intent intent = new Intent(context, PendingIntentReceiver.class);
    intent.setAction("no.nordicsemi.android.support.v18.ACTION_FOUND");
    int flags = 134217728;
    if (VERSION.SDK_INT >= 23) {
        flags |= 67108864;
    }

    return PendingIntent.getBroadcast(context, requestCode, intent, flags);
}

```



```

    }

    @NonNull
    android.bluetooth.le.ScanSettings toNativeScanSettings(@NonNull
BluetoothAdapter adapter, @NonNull ScanSettings settings, boolean exactCopy) {
        Builder builder = new Builder();
        if (exactCopy || adapter.isOffloadedScanBatchingSupported() &&
settings.getUseHardwareBatchingIfSupported()) {
            builder.setReportDelay(settings.getReportDelayMillis());
        }

        if (exactCopy || settings.getUseHardwareCallbackTypesIfSupported()) {

builder.setCallbackType(settings.getCallbackType()).setMatchMode(settings.getMat
chMode()).setNumOfMatches(settings.getNumOfMatches());
        }

builder.setScanMode(settings.getScanMode()).setLegacy(settings.getLegacy()).setP
hy(settings.getPhy());
        return builder.build();
    }

    @NonNull
    ScanSettings fromNativeScanSettings(@NonNull
android.bluetooth.le.ScanSettings settings, boolean
useHardwareBatchingIfSupported, boolean useHardwareFilteringIfSupported, boolean
useHardwareCallbackTypesIfSupported, long matchLostDeviceTimeout, long
matchLostTaskInterval, int matchMode, int numOfMatches) {
        ScanSettings.Builder builder = (new
ScanSettings.Builder()).setLegacy(settings.getLegacy()).setPhy(settings.getPhy())
.setCallbackType(settings.getCallbackType()).setScanMode(settings.getScanMode())
.setReportDelay(settings.getReportDelayMillis()).setUseHardwareBatchingIfSupport
ed(useHardwareBatchingIfSupported).setUseHardwareFilteringIfSupported(useHardwa
reFilteringIfSupported).setUseHardwareCallbackTypesIfSupported(useHardwareCallba
ckTypesIfSupported).setMatchOptions(matchLostDeviceTimeout,
matchLostTaskInterval).setMatchMode(matchMode).setNumOfMatches(numOfMatches);
        return builder.build();
    }

    @NonNull
    ArrayList<ScanFilter> fromNativeScanFilters(@NonNull
List<android.bluetooth.le.ScanFilter> filters) {
        ArrayList<ScanFilter> nativeScanFilters = new ArrayList();
        Iterator var3 = filters.iterator();

        while(var3.hasNext()) {
            android.bluetooth.le.ScanFilter filter =
(android.bluetooth.le.ScanFilter)var3.next();
            nativeScanFilters.add(this.fromNativeScanFilter(filter));
        }

        return nativeScanFilters;
    }
}

```

```

        @NonNull
        ScanFilter fromNativeScanFilter(@NonNull android.bluetooth.le.ScanFilter
filter) {
            ScanFilter.Builder builder = new ScanFilter.Builder();

            builder.setDeviceAddress(filter.getDeviceAddress()).setDeviceName(filter.getDeviceName()).setServiceUuid(filter.getServiceUuid(),
filter.getServiceUuidMask()).setManufacturerData(filter.getManufacturerId(),
filter.getManufacturerData(), filter.getManufacturerDataMask());
            if (filter.getServiceDataUuid() != null) {
                builder.setServiceData(filter.getServiceDataUuid(),
filter.getServiceData(), filter.getServiceDataMask());
            }

            return builder.build();
        }

        @NonNull
        ScanResult fromNativeScanResult(@NonNull android.bluetooth.le.ScanResult
result) {
            int eventType = result.getDataStatus() << 5 | (result.isLegacy() ? 16 : 0)
| (result.isConnectable() ? 1 : 0);
            byte[] data = result.getScanRecord() != null ?
result.getScanRecord().getBytes() : null;
            return new ScanResult(result.getDevice(), eventType,
result.getPrimaryPhy(), result.getSecondaryPhy(), result.getAdvertisingSid(),
result.getTxPower(), result.getRssi(), result.getPeriodicAdvertisingInterval(),
ScanRecord.parseFromBytes(data), result.getTimestampNanos());
        }

        static class PendingIntentExecutorWrapper extends
BluetoothLeScannerCompat.ScanCallbackWrapper {
            @NonNull
            final PendingIntentExecutor executor;

            PendingIntentExecutorWrapper(boolean offloadedBatchingSupported, boolean
offloadedFilteringSupported, @NonNull List<ScanFilter> filters, @NonNull
ScanSettings settings, @NonNull PendingIntentExecutor executor) {
                super(offloadedBatchingSupported, offloadedFilteringSupported, filters,
settings, executor, new Handler());
                this.executor = executor;
            }
        }
    }

    // Directory: app
    // Subdirectory:
    classes.jar_Decompile.com\no\nordicsemi\android\support\v18\scanner
    // File: BluetoothLeUtils.java

    package no.nordicsemi.android.support.v18.scanner;

```

```

import android.util.SparseArray;
import androidx.annotation.Nullable;
import java.util.Arrays;
import java.util.Iterator;
import java.util.Map;
import java.util.Set;
import java.util.Map.Entry;

class BluetoothLeUtils {
    static String toString(@Nullable SparseArray<byte[]> array) {
        if (array == null) {
            return "null";
        } else if (array.size() == 0) {
            return "{}";
        } else {
            StringBuilder buffer = new StringBuilder();
            buffer.append('{');

            for(int i = 0; i < array.size(); ++i) {

buffer.append(array.keyAt(i)).append("=").append(Arrays.toString((byte[])array.v
alueAt(i)));
                }

            buffer.append('}');
            return buffer.toString();
        }
    }

    static <T> String toString(@Nullable Map<T, byte[]> map) {
        if (map == null) {
            return "null";
        } else if (map.isEmpty()) {
            return "{}";
        } else {
            StringBuilder buffer = new StringBuilder();
            buffer.append('{');
            Iterator it = map.entrySet().iterator();

            while(it.hasNext()) {
                Entry<T, byte[]> entry = (Entry)it.next();
                Object key = entry.getKey();

buffer.append(key).append("=").append(Arrays.toString((byte[])map.get(key)));
                if (it.hasNext()) {
                    buffer.append(", ");
                }
            }

            buffer.append('}');
            return buffer.toString();
        }
    }
}

```

```

    static boolean equals(@Nullable SparseArray<byte[]> array, @Nullable
SparseArray<byte[]> otherArray) {
        if (array == otherArray) {
            return true;
        } else if (array != null && otherArray != null) {
            if (array.size() != otherArray.size()) {
                return false;
            } else {
                for(int i = 0; i < array.size(); ++i) {
                    if (array.keyAt(i) != otherArray.keyAt(i) ||
!Arrays.equals((byte[])array.valueAt(i), (byte[])otherArray.valueAt(i))) {
                        return false;
                    }
                }

                return true;
            }
        } else {
            return false;
        }
    }
}

```

```

    static <T> boolean equals(@Nullable Map<T, byte[]> map, Map<T, byte[]>
otherMap) {
        if (map == otherMap) {
            return true;
        } else if (map != null && otherMap != null) {
            if (map.size() != otherMap.size()) {
                return false;
            } else {
                Set<T> keys = map.keySet();
                if (!keys.equals(otherMap.keySet())) {
                    return false;
                } else {
                    Iterator var3 = keys.iterator();

                    Object key;
                    do {
                        if (!var3.hasNext()) {
                            return true;
                        }

                        key = var3.next();
                    } while(Objects.deepEquals(map.get(key), otherMap.get(key)));

                    return false;
                }
            }
        } else {
            return false;
        }
    }
}

```

```

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\no\nordicsemi\android\support\v18\scanner
// File: BluetoothUuid.java

package no.nordicsemi.android.support.v18.scanner;

import android.os.ParcelUuid;
import java.nio.ByteBuffer;
import java.nio.ByteOrder;
import java.util.UUID;

final class BluetoothUuid {
    private static final ParcelUuid BASE_UUID =
ParcelUuid.fromString("00000000-0000-1000-8000-00805F9B34FB");
    static final int UUID_BYTES_16_BIT = 2;
    static final int UUID_BYTES_32_BIT = 4;
    static final int UUID_BYTES_128_BIT = 16;

    static ParcelUuid parseUuidFrom(byte[] uuidBytes) {
        if (uuidBytes == null) {
            throw new IllegalArgumentException("uuidBytes cannot be null");
        } else {
            int length = uuidBytes.length;
            if (length != 2 && length != 4 && length != 16) {
                throw new IllegalArgumentException("uuidBytes length invalid - " +
length);
            } else if (length == 16) {
                ByteBuffer buf =
ByteBuffer.wrap(uuidBytes).order(ByteOrder.LITTLE_ENDIAN);
                long msb = buf.getLong(8);
                long lsb = buf.getLong(0);
                return new ParcelUuid(new UUID(msb, lsb));
            } else {
                long shortUuid;
                if (length == 2) {
                    shortUuid = (long)(uuidBytes[0] & 255);
                    shortUuid += (long)((uuidBytes[1] & 255) << 8);
                } else {
                    shortUuid = (long)(uuidBytes[0] & 255);
                    shortUuid += (long)((uuidBytes[1] & 255) << 8);
                    shortUuid += (long)((uuidBytes[2] & 255) << 16);
                    shortUuid += (long)((uuidBytes[3] & 255) << 24);
                }

                long msb = BASE_UUID.getUuid().getMostSignificantBits() + (shortUuid
<< 32);
                long lsb = BASE_UUID.getUuid().getLeastSignificantBits();
                return new ParcelUuid(new UUID(msb, lsb));
            }
        }
    }
}

```

```
// Directory: app
// Subdirectory:
classes.jar_Decompiler.com\no\nordicsemi\android\support\v18\scanner
// File: BuildConfig.java
```

```
package no.nordicsemi.android.support.v18.scanner;

public final class BuildConfig {
    public static final boolean DEBUG = false;
    public static final String LIBRARY_PACKAGE_NAME =
    "no.nordicsemi.android.support.v18.scanner";
    public static final String BUILD_TYPE = "release";
}
```

```
// Directory: app
// Subdirectory:
classes.jar_Decompiler.com\no\nordicsemi\android\support\v18\scanner
// File: Objects.java
```

```
package no.nordicsemi.android.support.v18.scanner;

import java.util.Arrays;

class Objects {
    static boolean deepEquals(Object a, Object b) {
        if (a != null && b != null) {
            if (a instanceof Object[] && b instanceof Object[]) {
                return Arrays.deepEquals((Object[])a, (Object[])b);
            } else if (a instanceof boolean[] && b instanceof boolean[]) {
                return Arrays.equals((boolean[])a, (boolean[])b);
            } else if (a instanceof byte[] && b instanceof byte[]) {
                return Arrays.equals((byte[])a, (byte[])b);
            } else if (a instanceof char[] && b instanceof char[]) {
                return Arrays.equals((char[])a, (char[])b);
            } else if (a instanceof double[] && b instanceof double[]) {
                return Arrays.equals((double[])a, (double[])b);
            } else if (a instanceof float[] && b instanceof float[]) {
                return Arrays.equals((float[])a, (float[])b);
            } else if (a instanceof int[] && b instanceof int[]) {
                return Arrays.equals((int[])a, (int[])b);
            } else if (a instanceof long[] && b instanceof long[]) {
                return Arrays.equals((long[])a, (long[])b);
            } else {
                return a instanceof short[] && b instanceof short[] ?
                Arrays.equals((short[])a, (short[])b) : a.equals(b);
            }
        } else {
            return a == b;
        }
    }
}
```

```

static boolean equals(Object a, Object b) {
    return a == null ? b == null : a.equals(b);
}

static int hash(Object... values) {
    return Arrays.hashCode(values);
}

static String toString(Object o) {
    return o == null ? "null" : o.toString();
}
}

```

```

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\no\nordicsemi\android\support\v18\scanner
// File: PendingIntentExecutor.java

```

```

package no.nordicsemi.android.support.v18.scanner;

```

```

import android.app.PendingIntent;
import android.app.Service;
import android.app.PendingIntent.CanceledException;
import android.content.Context;
import android.content.Intent;
import android.os.SystemClock;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

```

```

class PendingIntentExecutor extends ScanCallback {
    @NonNull
    private final PendingIntent callbackIntent;
    @Nullable
    private Context context;
    @Nullable
    private Context service;
    private long lastBatchTimestamp;
    private final long reportDelay;

    PendingIntentExecutor(@NonNull PendingIntent callbackIntent, @NonNull
ScanSettings settings) {
        this.callbackIntent = callbackIntent;
        this.reportDelay = settings.getReportDelayMillis();
    }

    PendingIntentExecutor(@NonNull PendingIntent callbackIntent, @NonNull
ScanSettings settings, @NonNull Service service) {
        this.callbackIntent = callbackIntent;
        this.reportDelay = settings.getReportDelayMillis();
        this.service = service;
    }
}

```

```

    }

    void setTemporaryContext(@Nullable Context context) {
        this.context = context;
    }

    public void onScanResult(int callbackType, @NonNull ScanResult result) {
        Context context = this.context != null ? this.context : this.service;
        if (context != null) {
            try {
                Intent extrasIntent = new Intent();
                extrasIntent.putExtra("android.bluetooth.le.extra.CALLBACK_TYPE",
callbackType);

                extrasIntent.putParcelableArrayListExtra("android.bluetooth.le.extra.LIST_SCAN_R
ESULT", new ArrayList(Collections.singletonList(result)));
                this.callbackIntent.send(context, 0, extrasIntent);
            } catch (CanceledException var5) {
            }

        }
    }

    public void onBatchScanResults(@NonNull List<ScanResult> results) {
        Context context = this.context != null ? this.context : this.service;
        if (context != null) {
            long now = SystemClock.elapsedRealtime();
            if (this.lastBatchTimestamp <= now - this.reportDelay + 5L) {
                this.lastBatchTimestamp = now;

                try {
                    Intent extrasIntent = new Intent();
                    extrasIntent.putExtra("android.bluetooth.le.extra.CALLBACK_TYPE",
1);

                    extrasIntent.putParcelableArrayListExtra("android.bluetooth.le.extra.LIST_SCAN_R
ESULT", new ArrayList(results));

                    extrasIntent.setExtrasClassLoader(ScanResult.class.getClassLoader());
                    this.callbackIntent.send(context, 0, extrasIntent);
                } catch (CanceledException var6) {
                }

            }
        }
    }

    public void onScanFailed(int errorCode) {
        Context context = this.context != null ? this.context : this.service;
        if (context != null) {
            try {
                Intent extrasIntent = new Intent();
                extrasIntent.putExtra("android.bluetooth.le.extra.ERROR_CODE",
errorCode);

```



```

        this.callbackIntent.send(context, 0, extrasIntent);
    } catch (CanceledException var4) {
    }
}
}
}
}

```

```

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\no\nordicsemi\android\support\v18\scanner
// File: PendingIntentReceiver.java

```

```

package no.nordicsemi.android.support.v18.scanner;

import android.app.PendingIntent;
import android.bluetooth.BluetoothAdapter;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import androidx.annotation.RequiresApi;
import java.util.ArrayList;
import java.util.List;

public class PendingIntentReceiver extends BroadcastReceiver {
    static final String ACTION =
"no.nordicsemi.android.support.v18.ACTION_FOUND";
    static final String EXTRA_PENDING_INTENT =
"no.nordicsemi.android.support.v18.EXTRA_PENDING_INTENT";
    static final String EXTRA_FILTERS =
"no.nordicsemi.android.support.v18.EXTRA_FILTERS";
    static final String EXTRA_SETTINGS =
"no.nordicsemi.android.support.v18.EXTRA_SETTINGS";
    static final String EXTRA_USE_HARDWARE_BATCHING =
"no.nordicsemi.android.support.v18.EXTRA_USE_HARDWARE_BATCHING";
    static final String EXTRA_USE_HARDWARE_FILTERING =
"no.nordicsemi.android.support.v18.EXTRA_USE_HARDWARE_FILTERING";
    static final String EXTRA_USE_HARDWARE_CALLBACK_TYPES =
"no.nordicsemi.android.support.v18.EXTRA_USE_HARDWARE_CALLBACK_TYPES";
    static final String EXTRA_MATCH_LOST_TIMEOUT =
"no.nordicsemi.android.support.v18.EXTRA_MATCH_LOST_TIMEOUT";
    static final String EXTRA_MATCH_LOST_INTERVAL =
"no.nordicsemi.android.support.v18.EXTRA_MATCH_LOST_INTERVAL";
    static final String EXTRA_MATCH_MODE =
"no.nordicsemi.android.support.v18.EXTRA_MATCH_MODE";
    static final String EXTRA_NUM_OF_MATCHES =
"no.nordicsemi.android.support.v18.EXTRA_NUM_OF_MATCHES";

    @RequiresApi(
        api = 26
    )
    public void onReceive(Context context, Intent intent) {
        if (context != null && intent != null) {

```

```

        PendingIntent callbackIntent =
(PendingIntent)intent.getParcelableExtra("no.nordicsemi.android.support.v18.EXTRA_PENDING_INTENT");
        if (callbackIntent != null) {
            ArrayList<android.bluetooth.le.ScanFilter> nativeScanFilters =
intent.getParcelableArrayListExtra("no.nordicsemi.android.support.v18.EXTRA_FILTERS");
            android.bluetooth.le.ScanSettings nativeScanSettings =
(android.bluetooth.le.ScanSettings)intent.getParcelableExtra("no.nordicsemi.android.support.v18.EXTRA_SETTINGS");
            if (nativeScanFilters != null && nativeScanSettings != null) {
                boolean useHardwareBatchingIfSupported =
intent.getBooleanExtra("no.nordicsemi.android.support.v18.EXTRA_USE_HARDWARE_BATCHING", true);
                boolean useHardwareFilteringIfSupported =
intent.getBooleanExtra("no.nordicsemi.android.support.v18.EXTRA_USE_HARDWARE_FILTERING", true);
                boolean useHardwareCallbackTypesIfSupported =
intent.getBooleanExtra("no.nordicsemi.android.support.v18.EXTRA_USE_HARDWARE_CALLBACK_TYPES", true);
                long matchLostDeviceTimeout =
intent.getLongExtra("no.nordicsemi.android.support.v18.EXTRA_MATCH_LOST_TIMEOUT", 10000L);
                long matchLostTaskInterval =
intent.getLongExtra("no.nordicsemi.android.support.v18.EXTRA_MATCH_LOST_INTERVAL", 10000L);
                int matchMode =
intent.getIntExtra("no.nordicsemi.android.support.v18.EXTRA_MATCH_MODE", 1);
                int numOfMatches =
intent.getIntExtra("no.nordicsemi.android.support.v18.EXTRA_NUM_OF_MATCHES", 3);
                BluetoothLeScannerCompat scanner =
BluetoothLeScannerCompat.getScanner();
                BluetoothLeScannerImplOreo scannerImpl =
(BluetoothLeScannerImplOreo)scanner;
                ArrayList<ScanFilter> filters =
scannerImpl.fromNativeScanFilters(nativeScanFilters);
                ScanSettings settings =
scannerImpl.fromNativeScanSettings(nativeScanSettings,
useHardwareBatchingIfSupported, useHardwareFilteringIfSupported,
useHardwareCallbackTypesIfSupported, matchLostDeviceTimeout,
matchLostTaskInterval, matchMode, numOfMatches);
                BluetoothAdapter adapter = BluetoothAdapter.getDefaultAdapter();
                boolean offloadedBatchingSupported =
adapter.isOffloadedScanBatchingSupported();
                boolean offloadedFilteringSupported =
adapter.isOffloadedFilteringSupported();
                BluetoothLeScannerImplOreo.PendingIntentExecutorWrapper wrapper;
                synchronized(scanner) {
                    try {
                        wrapper = scannerImpl.getWrapper(callbackIntent);
                    } catch (IllegalStateException var26) {
                        return;
                    }
                }
            }
        }
    }
}

```



```

    public static final int SCAN_FAILED_INTERNAL_ERROR = 3;
    public static final int SCAN_FAILED_FEATURE_UNSUPPORTED = 4;
    public static final int SCAN_FAILED_OUT_OF_HARDWARE_RESOURCES = 5;
    public static final int SCAN_FAILED_SCANNING_TOO_FREQUENTLY = 6;
    static final int NO_ERROR = 0;

    public void onScanResult(int callbackType, @NonNull ScanResult result) {
    }

    public void onBatchScanResults(@NonNull List<ScanResult> results) {
    }

    public void onScanFailed(int errorCode) {
    }
}

```

```

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\no\nordicsemi\android\support\v18\scanner
// File: ScanCallbackWrapperSet.java

```

```

package no.nordicsemi.android.support.v18.scanner;

```

```

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import java.util.HashSet;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;
import java.util.Set;

```

```

class ScanCallbackWrapperSet<W extends
BluetoothLeScannerCompat.ScanCallbackWrapper> {
    @NonNull
    private final Set<W> wrappers = new HashSet();

    @NonNull
    public Set<W> values() {
        return this.wrappers;
    }

    boolean isEmpty() {
        return this.wrappers.isEmpty();
    }

    void add(@NonNull W wrapper) {
        this.wrappers.add(wrapper);
    }

    boolean contains(@NonNull ScanCallback callback) {
        Iterator var2 = this.wrappers.iterator();

        while(var2.hasNext()) {

```

```

        W wrapper = (BluetoothLeScannerCompat.ScanCallbackWrapper)var2.next();
        if (wrapper.scanCallback == callback) {
            return true;
        }

        if (wrapper.scanCallback instanceof UserScanCallbackWrapper) {
            UserScanCallbackWrapper callbackWrapper =
(UserScanCallbackWrapper)wrapper.scanCallback;
            if (callbackWrapper.get() == callback) {
                return true;
            }
        }

        return false;
    }

    @Nullable
    W get(@NonNull ScanCallback callback) {
        Iterator var2 = this.wrappers.iterator();

        while(var2.hasNext()) {
            W wrapper = (BluetoothLeScannerCompat.ScanCallbackWrapper)var2.next();
            if (wrapper.scanCallback == callback) {
                return wrapper;
            }

            if (wrapper.scanCallback instanceof UserScanCallbackWrapper) {
                UserScanCallbackWrapper callbackWrapper =
(UserScanCallbackWrapper)wrapper.scanCallback;
                if (callbackWrapper.get() == callback) {
                    return wrapper;
                }
            }
        }

        return null;
    }

    @Nullable
    W remove(@NonNull ScanCallback callback) {
        Iterator var2 = this.wrappers.iterator();

        while(var2.hasNext()) {
            W wrapper = (BluetoothLeScannerCompat.ScanCallbackWrapper)var2.next();
            if (wrapper.scanCallback == callback) {
                return wrapper;
            }

            if (wrapper.scanCallback instanceof UserScanCallbackWrapper) {
                UserScanCallbackWrapper callbackWrapper =
(UserScanCallbackWrapper)wrapper.scanCallback;
                if (callbackWrapper.get() == callback) {
                    this.wrappers.remove(wrapper);
                }
            }
        }
    }

```

```

        return wrapper;
    }
}

this.cleanUp();
return null;
}

private void cleanUp() {
    List<W> deadWrappers = new LinkedList();
    Iterator var2 = this.wrappers.iterator();

    BluetoothLeScannerCompat.ScanCallbackWrapper wrapper;
    while(var2.hasNext()) {
        wrapper = (BluetoothLeScannerCompat.ScanCallbackWrapper)var2.next();
        if (wrapper.scanCallback instanceof UserScanCallbackWrapper) {
            UserScanCallbackWrapper callbackWrapper =
(UserScanCallbackWrapper)wrapper.scanCallback;
            if (callbackWrapper.isDead()) {
                deadWrappers.add(wrapper);
            }
        }
    }

    var2 = deadWrappers.iterator();

    while(var2.hasNext()) {
        wrapper = (BluetoothLeScannerCompat.ScanCallbackWrapper)var2.next();
        this.wrappers.remove(wrapper);
    }
}
}

```

```

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\no\nordicsemi\android\support\v18\scanner
// File: ScanFilter.java

```

```

package no.nordicsemi.android.support.v18.scanner;

```

```

import android.bluetooth.BluetoothAdapter;
import android.bluetooth.BluetoothDevice;
import android.os.Parcel;
import android.os.ParcelUuid;
import android.os.Parcelable;
import android.os.Parcelable.Creator;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import java.util.Arrays;
import java.util.Iterator;
import java.util.List;

```

```

import java.util.UUID;

public final class ScanFilter implements Parcelable {
    @Nullable
    private final String deviceName;
    @Nullable
    private final String deviceAddress;
    @Nullable
    private final ParcelUuid serviceUuid;
    @Nullable
    private final ParcelUuid serviceUuidMask;
    @Nullable
    private final ParcelUuid serviceDataUuid;
    @Nullable
    private final byte[] serviceData;
    @Nullable
    private final byte[] serviceDataMask;
    private final int manufacturerId;
    @Nullable
    private final byte[] manufacturerData;
    @Nullable
    private final byte[] manufacturerDataMask;
    private static final ScanFilter EMPTY = (new ScanFilter.Builder()).build();
    public static final Creator<ScanFilter> CREATOR = new Creator<ScanFilter>() {
        public ScanFilter[] newArray(int size) {
            return new ScanFilter[size];
        }
    }

    public ScanFilter createFromParcel(Parcel in) {
        ScanFilter.Builder builder = new ScanFilter.Builder();
        if (in.readInt() == 1) {
            builder.setDeviceName(in.readString());
        }

        if (in.readInt() == 1) {
            builder.setDeviceAddress(in.readString());
        }

        ParcelUuid serviceDataUuid;
        if (in.readInt() == 1) {
            serviceDataUuid =
(ParcelUuid)in.readParcelable(ParcelUuid.class.getClassLoader());
            builder.setServiceUuid(serviceDataUuid);
            if (in.readInt() == 1) {
                ParcelUuid uuidMask =
(ParcelUuid)in.readParcelable(ParcelUuid.class.getClassLoader());
                builder.setServiceUuid(serviceDataUuid, uuidMask);
            }
        }

        byte[] manufacturerData;
        int manufacturerDataMaskLength;
        byte[] manufacturerDataMask;
        int manufacturerDataLength;
    }

```

```

        if (in.readInt() == 1) {
            serviceDataUuid =
(ParcelUuid)in.readParcelable(ParcelUuid.class.getClassLoader());
            if (in.readInt() == 1) {
                manufacturerDataLength = in.readInt();
                manufacturerData = new byte[manufacturerDataLength];
                in.readByteArray(manufacturerData);
                if (in.readInt() == 0) {
                    builder.setServiceData(serviceDataUuid, manufacturerData);
                } else {
                    manufacturerDataMaskLength = in.readInt();
                    manufacturerDataMask = new byte[manufacturerDataMaskLength];
                    in.readByteArray(manufacturerDataMask);
                    builder.setServiceData(serviceDataUuid, manufacturerData,
manufacturerDataMask);
                }
            }
        }

        int manufacturerId = in.readInt();
        if (in.readInt() == 1) {
            manufacturerDataLength = in.readInt();
            manufacturerData = new byte[manufacturerDataLength];
            in.readByteArray(manufacturerData);
            if (in.readInt() == 0) {
                builder.setManufacturerData(manufacturerId, manufacturerData);
            } else {
                manufacturerDataMaskLength = in.readInt();
                manufacturerDataMask = new byte[manufacturerDataMaskLength];
                in.readByteArray(manufacturerDataMask);
                builder.setManufacturerData(manufacturerId, manufacturerData,
manufacturerDataMask);
            }
        }

        return builder.build();
    }
};

```

```

private ScanFilter(@Nullable String name, @Nullable String deviceAddress,
@Nullable ParcelUuid uuid, @Nullable ParcelUuid uuidMask, @Nullable ParcelUuid
serviceDataUuid, @Nullable byte[] serviceData, @Nullable byte[] serviceDataMask,
int manufacturerId, @Nullable byte[] manufacturerData, @Nullable byte[]
manufacturerDataMask) {
    this.deviceName = name;
    this.serviceUuid = uuid;
    this.serviceUuidMask = uuidMask;
    this.deviceAddress = deviceAddress;
    this.serviceDataUuid = serviceDataUuid;
    this.serviceData = serviceData;
    this.serviceDataMask = serviceDataMask;
    this.manufacturerId = manufacturerId;
    this.manufacturerData = manufacturerData;
    this.manufacturerDataMask = manufacturerDataMask;
}

```



```

}

public int describeContents() {
    return 0;
}

public void writeToParcel(Parcel dest, int flags) {
    dest.writeInt(this.deviceName == null ? 0 : 1);
    if (this.deviceName != null) {
        dest.writeString(this.deviceName);
    }

    dest.writeInt(this.deviceAddress == null ? 0 : 1);
    if (this.deviceAddress != null) {
        dest.writeString(this.deviceAddress);
    }

    dest.writeInt(this.serviceUuid == null ? 0 : 1);
    if (this.serviceUuid != null) {
        dest.writeParcelable(this.serviceUuid, flags);
        dest.writeInt(this.serviceUuidMask == null ? 0 : 1);
        if (this.serviceUuidMask != null) {
            dest.writeParcelable(this.serviceUuidMask, flags);
        }
    }

    dest.writeInt(this.serviceDataUuid == null ? 0 : 1);
    if (this.serviceDataUuid != null) {
        dest.writeParcelable(this.serviceDataUuid, flags);
        dest.writeInt(this.serviceData == null ? 0 : 1);
        if (this.serviceData != null) {
            dest.writeInt(this.serviceData.length);
            dest.writeByteArray(this.serviceData);
            dest.writeInt(this.serviceDataMask == null ? 0 : 1);
            if (this.serviceDataMask != null) {
                dest.writeInt(this.serviceDataMask.length);
                dest.writeByteArray(this.serviceDataMask);
            }
        }
    }

    dest.writeInt(this.manufacturerId);
    dest.writeInt(this.manufacturerData == null ? 0 : 1);
    if (this.manufacturerData != null) {
        dest.writeInt(this.manufacturerData.length);
        dest.writeByteArray(this.manufacturerData);
        dest.writeInt(this.manufacturerDataMask == null ? 0 : 1);
        if (this.manufacturerDataMask != null) {
            dest.writeInt(this.manufacturerDataMask.length);
            dest.writeByteArray(this.manufacturerDataMask);
        }
    }
}
}

```

```
@Nullable
public String getDeviceName() {
    return this.deviceName;
}

@Nullable
public ParcelUuid getServiceUuid() {
    return this.serviceUuid;
}

@Nullable
public ParcelUuid getServiceUuidMask() {
    return this.serviceUuidMask;
}

@Nullable
public String getDeviceAddress() {
    return this.deviceAddress;
}

@Nullable
public byte[] getServiceData() {
    return this.serviceData;
}

@Nullable
public byte[] getServiceDataMask() {
    return this.serviceDataMask;
}

@Nullable
public ParcelUuid getServiceDataUuid() {
    return this.serviceDataUuid;
}

public int getManufacturerId() {
    return this.manufacturerId;
}

@Nullable
public byte[] getManufacturerData() {
    return this.manufacturerData;
}

@Nullable
public byte[] getManufacturerDataMask() {
    return this.manufacturerDataMask;
}

public boolean matches(@Nullable ScanResult scanResult) {
    if (scanResult == null) {
        return false;
    } else {
```

```

        BluetoothDevice device = scanResult.getDevice();
        if (this.deviceAddress != null &&
!this.deviceAddress.equals(device.getAddress())) {
            return false;
        } else {
            ScanRecord scanRecord = scanResult.getScanRecord();
            if (scanRecord != null || this.deviceName == null &&
this.serviceUuid == null && this.manufacturerData == null && this.serviceData ==
null) {
                if (this.deviceName != null &&
!this.deviceName.equals(scanRecord.getDeviceName())) {
                    return false;
                } else if (this.serviceUuid != null &&
!matchesServiceUuids(this.serviceUuid, this.serviceUuidMask,
scanRecord.getServiceUuids())) {
                    return false;
                } else if (this.serviceDataUuid != null && scanRecord != null &&
!this.matchesPartialData(this.serviceData, this.serviceDataMask,
scanRecord.getServiceData(this.serviceDataUuid))) {
                    return false;
                } else {
                    return this.manufacturerId < 0 || scanRecord == null ||
this.matchesPartialData(this.manufacturerData, this.manufacturerDataMask,
scanRecord.getManufacturerSpecificData(this.manufacturerId));
                }
            } else {
                return false;
            }
        }
    }
}

```

```

private static boolean matchesServiceUuids(@Nullable ParcelUuid uuid,
@Nullable ParcelUuid parcelUuidMask, @Nullable List<ParcelUuid> uuids) {
    if (uuid == null) {
        return true;
    } else if (uuids == null) {
        return false;
    } else {
        Iterator var3 = uuids.iterator();

        ParcelUuid parcelUuid;
        UUID uuidMask;
        do {
            if (!var3.hasNext()) {
                return false;
            }

            parcelUuid = (ParcelUuid)var3.next();
            uuidMask = parcelUuidMask == null ? null : parcelUuidMask.getUuid();
        } while(!matchesServiceUuid(uuid.getUuid(), uuidMask,
parcelUuid.getUuid()));

        return true;
    }
}

```

```

    }
}

private static boolean matchesServiceUuid(@NonNull UUID uuid, @Nullable UUID
mask, @NonNull UUID data) {
    if (mask == null) {
        return uuid.equals(data);
    } else if ((uuid.getLeastSignificantBits() &
mask.getLeastSignificantBits()) != (data.getLeastSignificantBits() &
mask.getLeastSignificantBits())) {
        return false;
    } else {
        return (uuid.getMostSignificantBits() & mask.getMostSignificantBits())
== (data.getMostSignificantBits() & mask.getMostSignificantBits());
    }
}

private boolean matchesPartialData(@Nullable byte[] data, @Nullable byte[]
dataMask, @Nullable byte[] parsedData) {
    if (data == null) {
        return parsedData != null;
    } else if (parsedData != null && parsedData.length >= data.length) {
        int i;
        if (dataMask == null) {
            for(i = 0; i < data.length; ++i) {
                if (parsedData[i] != data[i]) {
                    return false;
                }
            }

            return true;
        } else {
            for(i = 0; i < data.length; ++i) {
                if ((dataMask[i] & parsedData[i]) != (dataMask[i] & data[i])) {
                    return false;
                }
            }

            return true;
        }
    } else {
        return false;
    }
}

public String toString() {
    return "BluetoothLeScanFilter [deviceName=" + this.deviceName + ",
deviceAddress=" + this.deviceAddress + ", mUuid=" + this.serviceUuid + ",
uuidMask=" + this.serviceUuidMask + ", serviceDataUuid=" +
Objects.toString(this.serviceDataUuid) + ", serviceData=" +
Arrays.toString(this.serviceData) + ", serviceDataMask=" +
Arrays.toString(this.serviceDataMask) + ", manufacturerId=" +
this.manufacturerId + ", manufacturerData=" +
Arrays.toString(this.manufacturerData) + ", manufacturerDataMask=" +

```

```

Arrays.toString(this.manufacturerDataMask) + "]]";
    }

    public int hashCode() {
        return Objects.hash(this.deviceName, this.deviceAddress,
this.manufacturerId, Arrays.hashCode(this.manufacturerData),
Arrays.hashCode(this.manufacturerDataMask), this.serviceDataUuid,
Arrays.hashCode(this.serviceData), Arrays.hashCode(this.serviceDataMask),
this.serviceUuid, this.serviceUuidMask);
    }

    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        } else if (obj != null && this.getClass() == obj.getClass()) {
            ScanFilter other = (ScanFilter)obj;
            return Objects.equals(this.deviceName, other.deviceName) &&
Objects.equals(this.deviceAddress, other.deviceAddress) && this.manufacturerId
== other.manufacturerId && Objects.deepEquals(this.manufacturerData,
other.manufacturerData) && Objects.deepEquals(this.manufacturerDataMask,
other.manufacturerDataMask) && Objects.equals(this.serviceDataUuid,
other.serviceDataUuid) && Objects.deepEquals(this.serviceData,
other.serviceData) && Objects.deepEquals(this.serviceDataMask,
other.serviceDataMask) && Objects.equals(this.serviceUuid, other.serviceUuid) &&
Objects.equals(this.serviceUuidMask, other.serviceUuidMask);
        } else {
            return false;
        }
    }

    boolean isEmpty() {
        return EMPTY.equals(this);
    }

    // $FF: synthetic method
    ScanFilter(String x0, String x1, ParcelUuid x2, ParcelUuid x3, ParcelUuid x4,
byte[] x5, byte[] x6, int x7, byte[] x8, byte[] x9, Object x10) {
        this(x0, x1, x2, x3, x4, x5, x6, x7, x8, x9);
    }

    public static final class Builder {
        private String deviceName;
        private String deviceAddress;
        private ParcelUuid serviceUuid;
        private ParcelUuid uuidMask;
        private ParcelUuid serviceDataUuid;
        private byte[] serviceData;
        private byte[] serviceDataMask;
        private int manufacturerId = -1;
        private byte[] manufacturerData;
        private byte[] manufacturerDataMask;

        public ScanFilter.Builder setDeviceName(@Nullable String deviceName) {
            this.deviceName = deviceName;
        }
    }

```

```

        return this;
    }

    public ScanFilter.Builder setDeviceAddress(@Nullable String deviceAddress)
    {
        if (deviceAddress != null &&
!BluetoothAdapter.checkBluetoothAddress(deviceAddress)) {
            throw new IllegalArgumentException("invalid device address " +
deviceAddress);
        } else {
            this.deviceAddress = deviceAddress;
            return this;
        }
    }

    public ScanFilter.Builder setServiceUuid(@Nullable ParcelUuid serviceUuid)
    {
        this.serviceUuid = serviceUuid;
        this.uuidMask = null;
        return this;
    }

    public ScanFilter.Builder setServiceUuid(@Nullable ParcelUuid serviceUuid,
@Nullable ParcelUuid uuidMask) {
        if (uuidMask != null && serviceUuid == null) {
            throw new IllegalArgumentException("uuid is null while uuidMask is
not null!");
        } else {
            this.serviceUuid = serviceUuid;
            this.uuidMask = uuidMask;
            return this;
        }
    }

    public ScanFilter.Builder setServiceData(@NonNull ParcelUuid
serviceDataUuid, @Nullable byte[] serviceData) {
        if (serviceDataUuid == null) {
            throw new IllegalArgumentException("serviceDataUuid is null!");
        } else {
            this.serviceDataUuid = serviceDataUuid;
            this.serviceData = serviceData;
            this.serviceDataMask = null;
            return this;
        }
    }

    public ScanFilter.Builder setServiceData(@NonNull ParcelUuid
serviceDataUuid, @Nullable byte[] serviceData, @Nullable byte[] serviceDataMask)
    {
        if (serviceDataUuid == null) {
            throw new IllegalArgumentException("serviceDataUuid is null");
        } else {
            if (serviceDataMask != null) {
                if (serviceData == null) {

```

```

        throw new IllegalArgumentException("serviceData is null while
serviceDataMask is not null");
    }

    if (serviceData.length != serviceDataMask.length) {
        throw new IllegalArgumentException("size mismatch for service
data and service data mask");
    }
}

this.serviceDataUuid = serviceDataUuid;
this.serviceData = serviceData;
this.serviceDataMask = serviceDataMask;
return this;
}
}

public ScanFilter.Builder setManufacturerData(int manufacturerId,
@Nullable byte[] manufacturerData) {
    if (manufacturerData != null && manufacturerId < 0) {
        throw new IllegalArgumentException("invalid manufacture id");
    } else {
        this.manufacturerId = manufacturerId;
        this.manufacturerData = manufacturerData;
        this.manufacturerDataMask = null;
        return this;
    }
}

public ScanFilter.Builder setManufacturerData(int manufacturerId,
@Nullable byte[] manufacturerData, @Nullable byte[] manufacturerDataMask) {
    if (manufacturerData != null && manufacturerId < 0) {
        throw new IllegalArgumentException("invalid manufacture id");
    } else {
        if (manufacturerDataMask != null) {
            if (manufacturerData == null) {
                throw new IllegalArgumentException("manufacturerData is null
while manufacturerDataMask is not null");
            }

            if (manufacturerData.length != manufacturerDataMask.length) {
                throw new IllegalArgumentException("size mismatch for
manufacturerData and manufacturerDataMask");
            }
        }

        this.manufacturerId = manufacturerId;
        this.manufacturerData = manufacturerData;
        this.manufacturerDataMask = manufacturerDataMask;
        return this;
    }
}

public ScanFilter build() {

```

```

        return new ScanFilter(this.deviceName, this.deviceAddress,
this.serviceUuid, this.uuidMask, this.serviceDataUuid, this.serviceData,
this.serviceDataMask, this.manufacturerId, this.manufacturerData,
this.manufacturerDataMask);
    }
}
}

```

```

// Directory: app
// Subdirectory:
classes.jar_Decompiler.com\no\nordicsemi\android\support\v18\scanner
// File: ScannerService.java

```

```

package no.nordicsemi.android.support.v18.scanner;

import android.app.PendingIntent;
import android.app.Service;
import android.content.Intent;
import android.os.Handler;
import android.os.IBinder;
import android.util.Log;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.annotation.RequiresPermission;
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;

public class ScannerService extends Service {
    private static final String TAG = "ScannerService";
    static final String EXTRA_PENDING_INTENT =
"no.nordicsemi.android.support.v18.EXTRA_PENDING_INTENT";
    static final String EXTRA_REQUEST_CODE =
"no.nordicsemi.android.support.v18.REQUEST_CODE";
    static final String EXTRA_FILTERS =
"no.nordicsemi.android.support.v18.EXTRA_FILTERS";
    static final String EXTRA_SETTINGS =
"no.nordicsemi.android.support.v18.EXTRA_SETTINGS";
    static final String EXTRA_START =
"no.nordicsemi.android.support.v18.EXTRA_START";
    @NonNull
    private final Object LOCK = new Object();
    private HashMap<Integer, ScanCallback> callbacks;
    private Handler handler;

    public void onCreate() {
        super.onCreate();
        this.callbacks = new HashMap();
        this.handler = new Handler();
    }
}

```



```

    @RequiresPermission(
        allOf = {"android.permission.BLUETOOTH_ADMIN",
"android.permission.BLUETOOTH"}
    )
    public int onStartCommand(Intent intent, int flags, int startId) {
        if (intent != null) {
            PendingIntent callbackIntent =
(PendingIntent)intent.getParcelableExtra("no.nordicsemi.android.support.v18.EXTRA
A_PENDING_INTENT");
            int requestCode =
intent.getIntExtra("no.nordicsemi.android.support.v18.REQUEST_CODE", 0);
            boolean start =
intent.getBooleanExtra("no.nordicsemi.android.support.v18.EXTRA_START", false);
            boolean stop = !start;
            boolean knownCallback;
            if (callbackIntent == null) {
                synchronized(this.LOCK) {
                    knownCallback = this.callbacks.isEmpty();
                }

                if (knownCallback) {
                    this.stopSelf();
                }

                return 2;
            }

            synchronized(this.LOCK) {
                knownCallback = this.callbacks.containsKey(requestCode);
            }

            if (start && !knownCallback) {
                ArrayList<ScanFilter> filters =
intent.getParcelableArrayListExtra("no.nordicsemi.android.support.v18.EXTRA_FILT
ERS");
                ScanSettings settings =
(ScanSettings)intent.getParcelableExtra("no.nordicsemi.android.support.v18.EXTRA
_SETTINGS");
                this.startScan((List)(filters != null ? filters :
Collections.emptyList()), settings != null ? settings : (new
ScanSettings.Builder()).build(), callbackIntent, requestCode);
            } else if (stop && knownCallback) {
                this.stopScan(requestCode);
            }
        }

        return 2;
    }

    @Nullable
    public IBinder onBind(Intent intent) {
        return null;
    }

```

```

    public void onTaskRemoved(Intent rootIntent) {
        super.onTaskRemoved(rootIntent);
    }

    @RequiresPermission(
        allOf = {"android.permission.BLUETOOTH_ADMIN",
"android.permission.BLUETOOTH"}
    )
    public void onDestroy() {
        BluetoothLeScannerCompat scannerCompat =
BluetoothLeScannerCompat.getScanner();
        Iterator var2 = this.callbacks.values().iterator();

        while(var2.hasNext()) {
            ScanCallback callback = (ScanCallback)var2.next();

            try {
                scannerCompat.stopScan(callback);
            } catch (Exception var5) {
            }
        }

        this.callbacks.clear();
        this.callbacks = null;
        this.handler = null;
        super.onDestroy();
    }

    @RequiresPermission(
        allOf = {"android.permission.BLUETOOTH_ADMIN",
"android.permission.BLUETOOTH"}
    )
    private void startScan(@NonNull List<ScanFilter> filters, @NonNull
ScanSettings settings, @NonNull PendingIntent callbackIntent, int requestCode) {
        PendingIntentExecutor executor = new PendingIntentExecutor(callbackIntent,
settings, this);
        synchronized(this.LOCK) {
            this.callbacks.put(requestCode, executor);
        }

        try {
            BluetoothLeScannerCompat scannerCompat =
BluetoothLeScannerCompat.getScanner();
            scannerCompat.startScanInternal(filters, settings, executor,
this.handler);
        } catch (Exception var8) {
            Log.w("ScannerService", "Starting scanning failed", var8);
        }
    }

    @RequiresPermission(
        allOf = {"android.permission.BLUETOOTH_ADMIN",
"android.permission.BLUETOOTH"}
    )

```

```

    )
    private void stopScan(int requestCode) {
        ScanCallback callback;
        boolean shouldStop;
        synchronized(this.LOCK) {
            callback = (ScanCallback)this.callbacks.remove(requestCode);
            shouldStop = this.callbacks.isEmpty();
        }

        if (callback != null) {
            try {
                BluetoothLeScannerCompat scannerCompat =
BluetoothLeScannerCompat.getScanner();
                scannerCompat.stopScan(callback);
            } catch (Exception var6) {
                Log.w("ScannerService", "Stopping scanning failed", var6);
            }

            if (shouldStop) {
                this.stopSelf();
            }
        }
    }
}

```

```

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\no\nordicsemi\android\support\v18\scanner
// File: ScanRecord.java

```

```

package no.nordicsemi.android.support.v18.scanner;

import android.os.ParcelUuid;
import android.util.Log;
import android.util.SparseArray;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public final class ScanRecord {
    private static final String TAG = "ScanRecord";
    private static final int DATA_TYPE_FLAGS = 1;
    private static final int DATA_TYPE_SERVICE_UUIDS_16_BIT_PARTIAL = 2;
    private static final int DATA_TYPE_SERVICE_UUIDS_16_BIT_COMPLETE = 3;
    private static final int DATA_TYPE_SERVICE_UUIDS_32_BIT_PARTIAL = 4;
    private static final int DATA_TYPE_SERVICE_UUIDS_32_BIT_COMPLETE = 5;
    private static final int DATA_TYPE_SERVICE_UUIDS_128_BIT_PARTIAL = 6;
    private static final int DATA_TYPE_SERVICE_UUIDS_128_BIT_COMPLETE = 7;
}

```

```

private static final int DATA_TYPE_LOCAL_NAME_SHORT = 8;
private static final int DATA_TYPE_LOCAL_NAME_COMPLETE = 9;
private static final int DATA_TYPE_TX_POWER_LEVEL = 10;
private static final int DATA_TYPE_SERVICE_DATA_16_BIT = 22;
private static final int DATA_TYPE_SERVICE_DATA_32_BIT = 32;
private static final int DATA_TYPE_SERVICE_DATA_128_BIT = 33;
private static final int DATA_TYPE_MANUFACTURER_SPECIFIC_DATA = 255;
private final int advertiseFlags;
@Nullable
private final List<ParcelUuid> serviceUuids;
@Nullable
private final SparseArray<byte[]> manufacturerSpecificData;
@Nullable
private final Map<ParcelUuid, byte[]> serviceData;
private final int txPowerLevel;
private final String deviceName;
private final byte[] bytes;

public int getAdvertiseFlags() {
    return this.advertiseFlags;
}

@Nullable
public List<ParcelUuid> getServiceUuids() {
    return this.serviceUuids;
}

@Nullable
public SparseArray<byte[]> getManufacturerSpecificData() {
    return this.manufacturerSpecificData;
}

@Nullable
public byte[] getManufacturerSpecificData(int manufacturerId) {
    return this.manufacturerSpecificData == null ? null :
(byte[])this.manufacturerSpecificData.get(manufacturerId);
}

@Nullable
public Map<ParcelUuid, byte[]> getServiceData() {
    return this.serviceData;
}

@Nullable
public byte[] getServiceData(@NonNull ParcelUuid serviceDataUuid) {
    return serviceDataUuid != null && this.serviceData != null ?
(byte[])this.serviceData.get(serviceDataUuid) : null;
}

public int getTxPowerLevel() {
    return this.txPowerLevel;
}

@Nullable

```

```

public String getDeviceName() {
    return this.deviceName;
}

@Nullable
public byte[] getBytes() {
    return this.bytes;
}

private ScanRecord(@Nullable List<ParcelUuid> serviceUids, @Nullable
SparseArray<byte[]> manufacturerData, @Nullable Map<ParcelUuid, byte[]>
serviceData, int advertiseFlags, int txPowerLevel, String localName, byte[]
bytes) {
    this.serviceUids = serviceUids;
    this.manufacturerSpecificData = manufacturerData;
    this.serviceData = serviceData;
    this.deviceName = localName;
    this.advertiseFlags = advertiseFlags;
    this.txPowerLevel = txPowerLevel;
    this.bytes = bytes;
}

@Nullable
static ScanRecord parseFromBytes(@Nullable byte[] scanRecord) {
    if (scanRecord == null) {
        return null;
    } else {
        int currentPos = 0;
        int advertiseFlag = -1;
        int txPowerLevel = Integer.MIN_VALUE;
        String localName = null;
        List<ParcelUuid> serviceUids = null;
        SparseArray<byte[]> manufacturerData = null;
        HashMap serviceData = null;

        try {
            int dataLength;
            for(; currentPos < scanRecord.length; currentPos += dataLength) {
                int length = scanRecord[currentPos++] & 255;
                if (length == 0) {
                    break;
                }

                dataLength = length - 1;
                int fieldType = scanRecord[currentPos++] & 255;
                switch(fieldType) {
                    case 1:
                        advertiseFlag = scanRecord[currentPos] & 255;
                        break;
                    case 2:
                    case 3:
                        if (serviceUids == null) {
                            serviceUids = new ArrayList();
                        }
                }
            }
        }
    }
}

```

```

        parseServiceUuid(scanRecord, currentPos, dataLength, 2,
serviceUids);
        break;
    case 4:
    case 5:
        if (serviceUids == null) {
            serviceUids = new ArrayList();
        }

        parseServiceUuid(scanRecord, currentPos, dataLength, 4,
serviceUids);
        break;
    case 6:
    case 7:
        if (serviceUids == null) {
            serviceUids = new ArrayList();
        }

        parseServiceUuid(scanRecord, currentPos, dataLength, 16,
serviceUids);
        break;
    case 8:
    case 9:
        localName = new String(extractBytes(scanRecord, currentPos,
dataLength));
        break;
    case 10:
        txPowerLevel = scanRecord[currentPos];
        break;
    case 22:
    case 32:
    case 33:
        int serviceUuidLength = 2;
        if (fieldType == 32) {
            serviceUuidLength = 4;
        } else if (fieldType == 33) {
            serviceUuidLength = 16;
        }

        byte[] serviceDataUuidBytes = extractBytes(scanRecord,
currentPos, serviceUuidLength);
        ParcelUuid serviceDataUuid =
BluetoothUuid.parseUuidFrom(serviceDataUuidBytes);
        byte[] serviceDataArray = extractBytes(scanRecord, currentPos
+ serviceUuidLength, dataLength - serviceUuidLength);
        if (serviceData == null) {
            serviceData = new HashMap();
        }

        serviceData.put(serviceDataUuid, serviceDataArray);
        break;
    case 255:
        int manufacturerId = ((scanRecord[currentPos + 1] & 255) << 8)

```

```

+ (scanRecord[currentPos] & 255);
        byte[] manufacturerDataBytes = extractBytes(scanRecord,
currentPos + 2, dataLength - 2);
        if (manufacturerData == null) {
            manufacturerData = new SparseArray();
        }

        manufacturerData.put(manufacturerId, manufacturerDataBytes);
    }
}

return new ScanRecord(serviceUids, manufacturerData, serviceData,
advertiseFlag, txPowerLevel, localName, scanRecord);
    } catch (Exception var17) {
        Log.e("ScanRecord", "unable to parse scan record: " +
Arrays.toString(scanRecord));
        return new ScanRecord((List)null, (SparseArray)null, (Map)null, -1,
Integer.MIN_VALUE, (String)null, scanRecord);
    }
}
}

public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    } else if (obj != null && this.getClass() == obj.getClass()) {
        ScanRecord other = (ScanRecord)obj;
        return Arrays.equals(this.bytes, other.bytes);
    } else {
        return false;
    }
}

public String toString() {
    return "ScanRecord [advertiseFlags=" + this.advertiseFlags + ",
serviceUids=" + this.serviceUids + ", manufacturerSpecificData=" +
BluetoothLeUtils.toString(this.manufacturerSpecificData) + ", serviceData=" +
BluetoothLeUtils.toString(this.serviceData) + ", txPowerLevel=" +
this.txPowerLevel + ", deviceName=" + this.deviceName + "]";
}

private static int parseServiceUuid(@NonNull byte[] scanRecord, int
currentPos, int dataLength, int uuidLength, @NonNull List<ParcelUuid>
serviceUids) {
    while(dataLength > 0) {
        byte[] uuidBytes = extractBytes(scanRecord, currentPos, uuidLength);
        serviceUids.add(BluetoothUuid.parseUuidFrom(uuidBytes));
        dataLength -= uuidLength;
        currentPos += uuidLength;
    }

    return currentPos;
}

```

```

        private static byte[] extractBytes(@NonNull byte[] scanRecord, int start, int
length) {
            byte[] bytes = new byte[length];
            System.arraycopy(scanRecord, start, bytes, 0, length);
            return bytes;
        }
    }
}

```

```

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\no\nordicsemi\android\support\v18\scanner
// File: ScanResult.java

```

```

package no.nordicsemi.android.support.v18.scanner;

```

```

import android.bluetooth.BluetoothDevice;
import android.os.Parcel;
import android.os.Parcelable;
import android.os.Parcelable.Creator;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;

```

```

public final class ScanResult implements Parcelable {
    public static final int DATA_COMPLETE = 0;
    public static final int DATA_TRUNCATED = 2;
    public static final int PHY_UNUSED = 0;
    public static final int SID_NOT_PRESENT = 255;
    public static final int TX_POWER_NOT_PRESENT = 127;
    public static final int PERIODIC_INTERVAL_NOT_PRESENT = 0;
    static final int ET_LEGACY_MASK = 16;
    static final int ET_CONNECTABLE_MASK = 1;
    @NonNull
    private final BluetoothDevice device;
    @Nullable
    private ScanRecord scanRecord;
    private final int rssi;
    private final long timestampNanos;
    private final int eventType;
    private final int primaryPhy;
    private final int secondaryPhy;
    private final int advertisingSid;
    private final int txPower;
    private final int periodicAdvertisingInterval;
    public static final Creator<ScanResult> CREATOR = new Creator<ScanResult>() {
        public ScanResult createFromParcel(Parcel source) {
            return new ScanResult(source);
        }
    }

    public ScanResult[] newArray(int size) {
        return new ScanResult[size];
    }
};

```



```

    /** @deprecated */
    public ScanResult(@NonNull BluetoothDevice device, @Nullable ScanRecord
scanRecord, int rssi, long timestampNanos) {
        this.device = device;
        this.scanRecord = scanRecord;
        this.rssi = rssi;
        this.timestampNanos = timestampNanos;
        this.eventType = 17;
        this.primaryPhy = 1;
        this.secondaryPhy = 0;
        this.advertisingSid = 255;
        this.txPower = 127;
        this.periodicAdvertisingInterval = 0;
    }

    public ScanResult(@NonNull BluetoothDevice device, int eventType, int
primaryPhy, int secondaryPhy, int advertisingSid, int txPower, int rssi, int
periodicAdvertisingInterval, @Nullable ScanRecord scanRecord, long
timestampNanos) {
        this.device = device;
        this.eventType = eventType;
        this.primaryPhy = primaryPhy;
        this.secondaryPhy = secondaryPhy;
        this.advertisingSid = advertisingSid;
        this.txPower = txPower;
        this.rssi = rssi;
        this.periodicAdvertisingInterval = periodicAdvertisingInterval;
        this.scanRecord = scanRecord;
        this.timestampNanos = timestampNanos;
    }

    private ScanResult(Parcel in) {
        this.device =
(BluetoothDevice)BluetoothDevice.CREATOR.createFromParcel(in);
        if (in.readInt() == 1) {
            this.scanRecord = ScanRecord.parseFromBytes(in.createByteArray());
        }

        this.rssi = in.readInt();
        this.timestampNanos = in.readLong();
        this.eventType = in.readInt();
        this.primaryPhy = in.readInt();
        this.secondaryPhy = in.readInt();
        this.advertisingSid = in.readInt();
        this.txPower = in.readInt();
        this.periodicAdvertisingInterval = in.readInt();
    }

    public void writeToParcel(Parcel dest, int flags) {
        this.device.writeToParcel(dest, flags);
        if (this.scanRecord != null) {
            dest.writeInt(1);
            dest.writeByteArray(this.scanRecord.getBytes());
        } else {

```

```

        dest.writeInt(0);
    }

    dest.writeInt(this.rssi);
    dest.writeLong(this.timestampNanos);
    dest.writeInt(this.eventType);
    dest.writeInt(this.primaryPhy);
    dest.writeInt(this.secondaryPhy);
    dest.writeInt(this.advertisingSid);
    dest.writeInt(this.txPower);
    dest.writeInt(this.periodicAdvertisingInterval);
}

public int describeContents() {
    return 0;
}

@NonNull
public BluetoothDevice getDevice() {
    return this.device;
}

@Nullable
public ScanRecord getScanRecord() {
    return this.scanRecord;
}

public int getRssi() {
    return this.rssi;
}

public long getTimestampNanos() {
    return this.timestampNanos;
}

public boolean isLegacy() {
    return (this.eventType & 16) != 0;
}

public boolean isConnectable() {
    return (this.eventType & 1) != 0;
}

public int getDataStatus() {
    return this.eventType >> 5 & 3;
}

public int getPrimaryPhy() {
    return this.primaryPhy;
}

public int getSecondaryPhy() {
    return this.secondaryPhy;
}

```

```

    public int getAdvertisingSid() {
        return this.advertisingSid;
    }

    public int getTxPower() {
        return this.txPower;
    }

    public int getPeriodicAdvertisingInterval() {
        return this.periodicAdvertisingInterval;
    }

    public int hashCode() {
        return Objects.hash(this.device, this.rssi, this.scanRecord,
this.timestampNanos, this.eventType, this.primaryPhy, this.secondaryPhy,
this.advertisingSid, this.txPower, this.periodicAdvertisingInterval);
    }

    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        } else if (obj != null && this.getClass() == obj.getClass()) {
            ScanResult other = (ScanResult)obj;
            return Objects.equals(this.device, other.device) && this.rssi ==
other.rssi && Objects.equals(this.scanRecord, other.scanRecord) &&
this.timestampNanos == other.timestampNanos && this.eventType == other.eventType
&& this.primaryPhy == other.primaryPhy && this.secondaryPhy ==
other.secondaryPhy && this.advertisingSid == other.advertisingSid &&
this.txPower == other.txPower && this.periodicAdvertisingInterval ==
other.periodicAdvertisingInterval;
        } else {
            return false;
        }
    }

    public String toString() {
        return "ScanResult{device=" + this.device + ", scanRecord=" +
Objects.toString(this.scanRecord) + ", rssi=" + this.rssi + ", timestampNanos="
+ this.timestampNanos + ", eventType=" + this.eventType + ", primaryPhy=" +
this.primaryPhy + ", secondaryPhy=" + this.secondaryPhy + ", advertisingSid=" +
this.advertisingSid + ", txPower=" + this.txPower + ",
periodicAdvertisingInterval=" + this.periodicAdvertisingInterval + '}';
    }

    // $FF: synthetic method
    ScanResult(Parcel x0, Object x1) {
        this(x0);
    }
}

// Directory: app
// Subdirectory:

```

```
classes.jar_Decompile.com\no\nordicsemi\android\support\v18\scanner  
// File: ScanSettings.java
```

```
package no.nordicsemi.android.support.v18.scanner;  
  
import android.os.Parcel;  
import android.os.Parcelable;  
import android.os.Parcelable.Creator;  
import androidx.annotation.NonNull;  
  
public final class ScanSettings implements Parcelable {  
    public static final long MATCH_LOST_DEVICE_TIMEOUT_DEFAULT = 10000L;  
    public static final long MATCH_LOST_TASK_INTERVAL_DEFAULT = 10000L;  
    public static final int SCAN_MODE_OPPORTUNISTIC = -1;  
    public static final int SCAN_MODE_LOW_POWER = 0;  
    public static final int SCAN_MODE_BALANCED = 1;  
    public static final int SCAN_MODE_LOW_LATENCY = 2;  
    public static final int CALLBACK_TYPE_ALL_MATCHES = 1;  
    public static final int CALLBACK_TYPE_FIRST_MATCH = 2;  
    public static final int CALLBACK_TYPE_MATCH_LOST = 4;  
    public static final int MATCH_NUM_ONE_ADVERTISEMENT = 1;  
    public static final int MATCH_NUM_FEW_ADVERTISEMENT = 2;  
    public static final int MATCH_NUM_MAX_ADVERTISEMENT = 3;  
    public static final int MATCH_MODE_AGGRESSIVE = 1;  
    public static final int MATCH_MODE_STICKY = 2;  
    public static final int PHY_LE_ALL_SUPPORTED = 255;  
    private final long powerSaveScanInterval;  
    private final long powerSaveRestInterval;  
    private final int scanMode;  
    private final int callbackType;  
    private final long reportDelayMillis;  
    private final int matchMode;  
    private final int numOfMatchesPerFilter;  
    private final boolean useHardwareFilteringIfSupported;  
    private final boolean useHardwareBatchingIfSupported;  
    private boolean useHardwareCallbackTypesIfSupported;  
    private final long matchLostDeviceTimeout;  
    private final long matchLostTaskInterval;  
    private final boolean legacy;  
    private final int phy;  
    public static final Creator<ScanSettings> CREATOR = new  
    Creator<ScanSettings>() {  
        public ScanSettings[] newArray(int size) {  
            return new ScanSettings[size];  
        }  
  
        public ScanSettings createFromParcel(Parcel in) {  
            return new ScanSettings(in);  
        }  
    };  
  
    public int getScanMode() {  
        return this.scanMode;  
    }  
}
```

```

public int getCallbackType() {
    return this.callbackType;
}

public int getMatchMode() {
    return this.matchMode;
}

public int getNumOfMatches() {
    return this.numOfMatchesPerFilter;
}

public boolean getUseHardwareFilteringIfSupported() {
    return this.useHardwareFilteringIfSupported;
}

public boolean getUseHardwareBatchingIfSupported() {
    return this.useHardwareBatchingIfSupported;
}

public boolean getUseHardwareCallbackTypesIfSupported() {
    return this.useHardwareCallbackTypesIfSupported;
}

void disableUseHardwareCallbackTypes() {
    this.useHardwareCallbackTypesIfSupported = false;
}

public long getMatchLostDeviceTimeout() {
    return this.matchLostDeviceTimeout;
}

public long getMatchLostTaskInterval() {
    return this.matchLostTaskInterval;
}

public boolean getLegacy() {
    return this.legacy;
}

public int getPhy() {
    return this.phy;
}

public long getReportDelayMillis() {
    return this.reportDelayMillis;
}

```

```

private ScanSettings(int scanMode, int callbackType, long reportDelayMillis,
int matchMode, int numOfMatchesPerFilter, boolean legacy, int phy, boolean
hardwareFiltering, boolean hardwareBatching, boolean hardwareCallbackTypes, long
matchTimeout, long taskInterval, long powerSaveScanInterval, long
powerSaveRestInterval) {

```

```

        this.scanMode = scanMode;
        this.callbackType = callbackType;
        this.reportDelayMillis = reportDelayMillis;
        this.numOfMatchesPerFilter = numOfMatchesPerFilter;
        this.matchMode = matchMode;
        this.legacy = legacy;
        this.phy = phy;
        this.useHardwareFilteringIfSupported = hardwareFiltering;
        this.useHardwareBatchingIfSupported = hardwareBatching;
        this.useHardwareCallbackTypesIfSupported = hardwareCallbackTypes;
        this.matchLostDeviceTimeout = matchTimeout * 1000000L;
        this.matchLostTaskInterval = taskInterval;
        this.powerSaveScanInterval = powerSaveScanInterval;
        this.powerSaveRestInterval = powerSaveRestInterval;
    }

    private ScanSettings(Parcel in) {
        this.scanMode = in.readInt();
        this.callbackType = in.readInt();
        this.reportDelayMillis = in.readLong();
        this.matchMode = in.readInt();
        this.numOfMatchesPerFilter = in.readInt();
        this.legacy = in.readInt() != 0;
        this.phy = in.readInt();
        this.useHardwareFilteringIfSupported = in.readInt() == 1;
        this.useHardwareBatchingIfSupported = in.readInt() == 1;
        this.matchLostDeviceTimeout = in.readLong();
        this.matchLostTaskInterval = in.readLong();
        this.powerSaveScanInterval = in.readLong();
        this.powerSaveRestInterval = in.readLong();
    }

    public void writeToParcel(Parcel dest, int flags) {
        dest.writeInt(this.scanMode);
        dest.writeInt(this.callbackType);
        dest.writeLong(this.reportDelayMillis);
        dest.writeInt(this.matchMode);
        dest.writeInt(this.numOfMatchesPerFilter);
        dest.writeInt(this.legacy ? 1 : 0);
        dest.writeInt(this.phy);
        dest.writeInt(this.useHardwareFilteringIfSupported ? 1 : 0);
        dest.writeInt(this.useHardwareBatchingIfSupported ? 1 : 0);
        dest.writeLong(this.matchLostDeviceTimeout);
        dest.writeLong(this.matchLostTaskInterval);
        dest.writeLong(this.powerSaveScanInterval);
        dest.writeLong(this.powerSaveRestInterval);
    }

    public int describeContents() {
        return 0;
    }

    public boolean hasPowerSaveMode() {
        return this.powerSaveRestInterval > 0L && this.powerSaveScanInterval > 0L;
    }

```

```

    }

    public long getPowerSaveRest() {
        return this.powerSaveRestInterval;
    }

    public long getPowerSaveScan() {
        return this.powerSaveScanInterval;
    }

    // $FF: synthetic method
    ScanSettings(Parcel x0, Object x1) {
        this(x0);
    }

    // $FF: synthetic method
    ScanSettings(int x0, int x1, long x2, int x3, int x4, boolean x5, int x6,
boolean x7, boolean x8, boolean x9, long x10, long x11, long x12, long x13,
Object x14) {
        this(x0, x1, x2, x3, x4, x5, x6, x7, x8, x9, x10, x11, x12, x13);
    }

    public static final class Builder {
        private int scanMode = 0;
        private int callbackType = 1;
        private long reportDelayMillis = 0L;
        private int matchMode = 1;
        private int numOfMatchesPerFilter = 3;
        private boolean legacy = true;
        private int phy = 255;
        private boolean useHardwareFilteringIfSupported = true;
        private boolean useHardwareBatchingIfSupported = true;
        private boolean useHardwareCallbackTypesIfSupported = true;
        private long matchLostDeviceTimeout = 10000L;
        private long matchLostTaskInterval = 10000L;
        private long powerSaveRestInterval = 0L;
        private long powerSaveScanInterval = 0L;

        @NonNull
        public ScanSettings.Builder setScanMode(int scanMode) {
            if (scanMode >= -1 && scanMode <= 2) {
                this.scanMode = scanMode;
                return this;
            } else {
                throw new IllegalArgumentException("invalid scan mode " + scanMode);
            }
        }

        @NonNull
        public ScanSettings.Builder setCallbackType(int callbackType) {
            if (!this.isValidCallbackType(callbackType)) {
                throw new IllegalArgumentException("invalid callback type - " +
callbackType);
            } else {

```

```

        this.callbackType = callbackType;
        return this;
    }
}

private boolean isValidCallbackType(int callbackType) {
    if (callbackType != 1 && callbackType != 2 && callbackType != 4) {
        return callbackType == 6;
    } else {
        return true;
    }
}

@NonNull
public ScanSettings.Builder setReportDelay(long reportDelayMillis) {
    if (reportDelayMillis < 0L) {
        throw new IllegalArgumentException("reportDelay must be > 0");
    } else {
        this.reportDelayMillis = reportDelayMillis;
        return this;
    }
}

@NonNull
public ScanSettings.Builder setNumOfMatches(int numOfMatches) {
    if (numOfMatches >= 1 && numOfMatches <= 3) {
        this.numOfMatchesPerFilter = numOfMatches;
        return this;
    } else {
        throw new IllegalArgumentException("invalid numOfMatches " +
numOfMatches);
    }
}

@NonNull
public ScanSettings.Builder setMatchMode(int matchMode) {
    if (matchMode >= 1 && matchMode <= 2) {
        this.matchMode = matchMode;
        return this;
    } else {
        throw new IllegalArgumentException("invalid matchMode " +
matchMode);
    }
}

@NonNull
public ScanSettings.Builder setLegacy(boolean legacy) {
    this.legacy = legacy;
    return this;
}

@NonNull
public ScanSettings.Builder setPhy(int phy) {
    this.phy = phy;
}

```



```

        return this;
    }

    @NonNull
    public ScanSettings.Builder setUseHardwareFilteringIfSupported(boolean
use) {
        this.useHardwareFilteringIfSupported = use;
        return this;
    }

    @NonNull
    public ScanSettings.Builder setUseHardwareBatchingIfSupported(boolean use)
{
        this.useHardwareBatchingIfSupported = use;
        return this;
    }

    @NonNull
    public ScanSettings.Builder setUseHardwareCallbackTypesIfSupported(boolean
use) {
        this.useHardwareCallbackTypesIfSupported = use;
        return this;
    }

    @NonNull
    public ScanSettings.Builder setMatchOptions(long deviceTimeoutMillis, long
taskIntervalMillis) {
        if (deviceTimeoutMillis > 0L && taskIntervalMillis > 0L) {
            this.matchLostDeviceTimeout = deviceTimeoutMillis;
            this.matchLostTaskInterval = taskIntervalMillis;
            return this;
        } else {
            throw new IllegalArgumentException("maxDeviceAgeMillis and
taskIntervalMillis must be > 0");
        }
    }

    @NonNull
    public ScanSettings.Builder setPowerSave(long scanInterval, long
restInterval) {
        if (scanInterval > 0L && restInterval > 0L) {
            this.powerSaveScanInterval = scanInterval;
            this.powerSaveRestInterval = restInterval;
            return this;
        } else {
            throw new IllegalArgumentException("scanInterval and restInterval
must be > 0");
        }
    }

    @NonNull
    public ScanSettings build() {
        if (this.powerSaveRestInterval == 0L && this.powerSaveScanInterval ==
0L) {

```

```

        this.updatePowerSaveSettings();
    }

    return new ScanSettings(this.scanMode, this.callbackType,
this.reportDelayMillis, this.matchMode, this.numOfMatchesPerFilter, this.legacy,
this.phy, this.useHardwareFilteringIfSupported,
this.useHardwareBatchingIfSupported, this.useHardwareCallbackTypesIfSupported,
this.matchLostDeviceTimeout, this.matchLostTaskInterval,
this.powerSaveScanInterval, this.powerSaveRestInterval);
    }

    private void updatePowerSaveSettings() {
        switch(this.scanMode) {
            case -1:
            case 0:
            default:
                this.powerSaveScanInterval = 500L;
                this.powerSaveRestInterval = 4500L;
                break;
            case 1:
                this.powerSaveScanInterval = 2000L;
                this.powerSaveRestInterval = 3000L;
                break;
            case 2:
                this.powerSaveScanInterval = 0L;
                this.powerSaveRestInterval = 0L;
        }
    }
}
}
}
}

```

```

// Directory: app
// Subdirectory:
classes.jar_Decompile.com\no\nordicsemi\android\support\v18\scanner
// File: UserScanCallbackWrapper.java

```

```

package no.nordicsemi.android.support.v18.scanner;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import java.lang.ref.WeakReference;
import java.util.List;

class UserScanCallbackWrapper extends ScanCallback {
    private final WeakReference<ScanCallback> weakScanCallback;

    UserScanCallbackWrapper(@NonNull ScanCallback userCallback) {
        this.weakScanCallback = new WeakReference(userCallback);
    }

    boolean isDead() {
        return this.weakScanCallback.get() == null;
    }
}

```

```

    }

    @Nullable
    ScanCallback get() {
        return (ScanCallback)this.weakScanCallback.get();
    }

    public void onScanResult(int callbackType, @NonNull ScanResult result) {
        ScanCallback userCallback = (ScanCallback)this.weakScanCallback.get();
        if (userCallback != null) {
            userCallback.onScanResult(callbackType, result);
        }
    }

    public void onBatchScanResults(@NonNull List<ScanResult> results) {
        ScanCallback userCallback = (ScanCallback)this.weakScanCallback.get();
        if (userCallback != null) {
            userCallback.onBatchScanResults(results);
        }
    }

    public void onScanFailed(int errorCode) {
        ScanCallback userCallback = (ScanCallback)this.weakScanCallback.get();
        if (userCallback != null) {
            userCallback.onScanFailed(errorCode);
        }
    }
}

```

```

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample
// File: App.java

```

```

package com.chileaf.cl831.sample;

import android.app.Application;
import android.os.Build;

import no.nordicsemi.android.dfu.DfuServiceInitiator;
import timber.log.Timber;

public class App extends Application {

    @Override
    public void onCreate() {
        super.onCreate();
        if (BuildConfig.DEBUG) {
            Timber.plant(new Timber.DebugTree());
        }
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {

```

```

        DfuServiceInitiator.createDfuNotificationChannel(this);
    }
}

```

```

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample
// File: BaseActivity.java

```

```
package com.chileaf.cl831.sample;
```

```

import android.Manifest;
import android.annotation.SuppressLint;
import android.bluetooth.BluetoothAdapter;
import android.bluetooth.BluetoothManager;
import android.content.Context;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.net.Uri;
import android.os.Build;
import android.os.Bundle;
import android.os.Handler;
import android.os.Looper;
import android.provider.Settings;
import android.widget.ImageView;
import android.widget.TextView;
import android.widget.Toast;

```

```

import androidx.annotation.LayoutRes;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.appcompat.app.AppCompatActivity;

```

```

import com.afollestad.materialdialogs.MaterialDialog;
import com.android.chileaf.WearManager;
import com.hjq.permissions.XXPermissions;

```

```

public abstract class BaseActivity extends AppCompatActivity {

    protected WearManager mManager;
    protected LoadingDialog mLoading;
    protected Handler mHandler = new Handler(Looper.getMainLooper());

    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(layoutId());
        initialize();
        initView();
        initData(savedInstanceState);
    }
}

```

```

@LayoutRes
protected abstract int layoutId();

protected abstract void initView();

protected abstract void initData(Bundle savedInstanceState);

private void initialize() {
    ImageView ivBack = findViewById(R.id.iv_toolbar_back);
    if (ivBack != null) {
        ivBack.setOnClickListener(v -> onBackPressed());
    }
    mManager = WearManager.getInstance(this);
    mManager.setDebug(BuildConfig.DEBUG);
}

protected void setTitle(String title) {
    TextView tvTitle = findViewById(R.id.tv_toolbar_title);
    if (tvTitle != null) {
        tvTitle.setText(title);
    }
}

protected void launchDetail(int type, long stamp) {
    Intent history = new Intent(this, HistoryDetailActivity.class);
    history.putExtra(HistoryDetailActivity.EXTRA_TYPE, type);
    history.putExtra(HistoryDetailActivity.EXTRA_STAMP, stamp);
    startActivity(history);
}

protected void showLoading() {
    showLoading(getString(R.string.loading));
}

protected void showLoading(String message) {
    mLoading = LoadingDialog.Builder(this)
        .setMessage(message)
        .build();
    mLoading.show();
}

protected void showLoadingAutoDismiss(final long delay) {
    showLoading();
    mHandler.postDelayed(this::hideLoading, delay);
}

protected void hideLoading() {
    if (mLoading != null && mLoading.isShowing()) {
        mLoading.dismiss();
    }
}

protected void showToast(final int messageResId) {
    Toast.makeText(this, messageResId, Toast.LENGTH_SHORT).show();
}

```

```

    }

    protected void showToast(final String message) {
        runOnUiThread(() -> Toast.makeText(this, message,
Toast.LENGTH_SHORT).show());
    }

    protected void isBLESupported() {
        if
(!getPackageManager().hasSystemFeature(PackageManager.FEATURE_BLUETOOTH_LE)) {
            new MaterialDialog.Builder(this)
                .title(R.string.no_ble)
                .positiveText(R.string.scanner_action_cancel)
                .positiveColorRes(R.color.colorPrimary)
                .onPositive((dialog, which) -> finish())
                .show();
        }
    }

    protected boolean isBLEEnabled() {
        final BluetoothManager bluetoothManager = (BluetoothManager)
getSystemService(Context.BLUETOOTH_SERVICE);
        final BluetoothAdapter adapter = bluetoothManager.getAdapter();
        return adapter != null && adapter.isEnabled();
    }

    protected String[] getPermissions() {
        int targetSdkVersion = getApplicationInfo().targetSdkVersion;
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S && targetSdkVersion
>= Build.VERSION_CODES.S) {
            return new String[]{android.Manifest.permission.BLUETOOTH_SCAN,
android.Manifest.permission.BLUETOOTH_CONNECT,
android.Manifest.permission.ACCESS_FINE_LOCATION};
        } else if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q &&
targetSdkVersion >= Build.VERSION_CODES.Q) {
            return new
String[]{android.Manifest.permission.ACCESS_FINE_LOCATION};
        } else {
            return new String[]{Manifest.permission.ACCESS_COARSE_LOCATION};
        }
    }

    @SuppressWarnings("MissingPermission")
    protected void showBLEDialog() {
        XXPermissions.with(this)
            .permission(getPermissions())
            .request((permissions, allGranted) -> {
                if (allGranted) {
                    startActivity(new
Intent(BluetoothAdapter.ACTION_REQUEST_ENABLE));
                } else {
                    showToast(getString(R.string.no_required_permission));
                }
            });
    }

```

```

    }

    protected boolean isLocationEnabled(@NonNull final Context context) {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
            int locationMode = Settings.Secure.LOCATION_MODE_OFF;
            try {
                locationMode =
Settings.Secure.getInt(context.getContentResolver(),
Settings.Secure.LOCATION_MODE);
            } catch (final Settings.SettingNotFoundException e) {
                // do nothing
            }
            return locationMode != Settings.Secure.LOCATION_MODE_OFF;
        }
        return true;
    }

    protected void onEnableLocation() {
        final Intent intent = new
Intent(Settings.ACTION_LOCATION_SOURCE_SETTINGS);
        startActivity(intent);
    }

    protected void onPermissionSettings() {
        final Intent intent = new
Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS);
        intent.setData(Uri.fromParts("package", getPackageName(), null));
        startActivity(intent);
    }

    // Get emotion value. (int, as emotion type)
    // 0 - <result not ready>
    // 1 - Calm
    // 2 - Stressed
    // 3 - Happy
    // 4 - Tense
    // 5 - Angry
    protected String getEmotion(int level) {
        if (level == 0) {
            return "Not ready";
        } else if (level == 1) {
            return "Calm";
        } else if (level == 2) {
            return "Stressed";
        } else if (level == 3) {
            return "Happy";
        } else if (level == 4) {
            return "Tense";
        } else if (level == 5) {
            return "Angry";
        } else {
            return "Unknown";
        }
    }
}

```

```

// 0 - <result not ready>
// 1 - Normal
// 2 - Moderate fatigue
// 3 - Severe fatigue
protected String getStamina(int stamina) {
    if (stamina == 0) {
        return "Not ready";
    } else if (stamina == 1) {
        return "Normal";
    } else if (stamina == 2) {
        return "Moderate fatigue";
    } else if (stamina == 3) {
        return "Severe fatigue";
    } else {
        return "Unknown";
    }
}

protected String getMode(int mode) {
    if (mode == 0) {
        return "Indoor running";
    } else if (mode == 1) {
        return "Outdoor running";
    } else if (mode == 2) {
        return "Outdoor cycling";
    } else if (mode == 3) {
        return "Spinning bike";
    } else if (mode == 4) {
        return "Free training";
    } else if (mode == 5) {
        return "Skipping rope";
    } else {
        return "None";
    }
}
}

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample
// File: BloodOxygenActivity.java

package com.chileaf.cl831.sample;

import android.bluetooth.BluetoothDevice;
import android.os.Bundle;
import android.widget.EditText;
import android.widget.Switch;

import androidx.annotation.NonNull;
import androidx.appcompat.widget.AppCompatTextView;

```



```

import com.android.chileaf.fitness.callback.BloodOxygenCallback;

public class BloodOxygenActivity extends BaseActivity implements
BloodOxygenCallback {

    private Switch aSwitch;
    private AppCompatActivity text1;
    private AppCompatActivity text2;
    private AppCompatActivity text3;
    private AppCompatActivity text4;

    @Override
    protected int layoutId() {
        return R.layout.activity_blood_oxygen;
    }

    @Override
    protected void initView() {
        aSwitch = findViewById(R.id.sh_blood);
        aSwitch.setOnCheckedChangeListener((compoundButton, b) -> {
            if (b) {
                mManager.setBloodOxygen(1);
            } else {
                mManager.setBloodOxygen(0);
            }
        });
        text1 = findViewById(R.id.tv_blood);
        text2 = findViewById(R.id.tv_wrist);
        text3 = findViewById(R.id.tv_pi);
        text4 = findViewById(R.id.tv_onwrist);
        showToast("Please wear tightly and relax your body.");
    }

    @Override
    protected void initData(Bundle savedInstanceState) {
        mManager.addBloodOxygenCallback(this);
    }

    @Override
    public void onBloodOxygenReceived(@NonNull BluetoothDevice device, int
bSwitch, String value, int gesture, int piValue, int onwrist) {
        runOnUiThread(() -> {
            aSwitch.setChecked(bSwitch == 1);
            if (value == "" || value == null) {
                return;
            }
            text1.setText(value + "");

            if (gesture == 0) {
                text2.setText("Wrong wrist posture");
            } else if (gesture == 1) {
                text2.setText("Wear the correct posture");
            }
        });
    }
}

```

```

        if (piValue == 0) {
            text3.setText("No pulse detected");
        } else if (piValue < 8) {
            text3.setText("Weak signal");
        } else if (piValue < 15) {
            text3.setText("Good signal");
        } else if (piValue >= 15) {
            text3.setText("Excellent signal");
        }

        if (onwrist == 0) {
            text4.setText("Off the wrist");
        } else if (onwrist == 1) {
            text4.setText("Worn");
        }
    });
}

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample
// File: History3DAdapter.java

package com.chileaf.cl831.sample;

import com.android.chileaf.model.HistoryOf3D;
import com.android.chileaf.model.HistoryOfSport;
import com.chad.library.adapter.base.BaseQuickAdapter;
import com.chad.library.adapter.base.viewholder.BaseViewHolder;

import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.Locale;

public class History3DAdapter extends BaseQuickAdapter<HistoryOf3D,
BaseViewHolder> {

    private SimpleDateFormat mDateFormat;

    public History3DAdapter() {
        super(R.layout.item_history, new ArrayList<>());
        mDateFormat = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss",
Locale.getDefault());
    }

    @Override
    protected void convert(BaseViewHolder helper, HistoryOf3D history) {
        StringBuilder item = new StringBuilder();
        // String date = mDateFormat.format(new Date(history.stamp));
        item
        // .append("Date time:").append(date).append("\n")
        .append("X,Y,Z:[ ")

```

```

        .append(history.accX).append(", ")
//        .append("AccY:")
        .append(history.accY).append(", ")
//        .append("AccZ:")
        .append(history.accZ).append(" ]");
        helper.setText(R.id.tv_history, item.toString());
    }
}

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample
// File: HistoryActivity.java

package com.chileaf.cl831.sample;

import android.os.Bundle;

import androidx.recyclerview.widget.DefaultItemAnimator;
import androidx.recyclerview.widget.DividerItemDecoration;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.android.chileaf.model.HistoryOfRecord;

/**
 * History record
 */
public class HistoryActivity extends BaseActivity {

    public static final String EXTRA_HISTORY = "extra_history";

    public static final int TYPE_SPORT = 0x02;
    public static final int TYPE_HEART = 0x04;
    public static final int TYPE_HEART_RR = 0x06;
    public static final int TYPE_INTERVAL = 0x08;
    public static final int TYPE_SINGLE = 0x10;

    public static final int TYPE_3D = 0x12;

    private RecyclerView mRvHistory;

    @Override
    protected int layoutId() {
        return R.layout.activity_history;
    }

    @Override
    protected void initView() {
        mRvHistory = findViewById(R.id.rv_history);
        mRvHistory.setLayoutManager(new LinearLayoutManager(this));
        mRvHistory.setItemAnimator(new DefaultItemAnimator());
        mRvHistory.addItemDecoration(new DividerItemDecoration(this,

```

```

DividerItemDecoration.VERTICAL));
    mRvHistory.setHasFixedSize(true);
}

@Override
protected void initData(Bundle savedInstanceState) {
    int type = getIntent().getIntExtra(EXTRA_HISTORY, 0);
    if (type == TYPE_SPORT) {
        setTitle("7 days sport history");
        showLoadingAutoDismiss(2000);
        HistorySportAdapter adapter = new HistorySportAdapter();
        mRvHistory.setAdapter(adapter);
        mManager.addHistoryOfSportCallback((device, sports) -> {
            runOnUiThread(() -> {
                adapter.replaceData(sports);
                hideLoading();
            });
        });
        mManager.getHistoryOfSport();
    } else if (type == TYPE_HEART) {
        setTitle("Heart rate history record");
        showLoadingAutoDismiss(2000);
        HistoryRecordAdapter adapter = new HistoryRecordAdapter();
        adapter.setOnItemClickListener((adapter1, view, position) -> {
            HistoryOfRecord history = (HistoryOfRecord)
adapter1.getData().get(position);
            launchDetail(HistoryDetailActivity.TYPE_HR, history.stamp);
        });
        mRvHistory.setAdapter(adapter);
        mManager.addHistoryOfHRRRecordCallback((device, records) -> {
            runOnUiThread(() -> {
                adapter.replaceData(records);
                hideLoading();
            });
        });
        mManager.getHistoryOfHRRRecord();
    } else if (type == TYPE_HEART_RR) {
        setTitle("RR history record");
        showLoadingAutoDismiss(2000);
        HistoryRecordAdapter adapter = new HistoryRecordAdapter();
        adapter.setOnItemClickListener((adapter1, view, position) -> {
            HistoryOfRecord history = (HistoryOfRecord)
adapter1.getData().get(position);
            launchDetail(HistoryDetailActivity.TYPE_RR, history.stamp);
        });
        mRvHistory.setAdapter(adapter);
        mManager.addHistoryOfRRRRecordCallback((device, records) -> {
            runOnUiThread(() -> {
                adapter.replaceData(records);
                hideLoading();
            });
        });
        mManager.getHistoryOfRRRRecord();
    } else if (type == TYPE_INTERVAL) {

```

```

        setTitle("Interval steps record");
        showLoadingAutoDismiss(5000);
        IntervalStepAdapter adapter = new IntervalStepAdapter();
        mRvHistory.setAdapter(adapter);
        mManager.addIntervalStepCallback((device, steps) -> {
            runOnUiThread(() -> {
                adapter.replaceData(steps);
                hideLoading();
            });
        });
        mManager.getIntervalSteps();
    } else if (type == TYPE_SINGLE) {
        setTitle("Single pressed record");
        showLoadingAutoDismiss(5000);
        HistoryRecordAdapter adapter = new HistoryRecordAdapter();
        mRvHistory.setAdapter(adapter);
        mManager.addSingleTapRecordCallback((device, records) -> {
            runOnUiThread(() -> {
                adapter.replaceData(records);
                hideLoading();
            });
        });
        mManager.getSingleTapRecords();
    } else if (type == TYPE_3D) {
        setTitle("3D History");
        showLoadingAutoDismiss(2000);
        History3DAdapter adapter = new History3DAdapter();
        mRvHistory.setAdapter(adapter);
        mManager.addHistoryOf3DDataCallback((device, history, finish) -> {
            runOnUiThread(() -> {
                adapter.addData(history);
                hideLoading();
                if (finish) {
                    showToast("Complete!");
                }
            });
        });
        mManager.getHistoryOf3D();
    }
}
}

```

```

}

```

```

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample
// File: HistoryDetailActivity.java

```

```

package com.chileaf.cl831.sample;

```

```

import android.graphics.Color;
import android.os.Bundle;

```

```

import androidx.appcompat.widget.AppCompatTextView;

```

```
import com.android.chileaf.model.HistoryOfHeartRate;
import com.android.chileaf.model.HistoryOfRespiratoryRate;
import com.github.mikephil.charting.charts.LineChart;
import com.github.mikephil.charting.components.XAxis;
import com.github.mikephil.charting.data.Entry;
import com.github.mikephil.charting.data.LineData;
import com.github.mikephil.charting.data.LineDataSet;
import com.github.mikephil.charting.formatter.ValueFormatter;
```

```
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.Locale;
```

```
import timber.log.Timber;
```

```
/**
```

```
 * History detail
```

```
 */
```

```
public class HistoryDetailActivity extends BaseActivity {
```

```
    public static final String EXTRA_TYPE = "extra_type";
    public static final String EXTRA_STAMP = "extra_stamp";
```

```
    public static final int TYPE_HR = 0x03;
    public static final int TYPE_RR = 0x05;
```

```
    private LineChart mChart;
    private AppCompatTextView mTvHistory;
    private SimpleDateFormat mDateFormat;
```

```
    @Override
```

```
    protected int layoutId() {
        return R.layout.activity_record_detail;
    }
```

```
    @Override
```

```
    protected void initView() {
        mTvHistory = findViewById(R.id.tv_history);
        mChart = findViewById(R.id.chart_history);
        initChart();
    }
```

```
    @Override
```

```
    protected void initData(Bundle savedInstanceState) {
        showLoadingAutoDismiss(2000);
        int type = getIntent().getIntExtra(EXTRA_TYPE, 0);
        long stamp = getIntent().getLongExtra(EXTRA_STAMP, 0);
        mDateFormat = new SimpleDateFormat("MM-dd HH:mm:ss",
Locale.getDefault());
        if (type == TYPE_HR) {
            setTitle("HR history detail");
        }
    }
}
```

```

        mManager.addHistoryOfHRDataCallback((device, heartRates) -> {
            runOnUiThread(() -> {
                Timber.d("heartRates:%d %s", heartRates.size(),
heartRates.toString());
                updateHeartRates(heartRates);
                hideLoading();
            });
        });
        mManager.getHistoryOfHRData(stamp);
//        mManager.addHistoryOfSingleRecordCallback((device, stamp1, step,
distance, calorie) -> {
//            runOnUiThread(() -> {
//                mTvHistory.setText("Step:" + step + "\ndistance: " +
distance/ 100f + "m\ncalorie:" + calorie/10f + "kcal");
//            });
//        });
//        mManager.getHistoryOfSingleRecord(stamp);
    } else if (type == TYPE_RR) {
        setTitle("RR history detail");
        mManager.addHistoryOfRRDataCallback((device, respiratoryRates) -> {
            runOnUiThread(() -> {
                Timber.d("respiratoryRates:%d %s", respiratoryRates.size(),
respiratoryRates.toString());
                updateRespiratoryRates(respiratoryRates);
                hideLoading();
            });
        });
        mManager.getHistoryOfRRData(stamp);
    }
}

```

```

private void initChart() {
    mChart.setNoDataText("");
    mChart.setTouchEnabled(true);
    mChart.setScaleEnabled(true);
    mChart.setPinchZoom(false);
    mChart.getDescription().setEnabled(false);
    mChart.getLegend().setEnabled(true);
    mChart.setScaleYEnabled(false);
    mChart.setScaleXEnabled(true);
    mChart.setDragEnabled(true);

    mChart.getAxisLeft().setDrawGridLines(true);
    mChart.getAxisLeft().setDrawAxisLine(true);
    mChart.getAxisLeft().setEnabled(true);
    mChart.getAxisLeft().setAxisMinimum(0f);

    mChart.getAxisRight().setEnabled(false);
    mChart.getXAxis().setTextSize(8);
    mChart.getXAxis().setGranularity(1f);
    mChart.getXAxis().setDrawAxisLine(true);
    mChart.getXAxis().setDrawGridLines(false);
    mChart.getXAxis().setPosition(XAxis.XAxisPosition.BOTTOM);
}

```

```

private void updateHeartRates(List<HistoryOfHeartRate> heartRates) {
    final List<Entry> values = new ArrayList<>();
    final List<String> stamps = new ArrayList<>();
    for (int i = 0; i < heartRates.size(); i++) {
        HistoryOfHeartRate history = heartRates.get(i);
        values.add(new Entry(i, history.heartRate));
        stamps.add(mDateFormat.format(new Date(history.stamp)));
    }
    mChart.resetTracking();
    LineDataSet dataSet = new LineDataSet(values, "Heart rate");
    dataSet.setValueTextSize(8);
    dataSet.setCircleRadius(1.5f);
    dataSet.setColor(Color.RED);
    dataSet.setFillColor(Color.RED);
    dataSet.setCircleColor(Color.RED);
    dataSet.setCircleHoleColor(Color.RED);
    dataSet.setValueTextColor(Color.RED);
    dataSet.setLineWidth(1f);
    dataSet.setDrawValues(true);
    dataSet.setDrawCircles(true);
    dataSet.setHighlightEnabled(false);
    dataSet.setMode(LineDataSet.Mode.LINEAR);
    dataSet.setDrawFilled(false);

    mChart.getXAxis().setValueFormatter(new StampValueFormatter(stamps));

    LineData data = new LineData(dataSet);
    mChart.setData(data);
    mChart.invalidate();
}

private void updateRespiratoryRates(List<HistoryOfRespiratoryRate>
respiratoryRates) {
    final List<Entry> values = new ArrayList<>();
    final List<String> stamps = new ArrayList<>();
    for (int i = 0; i < respiratoryRates.size(); i++) {
        HistoryOfRespiratoryRate history = respiratoryRates.get(i);
        values.add(new Entry(i, history.respiratoryRate));
        stamps.add(mDateFormat.format(new Date(history.stamp)));
    }
    mChart.resetTracking();
    LineDataSet dataSet = new LineDataSet(values, "RR");
    dataSet.setValueTextSize(8);
    dataSet.setCircleRadius(1.5f);
    dataSet.setColor(Color.BLUE);
    dataSet.setFillColor(Color.BLUE);
    dataSet.setCircleColor(Color.BLUE);
    dataSet.setValueTextColor(Color.BLUE);
    dataSet.setLineWidth(1f);
    dataSet.setDrawValues(true);
    dataSet.setDrawCircles(true);
    dataSet.setHighlightEnabled(false);
    dataSet.setMode(LineDataSet.Mode.LINEAR);
}

```



```

        dataSet.setDrawFilled(false);

        mChart.getXAxis().setValueFormatter(new StampValueFormatter(stamps));

        LineData data = new LineData(dataSet);
        mChart.setData(data);
        mChart.invalidate();
    }

    private static class StampValueFormatter extends ValueFormatter {

        private final List<String> stamps;

        private StampValueFormatter(List<String> stamps) {
            this.stamps = stamps;
        }

        @Override
        public String getFormattedValue(float value) {
            int index = (int) value;
            if (index >= 0 && index < stamps.size()) {
                return stamps.get(index);
            } else {
                return "";
            }
        }
    }
}

```

```

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample
// File: HistoryRecordAdapter.java

```

```

package com.chileaf.cl831.sample;

import android.widget.TextView;

import com.android.chileaf.model.HistoryOfRecord;
import com.chad.library.adapter.base.BaseQuickAdapter;
import com.chad.library.adapter.base.viewholder.BaseViewHolder;

import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.Locale;

public class HistoryRecordAdapter extends BaseQuickAdapter<HistoryOfRecord,
BaseViewHolder> {

    private SimpleDateFormat mDateFormat;

    public HistoryRecordAdapter() {
        super(R.layout.item_history, new ArrayList<>());
    }

```

```

        mDateFormat = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss",
Locale.getDefault());
    }

    @Override
    protected void convert(BaseViewHolder helper, HistoryOfRecord history) {
        String date = mDateFormat.format(new Date(history.record));
        TextView tvHistory = helper.getView(R.id.tv_history);
        tvHistory.setTextSize(20);
        tvHistory.setText(date);
    }
}

```

```

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample
// File: HistorySleepActivity.java

```

```

package com.chileaf.cl831.sample;

import android.bluetooth.BluetoothDevice;
import android.os.Bundle;
import android.util.Log;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.TextView;

import androidx.annotation.NonNull;

import com.android.chileaf.fitness.callback.HistoryOfSleepCallback;
import com.android.chileaf.model.HistorySleep;

import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.Locale;

public class HistorySleepActivity extends BaseActivity implements
HistoryOfSleepCallback {

    private ListView mListView;
    private ArrayAdapter mArrayAdapter;
    private List<String> strings = new ArrayList<>();
    private static final SimpleDateFormat mFormat = new
SimpleDateFormat("yyyy-MM-dd HH:mm:ss", Locale.getDefault());

    @Override
    protected int layoutId() {
        return R.layout.activity_sleepdata;
    }

    @Override
    protected void initView() {

```

```

        TextView mTvToolbarTitle = findViewById(R.id.tv_toolbar_title);
        mTvToolbarTitle.setText("Get Sleep Data");
        mListView = findViewById(R.id.listView);
        mArrayAdapter = new ArrayAdapter(this,
android.R.layout.simple_list_item_1, strings);
        mListView.setAdapter(mArrayAdapter);
        showLoadingAutoDismiss(2000);
    }

    @Override
    protected void initData(Bundle savedInstanceState) {
        mManager.addHistoryOfSleepCallback(this);
        mManager.getHistoryOfSleep();
    }

    @Override
    public void onHistoryOfSleepReceived(@NonNull BluetoothDevice device,
List<HistorySleep> sleeps) {
        runOnUiThread(() -> {
            for (int i = 0; i < sleeps.size(); i++) {
                HistorySleep sleep = sleeps.get(i);
                int[] actions = sleep.actions;
                int len = actions.length;
                long utc = sleep.utc;
                int index = 0;
                long utc2 = 0;
                String text = "";
                for (int i1 = 0; i1 < len; i1++) {
                    int action = actions[i1]; // one every five minutes
                    Log.d("", "action: " + action);
                    long utc1 = (utc + (i1 * 300000));
                    if (action > 20) { //wide awake
                        if (index >= 3) {
                            long utc3 = utc2 - (300000 * index);
                            for (int i2 = 0; i2 < index; i2++) {
                                text += "\nutc:" + millsToDate(utc3 + (i2 *
300000)) + "\naction Index: deep Sleep";
                            }
                        } else if (index > 0) {
                            long utc3 = utc2 - (300000 * index);
                            for (int i2 = 0; i2 < index; i2++) {
                                text += "\nutc:" + millsToDate(utc3 + (i2 *
300000)) + "\naction Index: light sleep";
                            }
                        }
                        index = 0;
                        utc2 = 0;
                        text += "\nutc:" + millsToDate(utc1) + "\naction Index:
not Sleep";
                    } else if (action <= 20 && action > 0) { //light sleep
                        if (index >= 3) {
                            long utc3 = utc2 - (300000 * index);
                            for (int i2 = 0; i2 < index; i2++) {
                                text += "\nutc:" + millsToDate(utc3 + (i2 *

```

```

300000)) + "\naction Index: deep Sleep";
        }

        } else if (index > 0) {
            long utc3 = utc2 - (300000 * index);
            for (int i2 = 0; i2 < index; i2++) {
                text += "\nutc:" + millsToDate(utc3 + (i2 *
300000)) + "\naction Index: light sleep";
            }
        }
        index = 0;
        utc2 = 0;
        text += "\nutc:" + millsToDate(utc1) + "\naction Index:
light sleep";
    } else {    //Deep sleep 3 >= 0 (3 consecutive zeros equals
deep sleep)

        index++;
        utc2 = utc1;
    }
}

if (index >= 3) {
    long utc3 = utc2 - (300000 * index);
    for (int i2 = 0; i2 < index; i2++) {
        text += "\nutc:" + millsToDate(utc3 + (i2 * 300000)) +
"\naction Index: deep Sleep";
    }

    } else if (index > 0) {
        long utc3 = utc2 - (300000 * index);
        for (int i2 = 0; i2 < index; i2++) {
            text += "\nutc:" + millsToDate(utc3 + (i2 * 300000)) +
"\naction Index: light sleep";
        }
    }
    strings.add(text);
}
mPagerAdapter.notifyDataSetChanged();
});
}

private String millsToDate(Long time) {
    Date dt = new Date(time);
    return mFormat.format(dt);
}

}

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample
// File: HistorySportAdapter.java

package com.chileaf.cl831.sample;

```

```

import com.android.chileaf.model.HistoryOfSport;
import com.chad.library.adapter.base.BaseQuickAdapter;
import com.chad.library.adapter.base.viewholder.BaseViewHolder;

import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.Locale;

public class HistorySportAdapter extends BaseQuickAdapter<HistoryOfSport,
BaseViewHolder> {

    private SimpleDateFormat mDateFormat;

    public HistorySportAdapter() {
        super(R.layout.item_history, new ArrayList<>());
        mDateFormat = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss",
Locale.getDefault());
    }

    @Override
    protected void convert(BaseViewHolder helper, HistoryOfSport history) {
        StringBuilder item = new StringBuilder();
        String date = mDateFormat.format(new Date(history.startTime));
        item.append("Date time:").append(date).append("\n")
            .append("Step:").append(history.step).append("歩\n")
            .append("Calorie:").append(String.format("%.1f", history.calorie
/ 10f)).append("CAL");
        helper.setText(R.id.tv_history, item.toString());
    }
}

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample
// File: IntervalStepAdapter.java

package com.chileaf.cl831.sample;

import com.android.chileaf.model.IntervalStep;
import com.chad.library.adapter.base.BaseQuickAdapter;
import com.chad.library.adapter.base.viewholder.BaseViewHolder;

import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.Locale;

public class IntervalStepAdapter extends BaseQuickAdapter<IntervalStep,
BaseViewHolder> {

```

```

private SimpleDateFormat mDateFormat;

public IntervalStepAdapter() {
    super(R.layout.item_history, new ArrayList<>());
    mDateFormat = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss",
Locale.getDefault());
}

@Override
protected void convert(BaseViewHolder helper, IntervalStep interval) {
    StringBuilder item = new StringBuilder();
    String date = mDateFormat.format(new Date(interval.stamp));
    item.append("Date time : ").append(date).append("\n")
        .append("Steps : ").append(interval.steps).append("\n");
    helper.setText(R.id.tv_history, item.toString());
}

}

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample
// File: LoadingDialog.java

package com.chileaf.cl831.sample;

import android.app.Activity;
import android.content.Context;
import android.os.Bundle;
import android.text.TextUtils;
import android.view.LayoutInflater;
import android.view.View;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

public class LoadingDialog extends AppCompatActivity {

    private View mRoot;

    public LoadingDialog(Bundle bundle) {
        this(bundle, bundle.getContext(), R.style.DialogStyle);
    }

    public LoadingDialog(Bundle bundle, Context context, int theme) {
        super(context, theme);
        mRoot = LayoutInflater.from(context).inflate(R.layout.dialog_loading,
null);
        TextView tvLoading = mRoot.findViewById(R.id.tv_loading);
        if (!TextUtils.isEmpty(bundle.getMessage())) {
            tvLoading.setVisibility(View.VISIBLE);
            tvLoading.setText(bundle.getMessage());
        } else {

```

```

        tvLoading.setVisibility(View.GONE);
    }
    setCanceledOnTouchOutside(false);
    setCancelable(false);
}

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(mRoot);
}

public static Builder Builder(Activity activity) {
    return new Builder(activity);
}

public static final class Builder {
    private Context context;
    private String message;

    private Builder(Context context) {
        this.context = context;
    }

    public Builder setMessage(String message) {
        this.message = message;
        return this;
    }

    public LoadingDialog build() {
        return new LoadingDialog(this);
    }
}
}

```

```

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample
// File: MainActivity.java

```

```

package com.chileaf.cl831.sample;

import android.bluetooth.BluetoothDevice;
import android.content.Intent;
import android.os.Bundle;
import android.text.TextUtils;
import android.widget.Button;
import android.widget.CompoundButton;
import android.widget.EditText;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.appcompat.app.AlertDialog;

```

```

import androidx.appcompat.widget.AppCompatEditText;
import androidx.appcompat.widget.SwitchCompat;

import com.android.chileaf.fitness.callback.BodyHealthCallback;
import com.android.chileaf.fitness.callback.Sensor6DFrequencyCallback;
import com.android.chileaf.fitness.callback.Sensor6DRawDataCallback;
import com.android.chileaf.fitness.callback.WearManagerCallbacks;
import com.android.chileaf.util.HexUtil;
import com.chileaf.cl831.sample.dfu.DfuActivity;
import com.chileaf.cl831.sample.multi.MultiConnectActivity;
import com.hjq.permissions.OnPermissionCallback;
import com.hjq.permissions.XXPermissions;

import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.HashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;

import timber.log.Timber;

public class MainActivity extends BaseActivity implements
ScannerFragment.OnDeviceSelectedListener, WearManagerCallbacks {

    private boolean mDeviceConnected = false;

    private TextView mTvDeviceName;
    private TextView mTvSDKVersion;
    private TextView mTvVersion;
    private TextView mTvRssi;
    private TextView mTvBattery;
    private TextView mTvSport;
    private TextView mTvHeartRate;
    private TextView mTvReceivedData;
    private TextView mTvAccelerometer;

    private TextView mTvHRStatus;
    private TextView mTvHRAAlertStatus;
    private EditText mEtMin;
    private EditText mEtMax;
    private EditText mEtGoal;
    private TextView mTvHRMax;
    private TextView mTv3DFrequency;
    private TextView mTv3DStatus;
    private EditText mEtHRMax;
    private TextView mTvHealth;
    private TextView mTv6DFrequency;
    private TextView mTv6DRawData;
    private Button mBtnConnect;
    private final SimpleDateFormat mDateFormat = new SimpleDateFormat("yy-MM-dd
HH:mm:ss", Locale.getDefault());
    private final Map<Integer, String> mFrequency3DMap = new HashMap<>();
    private final Map<Integer, String> mFrequency6DMap = new HashMap<>();

```



```

@Override
protected int layoutId() {
    return R.layout.activity_main;
}

@Override
protected void initView() {
    mTvDeviceName = findViewById(R.id.tv_device_name);
    mTvSDKVersion = findViewById(R.id.tv_sdk_version);

    mTvSDKVersion.setText("SDK Version: v" + BuildConfig.VERSION_NAME);

    mTvAccelerometer = findViewById(R.id.tv_accelerometer);

    mTvRssi = findViewById(R.id.tv_rssi);
    mTvVersion = findViewById(R.id.tv_version);
    mTvBattery = findViewById(R.id.tv_battery);
    mTvSport = findViewById(R.id.tv_sport);
    mTvHeartRate = findViewById(R.id.tv_hr);
    mTvReceivedData = findViewById(R.id.tv_received);

    mTvHRStatus = findViewById(R.id.tv_heart_rate);
    mTvHRAAlertStatus = findViewById(R.id.tv_hr_alert_status);

    mTvHRMax = findViewById(R.id.tv_hr_max);
    mTv3DFrequency = findViewById(R.id.tv_3d_frequency);
    mTv3DStatus = findViewById(R.id.tv_3d_status);

    mEtMin = findViewById(R.id.et_min);
    mEtMax = findViewById(R.id.et_max);
    mEtGoal = findViewById(R.id.et_goal);

    mEtHRMax = findViewById(R.id.et_hr_max);
    mTvHealth = findViewById(R.id.tv_health);

    mTv6DFrequency = findViewById(R.id.tv_6d_frequency);
    mTv6DRawData = findViewById(R.id.tv_6d_data);

    mBtnConnect = findViewById(R.id.btn_connect);

    //Multi connect
    findViewById(R.id.btn_multi).setOnClickListener(view ->
startActivity(new Intent(this, MultiConnectActivity.class)));
    //Sport health
    findViewById(R.id.btn_sport_health).setOnClickListener(view ->
startActivity(new Intent(this, SportHealthActivity.class)));
    //Get HeartRate Status
    findViewById(R.id.btn_heart_rate).setOnClickListener(view ->
mManager.getHeartRateStatus());
    //User information
    findViewById(R.id.btn_user_info).setOnClickListener(view ->
startActivity(new Intent(this, UserInfoActivity.class)));
    //Restoration

```

```

        findViewById(R.id.btn_restoration).setOnClickListener(view ->
mManager.restoration());
        //DFU upgrade
        findViewById(R.id.btn_dfu).setOnClickListener(view -> {
            if (!mManager.isConnected()) {
                showToast("请先连接设备");
                return;
            }
            startActivity(new Intent(this, DfuActivity.class));
        });
        //Get 7 days sport history
        findViewById(R.id.btn_history_sport).setOnClickListener(view ->
launchHistory(HistoryActivity.TYPE_SPORT));
        //Heart rate history record
        findViewById(R.id.btn_history_heart).setOnClickListener(view ->
launchHistory(HistoryActivity.TYPE_HEART));
        //Heart rate RR history record
        findViewById(R.id.btn_history_rr).setOnClickListener(view ->
launchHistory(HistoryActivity.TYPE_HEART_RR));
        //Get the number of steps in the interval
        findViewById(R.id.btn_interval).setOnClickListener(view ->
launchHistory(HistoryActivity.TYPE_INTERVAL));
        //Get historical data for a single key press
        findViewById(R.id.btn_single).setOnClickListener(view ->
launchHistory(HistoryActivity.TYPE_SINGLE));
        //Get historical data for 3d
        findViewById(R.id.btn_3d).setOnClickListener(view ->
launchHistory(HistoryActivity.TYPE_3D));

        //Set HeartRate Status
        findViewById(R.id.btn_hr_setting).setOnClickListener(view -> {
            int min = getValue(mEtMin);
            int max = getValue(mEtMax);
            int goal = getValue(mEtGoal);
            mManager.setHeartRateStatus(min, max, goal);
        });

        //Shutdown
        findViewById(R.id.btn_shut_down).setOnClickListener(view ->
mManager.shutdown());

        //Blood oxygen
        findViewById(R.id.btn_blood_oxygen).setOnClickListener(view ->
startActivity(new Intent(this, BloodOxygenActivity.class)));
        //Real time temperature
        findViewById(R.id.btn_temperature).setOnClickListener(view ->
startActivity(new Intent(this, TemperatureActivity.class)));

        //Heart Rate Alarm Switch
        SwitchCompat swAlarm = findViewById(R.id.sw_alarm);
        swAlarm.setOnCheckedChangeListener(new
CompoundButton.OnCheckedChangeListener() {
            @Override
            public void onCheckedChanged(CompoundButton buttonView, boolean

```

```

isChecked) {
    mManager.setHeartRateAlarm(isChecked);
    showToast(getAlarm(isChecked));
}
});

//Heart Rate Alert By Age
findViewById(R.id.btn_hr_alert).setOnClickListener(v ->
mManager.getHeartRateAlarm());
//Set HeartRate Max
findViewById(R.id.btn_hr_max).setOnClickListener(view -> {
    int max = getValue(mEtHRMax);
    mManager.setHeartRateMax(max);
});
//Get Heart Rate Max
findViewById(R.id.btn_get_hr_max).setOnClickListener(v ->
mManager.getHeartRateMax());
//Get Sleep Data
findViewById(R.id.btn_get_sleep_data).setOnClickListener(view ->
startActivity(new Intent(this, HistorySleepActivity.class)));
//Get 3D Frequency
findViewById(R.id.btn_3d_frequency).setOnClickListener(v ->
mManager.get3DFrequency());
//Setting 3D Frequency
findViewById(R.id.btn_3d_0).setOnClickListener(v ->
mManager.set3DFrequency(0)); //25HZ
findViewById(R.id.btn_3d_1).setOnClickListener(v ->
mManager.set3DFrequency(1)); //50HZ
findViewById(R.id.btn_3d_2).setOnClickListener(v ->
mManager.set3DFrequency(2)); //100HZ
findViewById(R.id.btn_3d_3).setOnClickListener(v ->
mManager.set3DFrequency(3)); //200HZ
findViewById(R.id.btn_3d_4).setOnClickListener(v ->
mManager.set3DFrequency(4)); //400HZ
//Get 6D Frequency
findViewById(R.id.btn_6d_frequency).setOnClickListener(view ->
mManager.get6DFrequency());
//Set 6D Frequency
findViewById(R.id.btn_6d_0).setOnClickListener(view ->
mManager.set6DFrequency(0)); //26hz
findViewById(R.id.btn_6d_1).setOnClickListener(view ->
mManager.set6DFrequency(1)); //52hz
findViewById(R.id.btn_6d_2).setOnClickListener(view ->
mManager.set6DFrequency(2)); //104hz
findViewById(R.id.btn_6d_3).setOnClickListener(view ->
mManager.set6DFrequency(3)); //208hz

//3D Status Switch
SwitchCompat sw3dStatus = findViewById(R.id.sw_3d_status);
sw3dStatus.setOnCheckedChangeListener(new
CompoundButton.OnCheckedChangeListener() {
    @Override
    public void onCheckedChanged(CompoundButton buttonView, boolean
isChecked) {

```

```

        mManager.set3DEnabled(isChecked);
        showToast(get3DStatus(isChecked));
    }
});

//Get 3D Status
findViewById(R.id.btn_3d_status).setOnClickListener(v ->
mManager.get3DStatus());

AppCompatActivity etFilter = findViewById(R.id.et_filter);
mBtnConnect.setOnClickListener(view -> {
    String filter = etFilter.getText().toString();
    if (!TextUtils.isEmpty(filter)) {
        mManager.setFilterNames(filter);
    } else {
        mManager.setFilterNames((String[]) null);
    }
    if (isBLEEnabled()) {
        if (!mDeviceConnected) {
            showDeviceScanningDialog();
        } else {
            mManager.disconnectDevice();
        }
    } else {
        showBLEDialog();
    }
});
}

private int getValue(EditText view) {
    String value = view.getText().toString();
    if (value.isEmpty()) {
        return 0;
    }
    return Integer.parseInt(value);
}

@Override
protected void initData(Bundle savedInstanceState) {
    isBLESupported();
    if (!isBLEEnabled()) {
        showBLEDialog();
    }

    mFrequency3DMap.put(0, "25HZ");
    mFrequency3DMap.put(1, "50HZ");
    mFrequency3DMap.put(2, "100HZ");
    mFrequency3DMap.put(3, "200HZ");
    mFrequency3DMap.put(4, "400HZ");

    mFrequency6DMap.put(0, "26HZ");
    mFrequency6DMap.put(1, "52HZ");
    mFrequency6DMap.put(2, "104HZ");
    mFrequency6DMap.put(3, "208HZ");

```

```

mManager.setManagerCallbacks(this);
mManager.addAccelerometerCallback((device, x, y, z) -> {
    runOnUiThread(() ->
mTvAccelerometer.setText(getString(R.string.accelerometer, x, y, z)));
});
mManager.addHeartRateStatusCallback((device, min, max, goal) -> {
    runOnUiThread(() -> mTvHRStatus.setText("HR Status Min:" + min + "
Max:" + max + " Goal:" + goal));
});
mManager.setCustomDataReceivedCallback((device, data) -> {
    runOnUiThread(() -> mTvReceivedData.setText("Received data:" +
HexUtil.bytes2HexString(data)));
});
mManager.addHeartRateAlarmCallback((device, stamp, enabled) -> {
    runOnUiThread(() -> {
        String status = getAlarm(enabled);
        mTvHRAAlertStatus.setText("HR Alarm:" + status + " \n(" +
mDateFormat.format(new Date(stamp)) + ")");
    });
});
mManager.addHeartRateMaxCallback((device, max) -> runOnUiThread(() ->
mTvHRMax.setText("HeartRate Max:" + max)));
mManager.addSensor3DFrequencyCallback((device, frequency) ->
runOnUiThread(() -> mTv3DFrequency.setText("3D Frequency:" +
mFrequency3DMap.get(frequency))));
mManager.addSensor3DStatusCallback((device, enabled) -> runOnUiThread(()
-> mTv3DStatus.setText("3D Status:" + (enabled ? "Enabled" : "Disabled"))));
mManager.addBodyHealthCallback(new BodyHealthCallback() {
    @Override
    public void onHealthReceived(@NonNull BluetoothDevice device, int
vo2Max, int breathRate, int emotionLevel, int stressPercent, int stamina, float
tp, float lf, float hf) {
        runOnUiThread(() -> mTvHealth.setText("Health vo2Max:" + vo2Max
+ " breathRate:" + breathRate + " emotionLevel:" + getEmotion(emotionLevel) +
" stressPercent:" + stressPercent + "% stamina:" +
getStamina(stamina) + " \nTP:" + tp + " LF:" + lf + " HF:" + hf));
    }
});
mManager.addSensor6DFrequencyCallback(new Sensor6DFrequencyCallback() {
    @Override
    public void onSensor6DFrequencyReceived(@NonNull BluetoothDevice
device, int frequency) {
        runOnUiThread(() -> mTv6DFrequency.setText("6D Frequency:" +
mFrequency6DMap.get(frequency)));
    }
});
mManager.addSensor6DRawDataCallback(new Sensor6DRawDataCallback() {
    @Override
    public void onSensor6DRawDataReceived(@NonNull BluetoothDevice
device, long utc, int sequence, int gyroscopeX, int gyroscopeY, int gyroscopeZ,
int accelerometerX, int accelerometerY, int accelerometerZ) {
        runOnUiThread(() -> {
            //UTC 0xFF:not supported timestamp

```

```

        String time = utc != 0xFF ? "\nUTC:" +
mDateFormat.format(new Date(utc)) + "(" + utc + ")" : "";
        mTv6DRawData.setText("Sensor:" + time + "\nSequence:" +
sequence + "\nGyroscopeX:" + gyroscopeX + "\nGyroscopeY:" + gyroscopeY +
"\nGyroscopeZ:" + gyroscopeZ
        + "\nAccelerometerX:" + accelerometerX +
"\nAccelerometerY:" + accelerometerY + "\nAccelerometerZ:" + accelerometerZ);
    });
    }
});
}

private void launchHistory(int type) {
    Intent history = new Intent(this, HistoryActivity.class);
    history.putExtra(HistoryActivity.EXTRA_HISTORY, type);
    startActivity(history);
}

private void showDeviceScanningDialog() {
    if (isLocationEnabled(this)) {
        XXPermissions.with(this)
            .permission(getPermissions())
            .request(new OnPermissionCallback() {
                @Override
                public void onGranted(@NonNull List<String> permissions,
boolean allGranted) {
                    if (allGranted) {
                        runOnUiThread(() -> {
                            final ScannerFragment dialog =
ScannerFragment.getInstance();
                            dialog.show(getSupportFragmentManager(),
"scan_fragment");
                        });
                    } else {
                        showToast("permission is denied");
                    }
                }
            })

        @Override
        public void onDenied(@NonNull List<String> permissions,
boolean doNotAskAgain) {
            if (doNotAskAgain) {
                new AlertDialog.Builder(MainActivity.this)

                .setTitle(getString(R.string.permission_required))

                .setMessage(getString(R.string.permission_location_info))

                .setPositiveButton(getString(R.string.yes), (dialog, which) -> {
                    onPermissionSettings();
                })

                .setNegativeButton(getString(R.string.no), null)
                    .show();
            }
        }
    }
}

```

```

        } else {
            showToast("permission is denied");
        }
    }
});
} else {
    new AlertDialog.Builder(this)
        .setTitle(getString(R.string.location_permission_title))
        .setMessage(getString(R.string.location_permission_info))
        .setPositiveButton("OK", (dialog, which) -> {
            onEnableLocation();
        })
        .setNegativeButton("Cancel", null)
        .show();
}
}

private void defaultUI() {
    mTvDeviceName.setText("Device name");
    mTvVersion.setText("Version:--");
    mTvRssi.setText("Rssi:--");
    mTvBattery.setText("Battery:--");
    mTvSport.setText("Sport:--");
    mTvAccelerometer.setText("Accelerometer:--");
    mTvHeartRate.setText("Heart Rate:--");
    mTvHRStatus.setText("HR Status Min:--");
    mTvHRAAlertStatus.setText("HR Alarm:--");
    mTvHRMax.setText("HeartRate Max:--");
    mTv3DFrequency.setText("3D Frequency:--");
    mTv3DStatus.setText("3D Status:--");
    mTvHealth.setText("Health:--");
    mTv6DFrequency.setText("6D Frequency:--");
    mTv6DRawData.setText("6D RawData:--");
    mBtnConnect.setText(getString(R.string.action_connect));
}

private String get3DStatus(boolean enabled) {
    return enabled ? "Enabled 3D" : "Disabled 3D";
}

private String getAlarm(boolean enabled) {
    return enabled ? "Alarm By Age" : "Alarm By High-Low";
}

@Override
public void onDeviceSelected(BluetoothDevice device, String name) {
    mManager.connectDevice(device);
    mTvDeviceName.setText(getString(R.string.device_name, name));
}

@Override
public void onError(@NonNull BluetoothDevice device, @NonNull String
message, int errorCode) {
    Timber.e("onError: (" + errorCode + ")");
}

```

```

    }

    @Override
    public void onDeviceNotSupported(@NonNull BluetoothDevice device) {
        showToast(getString(R.string.not_supported));
    }

    @Override
    public void onSoftwareVersion(@NonNull BluetoothDevice device, String
software) {
        runOnUiThread(() -> mTvVersion.setText("Software Version:" + software));
    }

    @Override
    public void onRssiRead(@NonNull BluetoothDevice device, int rssi) {
        runOnUiThread(() -> mTvRssi.setText("Rssi:" + rssi + "dBm"));
    }

    @Override
    public void onBatteryLevelChanged(@NonNull final BluetoothDevice device,
final int batteryLevel) {
        runOnUiThread(() -> mTvBattery.setText(getString(R.string.battery,
batteryLevel)));
    }

    @Override
    public void onHeartRateMeasurementReceived(@NonNull BluetoothDevice device,
int heartRate, @Nullable Boolean contactDetected, @Nullable Integer
energyExpanded, @Nullable List<Integer> rrIntervals) {
        runOnUiThread(() -> {
            mTvHeartRate.setText(getString(R.string.heart_rate,
heartRate));

            if (rrIntervals != null) {
                Timber.e("rrIntervals:%s", rrIntervals.toString());
            }
        });
    }

    @Override
    public void onSportReceived(@NonNull BluetoothDevice device, int step, int
distance, int calorie) {
        runOnUiThread(() -> mTvSport.setText(getString(R.string.sport, step,
distance / 100f, calorie / 10f)));
    }

    @Override
    public void onDeviceConnected(@NonNull BluetoothDevice device) {
        mDeviceConnected = true;
        runOnUiThread(() -> mBtnConnect.setText(R.string.action_disconnect));
    }

    @Override
    public void onDeviceDisconnected(@NonNull final BluetoothDevice device) {

```



```

        runOnUiThread(() -> defaultUI());
        mDeviceConnected = false;
        mManager.close();
    }

    @Override
    public void onLinkLossOccurred(@NonNull BluetoothDevice device) {
        runOnUiThread(() -> defaultUI());
        mDeviceConnected = false;
        mManager.close();
    }

    @Override
    public void onBackPressed() {
        mManager.disconnectDevice();
        super.onBackPressed();
    }
}

```

```

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample
// File: ScannerFragment.java

```

```

package com.chileaf.cl831.sample;

import android.Manifest;
import android.app.Dialog;
import android.bluetooth.BluetoothAdapter;
import android.bluetooth.BluetoothDevice;
import android.bluetooth.BluetoothManager;
import android.content.Context;
import android.content.DialogInterface;
import android.content.pm.PackageManager;
import android.os.Bundle;
import android.os.Handler;
import android.text.TextUtils;
import android.view.Gravity;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.BaseAdapter;
import android.widget.Button;
import android.widget.ListView;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AlertDialog;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
import androidx.fragment.app.DialogFragment;

```

```

import com.android.chileaf.WearManager;
import no.nordicsemi.android.support.v18.scanner.ScanResult;
import com.android.chileaf.fitness.common.FilterScanCallback;

import java.util.ArrayList;
import java.util.List;
import java.util.Set;

import timber.log.Timber;

/**
 * ScannerFragment class scan required BLE devices and shows them in a list.
 * This class scans and filter
 * * devices with standard BLE Service UUID and devices with custom BLE Service
 * UUID. It contains a
 * * list and a button to scan/cancel. There is a interface {@link
 * OnDeviceSelectedListener} which is
 * * implemented by activity in order to receive selected device. The scanning
 * will continue to scan
 * * for 5 seconds and then stop.
 */
public class ScannerFragment extends DialogFragment {

    private static final long SCAN_DURATION = 15000;
    private static final int REQUEST_PERMISSION_REQ_CODE = 34;
    // private static final String[] FILTER_NAMES = new String[]{"CL831",
    "CL833", "CL880", "Buff"};
    // private static final String[] FILTER_NAMES = null;

    private BluetoothAdapter mBluetoothAdapter;
    private OnDeviceSelectedListener mListener;
    private DeviceListAdapter mAdapter;
    private final Handler mHandler = new Handler();

    private Button mScanButton;

    private View mPermissionRationale;

    private boolean mIsScanning = false;
    private WearManager mManager;

    public static ScannerFragment getInstance() {
        final ScannerFragment fragment = new ScannerFragment();
        final Bundle args = new Bundle();
        fragment.setArguments(args);
        return fragment;
    }

    /**
     * Interface required to be implemented by activity.
     */
    public interface OnDeviceSelectedListener {
        /**

```

```

        * Fired when user selected the device.
        *
        * @param device the device to connect to
        * @param name the device name. Unfortunately on some devices {@link
BluetoothDevice#getName()}
        * always returns <code>null</code>, i.e. Sony Xperia Z1
(C6903) with Android 4.3.
        * The name has to be parsed manually form the
Advertisement packet.
    */
    void onDeviceSelected(final BluetoothDevice device, final String name);

    /**
     * Fired when scanner dialog has been cancelled without selecting a
device.
    */
    default void onDialogCanceled() {
    }

    /**
     * This will make sure that {@link OnDeviceSelectedListener} interface is
implemented by activity.
    */
    @Override
    public void onAttach(final Context context) {
        super.onAttach(context);
        try {
            this.mListener = (OnDeviceSelectedListener) context;
        } catch (final ClassCastException e) {
            throw new ClassCastException(context.toString() + " must implement
OnDeviceSelectedListener");
        }
    }

    @Override
    public void onCreate(final Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        final BluetoothManager manager = (BluetoothManager)
requireContext().getSystemService(Context.BLUETOOTH_SERVICE);
        if (manager != null) {
            mBluetoothAdapter = manager.getAdapter();
        }
    }

    @Override
    public void onDestroyView() {
        stopScan();
        super.onDestroyView();
    }

    @NonNull
    @Override

```

```

    public Dialog onCreateDialog(final Bundle savedInstanceState) {
        mManager = WearManager.getInstance(requireContext());
        final AlertDialog.Builder builder = new
AlertDialog.Builder(requireContext());
        final View dialogView =
LayoutInflater.from(getActivity()).inflate(R.layout.fragment_device_scan, null);
        final ListView listview = dialogView.findViewById(android.R.id.list);

        listview.setEmptyView(dialogView.findViewById(android.R.id.empty));
        listview.setAdapter(mAdapter = new DeviceListAdapter(getActivity()));

        builder.setTitle(R.string.scanner_title);
        final AlertDialog dialog = builder.setView(dialogView).create();
        listview.setOnItemClickListener((parent, view, position, id) -> {
            stopScan();
            dialog.dismiss();
            final ExtendedBluetoothDevice d = (ExtendedBluetoothDevice)
mAdapter.getItem(position);
            mListener.onDeviceSelected(d.device, d.name);
        });

        mPermissionRationale =
dialogView.findViewById(R.id.permission_rationale); // this is not null only on
API23+

        mScanButton = dialogView.findViewById(R.id.action_cancel);
        mScanButton.setOnClickListener(v -> {
            if (v.getId() == R.id.action_cancel) {
                if (mIsScanning) {
                    dialog.cancel();
                } else {
                    startScan();
                }
            }
        });

        addBoundDevices();
        if (savedInstanceState == null)
            startScan();
        return dialog;
    }

    @Override
    public void onCancel(DialogInterface dialog) {
        super.onCancel(dialog);
        mListener.onDialogCanceled();
    }

    @Override
    public void onRequestPermissionsResult(final int requestCode, final @NonNull
String[] permissions, final @NonNull int[] grantResults) {
        switch (requestCode) {
            case REQUEST_PERMISSION_REQ_CODE: {
                if (grantResults[0] == PackageManager.PERMISSION_GRANTED) {

```

```

        // We have been granted the
Manifest.permission.ACCESS_COARSE_LOCATION permission. Now we may proceed with
scanning.
        startScan();
    } else {
        mPermissionRationale.setVisibility(View.VISIBLE);
        Toast.makeText(getActivity(),
R.string.no_required_permission, Toast.LENGTH_SHORT).show();
    }
    break;
}
}
}

/**
 * Scan for 5 seconds and then stop scanning when a BluetoothLE device is
found then mLEScanCallback
 * is activated This will perform regular scan for custom BLE Service UUID
and then filter out.
 * using class ScannerServiceParser
 */
private void startScan() {
    // Since Android 6.0 we need to obtain either
Manifest.permission.ACCESS_COARSE_LOCATION or
Manifest.permission.ACCESS_FINE_LOCATION to be able to scan for
    // Bluetooth LE devices. This is related to beacons as proximity
devices.
    // On API older than Marshmallow the following code does nothing.
    if (ContextCompat.checkSelfPermission(requireContext(),
Manifest.permission.ACCESS_COARSE_LOCATION) !=
PackageManager.PERMISSION_GRANTED) {
        // When user pressed Deny and still wants to use this functionality,
show the rationale
        if
(ActivityCompat.shouldShowRequestPermissionRationale(requireActivity(),
Manifest.permission.ACCESS_COARSE_LOCATION) &&
mPermissionRationale.getVisibility() == View.GONE) {
            mPermissionRationale.setVisibility(View.VISIBLE);
            return;
        }

        requestPermissions(new
String[]{Manifest.permission.ACCESS_COARSE_LOCATION},
REQUEST_PERMISSION_REQ_CODE);
        return;
    }

    // Hide the rationale message, we don't need it anymore.
    if (mPermissionRationale != null)
        mPermissionRationale.setVisibility(View.GONE);

    mAdapter.clearDevices();
    mScanButton.setText(R.string.scanner_action_cancel);
}

```

```

        // mManager.setFilterNames(FILTER_NAMES);
        mManager.startScan(mScanCallback);

        mIsScanning = true;
        mHandler.postDelayed(() -> {
            if (mIsScanning) {
                stopScan();
            }
        }, SCAN_DURATION);
    }

    /**
     * Stop scan if user tap Cancel button
     */
    private void stopScan() {
        if (mIsScanning) {
            mScanButton.setText(R.string.scanner_action_scan);
            WearManager.getInstance(getActivity()).stopScan();
            mIsScanning = false;
        }
    }

    private FilterScanCallback mScanCallback = new FilterScanCallback() {
        @Override
        public void onFilterScanResults(@NonNull List<ScanResult> results) {
            mAdapter.update(results);
        }
    };

    private void addBoundDevices() {
        final Set<BluetoothDevice> devices =
mBluetoothAdapter.getBondedDevices();
        mAdapter.addBondedDevices(devices);
    }

    private static class DeviceListAdapter extends BaseAdapter {
        private static final int TYPE_TITLE = 0;
        private static final int TYPE_ITEM = 1;
        private static final int TYPE_EMPTY = 2;

        private final ArrayList<ExtendedBluetoothDevice> mListBondedValues = new
ArrayList<>();
        private final ArrayList<ExtendedBluetoothDevice> mListValues = new
ArrayList<>();
        private final Context mContext;

        public DeviceListAdapter(final Context context) {
            mContext = context;
        }

        /**
         * Sets a list of bonded devices.
         *
         * @param devices list of bonded devices.

```

```

        */
        private void addBondedDevices(final Set<BluetoothDevice> devices) {
            final List<ExtendedBluetoothDevice> bondedDevices =
mListBondedValues;
            for (BluetoothDevice device : devices) {
//                if (matchDeviceName(device.getName())) {
//                    bondedDevices.add(new ExtendedBluetoothDevice(device));
//                }
            }
            notifyDataSetChanged();
        }

//        private boolean matchDeviceName(String name) {
//            if (FILTER_NAMES == null) {
//                return true;
//            } else {
//                if (name != null && !TextUtils.isEmpty(name)) {
//                    for (String filterName : FILTER_NAMES) {
//                        if (name.toUpperCase().startsWith(filterName)) {
//                            return true;
//                        }
//                    }
//                }
//            }
//            return false;
//        }
//    }

/**
 * Updates the list of not bonded devices.
 *
 * @param results list of results from the scanner
 */
private void update(final List<ScanResult> results) {
    for (final ScanResult result : results) {
        Timber.e(result.toString());
        final ExtendedBluetoothDevice device = findDevice(result);
        if (device == null) {
            mListValues.add(new ExtendedBluetoothDevice(result));
        } else if (result.getScanRecord() != null) {
            device.name = result.getScanRecord().getDeviceName();
            device.rssi = result.getRssi();
        }
    }
    notifyDataSetChanged();
}

private ExtendedBluetoothDevice findDevice(final ScanResult result) {
    for (final ExtendedBluetoothDevice device : mListBondedValues)
        if (device.matches(result))
            return device;
    for (final ExtendedBluetoothDevice device : mListValues)
        if (device.matches(result))
            return device;
    return null;
}

```

```

    }

    private void clearDevices() {
        mListValues.clear();
        notifyDataSetChanged();
    }

    @Override
    public int getCount() {
        final int bondedCount = mListBondedValues.size() + 1; // 1 for the
title        final int availableCount = mListValues.isEmpty() ? 2 :
mListValues.size() + 1; // 1 for title, 1 for empty text
        if (bondedCount == 1)
            return availableCount;
        return bondedCount + availableCount;
    }

    @Override
    public Object getItem(int position) {
        final int bondedCount = mListBondedValues.size() + 1; // 1 for the
title        if (mListBondedValues.isEmpty()) {
            if (position == 0)
                return R.string.scanner_subtitle_not_bonded;
            else
                return mListValues.get(position - 1);
        } else {
            if (position == 0)
                return R.string.scanner_subtitle_bonded;
            if (position < bondedCount)
                return mListBondedValues.get(position - 1);
            if (position == bondedCount)
                return R.string.scanner_subtitle_not_bonded;
            return mListValues.get(position - bondedCount - 1);
        }
    }

    @Override
    public int getViewTypeCount() {
        return 3;
    }

    @Override
    public boolean areAllItemsEnabled() {
        return false;
    }

    @Override
    public boolean isEnabled(int position) {
        return getItemViewType(position) == TYPE_ITEM;
    }

    @Override

```



```

public int getItemViewType(int position) {
    if (position == 0)
        return TYPE_TITLE;

    if (!mListBondedValues.isEmpty() && position ==
mListBondedValues.size() + 1)
        return TYPE_TITLE;

    if (position == getCount() - 1 && mListValues.isEmpty())
        return TYPE_EMPTY;

    return TYPE_ITEM;
}

@Override
public long getItemId(int position) {
    return position;
}

@Override
public View getView(int position, View oldView, ViewGroup parent) {
    final LayoutInflater inflater = LayoutInflater.from(mContext);
    final int type = getItemViewType(position);

    View view = oldView;
    switch (type) {
        case TYPE_EMPTY:
            if (view == null) {
                view = new TextView(mContext);
                final TextView empty = (TextView) view;
                empty.setGravity(Gravity.CENTER_HORIZONTAL);

empty.setText(mContext.getString(R.string.scanner_empty));
            }
            break;
        case TYPE_TITLE:
            if (view == null) {
                view = new TextView(mContext);
            }
            final TextView title = (TextView) view;
            title.setGravity(Gravity.CENTER_HORIZONTAL);
            title.setText((Integer) getItem(position));
            break;
        default:
            if (view == null) {
                view = inflater.inflate(R.layout.item_device_list,
parent, false);

                final ViewHolder holder = new ViewHolder();
                holder.name = view.findViewById(R.id.name);
                holder.address = view.findViewById(R.id.address);
                holder.signal = view.findViewById(R.id.rssi);
                view.setTag(holder);
            }
    }
}

```

```

        final ExtendedBluetoothDevice device =
(ExtendedBluetoothDevice) getItem(position);
        final ViewHolder holder = (ViewHolder) view.getTag();
        final String name = device.name;
        holder.name.setText(name != null ? name :
mContext.getString(R.string.not_available));
        holder.address.setText(device.device.getAddress());
        if (!device.isBonded || device.rssi !=
ExtendedBluetoothDevice.NO_RSSI) {
            holder.signal.setText(device.rssi + "dBm");
            holder.signal.setVisibility(View.VISIBLE);
        } else {
            holder.signal.setVisibility(View.GONE);
        }
        break;
    }
    return view;
}

private class ViewHolder {
    private TextView name;
    private TextView address;
    private TextView signal;
}
}

```

```

private static class ExtendedBluetoothDevice {

    private static final int NO_RSSI = -1000;

    private String name;
    private int rssi;
    private boolean isBonded;
    private final BluetoothDevice device;

    private ExtendedBluetoothDevice(final ScanResult scanResult) {
        this.device = scanResult.getDevice();
        this.name = scanResult.getScanRecord() != null ?
scanResult.getScanRecord().getDeviceName() : null;
        this.rssi = scanResult.getRssi();
        this.isBonded = false;
    }

    private ExtendedBluetoothDevice(final BluetoothDevice device) {
        this.device = device;
        this.name = device.getName();
        this.rssi = NO_RSSI;
        this.isBonded = true;
    }

    private boolean matches(final ScanResult scanResult) {
        return
device.getAddress().equals(scanResult.getDevice().getAddress());
    }
}

```

```
}  
}
```

```
// Directory: app  
// Subdirectory: src\main\java\com\chileaf\cl831\sample  
// File: Sleep.java
```

```
package com.chileaf.cl831.sample;
```

```
public class Sleep {  
    private String date;  
    private int notSleepingTime;  
    private int lightSleepTime;  
    private int deepSleepTime;  
    private long totalTime;  
    private int index;  
  
    public Sleep(String date, int notSleepingTime, int lightSleepTime, int  
deepSleepTime, long totalTime, int index) {  
        this.date = date;  
        this.notSleepingTime = notSleepingTime;  
        this.lightSleepTime = lightSleepTime;  
        this.deepSleepTime = deepSleepTime;  
        this.totalTime = totalTime;  
        this.index = index;  
    }  
  
    public Sleep() {  
    }  
  
    public String getDate() {  
        return date;  
    }  
  
    public void setDate(String date) {  
        this.date = date;  
    }  
  
    public int getNotSleepingTime() {  
        return notSleepingTime;  
    }  
  
    public void setNotSleepingTime(int notSleepingTime) {  
        this.notSleepingTime = notSleepingTime;  
    }  
  
    public int getLightSleepTime() {  
        return lightSleepTime;  
    }  
  
    public void setLightSleepTime(int lightSleepTime) {  
        this.lightSleepTime = lightSleepTime;  
    }  
}
```

```

    public int getDeepSleepTime() {
        return deepSleepTime;
    }

    public void setDeepSleepTime(int deepSleepTime) {
        this.deepSleepTime = deepSleepTime;
    }

    public long getTotalTime() {
        return totalTime;
    }

    public void setTotalTime(long totalTime) {
        this.totalTime = totalTime;
    }

    public int getIndex() {
        return index;
    }

    public void setIndex(int index) {
        this.index = index;
    }

    @Override
    public String toString() {
        return
            "["+date+"]{"+
            "\nnot Sleeping Time = " + notSleepingTime +
            "(minute)\nlight Sleep Time = " + lightSleepTime +
            "(minute)\ndeep Sleep Time = " + deepSleepTime +
            "(minute)\ntotal Time = " + totalTime+"(minute)}";
    }
}

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample
// File: SportHealthActivity.java

package com.chileaf.cl831.sample;

import android.bluetooth.BluetoothDevice;
import android.os.Bundle;

import androidx.annotation.NonNull;
import androidx.appcompat.widget.AppCompatActivity;

import com.android.chileaf.fitness.callback.BodySportHealthCallback;

public class SportHealthActivity extends AppCompatActivity implements
BodySportHealthCallback {

```

```

private AppCompatActivity mTvSportHealth;

@Override
protected int layoutId() {
    return R.layout.activity_sport_health;
}

@Override
protected void initView() {
    setTitle("Sport health");
    mTvSportHealth = findViewById(R.id.tv_sport_health);
}

@Override
protected void initData(Bundle savedInstanceState) {
    mManager.addBodySportHealthCallback(this);
}

@Override
public void onSportHealthReceived(@NonNull BluetoothDevice device, int
vo2Max, int breathRate, int emotion, int pressure, int stamina) {
    runOnUiThread(() ->
        mTvSportHealth.setText("Sport health \nvo2Max:" + vo2Max +
"\nbreathRate:" + breathRate + "\nemotion:" +
        getEmotion(emotion) + "\npressure:" + pressure +
"%\nstamina:" + getStamina(stamina)));
    }
}

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample
// File: TemperatureActivity.java

package com.chileaf.cl831.sample;

import android.bluetooth.BluetoothDevice;
import android.os.Bundle;
import android.widget.EditText;

import androidx.annotation.NonNull;
import androidx.appcompat.widget.AppCompatActivity;

import com.android.chileaf.fitness.callback.TemperatureCallback;

public class TemperatureActivity extends AppCompatActivity implements
TemperatureCallback {

    private AppCompatActivity text1;
    private AppCompatActivity text2;
    private AppCompatActivity text3;

    @Override
    protected int layoutId() {
        return R.layout.activity_temperature;
    }

```

```

    }

    @Override
    protected void initView() {
        text1 = findViewById(R.id.tv_environment);
        text2 = findViewById(R.id.tv_wrist_temperature);
        text3 = findViewById(R.id.tv_temperature);
    }

    @Override
    protected void initData(Bundle savedInstanceState) {
        mManager.addTemperatureCallback(this);
    }

    @Override
    public void onTemperatureReceived(@NonNull BluetoothDevice device, float
environment, float wrist, float body) {
        runOnUiThread(() -> {
            text1.setText(environment + "");
            text2.setText(wrist + "");
            text3.setText(body + "");
        });
    }
}

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample
// File: UserInfoActivity.java

package com.chileaf.cl831.sample;

import android.bluetooth.BluetoothDevice;
import android.os.Bundle;
import android.text.Editable;
import android.text.TextUtils;
import android.text.TextWatcher;
import android.widget.EditText;
import android.widget.RadioButton;
import android.widget.RadioGroup;

import androidx.annotation.NonNull;

import com.android.chileaf.fitness.callback.UserInfoCallback;

/**
 * User information
 */
public class UserInfoActivity extends BaseActivity implements UserInfoCallback {

    private EditText mEtAge;
    private EditText mEtHeight;
    private EditText mEtWeight;
    private EditText mEtUserId;
    private RadioGroup mRgSex;

```

```

private RadioButton mRbMale;
private RadioButton mRbFemale;

private int mAge;//age
private int mSex;//sex
private int mHeight;//height
private int mWeight;//weigh
private long mUserId;//user id (phone number)

@Override
protected int layoutId() {
    return R.layout.activity_user_info;
}

@Override
protected void initView() {
    mEtAge = findViewById(R.id.et_age);
    mEtHeight = findViewById(R.id.et_height);
    mEtWeight = findViewById(R.id.et_weight);
    mEtUserId = findViewById(R.id.et_user_id);
    mRgSex = findViewById(R.id.rg_sex);
    mRbMale = findViewById(R.id.rb_male);
    mRbFemale = findViewById(R.id.rb_female);

    mEtAge.addTextChangedListener(new SimpleTextWatcher() {
        @Override
        public void onTextChanged(CharSequence s, int start, int before, int
count) {
            if (!TextUtils.isEmpty(s)) {
                mAge = Integer.parseInt(s.toString());
            }
        }
    });
    mRgSex.setOnCheckedChangeListener((group, checkedId) -> mSex = checkedId
== R.id.rb_male ? 1 : 0);
    mEtHeight.addTextChangedListener(new SimpleTextWatcher() {
        @Override
        public void onTextChanged(CharSequence s, int start, int before, int
count) {
            if (!TextUtils.isEmpty(s)) {
                mHeight = Integer.parseInt(s.toString());
            }
        }
    });
    mEtWeight.addTextChangedListener(new SimpleTextWatcher() {
        @Override
        public void onTextChanged(CharSequence s, int start, int before, int
count) {
            if (!TextUtils.isEmpty(s)) {
                mWeight = Integer.parseInt(s.toString());
            }
        }
    });
}

```

```

    });

    mEtUserId.addTextChangedListener(new SimpleTextWatcher() {
        @Override
        public void onTextChanged(CharSequence s, int start, int before, int
count) {
            if (!TextUtils.isEmpty(s)) {
                mUserId = Long.parseLong(s.toString());
            }
        }
    });
    //get user information
    findViewById(R.id.btn_get_user).setOnClickListener(view ->
mManager.getUserInfo());
    //set user information
    findViewById(R.id.btn_set_user).setOnClickListener(view -> {
        if (isEmpty(mEtAge, mAge)) {
            showToast("Please input the correct age");
        } else if (isEmpty(mEtHeight, mHeight)) {
            showToast("Please input the correct height");
        } else if (isEmpty(mEtWeight, mWeight)) {
            showToast("Please input the correct weigh");
        } else if (isEmpty(mEtUserId, mUserId)) {
            showToast("Please input the correct user id");
        } else {
            mManager.setUserInfo(mAge, mSex, mWeight, mHeight, mUserId);
            showToast("Set success");
        }
    });
}

@Override
protected void initData(Bundle savedInstanceState) {
    setTitle("User information");
    mManager.addUserInfoCallback(this);
}

private boolean isEmpty(EditText view, long value) {
    return view.getText().toString().isEmpty() || value == 0;
}

@Override
public void onUserInfoReceived(@NonNull BluetoothDevice device, int age, int
sex, int weight, int height, long userId) {
    runOnUiThread(() -> {
        mEtAge.setText(String.valueOf(age));
        mEtHeight.setText(String.valueOf(height));
        mEtWeight.setText(String.valueOf(weight));
        mEtUserId.setText(String.valueOf(userId));
        mRbMale.setChecked(sex == 1);
        mRbFemale.setChecked(sex == 0);
    });
}

```



```

        private abstract static class SimpleTextWatcher implements TextWatcher {

            @Override
            public void beforeTextChanged(CharSequence s, int start, int count, int
after) {

                }

            @Override
            public void afterTextChanged(Editable s) {

                }

        }

    }

```

```

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample\dfu
// File: DfuActivity.java

```

```

package com.chileaf.cl831.sample.dfu;

```

```

import android.annotation.SuppressLint;
import android.app.Activity;
import android.app.ActivityManager;
import android.app.ActivityManager.RunningServiceInfo;
import android.app.NotificationManager;
import android.bluetooth.BluetoothDevice;
import android.content.Context;
import android.content.Intent;
import android.content.SharedPreferences;
import android.database.Cursor;
import android.net.Uri;
import android.os.Build;
import android.os.Bundle;
import android.os.Handler;
import android.preference.PreferenceManager;
import android.provider.MediaStore;
import android.text.TextUtils;
import android.view.View;
import android.webkit.MimeTypeMap;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;

import androidx.activity.result.ActivityResultCallback;
import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContract;
import androidx.annotation.NonNull;
import androidx.localbroadcastmanager.content.LocalBroadcastManager;

import com.chileaf.cl831.sample.BaseActivity;

```

```

import com.chileaf.cl831.sample.R;
import com.hjq.permissions.OnPermissionCallback;
import com.hjq.permissions.Permission;
import com.hjq.permissions.XXPermissions;

import org.jetbrains.annotations.Nullable;

import java.io.File;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

import no.nordicsemi.android.dfu.DfuProgressListener;
import no.nordicsemi.android.dfu.DfuProgressListenerAdapter;
import no.nordicsemi.android.dfu.DfuServiceInitiator;
import no.nordicsemi.android.dfu.DfuServiceListenerHelper;
import no.nordicsemi.android.support.v18.scanner.BluetoothLeScannerCompat;
import no.nordicsemi.android.support.v18.scanner.ScanCallback;
import no.nordicsemi.android.support.v18.scanner.ScanFilter;
import no.nordicsemi.android.support.v18.scanner.ScanResult;
import no.nordicsemi.android.support.v18.scanner.ScanSettings;
import timber.log.Timber;

@SuppressLint("MissingPermission")
public class DfuActivity extends BaseActivity implements
UploadCancelFragment.CancelFragmentListener {

    private static final String TAG = "DfuActivity";

    private static final String PREFS_PREFIX = "com.chileaf.dfu";

    private static final String PREFS_FILE_NAME = PREFS_PREFIX +
".PREFS_FILE_NAME";
    private static final String PREFS_FILE_TYPE = PREFS_PREFIX +
".PREFS_FILE_TYPE";
    private static final String PREFS_FILE_SIZE = PREFS_PREFIX +
".PREFS_FILE_SIZE";

    private static final String DATA_STATUS = "status";
    private static final String DATA_DFU_COMPLETED = "dfu_completed";
    private static final String DATA_DFU_ERROR = "dfu_error";

    private TextView mFileNameView;
    private TextView mFileSizeView;
    private TextView mFileStatusView;
    private TextView mTextPercentage;
    private TextView mTextUploading;

    private Button mSelectFileButton;
    private Button mUploadButton;

    private Uri mFileUri;
    private boolean mStatusOk;
/**

```

```

        * Flag set to true in {@link #onRestart()} and to false in {@link
#onPause()}.
        */
        private boolean mResumed;
        /**
        * Flag set to true if DFU operation was completed while {@link #mResumed}
was false.
        */
        private boolean mDfuCompleted;
        /**
        * The error message received from DFU service while {@link #mResumed} was
false.
        */
        private String mDfuError;

        private final ScanCallback mScanCallback = new ScanCallback() {

            @Override
            public void onBatchScanResults(final List<ScanResult> results) {
                for (ScanResult result : results) {
                    BluetoothDevice device = result.getDevice();
                    if (isDfuDevice(device.getName())) {
                        showToast("Start Update...");
                        dfuUpdated(device);
                        break;
                    }
                }
            }
        };

        private static class ContentResultContract extends
ActivityResultContract<String, Uri> {
            @NonNull
            @Override
            public Intent createIntent(@NonNull Context context, String type) {
                return new Intent(Intent.ACTION_GET_CONTENT)
                    .addCategory(Intent.CATEGORY_OPENABLE)
                    .setType(type);
            }

            @Override
            public Uri parseResult(int resultCode, @Nullable Intent intent) {
                if (resultCode == Activity.RESULT_OK && intent != null) {
                    return intent.getData();
                }
                return null;
            }
        }

        private final ActivityResultLauncher<String> zipLauncher =
registerForActivityResult(new ContentResultContract(), new
ActivityResultCallback<Uri>() {
            @Override
            public void onActivityResult(Uri uri) {

```

```

        if (uri == null) {
            showToast("Uri is null");
        } else {
            if ("content".equalsIgnoreCase(uri.getScheme())) {
                Cursor query = getContentResolver().query(uri, null, null,
null, null);
                if (query != null && query.moveToNext()) {
                    int displayNameIndex =
query.getColumnIndex(MediaStore.MediaColumns.DISPLAY_NAME);
                    int fileSizeIndex =
query.getColumnIndex(MediaStore.MediaColumns.SIZE);
                    String fileName = query.getString(displayNameIndex);
                    int fileSize = query.getInt(fileSizeIndex);
                    updateFileInfo(uri, fileName, fileSize,
DfuService.TYPE_AUTO);
                    query.close();
                } else if ("file".equalsIgnoreCase(uri.getScheme())) {
                    String filePath = uri.getPath();
                    if (filePath != null) {
                        File file = new File(filePath);
                        updateFileInfo(uri, file.getName(), file.length(),
DfuService.TYPE_AUTO);
                    }
                }
                Timber.i("zipLauncher uri:%s", uri);
            }
        }
    });

    /**
     * The progress listener receives events from the DFU Service.
     * If is registered in onCreate() and unregistered in onDestroy() so methods
here may also be called
     * when the screen is locked or the app went to the background. This is
because the UI needs to have the
     * correct information after user comes back to the activity and this
information can't be read from the service
     * as it might have been killed already (DFU completed or finished with
error).
     */
    private final DfuProgressListener mDfuProgressListener = new
DfuProgressListenerAdapter() {
        @Override
        public void onDeviceConnecting(final String deviceAddress) {
            mTextPercentage.setText("Device Connecting...");
        }

        @Override
        public void onDfuProcessStarting(final String deviceAddress) {
            mTextPercentage.setText("Process Starting...");
        }

        @Override

```

```

public void onEnablingDfuMode(final String deviceAddress) {
    mTextPercentage.setText("Updating...");
}

@Override
public void onFirmwareValidating(final String deviceAddress) {
    mTextPercentage.setText("FirmwareValidating...");
}

@Override
public void onDeviceDisconnecting(final String deviceAddress) {
    mTextPercentage.setText("Device Disconnecting...");
}

@Override
public void onDfuCompleted(final String deviceAddress) {
    mTextPercentage.setText("Update Completed");
    if (mResumed) {
        // let's wait a bit until we cancel the notification. When
canceled immediately it will be recreated by service again.
        new Handler().postDelayed(() -> {
            onTransferCompleted();
            // if this activity is still open and upload process was
completed, cancel the notification
            final NotificationManager manager = (NotificationManager)
getSystemService(Context.NOTIFICATION_SERVICE);
            if (manager != null) {
                manager.cancel(DfuService.NOTIFICATION_ID);
            }
        }, 200);
    } else {
        // Save that the DFU process has finished
        mDfuCompleted = true;
    }
}

@Override
public void onDfuAborted(final String deviceAddress) {
    mTextPercentage.setText("Update Aborted");
    // let's wait a bit until we cancel the notification. When canceled
immediately it will be recreated by service again.
    new Handler().postDelayed(() -> {
        onUploadCanceled();
        // if this activity is still open and upload process was
completed, cancel the notification
        final NotificationManager manager = (NotificationManager)
getSystemService(Context.NOTIFICATION_SERVICE);
        if (manager != null) {
            manager.cancel(DfuService.NOTIFICATION_ID);
        }
    }, 200);
}

@Override

```

```

        public void onProgressChanged(final String deviceAddress, final int
percent, final float speed, final float avgSpeed, final int currentPart, final
int partsTotal) {
            mTextPercentage.setText(getString(R.string.dfu_uploading_percentage,
percent));
            if (partsTotal > 1)
                mTextUploading.setText(String.format(Locale.getDefault(),
"Uploading Progress:%d/%d", currentPart, partsTotal));
            else
                mTextUploading.setText("Updating...");
        }

        @Override
        public void onError(final String deviceAddress, final int error, final
int errorType, final String message) {
            if (mResumed) {
                showErrorMessage(message);
                // We have to wait a bit before canceling notification. This is
called before DfuService creates the last notification.
                new Handler().postDelayed(() -> {
                    // if this activity is still open and upload process was
completed, cancel the notification
                    final NotificationManager manager = (NotificationManager)
getSystemService(Context.NOTIFICATION_SERVICE);
                    if (manager != null) {
                        manager.cancel(DfuService.NOTIFICATION_ID);
                    }
                }, 200);
            } else {
                mDfuError = message;
            }
        }
    };

    @Override
    protected int layoutId() {
        return R.layout.activity_dfu;
    }

    @Override
    public void initView() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            DfuServiceInitiator.createDfuNotificationChannel(this);
        }
        if (!isBLEEnabled()) {
            showBLEDialog();
        }
        defaultUI();
        XXPermissions.with(this)
            .permission(Permission.MANAGE_EXTERNAL_STORAGE)
            .request(new OnPermissionCallback() {
                @Override
                public void onGranted(@NonNull List<String> permissions,
boolean allGranted) {

```

```

        showToast("permission is " + (allGranted ? "granted" :
"denied"));
    }
    });
    DfuServiceListenerHelper.registerProgressListener(this,
mDfuProgressListener);
}

@Override
public void initData(@Nullable Bundle savedInstanceState) {
    if (savedInstanceState != null) {
        mStatusOk = mStatusOk || savedInstanceState.getBoolean(DATA_STATUS);
        mUploadButton.setEnabled(mStatusOk);
        mDfuError = savedInstanceState.getString(DATA_DFU_ERROR);
        mDfuCompleted = savedInstanceState.getBoolean(DATA_DFU_COMPLETED);
    }
}

private boolean isDfuDevice(String name) {
    return name != null && !TextUtils.isEmpty(name) &&
(name.toUpperCase().endsWith("U"));
}

@Override
protected void onDestroy() {
    super.onDestroy();
    DfuServiceListenerHelper.unregisterProgressListener(this,
mDfuProgressListener);
}

@Override
protected void onSaveInstanceState(final Bundle outState) {
    super.onSaveInstanceState(outState);
    outState.putBoolean(DATA_STATUS, mStatusOk);
    outState.putString(DATA_DFU_ERROR, mDfuError);
    outState.putBoolean(DATA_DFU_COMPLETED, mDfuCompleted);
}

private void defaultUI() {
    mFileNameView = findViewById(R.id.file_name);
    mFileSizeView = findViewById(R.id.file_size);
    mFileStatusView = findViewById(R.id.file_status);
    mSelectFileButton = findViewById(R.id.action_select_file);
    mUploadButton = findViewById(R.id.action_upload);
    mTextPercentage = findViewById(R.id.action_progress);
    mTextUploading = findViewById(R.id.action_uploading);

    final SharedPreferences preferences =
PreferenceManager.getDefaultSharedPreferences(this);
    if (isDfuServiceRunning()) {
        // Restore image file information
        mFileNameView.setText(preferences.getString(PREFS_FILE_NAME, ""));
        mFileSizeView.setText(preferences.getString(PREFS_FILE_SIZE, ""));
        mFileStatusView.setText(R.string.dfu_file_status_ok);
    }
}

```

```

        mStatusOk = true;
        showProgressBar();
    }
}

@Override
protected void onResume() {
    super.onResume();
    mResumed = true;
    if (mDfuCompleted)
        onTransferCompleted();
    if (mDfuError != null)
        showErrorMessage(mDfuError);
    if (mDfuCompleted || mDfuError != null) {
        // if this activity is still open and upload process was completed,
cancel the notification
        final NotificationManager manager = (NotificationManager)
getSystemService(Context.NOTIFICATION_SERVICE);
        if (manager != null) {
            manager.cancel(DfuService.NOTIFICATION_ID);
        }
        mDfuCompleted = false;
        mDfuError = null;
    }
}

@Override
protected void onPause() {
    super.onPause();
    mResumed = false;
}

public void startScan(String address) {
    final BluetoothLeScannerCompat scanner =
BluetoothLeScannerCompat.getScanner();
    final ScanSettings settings = new ScanSettings.Builder()
        .setScanMode(ScanSettings.SCAN_MODE_LOW_LATENCY)
        .setUseHardwareBatchingIfSupported(false)
        .setReportDelay(1000)
        .setLegacy(false)
        .build();
    final List<ScanFilter> filters = new ArrayList<>();
    filters.add(new ScanFilter.Builder()
        .setDeviceAddress(address)
        .build());
    scanner.startScan(filters, settings, mScanCallback);
}

/**
 * Stop scan devices
 */
public void stopScan() {
    if (mScanCallback != null) {
        final BluetoothLeScannerCompat scanner =

```



```

BluetoothLeScannerCompat.getScanner();
    scanner.stopScan(mScanCallback);
}
}

/**
 * Updates the file information on UI
 *
 * @param fileName file name
 * @param fileSize file length
 */
private void updateFileInfo(Uri uri, final String fileName, final long
fileSize, final int fileType) {
    mFileUri = uri;
    mFileNameView.setText(fileName);
    mFileSizeView.setText(getString(R.string.dfu_file_size_text, fileSize));
    final String extension = fileType == DfuService.TYPE_AUTO ? "(?i)ZIP" :
"(?i)HEX|BIN"; // (?i) = case insensitive
    final boolean statusOk = mStatusOk =
MimeTypeMap.getFileExtensionFromUrl(fileName).matches(extension);
    mFileStatusView.setText(statusOk ? R.string.dfu_file_status_ok :
R.string.dfu_file_status_invalid);
    mUploadButton.setEnabled(statusOk);
    // Ask the user for the Init packet file if HEX or BIN files are
selected. In case of a ZIP file the Init packets should be included in the ZIP.
    if (statusOk) {
        onUploadClicked(null);
    }
}

public void onSelectFileClicked(final View view) {
    zipLauncher.launch(DfuService.MIME_TYPE_ZIP);
}

/**
 * Callback of UPDATE/CANCEL button on DfuActivity
 */
public void onUploadClicked(final View view) {
    if (isDfuServiceRunning()) {
        showUploadCancelDialog();
        return;
    }
    // Check whether the selected file is a HEX file (we are just checking
the extension)
    if (!mStatusOk) {
        Toast.makeText(this, R.string.dfu_file_status_invalid_message,
Toast.LENGTH_LONG).show();
        return;
    }
    String address = mManager.dfuMode();
    startScan(address);
    showLoadingAutoDismiss(30000L);
}
}

```

```

private void dfuUpdated(BluetoothDevice device) {
    stopScan();
    hideLoading();
    final SharedPreferences preferences =
PreferenceManager.getDefaultSharedPreferences(this);
    final SharedPreferences.Editor editor = preferences.edit();
    editor.putString(PREFS_FILE_NAME, mFileNameView.getText().toString());
    editor.putString(PREFS_FILE_SIZE, mFileSizeView.getText().toString());
    editor.apply();

    showProgressBar();

    final boolean keepBond = false;
    final boolean forceDfu = false;
    final boolean enablePRNs = Build.VERSION.SDK_INT <
Build.VERSION_CODES.M;
    String value = String.valueOf(DfuServiceInitiator.DEFAULT_PRN_VALUE);
    int numberOfPackets;
    try {
        numberOfPackets = Integer.parseInt(value);
    } catch (final NumberFormatException e) {
        numberOfPackets = DfuServiceInitiator.DEFAULT_PRN_VALUE;
    }
    final DfuServiceInitiator starter = new
DfuServiceInitiator(device.getAddress())
        .setUnsafeExperimentalButtonlessServiceInSecureDfuEnabled(true)
        .setPacketsReceiptNotificationsValue(numberOfPackets)
        .setPacketsReceiptNotificationsEnabled(enablePRNs)
        .setDeviceName(device.getName())
        .setKeepBond(keepBond)
        .setForceDfu(forceDfu);
    starter.setZip(mFileUri);
    Timber.v("dfuUpdated: %s - %s file uri:%s", device.getName(),
device.getAddress(), mFileUri);
    starter.start(this, DfuService.class);
}

private void showUploadCancelDialog() {
    final LocalBroadcastManager manager =
LocalBroadcastManager.getInstance(this);
    final Intent pauseAction = new Intent(DfuService.BROADCAST_ACTION);
    pauseAction.putExtra(DfuService.EXTRA_ACTION, DfuService.ACTION_PAUSE);
    manager.sendBroadcast(pauseAction);
    final UploadCancelFragment fragment =
UploadCancelFragment.getInstance();
    fragment.show(getSupportFragmentManager(), TAG);
}

private void showProgressBar() {
    mTextPercentage.setVisibility(View.VISIBLE);
    mTextPercentage.setText(null);
    mTextUploading.setVisibility(View.VISIBLE);
    mTextUploading.setText("Updating...");
    mSelectFileButton.setEnabled(false);
}

```

```

        mUploadButton.setEnabled(true);
        mUploadButton.setText(R.string.dfu_action_upload_cancel);
    }

    private void onTransferCompleted() {
        clearUI(true);
        showToast(getString(R.string.dfu_success));
    }

    public void onUploadCanceled() {
        clearUI(false);
        showToast(getString(R.string.dfu_status_aborted));
    }

    @Override
    public void onCancelUpload() {
        mTextUploading.setText("Cancel Upload...");
        mTextPercentage.setText(null);
    }

    private void showErrorMessage(final String message) {
        clearUI(false);
        showToast("Update Failed: " + message);
    }

    private void clearUI(final boolean clearDevice) {
        mTextPercentage.setVisibility(View.INVISIBLE);
        mTextUploading.setVisibility(View.INVISIBLE);
        mSelectFileButton.setEnabled(true);
        mUploadButton.setEnabled(false);
        mUploadButton.setText(R.string.dfu_action_upload);
        mFileStatusView.setText(null);
        mFileNameView.setText(null);
        mFileSizeView.setText(null);
        mStatusOk = false;
    }

    private boolean isDfuServiceRunning() {
        final ActivityManager manager = (ActivityManager)
getSystemService(Context.ACTIVITY_SERVICE);
        if (manager != null) {
            for (RunningServiceInfo service :
manager.getRunningServices(Integer.MAX_VALUE)) {
                if
(DfuService.class.getName().equals(service.service.getClassName())) {
                    return true;
                }
            }
        }
        return false;
    }
}

```

```

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample\dfu
// File: DfuService.java

package com.chileaf.cl831.sample.dfu;

import android.app.Activity;

import no.nordicsemi.android.dfu.DfuBaseService;

public class DfuService extends DfuBaseService {

    @Override
    protected Class<? extends Activity> getNotificationTarget() {
        /*
         * As a target activity the NotificationActivity is returned, not the
         MainActivity. This is because the notification must create a new task:
         *
         * intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
         *
         * when user press it. Using NotificationActivity we can check whether
         the new activity is a root activity (that means no other activity was open
         before)
         * or that there is other activity already open. In the later case the
         notificationActivity will just be closed. System will restore the previous
         activity.
         * However if the application has been closed during upload and user
         click the notification a NotificationActivity will be launched as a root
         activity.
         * It will create and start the main activity and terminate itself.
         *
         * This method may be used to restore the target activity in case the
         application was closed or is open. It may also be used to recreate an activity
         * history (see NotificationActivity).
         */
        return NotificationActivity.class;
    }

    @Override
    protected boolean isDebug() {
        return true;
    }
}

```

```

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample\dfu
// File: NotificationActivity.java

package com.chileaf.cl831.sample.dfu;

import android.app.Activity;
import android.content.Intent;

```

```

import android.os.Bundle;

import com.chileaf.cl831.sample.MainActivity;

public class NotificationActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        // If this activity is the root activity of the task, the app is
not running
        if (isTaskRoot()) {
            // Start the app before finishing
            final Intent parentIntent = new Intent(this,
MainActivity.class);
            parentIntent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
            final Intent startAppIntent = new Intent(this,
DfuActivity.class);
            startAppIntent.putExtras(getIntent().getExtras());
            startActivities(new Intent[] { parentIntent,
startAppIntent });
        }
        // Now finish, which will drop the user in to the activity that
was at the top
        // of the task stack
        finish();
    }
}

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample\dfu
// File: PermissionRationaleFragment.java

package com.chileaf.cl831.sample.dfu;

import android.app.Dialog;
import android.content.Context;
import android.os.Bundle;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AlertDialog;
import androidx.fragment.app.DialogFragment;

import com.chileaf.cl831.sample.R;

public class PermissionRationaleFragment extends DialogFragment {

    private static final String ARG_PERMISSION = "ARG_PERMISSION";
    private static final String ARG_TEXT = "ARG_TEXT";

    private PermissionDialogListener mListener;

    public interface PermissionDialogListener {

```

```

        void onRequestPermission(final String permission);
    }

    @Override
    public void onAttach(final Context context) {
        super.onAttach(context);

        if (context instanceof PermissionDialogListener) {
            mListener = (PermissionDialogListener) context;
        } else {
            throw new IllegalArgumentException("The parent activity must
implement PermissionDialogListener");
        }
    }

    @Override
    public void onDetach() {
        super.onDetach();
        mListener = null;
    }

    public static PermissionRationaleFragment getInstance(final int aboutResId,
final String permission) {
        final PermissionRationaleFragment fragment = new
PermissionRationaleFragment();

        final Bundle args = new Bundle();
        args.putInt(ARG_TEXT, aboutResId);
        args.putString(ARG_PERMISSION, permission);
        fragment.setArguments(args);

        return fragment;
    }

    @Override
    @NonNull
    public Dialog onCreateDialog(final Bundle savedInstanceState) {
        final Bundle args = getArguments();
        final StringBuilder text = new
StringBuilder(getString(args.getInt(ARG_TEXT)));
        return new
AlertDialog.Builder(getActivity()).setTitle(R.string.permission_required).setMes
sage(text)
                .setNegativeButton(R.string.no, null)
                .setPositiveButton(R.string.yes, (dialog, which) ->
mListener.onRequestPermission(args.getString(ARG_PERMISSION))).create();
    }
}

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample\dfu
// File: UploadCancelFragment.java

```

```

package com.chileaf.cl831.sample.dfu;

import android.app.Dialog;
import android.content.Context;
import android.content.DialogInterface;
import android.content.Intent;
import android.os.Bundle;
import android.util.Log;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AlertDialog;
import androidx.fragment.app.DialogFragment;
import androidx.localbroadcastmanager.content.LocalBroadcastManager;

import com.chileaf.cl831.sample.R;

/**
 * When cancel button is pressed during uploading this fragment shows uploading
 * cancel dialog
 */
public class UploadCancelFragment extends DialogFragment {

    private static final String TAG = "UploadCancelFragment";

    private CancelFragmentListener mListener;

    public interface CancelFragmentListener {
        void onCancelUpload();
    }

    public static UploadCancelFragment getInstance() {
        return new UploadCancelFragment();
    }

    @Override
    public void onAttach(final Context context) {
        super.onAttach(context);
        try {
            mListener = (CancelFragmentListener) context;
        } catch (final ClassCastException e) {
            Log.d(TAG, "The parent Activity must implement CancelFragmentListener interface");
        }
    }

    @NonNull
    @Override
    public Dialog onCreateDialog(final Bundle savedInstanceState) {
        return new
AlertDialog.Builder(getActivity()).setTitle(R.string.dfu_confirmation_dialog_title).setMessage(R.string.dfu_upload_dialog_cancel_message).setCancelable(false).setPositiveButton(R.string.yes, (dialog, whichButton) -> {
            final LocalBroadcastManager manager =

```

```

LocalBroadcastManager.getInstance(getActivity());
        final Intent pauseAction = new
Intent(DfuService.BROADCAST_ACTION);
        pauseAction.putExtra(DfuService.EXTRA_ACTION,
DfuService.ACTION_ABORT);
        manager.sendBroadcast(pauseAction);
        mListener.onCancelUpload();
    }).setNegativeButton(R.string.no, (dialog, which) ->
dialog.cancel()).create();
    }

    @Override
    public void onCancel(final DialogInterface dialog) {
        final LocalBroadcastManager manager =
LocalBroadcastManager.getInstance(getActivity());
        final Intent pauseAction = new Intent(DfuService.BROADCAST_ACTION);
        pauseAction.putExtra(DfuService.EXTRA_ACTION, DfuService.ACTION_RESUME);
        manager.sendBroadcast(pauseAction);
    }
}

```

```

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample\multi
// File: DeviceAdapter.java

```

```

package com.chileaf.cl831.sample.multi;

import android.annotation.SuppressLint;
import android.bluetooth.BluetoothDevice;
import android.widget.TextView;

import androidx.annotation.NonNull;

import com.chad.library.adapter.base.BaseQuickAdapter;
import com.chad.library.adapter.base.viewholder.BaseViewHolder;
import com.chileaf.cl831.sample.R;

import java.util.ArrayList;

@SuppressLint({"SetTextI18n", "MissingPermission"})
public class DeviceAdapter extends BaseQuickAdapter<DeviceItem, BaseViewHolder>
{

    public DeviceAdapter() {
        super(R.layout.item_device, new ArrayList<>());
        addChildClickViewIds(R.id.btn_disconnect);
    }

    @Override
    protected void convert(BaseViewHolder helper, DeviceItem item) {
        TextView tvName = helper.getView(R.id.name);
        TextView tvAddress = helper.getView(R.id.address);
        TextView tvHeart = helper.getView(R.id.heart);
    }
}

```



```

        TextView tvStep = helper.getView(R.id.step);
        TextView tvDistance = helper.getView(R.id.distance);
        TextView tvCalorie = helper.getView(R.id.calorie);
        tvName.setText(item.device.getName());
        tvAddress.setText(item.device.getAddress());
        tvHeart.setText(item.heartRate + "BPM");
        tvStep.setText(item.step + "steps");
        tvDistance.setText(item.distance / 100f + "m");
        tvCalorie.setText(item.calorie / 10f + "KCal");
    }

    public void addDevice(DeviceItem item) {
        getData().add(item);
        notifyDataSetChanged();
    }

    public void removeDevice(BluetoothDevice device) {
        DeviceItem item = getItem(device);
        if (item != null) {
            getData().remove(item);
            notifyDataSetChanged();
        }
    }

    public DeviceItem getItem(BluetoothDevice device) {
        for (DeviceItem item : getData()) {
            if (item.device.getAddress() == device.getAddress()) {
                return item;
            }
        }
        return null;
    }

    public void onSoftwareVersion(@NonNull BluetoothDevice device, String
software) {
        DeviceItem item = getItem(device);
        if (item != null) {
            item.version = software;
            notifyItemChanged(getData().indexOf(item));
        }
    }

    public void onBatteryLevelChanged(@NonNull final BluetoothDevice device,
final int batteryLevel) {
        DeviceItem item = getItem(device);
        if (item != null) {
            item.battery = batteryLevel;
            notifyItemChanged(getData().indexOf(item));
        }
    }

    public void onHeartRateMeasurementReceived(@NonNull BluetoothDevice device,
int heartRate) {
        DeviceItem item = getItem(device);

```

```

        if (item != null) {
            item.heartRate = heartRate;
            notifyItemChanged(getData().indexOf(item));
        }
    }

    public void onSportReceived(@NonNull BluetoothDevice device, int step, int
distance, int calorie) {
        DeviceItem item = getItem(device);
        if (item != null) {
            item.step = step;
            item.distance = distance;
            item.calorie = calorie;
            notifyItemChanged(getData().indexOf(item));
        }
    }
}

```

```

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample\multi
// File: DeviceItem.java

```

```

package com.chileaf.cl831.sample.multi;

import android.bluetooth.BluetoothDevice;

public class DeviceItem {

    public BluetoothDevice device;
    public String version;
    public int heartRate;
    public int step;
    public int distance;
    public int calorie;
    public int battery;

    public DeviceItem(BluetoothDevice device) {
        this.device = device;
    }
}

```

```

// Directory: app
// Subdirectory: src\main\java\com\chileaf\cl831\sample\multi
// File: MultiConnectActivity.java

```

```

package com.chileaf.cl831.sample.multi;

import android.annotation.SuppressLint;
import android.bluetooth.BluetoothDevice;
import android.bluetooth.BluetoothGatt;

```

```

import android.os.Bundle;
import android.widget.Button;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.appcompat.app.AlertDialog;
import androidx.recyclerview.widget.DefaultItemAnimator;
import androidx.recyclerview.widget.DividerItemDecoration;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.android.chileaf.WearManager;
import com.android.chileaf.fitness.FitnessManager;
import com.android.chileaf.fitness.callback.WearManagerCallbacks;
import com.chileaf.cl831.sample.BaseActivity;
import com.chileaf.cl831.sample.BuildConfig;
import com.chileaf.cl831.sample.R;
import com.chileaf.cl831.sample.ScannerFragment;
import com.hjq.permissions.OnPermissionCallback;
import com.hjq.permissions.XXPermissions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;

import timber.log.Timber;

@SuppressLint("MissingPermission")
public class MultiConnectActivity extends BaseActivity implements
ScannerFragment.OnDeviceSelectedListener, WearManagerCallbacks {

    private final List<BluetoothDevice> mManagedDevices = new ArrayList<>();
    private final HashMap<BluetoothDevice, FitnessManager<WearManagerCallbacks>>
mBleManagers = new HashMap<>();

    private Button mBtnConnect;
    private RecyclerView mRvDevice;

    private final DeviceAdapter mAdapter = new DeviceAdapter();

    @Override
    protected int layoutId() {
        return R.layout.activity_multi_connect;
    }

    @Override
    protected void initView() {
        mBtnConnect = findViewById(R.id.btn_connect);
        mRvDevice = findViewById(R.id.rv_device);

        mRvDevice.addItemDecoration(new DividerItemDecoration(this,
DividerItemDecoration.VERTICAL));
        mRvDevice.setLayoutManager(new LinearLayoutManager(this));

```

```

mRvDevice.setItemAnimator(new DefaultItemAnimator());
mRvDevice.setHasFixedSize(true);

mAdapter.setOnItemClickListener((adapter, view, position) -> {
    DeviceItem item = mAdapter.getItem(position);
    if (item != null) {
        disconnect(item.device);
    }
});

mRvDevice.setAdapter(mAdapter);

mBtnConnect.setOnClickListener(view -> {
    if (isBLEEnabled()) {
        showDeviceScanningDialog();
    } else {
        showBLEDialDialog();
    }
});
}

@Override
protected void initData(Bundle savedInstanceState) {
    isBLESupported();
    if (!isBLEEnabled()) {
        showBLEDialDialog();
    }
}

public WearManager getWearManager(final BluetoothDevice device) {
    return (WearManager) mBleManagers.get(device);
}

public List<BluetoothDevice> getConnectedDevices() {
    final List<BluetoothDevice> list = new ArrayList<>();
    for (BluetoothDevice device : mManagedDevices) {
        final WearManager manager = getWearManager(device);
        if (manager != null && manager.isConnected()) {
            list.add(device);
        }
    }
    return Collections.unmodifiableList(list);
}

public void connect(final BluetoothDevice device) {
    // If a device is in managed devices it means that it's already
connected, or was connected
    // using autoConnect and the link was lost but Android is already trying
to connect to it.
    FitnessManager<WearManagerCallbacks> manager = mBleManagers.get(device);
    if (manager == null) {
        manager = new WearManager(this);
    }
    mBleManagers.put(device, manager);
}

```

```

        manager.setManagerCallbacks(this);
        manager.setDebug(BuildConfig.DEBUG);
        if (!mManagedDevices.contains(device)) {
            mManagedDevices.add(device);
        }
        Timber.v("Connect:%s %s %s", device.getName(), device.getAddress(),
manager.toString());
        manager.connect(device)
            .retry(2, 100)
            .useAutoConnect(false)
            .timeout(10000)
            .fail((d, status) -> {
                mManagedDevices.remove(device);
                mBleManagers.remove(device);
            })
            .enqueue();
    }

    public final void disconnect(final BluetoothDevice device) {
        final WearManager manager = getWearManager(device);
        if (manager != null) {
            manager.disconnect().enqueue();
        }
    }

    public final void disconnectAll() {
        for (BluetoothDevice device : getConnectedDevices()) {
            disconnect(device);
        }
    }

    public final boolean isConnected(final BluetoothDevice device) {
        final WearManager manager = getWearManager(device);
        return manager != null && manager.isConnected();
    }

    public final boolean isReady(final BluetoothDevice device) {
        final WearManager manager = getWearManager(device);
        return manager != null && manager.isReady();
    }

    public final int getConnectionState(final BluetoothDevice device) {
        final WearManager manager = getWearManager(device);
        return manager != null ? manager.getConnectionState() :
BluetoothGatt.STATE_DISCONNECTED;
    }

    private void showDeviceScanningDialog() {
        if (isLocationEnabled(this)) {
            XXPermissions.with(this)
                .permission(getPermissions())
                .request(new OnPermissionCallback() {
                    @Override
                    public void onGranted(@NonNull List<String> permissions,

```

```

boolean allGranted) {
    if (allGranted) {
        runOnUiThread(() -> {
            final ScannerFragment dialog =
ScannerFragment.getInstance();
            dialog.show(getSupportFragmentManager(),
"scan_fragment");
        });
    } else {
        showToast("permission is denied");
    }
}

@Override
public void onDenied(@NonNull List<String> permissions,
boolean doNotAskAgain) {
    if (doNotAskAgain) {
        new
AlertDialog.Builder(MultiConnectActivity.this)
.setTitle(getString(R.string.permission_required))
.setMessage(getString(R.string.permission_location_info))
.setPositiveButton(getString(R.string.yes), (dialog, which) -> {
    onPermissionSettings();
})
.setNegativeButton(getString(R.string.no), null)
        .show();
    } else {
        showToast("permission is denied");
    }
}
});
} else {
    new AlertDialog.Builder(this)
        .setTitle(getString(R.string.location_permission_title))
        .setMessage(getString(R.string.location_permission_info))
        .setPositiveButton("OK", (dialog, which) -> {
            onEnableLocation();
        })
        .setNegativeButton("Cancel", null)
        .show();
}
}

@Override
public void onDeviceSelected(BluetoothDevice device, String name) {
    showToast("Connect " + device.getName());
    connect(device);
}

@Override

```

```

    public void onError(@NonNull BluetoothDevice device, @NonNull String
message, int errorCode) {
        Timber.e("onError: (" + errorCode + ")");
        showToast(message + " (" + errorCode + ")");
    }

    @Override
    public void onDeviceNotSupported(@NonNull BluetoothDevice device) {
        showToast(getString(R.string.not_supported));
    }

    @Override
    public void onSoftwareVersion(@NonNull BluetoothDevice device, String
software) {
        runOnUiThread(() -> mAdapter.onSoftwareVersion(device, software));
    }

    @Override
    public void onBatteryLevelChanged(@NonNull final BluetoothDevice device,
final int batteryLevel) {
        runOnUiThread(() -> mAdapter.onBatteryLevelChanged(device,
batteryLevel));
    }

    @Override
    public void onHeartRateMeasurementReceived(@NonNull BluetoothDevice device,
int heartRate, @Nullable Boolean contactDetected, @Nullable Integer
energyExpanded, @Nullable List<Integer> rrIntervals) {
        runOnUiThread(() -> mAdapter.onHeartRateMeasurementReceived(device,
heartRate));
    }

    @Override
    public void onSportReceived(@NonNull BluetoothDevice device, int step, int
distance, int calorie) {
        runOnUiThread(() -> mAdapter.onSportReceived(device, step, distance,
calorie));
    }

    @Override
    public void onDeviceConnected(@NonNull BluetoothDevice device) {
        showToast("Add " + device.getName());
        runOnUiThread(() -> mAdapter.addDevice(new DeviceItem(device)));
    }

    @Override
    public void onDeviceDisconnected(@NonNull final BluetoothDevice device) {
        showToast("Remove " + device.getName());
        runOnUiThread(() -> mAdapter.removeDevice(device));
    }

    @Override
    public void onLinkLossOccurred(@NonNull BluetoothDevice device) {
        showToast("LinkLoss " + device.getName());
    }

```

```
        runOnUiThread(() -> mAdapter.removeDevice(device));
    }

    @Override
    public void onBackPressed() {
        disconnectAll();
        super.onBackPressed();
    }
}
```